DEVOPS ENGINEERING MASTERY

IBM CLOUD & MULTI-CLOUD ARCHITECTURE

IBM Cloud Infrastructure, VPC, IAM, Virtual Servers, Object Storage, Red Hat OpenShift, Hybrid Cloud, Multi-Cloud Networking, Security & Enterprise Cloud Operations

VOLUME 14 • FIRST EDITION

```
nahid@technova:~$ ibmcloud login --sso
✓ Logged in to IBM Cloud — account: technova-prod
nahid@technova:~$ ibmcloud ks cluster ls
✓ technova-ocp — OpenShift 4.x — normal — 3 zones
nahid@technova:~$ ibmcloud is vpcs
```

IBM Cloud • VPC • IAM • OpenShift • Kubernetes • Terraform • Ansible

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Cloud Engineering — IBM Cloud & Multi-Cloud Architecture

DEVOPS ENGINEERING MASTERY — Volume 14

IBM Cloud Infrastructure, VPC, IAM, Virtual Servers, Object Storage, Red Hat OpenShift, Hybrid Cloud, Multi-Cloud Networking, Security ও Enterprise Cloud Operations

First Edition — Published 9 November 2026.

Hands-on environment: Ubuntu Server 24.04 LTS, IBM Cloud CLI (`ibmcloud`), Terraform with the IBM Cloud provider, Ansible, kubectl and the OpenShift CLI (`oc`), with a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...). Sample terminal outputs are illustrative. Cloud chapters name the resources that incur charges and always end with a cleanup step — never leave cloud resources running after a lab. Official provider documentation (IBM Cloud, AWS, Azure, Google Cloud, Terraform, Kubernetes, Red Hat) is the authority; no pricing or service limits are stated in this book. Multi-cloud is never presented as automatically better than single-cloud: every multi-cloud decision in this book is weighed against its operational complexity, egress cost and staffing cost.

How to use this book. Read the parts in order — each part builds on the previous one. Keep an IBM Cloud-ready machine beside you and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy commands blindly; read the explanation under each command and configuration field first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on production infrastructure: use disposable cloud accounts, never paste a real API key or secret into an example, and never commit credentials to Git.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test environment, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **TROUBLESHOOTING** helps you fix a failure, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Infrastructure changes should first be practised in a disposable environment, never directly on production systems. The author and publisher accept no liability for data loss, service disruption or cloud charges caused by running commands without a backup, a plan review or an approval gate.

Who this book is for

A learner who knows Linux, networking, Git and scripting, understands Docker and Kubernetes, has studied AWS, Azure and GCP fundamentals, and now wants to engineer on IBM Cloud and design enterprise hybrid and multi-cloud platforms — from a first VPC to a governed, multi-provider production platform with OpenShift, Terraform, centralized observability and disaster recovery.

Learning outcomes

By the end of this book you will be able to: explain IBM Cloud architecture, accounts, Resource Groups and IAM from first principles; design VPC networks with subnets, Security Groups, ACLs and private connectivity; deploy and operate Virtual Servers, load balancers, Object Storage and managed databases; run Kubernetes and Red Hat OpenShift workloads with Operators, Routes and RBAC; automate across clouds with Terraform and Ansible; compare single-cloud, hybrid and multi-cloud architectures honestly, including their costs; build centralized multi-cloud observability and security; and design an enterprise multi-cloud platform with disaster recovery, FinOps and operational runbooks.

সূচিপত্র — Contents

PART I — IBM Cloud Foundations & Architecture

Chapter 01 — IBM Cloud and Enterprise Cloud Computing 7

Chapter 02 — IBM Cloud Accounts and Governance 14

Chapter 03 — IBM Cloud IAM 21

Chapter 04 — IBM Cloud CLI and API Automation 28

Chapter 05 — Global Infrastructure, Responsibility and Billing 35

PART II — Networking & Infrastructure

Chapter 06 — IBM Cloud VPC Architecture 43

Chapter 07 — VPC Routing and Security 51

Chapter 08 — Load Balancing and DNS 57

Chapter 09 — IBM Cloud Virtual Servers 63

Chapter 10 — Hybrid Connectivity 69

Chapter 11 — High Availability and Resilience 75

PART III — Storage, Database & Application Services

Chapter 12 — IBM Cloud Object Storage 82

Chapter 13 — Block and File Storage 93

Chapter 14 — Managed Database Engineering 100

Chapter 15 — Kubernetes and Red Hat OpenShift 107

Chapter 16 — Container Registry and Application Hosting 113

PART IV — Multi-Cloud & Platform Engineering

Chapter 17 — Multi-Cloud Architecture Fundamentals 120

Chapter 18 — Hybrid Cloud Networking 129

Chapter 19 — Multi-Cloud Kubernetes and OpenShift 135

Chapter 20 — Cross-Cloud Automation 141

PART V — Enterprise Multi-Cloud Operations

Chapter 21 — Multi-Cloud Observability 148

Chapter 22 — Multi-Cloud Security 153

Chapter 23 — Disaster Recovery, Reliability and FinOps 159

Chapter 24 — Enterprise Multi-Cloud Architecture 165

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (150+ Terms) • CLI, IAM, VPC & Multi-Cloud References • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *IBM Cloud & Multi-Cloud Lab Workbook* . MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, CLI, IAM, VPC and multi-cloud references, checklists and decision trees are in the separate *Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

IBM Cloud Foundations & Architecture

IBM Cloud ও Enterprise Cloud Computing, Account ও Governance, IBM Cloud IAM, CLI ও API Automation, Global Infrastructure, Responsibility ও Billing।

অধ্যায় ১ • PART I — IBM Cloud Foundations & Architecture

Chapter 1 — IBM Cloud and Enterprise Cloud Computing

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক পারবেন — (১) ক্লাউড কম্পিউটিংয়ের মডেলগুলোকে প্রথম নীতি থেকে ব্যাখ্যা করতে; (২) IBM Cloud-এর অবস্থান, ইতিহাসগত প্রেক্ষাপট ও এন্টারপ্রাইজ বৈশিষ্ট্য চিহ্নিত করতে; (৩) পাবলিক, প্রাইভেট ও হাইব্রিড ক্লাউডের মধ্যে পার্থক্য ও ট্রেড-অফ বিশ্লেষণ করতে; (৪) রিজিয়ন (Region) ও জোন (Zone) ধারণা দিয়ে উচ্চ-উপলভ্যতা নকশা করতে; (৫) TechNova-র মতো একটি কাল্পনিক প্রতিষ্ঠানের জন্য ক্লাউড আর্কিটেকচারের নীতিমালা প্রয়োগ করতে। প্রতিটি উদ্দেশ্য পরের অধ্যায়গুলোর ভিত্তি — অ্যাকাউন্ট, IAM, CLI ও বিলিং বোঝার আগে এই মানসিক মডেলটি অপরিহার্য।

- ক্লাউডকে শুধু “অন্যের কম্পিউটার” হিসেবে নয়, একটি বিতরণকৃত অপারেটিং ব্যবস্থা হিসেবে বুঝতে পারা।
- On-demand self-service, resource pooling, rapid elasticity ও measured service — এই চার বৈশিষ্ট্য দিয়ে যেকোনো সেবাকে যাচাই করা।
- IBM Cloud-এর গ্লোবাল অবকাঠামোর মানচিত্র মনে মনে আঁকা ও ব্যর্থতার ডোমেইন আলাদা করা।
- এন্টারপ্রাইজ ব্যবহার-কেসে নিয়ন্ত্রণ, সম্মতি (compliance) ও ডেটা সার্বভৌমত্বের দাবি মেলানো।

১.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

এই অধ্যায়ে ঢোকার আগে পাঠকের জানা থাকা উচিত: একটি সার্ভার কী — CPU, মেমরি, ডিস্ক ও নেটওয়ার্কের সমন্বয়; ভার্চুয়ালাইজেশন কীভাবে একটি ভৌত মেশিনকে একাধিক ভার্চুয়াল মেশিনে ভাগ করে; নেটওয়ার্কের মৌলিক ধারণা — IP ঠিকানা, DNS, লোড ব্যালান্সার; এবং লিনাক্স টার্মিনালে মৌলিক কমান্ড চালানো। TechNova-র প্রসঙ্গটি মনে রাখুন: এটি একটি কাল্পনিক মাঝারি আকারের সফটওয়্যার কোম্পানি, যার একটি ওয়েব অ্যাপ্লিকেশন, একটি API সার্ভার ও একটি PostgreSQL ডেটাবেস আছে। তাদের ডোমেইন technova.example এবং অভ্যন্তরীণ ঠিকানা 203.0.113.x — এগুলো কেবল উদাহরণ, বাস্তব ঠিকানা নয়। যদি ভার্চুয়ালাইজেশন অস্পষ্ট মনে হয়, ভাবুন একটি বড় বাড়িকে ভাড়া দেওয়া অ্যাপার্টমেন্টে ভাগ করার মতো — প্রতিটি ভাড়াটে নিজের দরজা, চাবি ও বিদ্যুৎ মিটার পায়, কিন্তু ভবনের ভিত্তি একটাই। ক্লাউড সেই ভাগাভাগিকে স্বয়ংক্রিয়, পরিমাপযোগ্য ও চাহিদা অনুযায়ী করে তোলে।

আরও একটি পুনরালোচনা: ক্যাপেক্স (CapEx) বনাম ওপেক্স (OpEx)। নিজের ডেটা সেন্টার কেনা মানে বড় এককালীন খরচ (CapEx) ও দীর্ঘ প্রতিশ্রুতি; ক্লাউডে ভাড়া করা মানে ব্যবহার অনুযায়ী চলমান খরচ (OpEx)। এই পার্থক্য শুধু হিসাববিজ্ঞানের নয় — এটি সিদ্ধান্তের গতিকে বদলে দেয়। TechNova যখন নিজের সার্ভার কিনত, তখন একটি নতুন পরিবেশ পেতে সপ্তাহ লাগত; ক্লাউডে সেই একই পরিবেশ মিনিটে তৈরি হয়, আবার না লাগলে মুছে ফেলা যায়। এই গতিই এন্টারপ্রাইজ ক্লাউডের মূল প্রতিশ্রুতি, তবে এর সঙ্গে আসে নতুন দায়িত্ব — খরচ নিয়ন্ত্রণ, নিরাপত্তা ও পরিচালনার শৃঙ্খলা।

> **CONCEPT:** ক্লাউড কম্পিউটিং হলো নেটওয়ার্কের মাধ্যমে চাহিদা অনুযায়ী কনফিগারযোগ্য কম্পিউটিং সম্পদ (সার্ভার, স্টোরেজ, নেটওয়ার্ক, অ্যাপ্লিকেশন) পাওয়ার মডেল, যা দ্রুত সরবরাহ করা যায় এবং ন্যূনতম ব্যবস্থাপনা প্রচেষ্টায় ছেড়ে দেওয়া যায়। সংজ্ঞার প্রতিটি শব্দ গুরুত্বপূর্ণ — “চাহিদা অনুযায়ী” মানে অপেক্ষা নয়, “পরিমাপযোগ্য” মানে অদৃশ্য খরচ নয়।

১.৩ প্রথম নীতি থেকে ব্যাখ্যা: সমস্যাটি কী ছিল?

ক্লাউডের জন্ম কোনো ফ্যাশন থেকে হয়নি; এটি একটি অর্থনৈতিক ও প্রকৌশলগত সমস্যার সমাধান। ঐতিহ্যবাহী মডেলে প্রতিটি প্রতিষ্ঠানকে সর্বোচ্চ চাহিদা (peak load) অনুযায়ী সার্ভার কিনতে হতো। TechNova-র উদাহরণ নিন: সাধারণ দিনে তাদের অ্যাপে ১,০০০ ব্যবহারকারী, কিন্তু একটি প্রচারণার দিনে ২০,০০০। নিজের হার্ডওয়্যার কিনলে ২০,০০০-এর জন্য কিনতে হবে, অথচ বছরের ৩৫০ দিন তার ৯৫% বসে থাকবে। এটাই অপচয়। আবার কম কিনলে প্রচারণার দিনে সাইট ভেঙে পড়বে। ক্লাউড এই দ্বন্দ্ব ভাঙে — সম্পদকে একটি ভাগ করা পুল (shared pool) থেকে নেওয়া হয়, চাহিদা বাড়লে বাড়ানো হয়, কমলে ছেড়ে দেওয়া হয়।

দ্বিতীয় সমস্যা: সময়। একটি ভৌত সার্ভার অর্ডার, ডেলিভারি, র‍্যাকিং, কেবলিং ও কনফিগারেশনে সপ্তাহ লাগে। একটি স্টার্টআপ বা একটি নতুন পণ্য দলের কাছে সপ্তাহ মানে সুযোগ হারানো। ক্লাউডে ভার্চুয়াল সার্ভার একটি API কলের মাধ্যমে তৈরি হয় — কারণ অবকাঠামো আগেই তৈরি, কেবল বরাদ্দটুকু বাকি। তৃতীয় সমস্যা: ব্যর্থতা। একটি সার্ভার নষ্ট হলে পুরো সেবা বন্ধ; কিন্তু যদি সেবাটি একাধিক স্বাধীন ভবনে ছড়ানো থাকে, একটি ভবনের বিদ্যুৎ গেলেও অন্যটি চলবে। এই ধারণাই পরে রিজিয়ন ও জোনে পরিণত হয়।

ক্লাউডের তিনটি সেবা মডেল এই সমস্যার বিভিন্ন স্তরে উত্তর দেয়। IaaS (Infrastructure as a Service) দেয় কাঁচা অবকাঠামো — ভার্চুয়াল সার্ভার, স্টোরেজ, নেটওয়ার্ক — যেখানে অপারেটিং সিস্টেম থেকে ওপরের সবকিছু ব্যবহারকারীর। PaaS (Platform as a Service) দেয় প্রস্তুত প্ল্যাটফর্ম — ডেটাবেস, রানটাইম, কুবারনেটিস — যেখানে কোড লেখাই মূল কাজ, সার্ভার প্যাচ করা নয়। SaaS (Software as a Service) দেয় সম্পূর্ণ অ্যাপ্লিকেশন — ইমেইল বা CRM — যেখানে ব্যবহারকারী কেবল ব্যবহার করে। TechNova-র জন্য: তাদের কাস্টম API IaaS বা PaaS-এ চলতে পারে, তাদের ডেটাবেস পরিচালিত (managed) সেবা হিসেবে PaaS-এ নেওয়া যেতে পারে, আর অফিসের ইমেইল SaaS-এ। কোনো মডেলই সর্বত্র সেরা নয়; নিয়ন্ত্রণ বনাম সুবিধার ট্রেড-অফ প্রতিটি স্তরে বদলায়।

মডেল	প্রদানকারী পরিচালনা করে	ব্যবহারকারী পরিচালনা করে	TechNova উদাহরণ	কখন ব্যবহার করবেন
IaaS	ভৌত হার্ডওয়্যার, ভার্চুয়ালাইজেশন, নেটওয়ার্ক	OS, রানটাইম, অ্যাপ, ডেটা	Virtual Server, VPC	পূর্ণ নিয়ন্ত্রণ চাইলে, বিশেষ OS দরকার হলে
PaaS	OS, রানটাইম, প্যাচ, স্কেলিং	অ্যাপ কোড ও ডেটা	পরিচালিত ডেটাবেস, কনটেইনার প্ল্যাটফর্ম	দ্রুত ডেলিভারি, কম অপারেশন চাইলে
SaaS	প্রায় সবকিছু	কনফিগারেশন ও ডেটা	ইমেইল, টিকিটিং	মূল ব্যবসা নয় এমন কাজে
Serverless / FaaS	সার্ভার, স্কেলিং, প্যাচ	ফাংশন কোড	ইভেন্ট-চালিত কাজ, ছবি প্রক্রিয়াকরণ	অনিয়মিত, ছোট কাজে

ডেপ্লয়মেন্ট মডেলও একইভাবে বুঝতে হবে। পাবলিক ক্লাউডে সম্পদ বহু গ্রাহকের মধ্যে ভাগ করা হয়, তবে যৌক্তিকভাবে (logically) বিচ্ছিন্ন থাকে — যেমন একই ভবনে আলাদা অ্যাপার্টমেন্ট। প্রাইভেট ক্লাউডে সম্পদ কেবল একটি প্রতিষ্ঠানের জন্য — নিজের ডেটা সেন্টারে বা প্রদানকারীর ডেডিকেটেড পরিবেশে। হাইব্রিড ক্লাউডে দুটোকে সংযুক্ত করা হয় — সংবেদনশীল ডেটা প্রাইভেটে, পরিবর্তনশীল লোড পাবলিকে। মাল্টি-ক্লাউড হলো একাধিক প্রদানকারী ব্যবহার করা — এই বইয়ের পরের অংশে আলোচ্য। গুরুত্বপূর্ণ সতর্কতা: হাইব্রিড বা মাল্টি-ক্লাউড স্বয়ংক্রিয়ভাবে ভালো নয়; এটি জটিলতা, খরচ ও পরিচালনার বোঝা বাড়ায়। সুবিধা তখনই, যখন নির্দিষ্ট ব্যবসায়িক কারণ — যেমন নিয়ন্ত্রক প্রয়োজন, ঝুঁকি ছড়ানো, বা নির্দিষ্ট সেবার শ্রেষ্ঠত্ব — সেই জটিলতার দাম দেয়।

১.৪ IBM Cloud পরিচিতি: কেন এন্টারপ্রাইজ প্রসঙ্গে এটি আলাদা?

IBM Cloud একটি এন্টারপ্রাইজ-কেন্দ্রিক পাবলিক ক্লাউড প্ল্যাটফর্ম, যার শিকড় IBM-এর দীর্ঘ এন্টারপ্রাইজ, মেইনফ্রেম ও হাইব্রিড অভিজ্ঞতায়। এর পরিচয় তিনটি স্তম্ভে বোঝা যায়। প্রথম, হাইব্রিড ও ওপেন ভিত্তি: IBM Cloud-এর নকশায় অন-প্রিমিসেস, প্রাইভেট ও পাবলিক পরিবেশকে একসঙ্গে চালানোর ধারণা কেন্দ্রীয় — বিশেষ করে Red Hat OpenShift-ভিত্তিক কনটেইনার প্ল্যাটফর্মের মাধ্যমে, যা একই অ্যাপকে বিভিন্ন পরিবেশে চালাতে সাহায্য করে। দ্বিতীয়, এন্টারপ্রাইজ নিয়ন্ত্রণ: IAM, রিসোর্স গ্রুপ, অডিট লগ ও নীতি-ভিত্তিক শাসন (governance) শুরু থেকেই প্ল্যাটফর্মের অংশ। তৃতীয়, শিল্প-নির্দিষ্ট সম্মতি: ব্যাংকিং, স্বাস্থ্য ও সরকারি কাজে প্রয়োজনীয় নিয়ন্ত্রণ কাঠামোর দিকে বিশেষ মনোযোগ — যদিও কোনো নির্দিষ্ট সার্টিফিকেশনের দাবি করার আগে সর্বদা IBM Cloud-এর বর্তমান সম্মতি নথি যাচাই করতে হবে, কারণ সার্টিফিকেশন ও সুযোগ সময়ের সঙ্গে বদলায়।

IBM Cloud-এ ক্যাটালগ (Catalog) ধারণাটি কেন্দ্রীয়। ক্যাটালগ হলো সেবার বাজার — কম্পিউট, স্টোরেজ, ডেটাবেস, AI, ইন্টিগ্রেশন, নিরাপত্তা, পর্যবেক্ষণসহ শতাধিক সেবা। প্রতিটি সেবার একটি পরিকল্পনা (plan) থাকে — যেমন Lite (সীমিত বিনামূল্যের স্তর, যেখানে প্রযোজ্য), Standard বা Enterprise — এবং প্রতিটি পরিকল্পনার মূল্য ও সীমা আলাদা। এই বইয়ে কোনো নির্দিষ্ট মূল্য উল্লেখ করা হবে না; মূল্য যাচাই করতে হবে IBM Cloud ক্যাটালগ ও মূল্য ক্যালকুলেটরে, কারণ মূল্য অঞ্চল, পরিকল্পনা ও ব্যবহারের ওপর নির্ভর করে। TechNova যখন একটি পরিচালিত PostgreSQL নেবে, তারা প্রথমে ক্যাটালগে পরিকল্পনা, অঞ্চল ও মূল্য অনুমান দেখবে, তারপর সিদ্ধান্ত নেবে — অনুমানের ভিত্তিতে বাজেট নয়, ক্যালকুলেটরের ভিত্তিতে।

> **REAL WORLD:** TechNova তাদের পুরনো মনোলিথকে এক রাতে ক্লাউডে তুলতে চেয়েছিল। অভিজ্ঞ স্থপতি থামালেন: প্রথমে কেবল স্ট্যাটিক ওয়েবসাইট ও স্টেজিং পরিবেশ ক্লাউডে নেওয়া হলো, প্রোডাকশন ডেটাবেস পরে। তিন সপ্তাহে দল IAM, নেটওয়ার্ক ও বিলিং শিখল — তারপর প্রোডাকশন মাইগ্রেশন হলো কোনো বড় ঘটনা ছাড়াই। ক্লাউড মাইগ্রেশন একটি প্রকল্প নয়, একটি শেখার ক্রম।

১.৫ পাবলিক, প্রাইভেট ও হাইব্রিড ক্লাউড: গভীর তুলনা

পাবলিক ক্লাউডের শক্তি অর্থনীতি ও গতি; দুর্বলতা নিয়ন্ত্রণের সীমা। আপনি ভৌত হার্ডওয়্যার দেখতে পান না, ডেটা কোন ডিস্কে আছে জানেন না — তবে চুক্তি, এনক্রিপশন ও অঞ্চল নির্বাচনের মাধ্যমে নিয়ন্ত্রণ পুনরুদ্ধার করা যায়। প্রাইভেট ক্লাউডের শক্তি নিয়ন্ত্রণ ও অনুমানযোগ্যতা; দুর্বলতা খরচ, গতি ও স্কেলের সীমা। হাইব্রিডের প্রতিশ্রুতি উভয়ের সেরা অংশ; বাস্তবতা — দুটি পরিবেশের মধ্যে নেটওয়ার্ক, পরিচয় (identity) ও পর্যবেক্ষণের সামঞ্জস্য রাখা একটি স্থায়ী প্রকৌশল কাজ। TechNova-র ক্ষেত্রে: গ্রাহকের ব্যক্তিগত ডেটা সংরক্ষণের নিয়ম যদি বলে ডেটা দেশের ভেতরে থাকতে হবে, তবে অঞ্চল নির্বাচন ও হাইব্রিড নকশা আইনি প্রয়োজন হয়ে ওঠে, প্রযুক্তিগত পছন্দ নয়।

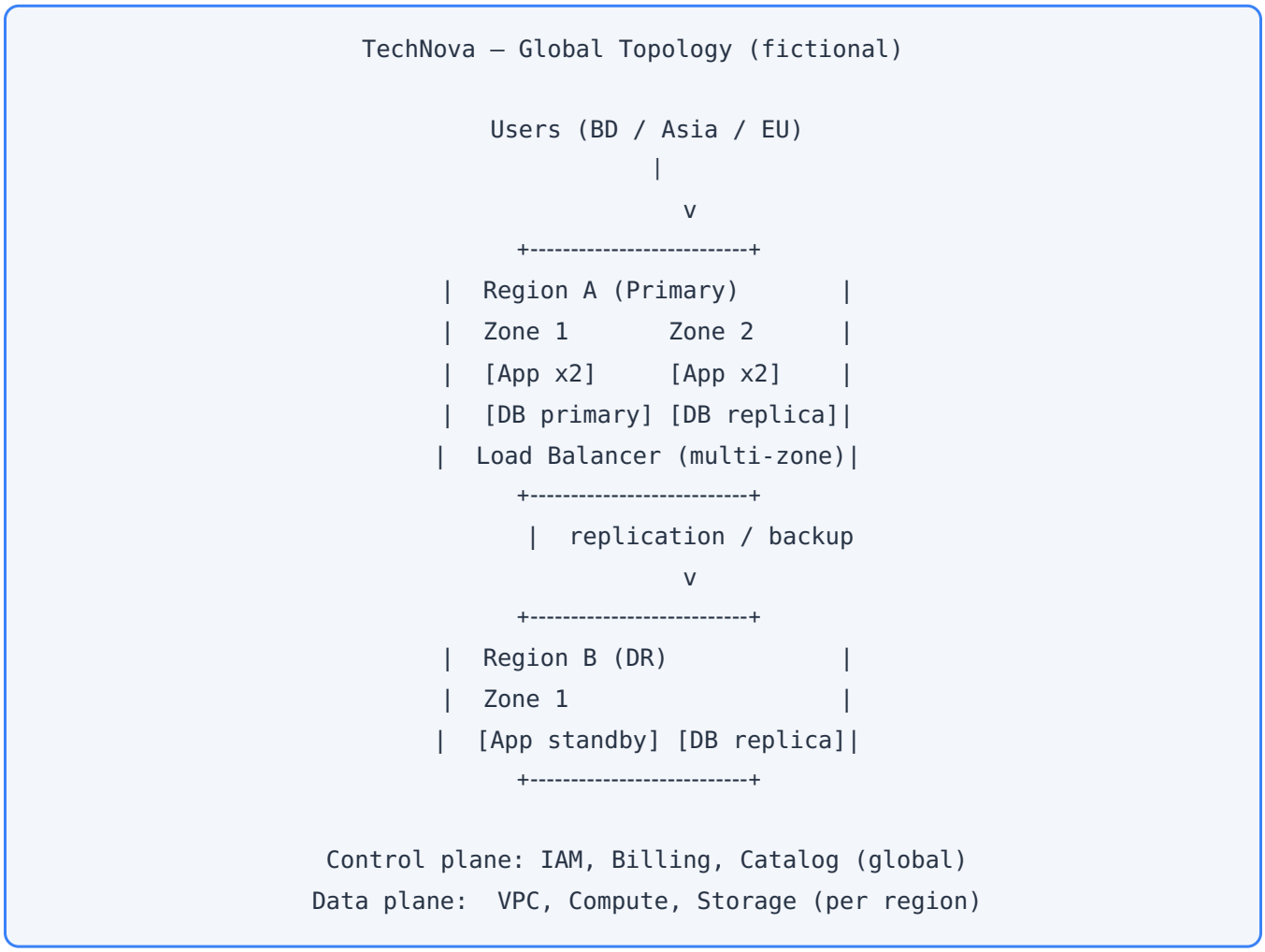
একটি সাধারণ ভুল: “প্রাইভেট মানেই নিরাপদ।” নিরাপত্তা আসে কনফিগারেশন, প্যাচ, পর্যবেক্ষণ ও প্রক্রিয়া থেকে — অবস্থান থেকে নয়। একটি খারাপভাবে পরিচালিত প্রাইভেট সার্ভার একটি ভালোভাবে পরিচালিত পাবলিক সেবার চেয়ে অনেক বেশি ঝুঁকিপূর্ণ। একইভাবে “পাবলিক মানেই সস্তা” ভুল। অলস (idle) ভার্চুয়াল সার্ভার, অপ্রয়োজনীয় স্টোরেজ ও ডেটা স্থানান্তর (egress) খরচ নীরবে বাড়ে। পরের অধ্যায়গুলোতে বিলিং ও ট্যাগিং দিয়ে এই খরচকে দৃশ্যমান করার উপায় শেখা হবে।

১.৬ গ্লোবাল অবকাঠামো: রিজিয়ন ও জোন

IBM Cloud-এর গ্লোবাল অবকাঠামোকে তিন স্তরে ভাবুন: রিজিয়ন, জোন ও ডেটা সেন্টার। একটি রিজিয়ন হলো একটি ভৌগোলিক এলাকায় অবস্থিত সেবার সমষ্টি — যেমন ইউরোপ বা উত্তর আমেরিকার একটি এলাকা। প্রতিটি রিজিয়নের

ভেতরে এক বা একাধিক জোন (Zone) থাকে, যেগুলো একে অপরের থেকে স্বাধীন ব্যর্থতার ডোমেইন — আলাদা ভবন, আলাদা বিদ্যুৎ ও কুলিং, আলাদা নেটওয়ার্ক পথ। এই বিচ্ছিন্নতাই উচ্চ-উপলভ্যতার ভিত্তি: একটি জোনে আগুন লাগলে বা বিদ্যুৎ গেলে অন্য জোন চলতে থাকে। তবে সব সেবা সব রিজিয়নে একইভাবে পাওয়া যায় না — সেবার উপলভ্যতা অঞ্চলভেদে ভিন্ন হয়। কোনো সেবা নির্দিষ্ট অঞ্চলে আছে বলে দাবি করার আগে ক্যাটালগে যাচাই করুন; এই বইয়ে অঞ্চলের নাম দিয়ে নির্দিষ্ট প্রতিশ্রুতি দেওয়া হবে না।

রিজিয়ন নির্বাচন চারটি বিষয়ের ভারসাম্য: লেটেন্সি (ব্যবহারকারীর কাছাকাছি হলে সাড়া দ্রুত), ডেটা আবাস (data residency — ডেটা কোথায় থাকতে পারে), সেবার উপলভ্যতা, ও মূল্য। TechNova-র ব্যবহারকারী যদি মূলত বাংলাদেশ ও দক্ষিণ এশিয়ায় থাকে, তবে কাছাকাছি অঞ্চল লেটেন্সি কমাবে; কিন্তু যদি তাদের গ্রাহকের চুক্তি বলে ডেটা একটি নির্দিষ্ট অঞ্চলে থাকতে হবে, তবে লেটেন্সির চেয়ে সম্মতি বড়। আর দুর্যোগ পুনরুদ্ধার (disaster recovery) ভাবতে হলে অন্তত দুটি অঞ্চলের কথা ভাবতে হবে — একটি প্রাথমিক, একটি ব্যাকআপ। এক অঞ্চলের ভেতরে একাধিক জোনে চালানো উচ্চ-উপলভ্যতা (high availability); দুটি অঞ্চলে চালানো দুর্যোগ সহনশীলতা (disaster resilience)। দুটি এক নয়, খরচও এক নয়।



কন্ট্রোল প্লেন বনাম ডেটা প্লেন পার্থক্যটি বুঝুন। IAM, বিলিং ও ক্যাটালগের মতো নিয়ন্ত্রণমূলক সেবা সাধারণত গ্লোবাল — এগুলো দিয়ে আপনি সব অঞ্চলের সম্পদ পরিচালনা করেন। কিন্তু আপনার অ্যাপ, ডেটাবেস ও স্টোরেজ থাকে নির্দিষ্ট অঞ্চলে — সেটাই ডেটা প্লেন। কন্ট্রোল প্লেনে সমস্যা হলে নতুন সম্পদ তৈরি করা কঠিন হতে পারে, কিন্তু চলমান অ্যাপ সাধারণত চলতে থাকে; ডেটা প্লেনে সমস্যা হলে সরাসরি ব্যবহারকারী আক্রান্ত হয়। এই পার্থক্য ট্রাবলশুটিংয়ের সময় দিকনির্দেশনা দেয়।

১.৭ এন্টারপ্রাইজ ক্লাউড ব্যবহার-কেস

এন্টারপ্রাইজে ক্লাউড পাঁচটি প্রধান কাজে ব্যবহৃত হয়। প্রথম, অ্যাপ্লিকেশন হোস্টিং ও আধুনিকীকরণ — পুরনো মনোলিথকে কনটেইনারে ভেঙে OpenShift বা কুবারনেটিসে চালানো। দ্বিতীয়, ডেটা ও বিশ্লেষণ — ডেটা লেক, ওয়্যারহাউস ও AI সেবায় ডেটা প্রক্রিয়াকরণ। তৃতীয়, দুর্যোগ পুনরুদ্ধার ও ব্যাকআপ — অন্য অঞ্চলে প্রতিলিপি রাখা। চতুর্থ, উন্নয়ন ও পরীক্ষা পরিবেশ — চাহিদা অনুযায়ী এপিমেরাল (ক্ষণস্থায়ী) পরিবেশ তৈরি ও মুছে ফেলা। পঞ্চম, হাইব্রিড ইন্টিগ্রেশন — অন-প্রিমিসেস সিস্টেমের সঙ্গে ক্লাউড সেবা সংযুক্ত করা, যেমন মেইনফ্রেমের ডেটা API দিয়ে প্রকাশ করা। TechNova-র ক্ষেত্রে সবচেয়ে মূল্যবান প্রথম ও চতুর্থটি: প্রোডাকশনকে স্থিতিশীল রেখে ডেভেলপমেন্টের গতি বাড়ানো।

প্রতিটি ব্যবহার-কেসের জন্য একই প্রশ্নগুলো করুন: সমস্যাটি কী? কেন ক্লাউড দরকার? অভ্যন্তরে কীভাবে কাজ করবে? অন্য সেবার সঙ্গে কীভাবে কথা বলবে? ব্যর্থ হলে কী হবে? কীভাবে সুরক্ষিত হবে? কীভাবে পর্যবেক্ষণ করা হবে? কীভাবে বিল হবে? কখন ব্যবহার করবেন, কখন এড়াবেন? বিকল্প কী? এই প্রশ্নমালা এই বইয়ের প্রতিটি সেবার বিশ্লেষণে ফিরে আসবে। উদাহরণ: পরিচালিত ডেটাবেসের সমস্যা হলো প্যাচ, ব্যাকআপ ও রেপ্লিকেশন পরিচালনা; এটি প্রয়োজন কারণ ডেটা হারানো ব্যবসা বন্ধ করে দেয়; অভ্যন্তরে এটি স্বয়ংক্রিয় ব্যাকআপ, স্ট্যান্ডবাই রেপ্লিকা ও পর্যবেক্ষণ চালায়; ব্যর্থ হলে রেপ্লিকা প্রোমোট হয়; নিরাপত্তা আসে IAM, নেটওয়ার্ক ও এনক্রিপশন থেকে; বিল হয় পরিকল্পনা ও ব্যবহার অনুযায়ী; ব্যবহার করুন যখন দল ছোট ও ডেটা গুরুত্বপূর্ণ, এড়ান যখন বিশেষ ইঞ্জিন বা কাস্টম এক্সটেনশন দরকার।

১.৮ ক্লাউড আর্কিটেকচারের নীতিমালা

ভালো ক্লাউড আর্কিটেকচারের সাতটি নীতি মনে রাখুন। (১) ব্যর্থতাকে স্বাভাবিক ধরুন (design for failure): কোনো একক উপাদানের ওপর নির্ভর করবেন না; স্বাস্থ্য পরীক্ষা (health check) ও স্বয়ংক্রিয় পুনরুদ্ধার নকশায় রাখুন। (২) ঢিলেঢালাভাবে সংযুক্ত করুন (loose coupling): সেবাগুলো API ও বার্তা কিউ দিয়ে কথা বলুক, সরাসরি ডেটাবেস ভাগ না করুক। (৩) অপরিবর্তনীয় অবকাঠামো (immutable infrastructure): সার্ভার প্যাচ করে ঠিক না করে, নতুন করে তৈরি করুন — এতে পরিবেশ পুনরুৎপাদনযোগ্য থাকে। (৪) স্বয়ংক্রিয় করুন: হাতে ক্লিক করে তৈরি পরিবেশ ডকুমেন্ট করা যায় না, কোড দিয়ে তৈরি পরিবেশ যায়। (৫) নিরাপত্তা শুরু থেকে: নেটওয়ার্ক বিভাজন, ন্যূনতম সুযোগ ও এনক্রিপশন পরে যোগ করা যায় না। (৬) পর্যবেক্ষণযোগ্যতা: লগ, মেট্রিক ও ট্রেস ছাড়া বিতরণকৃত ব্যবস্থা অন্ধ। (৭) খরচ সচেতনতা: প্রতিটি স্থাপত্য সিদ্ধান্তের মূল্য আছে; ট্যাগিং ও বাজেট ছাড়া খরচ অদৃশ্য থাকে।

> **WARNING:** একটিমাত্র জোনে প্রোডাকশন ডেটাবেস চালানো মানে একটিমাত্র ভবনের ওপর ব্যবসা বাজি রাখা। উন্নয়ন পরিবেশে এটি ঠিক আছে; প্রোডাকশনে নয়। মাল্টি-জোন স্থাপন শুরুতেই নকশা করুন — পরে যোগ করা অনেক কঠিন ও ঝুঁকিপূর্ণ।

১.৯ অভ্যন্তরীণ কার্যপ্রণালী: অনুরোধ কীভাবে সম্পদে পরিণত হয়?

যখন আপনি কনসোল বা CLI দিয়ে একটি ভার্চুয়াল সার্ভার চান, তখন যা ঘটে তা একটি পাইপলাইন: প্রমাণীকরণ (আপনি কে), অনুমোদন (আপনার কি অধিকার আছে), যাচাই (অঞ্চলে কোটা ও সক্ষমতা আছে কি), বরাদ্দ (হাইপারভাইজার একটি স্লট দেয়), নেটওয়ার্ক সংযুক্তি (VPC, সাবনেট, নিরাপত্তা গ্রুপ), স্টোরেজ সংযুক্তি, ও বুট। প্রতিটি ধাপে ব্যর্থতা সম্ভব — কোটা শেষ, অঞ্চলে সক্ষমতা নেই, নেটওয়ার্ক নীতি বাধা দিচ্ছে। তাই “সার্ভার তৈরি হয়নি” একটি উপসর্গ, রোগ নয়; লগ ও ত্রুটি কোড দেখে ধাপ চিহ্নিত করতে হয়। পরিচালিত সেবাগুলো এই পাইপলাইনের ওপরে আরও একটি স্তর যোগ করে: অপারেটর (operator) নামের স্বয়ংক্রিয় নিয়ন্ত্রক ক্রমাগত কাঙ্ক্ষিত অবস্থা (desired state) ও প্রকৃত অবস্থা মিলিয়ে দেখে — রেপ্লিকা কমে গেলে নতুন তৈরি করে, ডিস্ক ভরলে সতর্ক করে।

১.১০ CLI / API উদাহরণ

এই অধ্যায়ে CLI শুধু পরিচিতির জন্য; বিস্তারিত অধ্যায় ৪-এ। নিচের কমান্ডগুলো আকৃতি বোঝায় — চালানোর আগে লগইন ও টার্গেট যাচাই করুন।

```
ibmcloud login
ibmcloud target --cf
ibmcloud regions
ibmcloud catalog search "databases"
```

প্রথম কমান্ড প্রমাণীকরণ করে, দ্বিতীয়টি প্রসঙ্গ (কোন অ্যাকাউন্ট, অঞ্চল, রিসোর্স গ্রুপ) নির্ধারণ করে, তৃতীয়টি উপলভ্য অঞ্চল দেখায়, চতুর্থটি ক্যাটালগে সেবা খোঁজে। ভুল টার্গেটে সম্পদ তৈরি করা নবীনদের সবচেয়ে সাধারণ ভুল — একটি কমান্ডের আগে `ibmcloud target` দেখে নেওয়া অভ্যাস করুন।

১.১১ IaC উদাহরণ — যেখানে প্রাসঙ্গিক

এই অধ্যায়ের ধারণাকে কোডে প্রকাশ করার সবচেয়ে ছোট উদাহরণ হলো একটি রিসোর্স গ্রুপকে Terraform-এ ঘোষণা করা (বিস্তারিত অধ্যায় ৪-এ)। নীতিটি মনে রাখুন: ক্লিকে তৈরি পরিবেশ পুনরায় তৈরি করা যায় না; কোডে লেখা পরিবেশ যায়।

```
# Illustrative only — provider version & auth in Ch.4
resource "ibm_resource_group" "technova" {
  name = "technova-prod"
}
```

১.১২ প্রোডাকশন পরিস্থিতি: TechNova-র প্রথম নকশা

TechNova সিদ্ধান্ত নিল: প্রাইমারি অঞ্চলে দুটি জোনে অ্যাপ চালাবে, ডেটাবেসের রেপ্লিকা দ্বিতীয় জোনে, স্ট্যাটিক সম্পদ অবজেক্ট স্টোরেজে, ব্যাকআপ অন্য অঞ্চলে। স্টেজিং হবে একটিমাত্র জোনে, ছোট আকারে — কারণ স্টেজিংয়ে ব্যর্থতার খরচ কম। ডেভেলপমেন্ট পরিবেশ হবে এপিমেরাল: প্রতিটি ফিচার শাখার জন্য তৈরি হবে, মার্জের পর মুছে যাবে। এই পার্থক্য ইচ্ছাকৃত — প্রতিটি পরিবেশের উদ্দেশ্য অনুযায়ী স্থিতিস্থাপকতা ও খরচ নির্ধারণ করা হয়েছে। একটি বাস্তব সিদ্ধান্ত: প্রোডাকশনে মাল্টি-জোন লোড ব্যালান্সার ব্যবহার করা হলো, কিন্তু স্টেজিংয়ে নয় — সাশ্রয় ও সরলতার জন্য।

১.১৩ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: ডেটা বিশ্রামে (at rest) ও চলাচলে (in transit) এনক্রিপশন, নেটওয়ার্ক বিভাজন, IAM ন্যূনতম সুযোগ, অডিট লগ — এগুলো শুরু থেকে। Shared Responsibility Model-এর মূল কথা: প্রদানকারী ক্লাউডের নিরাপত্তা দেয়, আপনি ক্লাউডে যা রাখেন তার নিরাপত্তা আপনার। খরচ: মূল্য ক্যালকুলেটরে অনুমান করুন, ট্যাগ দিয়ে পরিবেশ ও দল অনুযায়ী খরচ ভাগ করুন, অলস সম্পদ নিয়মিত পর্যালোচনা করুন, বাজেট সতর্কতা (alert) বসান। কোনো নির্দিষ্ট মূল্য এই বইয়ে উল্লেখ করা হবে না — ক্যাটালগ ও ক্যালকুলেটরই কর্তৃপক্ষ।

১.১৪ সাধারণ ভুল

- সবকিছু একটি জোনে বা একটি অঞ্চলে রাখা, তারপর “ক্লাউড মানেই নিরাপদ” ভাবা।
- মূল্য না দেখে সবচেয়ে বড় আকারের সার্ভার নেওয়া, তারপর বিল দেখে অবাক হওয়া।
- স্টেজিং ও প্রোডাকশনকে একই অ্যাকাউন্ট ও নেটওয়ার্কে মেশানো।

- ডকুমেন্ট ছাড়া হাতে ক্লিক করে পরিবেশ তৈরি করা — পরে কেউ পুনরায় তৈরি করতে পারে না।
- হাইব্রিড/মাল্টি-ক্লাউডকে ডিফল্ট হিসেবে নেওয়া, ব্যবসায়িক কারণ ছাড়া জটিলতা বাড়ানো।

১.১৫ ট্রাবলশুটিং

TROUBLESHOOTING: সার্ভার তৈরি হচ্ছে না? ক্রমে যাচাই করুন: (১) `ibmcloud target` — সঠিক অ্যাকাউন্ট/অঞ্চল/রিসোর্স গ্রুপ কি? (২) কোটা ও সেবার উপলভ্যতা — ঐ অঞ্চলে ঐ আকার পাওয়া যায় কি? (৩) IAM অনুমতি — রিসোর্স তৈরি করার অধিকার আছে কি? (৪) নেটওয়ার্ক — VPC ও সাবনেট ঠিক আছে কি? ত্রুটি বার্তার কোডটি নোট করুন; একই বার্তা দিয়ে সমর্থন (support) খোঁজা অনেক দ্রুত হয়।

অ্যাপ ধীর চলছে? প্রথমে লেটেন্সি কোথায় তা আলাদা করুন: ব্যবহারকারী থেকে লোড ব্যালান্সার, ব্যালান্সার থেকে অ্যাপ, অ্যাপ থেকে ডেটাবেস — তিনটি পর্ব আলাদা করে মাপুন। অধিকাংশ সময় সমস্যা ডেটাবেস কুয়েরি বা ভুল অঞ্চল নির্বাচনে থাকে, কম্পিউট শক্তিতে নয়। লগ দেখার আগে অনুমান করে সার্ভার বড় করবেন না।

১.১৬ সারসংক্ষেপ

ক্লাউড হলো চাহিদা অনুযায়ী, পরিমাপযোগ্য, ভাগ করা কম্পিউটিং সম্পদের মডেল। IaaS/PaaS/SaaS নিয়ন্ত্রণ বনাম সুবিধার বিভিন্ন স্তর দেয়; পাবলিক/প্রাইভেট/হাইব্রিড বিভিন্ন নিয়ন্ত্রণ ও খরচের ভারসাম্য দেয়। IBM Cloud এন্টারপ্রাইজ হাইব্রিড ও ওপেন ভিত্তির ওপর দাঁড়িয়ে আছে, যার গ্লোবাল অবকাঠামো রিজিয়ন ও জোনে বিভক্ত। ভালো আর্কিটেকচার ব্যর্থতা ধরে নিয়ে নকশা করে, ঢিলেঢালাভাবে সংযুক্ত করে, স্বয়ংক্রিয় করে ও শুরু থেকে নিরাপদ রাখে। TechNova-র প্রথম নকশা এই নীতিগুলোর প্রয়োগ — মাল্টি-জোন প্রোডাকশন, ছোট স্টেজিং, এপিমেরাল ডেভেলপমেন্ট।

১.১৭ পর্যালোচনা প্রশ্ন

1. IaaS, PaaS ও SaaS-এর মধ্যে নিয়ন্ত্রণের পার্থক্য TechNova-র API-র ক্ষেত্রে কীভাবে সিদ্ধান্ত বদলায়?
2. একটি জোন ব্যর্থ হলে মাল্টি-জোন স্থাপন কেন চলতে থাকে, আর একক-জোন স্থাপন কেন থেমে যায়?
3. হাইব্রিড ক্লাউড কখন যুক্তিযুক্ত, আর কখন এটি কেবল জটিলতা বাড়ায়?
4. কন্ট্রোল প্লেন ও ডেটা প্লেনের পার্থক্য ট্রাবলশুটিংয়ে কীভাবে সাহায্য করে?
5. রিজিয়ন নির্বাচনে লেটেন্সি, ডেটা আবাস ও সেবার উপলভ্যতার মধ্যে কীভাবে ভারসাম্য করবেন?

১.১৮ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য এক পৃষ্ঠার স্থাপত্য নোট লিখুন: কোন অংশ কোন অঞ্চল/জোনে থাকবে, কেন, এবং ব্যর্থ হলে কী হবে — মূল্য উল্লেখ না করে ক্যাটালগে যাচাইয়ের ধাপ লিখুন।
2. IBM Cloud ক্যাটালগ ঘুরে তিনটি সেবা (একটি কম্পিউট, একটি ডেটাবেস, একটি স্টোরেজ) চিহ্নিত করুন এবং প্রতিটির জন্য এই বইয়ের প্রশ্নমালা — সমস্যা, ব্যর্থতা, নিরাপত্তা, বিল — দিয়ে এক প্যারাগ্রাফ করে লিখুন।

অধ্যায় ২ • PART I — IBM Cloud Foundations & Architecture

Chapter 2 — IBM Cloud Accounts and Governance

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক পারবেন — (১) IBM Cloud অ্যাকাউন্টের কাঠামো ব্যাখ্যা করতে; (২) Resource Group দিয়ে সম্পদ সংগঠিত করতে; (৩) প্রজেক্ট ও সম্পদ-সংগঠনের ধারণা প্রয়োগ করতে; (৪) IAM-সক্ষম সম্পদ ও অ্যাকাউন্ট গভর্ন্যান্সের সম্পর্ক বোঝাতে; (৫) ডেভেলপমেন্ট, স্টেজিং ও প্রোডাকশন পরিবেশ আলাদা করতে; (৬) বিলিং সংগঠনের মাধ্যমে খরচের দায়িত্ব নির্ধারণ করতে। TechNova-র প্রসঙ্গে: একটি অ্যাকাউন্টে সবকিছু রাখা মানে একটি চাবি দিয়ে পুরো বাড়ি খোলা — আলাদা করা মানে আগুন এক ঘরে আটকে রাখা।

- অ্যাকাউন্ট, রিসোর্স গ্রুপ ও ট্যাগের মধ্যে স্তরভেদ চিহ্নিত করা।
- এন্টারপ্রাইজে একাধিক অ্যাকাউন্ট কখন দরকার, কখন অপ্রয়োজনীয় জটিলতা — তা বিচার করা।
- পরিবেশ পৃথকীকরণের নকশা লেখা ও তার ট্রেড-অফ ব্যাখ্যা করা।
- বিলিংকে সংগঠন কাঠামোর সঙ্গে মেলানো, যাতে প্রতিটি খরচের মালিক থাকে।

২.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

অধ্যায় ১ থেকে মনে করুন: রিজিয়ন ও জোন, IaaS/PaaS পার্থক্য, Shared Responsibility-র ধারণা। এবার যোগ করুন সংগঠনের ধারণা: একটি কোম্পানিতে দল, প্রকল্প ও পরিবেশ থাকে — অর্থ দল, পণ্য দল, প্ল্যাটফর্ম দল। ক্লাউডে এই সংগঠনকে প্রতিফলিত না করলে দুটি সমস্যা হয়: কে কী তৈরি করল তা জানা যায় না, আর বিল এলে কে দেবে তা জানা যায় না। একটি ফোল্ডার কাঠামোর কথা ভাবুন: কম্পিউটারে ফাইল ছড়িয়ে রাখলে খুঁজে পাওয়া যায় না; ফোল্ডার, নামকরণ ও লেবেল থাকলে খুঁজে পাওয়া যায়। Resource Group ও ট্যাগ সেই ফোল্ডার ও লেবেলের ক্লাউড সংস্করণ। আর IAM (পরের অধ্যায়) হলো সেই ফোল্ডারের তালা — কে কোন ফোল্ডারে ঢুকতে পারবে। এই অধ্যায়ে তালা নয়, ঘরের বিন্যাস শেখা হবে।

২.৩ প্রথম নীতি থেকে ব্যাখ্যা: সংগঠন কেন প্রথম কাজ?

ক্লাউডে প্রথম সম্পদ তৈরি করা সহজ; শততম সম্পদ পরিচালনা করা কঠিন। সমস্যাটি স্কেলের: পাঁচটি সার্ভার মনে রাখা যায়, পাঁচশোটি যায় না। সংগঠন ছাড়া তিনটি ব্যর্থতা অনিবার্য। প্রথম, বিস্ফোরণ ব্যাসার্ধ (blast radius): ডেভেলপমেন্টে একটি ভুল কমান্ড প্রোডাকশন ডেটাবেস মুছে ফেলে — কারণ তারা একই জায়গায় ছিল। দ্বিতীয়, অদৃশ্য খরচ: কোন দল কত খরচ করছে জানা যায় না, তাই কেউ দায় নেয় না। তৃতীয়, অডিট অসম্ভব: নিরাপত্তা পরীক্ষায় কে কী করেছে প্রমাণ করা যায় না। অ্যাকাউন্ট ও রিসোর্স গ্রুপ এই তিনটির উত্তর — বিচ্ছিন্নতা, দৃশ্যমানতা ও জবাবদিহি।

একটি নীতি মনে রাখুন: পরিবেশ আলাদা করা মানে শুধু নাম আলাদা করা নয় — পরিচয়, নেটওয়ার্ক, অনুমতি ও বিলিংয়ের সীমানা আলাদা করা। “technova-prod” নামের একটি রিসোর্স গ্রুপে প্রোডাকশন রাখা ভালো শুরু, কিন্তু যদি একই ব্যক্তি ডেভ ও প্রোড উভয়ে অ্যাডমিন হয়, তবে সীমানা কেবল নামের। সত্যিকারের পৃথকীকরণের জন্য অ্যাকাউন্ট বা অন্তত আলাদা অ্যাক্সেস নীতি দরকার। ছোট দলে শুরুতে রিসোর্স গ্রুপ যথেষ্ট হতে পারে; দল ও ঝুঁকি বাড়লে অ্যাকাউন্ট পৃথকীকরণ বিবেচনা করুন — তবে অকারণে অনেক অ্যাকাউন্ট তৈরি করা নিজেই পরিচালনার বোঝা।

> **CONCEPT:** অ্যাকাউন্ট হলো বিলিং ও পরিচয়ের সর্বোচ্চ সীমানা; Resource Group হলো অ্যাকাউন্টের ভেতরে সম্পদ গুচ্ছ করার ও অ্যাক্সেস ও বিলিং দৃশ্যমানতা দেওয়ার একক। ট্যাগ হলো গ্রুপের ওপরে লাগানো লেবেল — পরিবেশ, দল, খরচ-কেন্দ্র চিহ্নিত করতে। তিনটি স্তর একসঙ্গে কাজ করে, একটি অন্যটির বিকল্প নয়।

২.৪ IBM Cloud অ্যাকাউন্ট: কী, কীভাবে কাজ করে?

IBM Cloud অ্যাকাউন্ট একটি প্রতিষ্ঠান বা ব্যক্তির ক্লাউড পরিচয় ও বিলিং সত্তা। এর ভেতরে থাকে: ব্যবহারকারী ও তাদের পরিচয় (IAM-এর বিষয়), রিসোর্স গ্রুপ, সম্পদ, বিলিং তথ্য, ও গভর্ন্যান্স নীতি। অ্যাকাউন্ট তৈরি করার সময় যে ইমেইল ও প্রতিষ্ঠানের তথ্য দেওয়া হয়, তা পরে বিলিং ও সমর্থনের ভিত্তি হয় — তাই এটি একটি প্রশাসনিক সিদ্ধান্ত, কেবল প্রযুক্তিগত নয়। এন্টারপ্রাইজ অ্যাকাউন্টে একাধিক অ্যাকাউন্টকে একটি এন্টারপ্রাইজ কাঠামোর অধীনে আনা যায়, যাতে কেন্দ্রীয় বিলিং ও নীতি প্রয়োগ করা যায় — এই ধারণাটি ২.৮-এ বিস্তারিত।

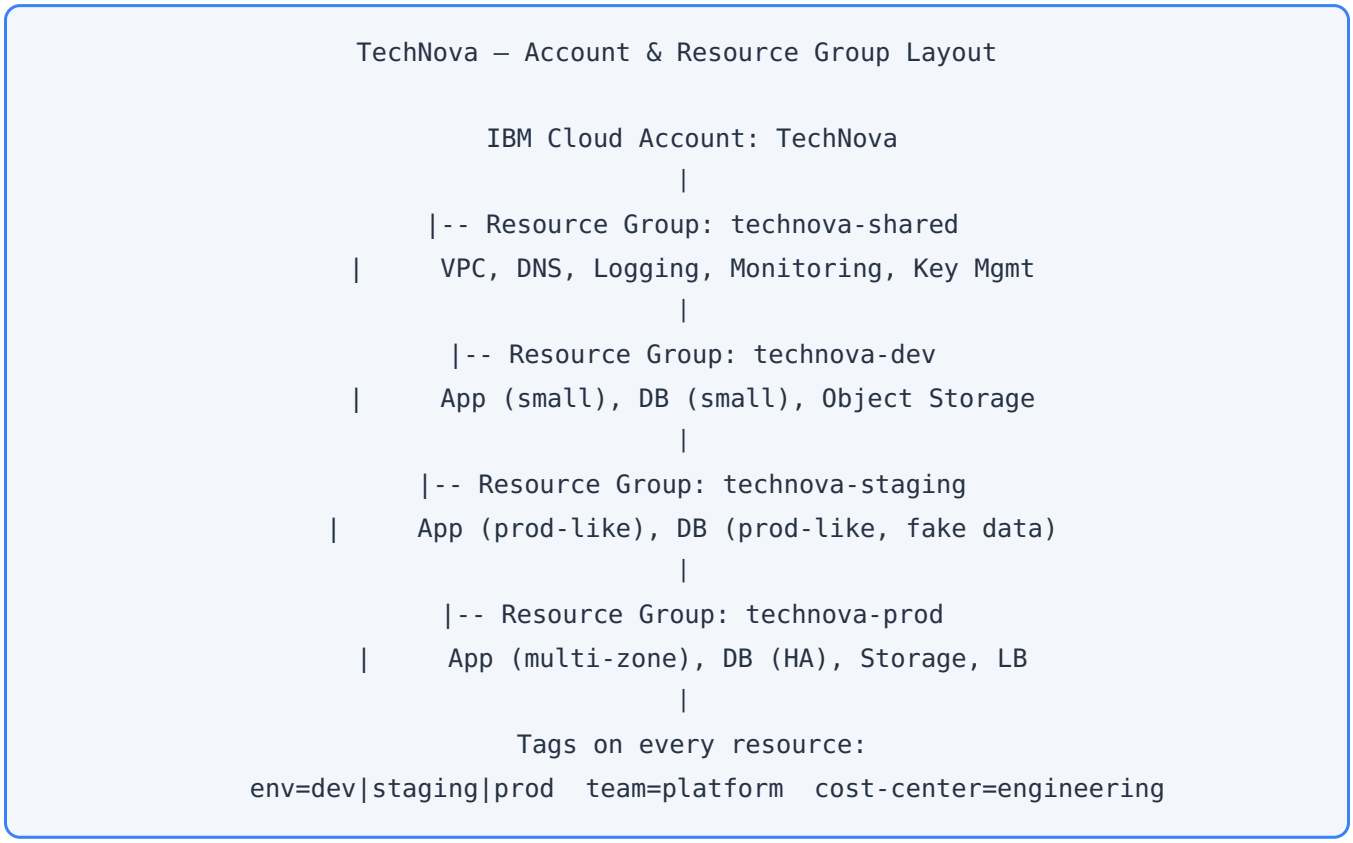
অ্যাকাউন্টের অভ্যন্তরীণ কাজ: যখন আপনি লগইন করেন, IBM Cloud আপনার পরিচয় যাচাই করে একটি টোকেন দেয়; সেই টোকেন দিয়ে আপনি যে অ্যাকাউন্টে সুইচ করেন, তার প্রসঙ্গে কাজ করেন। প্রতিটি API কল সেই অ্যাকাউন্ট প্রসঙ্গ বহন করে। এর মানে: ভুল অ্যাকাউন্টে থাকলে সঠিক কমান্ডও ভুল জায়গায় সম্পদ তৈরি করবে। তাই CLI-তে `ibmcloud target` ও কনসোলে অ্যাকাউন্ট নির্বাচক (account switcher) সবসময় যাচাই করুন। অ্যাকাউন্ট ব্যর্থ হলে কী হয়? অ্যাকাউন্ট স্থগিত হলে সম্পদ তৈরি ও পরিবর্তন বন্ধ হতে পারে, কিন্তু চলমান সম্পদের আচরণ নীতি ও বিলিং অবস্থার ওপর নির্ভর করে — তাই বিলিং সতর্কতা ও অ্যাকাউন্ট প্রশাসকের যোগাযোগ তথ্য হালনাগাদ রাখা অপারেশনাল দায়িত্ব।

স্তর	কী নিয়ন্ত্রণ করে	TechNova উদাহরণ	ব্যর্থ হলে
অ্যাকাউন্ট	বিলিং, পরিচয়, সামগ্রিক নীতি	TechNova মূল অ্যাকাউন্ট	বিলিং/অ্যাক্সেস বন্ধ হতে পারে
Resource Group	সম্পদ গুচ্ছ, অ্যাক্সেস, খরচ দৃশ্য	technova-prod, technova-dev	একটি গ্রুপের ভুল অন্য গ্রুপে ছড়ায় না
ট্যাগ	খরচ বরাদ্দ, অনুসন্ধান, নীতি	env:prod, team:platform	খরচের মালিকানা হারিয়ে যায়
প্রজেক্ট ধারণা	কাজের পরিসর, সংশ্লিষ্ট সম্পদ	“পেমেন্ট পুনর্লিখন” প্রকল্প	পরিসর অস্পষ্ট হয়

২.৫ Resource Groups: নকশা ও ব্যবহার

Resource Group হলো একটি অ্যাকাউন্টের ভেতরে সম্পদ সংগঠিত করার প্রধান একক। প্রতিটি সম্পদ ঠিক একটি রিসোর্স গ্রুপে থাকে। গ্রুপ দিয়ে আপনি করতে পারেন: (১) অ্যাক্সেস দেওয়া — একটি দলকে কেবল তাদের গ্রুপে সম্পদ দেখতে/পরিচালনা করতে দেওয়া; (২) খরচ দেখা — গ্রুপ অনুযায়ী বিল ভাগ করা; (৩) জীবনচক্র পরিচালনা — একটি প্রকল্প শেষ হলে তার গ্রুপের সম্পদ পর্যালোচনা করে মুছে ফেলা। TechNova-র সুপারিশকৃত শুরু: `technova-dev`, `technova-staging`, `technova-prod` — তিনটি গ্রুপ, তিনটি পরিবেশ। এরপর দল বাড়লে দল-ভিত্তিক গ্রুপ যোগ করা যেতে পারে, তবে পরিবেশ-ভিত্তিক বিভাজনই প্রথম।

নামকরণের শৃঙ্খলা গুরুত্বপূর্ণ: ছোট হাতের অক্ষর, হাইফেন, পরিবেশের নাম শেষে — যেমন `technova-prod`। নামে তারিখ বা ব্যক্তির নাম রাখবেন না। একটি গ্রুপে কী রাখবেন, আর কী রাখবেন না — তা লিখিত নীতিতে রাখুন। উদাহরণ নীতি: প্রোডাকশন গ্রুপে কেবল প্রোডাকশন সম্পদ; শেয়ার্ড নেটওয়ার্কিং ও নিরাপত্তা সেবা একটি আলাদা `technova-shared` গ্রুপে। শেয়ার্ড গ্রুপের ধারণা: VPC, DNS, লগিংয়ের মতো সব পরিবেশে ব্যবহৃত সম্পদ আলাদা রাখা, যাতে একটি পরিবেশ মুছলে শেয়ার্ড ভিত্তি না মুছে যায়।



২.৬ প্রজেক্ট ও সম্পদ-সংগঠনের ধারণা

“প্রজেক্ট” শব্দটি IBM Cloud-এ বিভিন্ন প্রসঙ্গে ব্যবহৃত হয় — কখনও একটি কাজের পরিসর, কখনও নির্দিষ্ট সেবার (যেমন Code Engine প্রজেক্ট বা Schematics workspace) সংগঠন একক। এই বইয়ে প্রজেক্টকে সাধারণ ধারণা হিসেবে বুঝুন: একটি নির্দিষ্ট উদ্দেশ্যে একত্রিত সম্পদ, দল ও সময়সীমার সমষ্টি। একটি প্রজেক্টের সম্পদ একাধিক রিসোর্স গ্রুপে ছড়াতে পারে, তবে ট্যাগ দিয়ে তাদের একত্রে চিহ্নিত করা যায় — যেমন `project:payment-rewrite`। প্রজেক্ট শেষ হলে ট্যাগ দিয়ে সেই সম্পদগুলো খুঁজে পর্যালোচনা করুন; অপ্রয়োজনীয়গুলো মুছুন। প্রজেক্ট-ভিত্তিক সংগঠনের বিকল্প হলো কেবল পরিবেশ-ভিত্তিক সংগঠন — ছোট দলে পরিবেশই যথেষ্ট, বড় দলে প্রজেক্ট ট্যাগ যোগ করুন।

২.৭ IAM-সক্ষম সম্পদ: অ্যাকাউন্টের সঙ্গে সম্পর্ক

সব IBM Cloud সম্পদ একইভাবে অনুমতি পরিচালনা করে না। IAM-সক্ষম (IAM-enabled) সম্পদ হলো সেগুলো যাদের অ্যাক্সেস IBM Cloud IAM নীতি দিয়ে নিয়ন্ত্রিত হয় — রিসোর্স গ্রুপ, ভূমিকা (role) ও নীতি (policy) দিয়ে। পুরনো Cloud Foundry-ভিত্তিক সম্পদে ভিন্ন মডেল থাকতে পারে। নতুন স্থাপনায় IAM-সক্ষম সেবা বেছে নেওয়া উচিত, কারণ এতে একই নীতি কাঠামো সব সেবায় প্রযোজ্য হয়। একটি সম্পদ তৈরি করার আগে ক্যাটালগে দেখুন সেটি IAM-সক্ষম কি না; যদি না হয়, তবে তার অ্যাক্সেস মডেল আলাদাভাবে নথিভুক্ত করুন। অভ্যন্তরে, IAM-সক্ষম সম্পদ প্রতিটি API কলে কলারের টোকেন ও নীতি যাচাই করে — অনুমতি না থাকলে কল প্রত্যাখ্যাত হয়, সম্পদে পৌঁছানোর আগেই।

২.৮ অ্যাকাউন্ট গভর্ন্যান্স ও এন্টারপ্রাইজ অ্যাকাউন্ট পৃথকীকরণ

গভর্ন্যান্স মানে নিয়ম, নীতি ও জবাবদিহির কাঠামো — কে কী করতে পারে, কীভাবে প্রমাণিত হয়, কীভাবে নিরীক্ষা হয়। এন্টারপ্রাইজে একাধিক অ্যাকাউন্ট ব্যবহারের কারণ: (১) বিস্ফোরণ ব্যাসার্ধ কমানো — একটি অ্যাকাউন্টের সমস্যা অন্যটিকে স্পর্শ করে না; (২) বিলিং আলাদা করা — ব্যবসায়িক ইউনিট অনুযায়ী; (৩) সম্মতি — নিয়ন্ত্রিত ডেটা আলাদা অ্যাকাউন্টে; (৪) স্বায়ত্তশাসন — দলগুলো নিজেদের গতিতে কাজ করতে পারে। বিপরীতে, অনেক অ্যাকাউন্টের খরচ:

কেন্দ্রীয় দৃশ্যমানতা কমে, নীতি ছড়িয়ে যায়, শেয়ার্ড সেবা জটিল হয়। TechNova-র মতো মাঝারি দলের জন্য শুরুতে একটি অ্যাকাউন্ট ও তিনটি রিসোর্স গ্রুপ; ব্যবসা বাড়লে বা নিয়ন্ত্রক দাবি এলে অ্যাকাউন্ট ভাগ করুন — আগেভাগে নয়। এন্টারপ্রাইজ অ্যাকাউন্ট কাঠামোতে মূল (parent) অ্যাকাউন্ট থেকে সন্তান অ্যাকাউন্ট তৈরি ও কেন্দ্রীয় বিলিং করা যায় — বিস্তারিত সেটআপের জন্য IBM Cloud-এর বর্তমান এন্টারপ্রাইজ নথি দেখুন, কারণ কাঠামো ও সীমা পরিবর্তিত হয়।

> **REAL WORLD:** একটি দল ডেভ ও প্রোড একই রিসোর্স গ্রুপে রেখেছিল কারণ "ছোট দল, আলাদা করার সময় নেই।" একদিন একজন ডেভেলপার পরীক্ষার জন্য একটি স্ক্রিপ্ট চালিয়ে প্রোডাকশন ডেটাবেসের সংযোগ স্ট্রিং মুছে ফেললেন। ডাউনটাইম ছিল ৪০ মিনিট; শিক্ষা ছিল স্থায়ী — পরদিনই আলাদা গ্রুপ ও আলাদা অ্যাক্সেস নীতি তৈরি হলো। পৃথকীকরণের খরচ একদিনের কাজ; না করার খরচ একটি ঘটনা।

২.৯ ডেভেলপমেন্ট, স্টেজিং ও প্রোডাকশন

তিনটি পরিবেশের উদ্দেশ্য আলাদা, তাই তাদের আকার, ডেটা ও সুরক্ষা আলাদা হওয়া উচিত। ডেভেলপমেন্ট: দ্রুত পরীক্ষা, ভাঙা চলবে, ভুয়া ডেটা, ছোট আকার, স্বয়ংক্রিয়ভাবে মুছে ফেলা যায়। স্টেজিং: প্রোডাকশনের মতো আকৃতি ও কনফিগারেশন, কিন্তু ভুয়া বা ছদ্মবেশী ডেটা, প্রোডাকশনের মতো নেটওয়ার্ক নীতি পরীক্ষা করা যায়। প্রোডাকশন: বাস্তব ডেটা, মাল্টি-জোন, ব্যাকআপ, কঠোর অ্যাক্সেস, পরিবর্তনের অনুমোদন। একটি সাধারণ ভুল: স্টেজিংকে প্রোডাকশনের চেয়ে অনেক ছোট রাখা — তখন স্টেজিংয়ে পাস করা পরিবর্তন প্রোডাকশনে ব্যর্থ হয়, কারণ স্কেল ও সময়সীমা ভিন্ন। স্টেজিং প্রোডাকশনের আয়না হওয়া উচিত, খেলনা নয়। আরেকটি ভুল: প্রোডাকশন ডেটা স্টেজিংয়ে কপি করা — এতে গোপনীয়তা ঝুঁকি তৈরি হয়; ছদ্মবেশী (masked) বা সংশ্লেষিত (synthetic) ডেটা ব্যবহার করুন।

বিষয়	Development	Staging	Production
উদ্দেশ্য	দ্রুত পরীক্ষা	প্রোড-সদৃশ যাচাই	বাস্তব ব্যবহারকারী
ডেটা	ভুয়া	ছদ্মবেশী/সংশ্লেষিত	বাস্তব, ব্যাকআপসহ
উপলভ্যতা	একক জোন যথেষ্ট	প্রোড-সদৃশ	মাল্টি-জোন
অ্যাক্সেস	দল, প্রশস্ত	সীমিত	কঠোর, অনুমোদনসহ
খরচ	সর্বনিম্ন, এপিমেরাল	মাঝারি	প্রয়োজন অনুযায়ী

২.১০ বিলিং সংগঠন: খরচের মালিকানা

বিলিং সংগঠন মানে প্রতিটি খরচকে একটি মালিকের সঙ্গে যুক্ত করা — রিসোর্স গ্রুপ, ট্যাগ (যেমন cost-center, team, project) ও অ্যাকাউন্ট কাঠামো দিয়ে। IBM Cloud কনসোলে বিলিং ও ব্যবহারের বিবরণ দেখা যায়; সেখানে গ্রুপ ও ট্যাগ অনুযায়ী ফিল্টার করে কোন পরিবেশ বা দল কত খরচ করছে তা জানা যায়। TechNova-র নিয়ম: প্রতিটি সম্পদে `env`, `team` ও `cost-center` ট্যাগ বাধ্যতামূলক; ট্যাগ ছাড়া সম্পদ তৈরি হলে সাপ্তাহিক পর্যালোচনায় ধরা পড়ে। বিলিংয়ের অভ্যন্তরীণ কাজ: প্রতিটি সম্পদ ব্যবহারের মিটারিং ডেটা তৈরি করে, তা বিলিং ব্যবস্থায় জমা হয়, তারপর পরিকল্পনা ও অঞ্চল অনুযায়ী মূল্যায়ন করা হয়। কোনো নির্দিষ্ট মূল্য এখানে দেওয়া হবে না — ক্যাটালগ ও মূল্য ক্যালকুলেটরে যাচাই করুন।

২.১১ অভ্যন্তরীণ কার্যপ্রণালী ও যোগাযোগ

একটি সম্পদ তৈরি হলে রিসোর্স কন্ট্রোলার (Resource Controller) তার রেকর্ড রাখে: কোন অ্যাকাউন্ট, কোন গ্রুপ, কোন অঞ্চল, কোন পরিকল্পনা। IAM নীতি সেই রেকর্ডের সঙ্গে মিলিয়ে অ্যাক্সেস দেয়। বিলিং সেই রেকর্ডের সঙ্গে মিটারিং যুক্ত করে।

একটি সম্পদ মুছলে রেকর্ড মুছে যায়, কিন্তু বিলিং ইতিহাস থেকে যায় — তাই "মুছে ফেলেছি, বিল আসবে না" সবসময় সত্য নয়; চলমান ব্যবহারের বিল থেকে যায়, আর কিছু সেবায় মুছে ফেলার পরেও স্ন্যাপশট বা ব্যাকআপের বিল আসতে পারে। তাই মুছে ফেলার আগে সংশ্লিষ্ট স্টোরেজ, স্ন্যাপশট ও IP-র তালিকা পরীক্ষা করুন।

২.১২ CLI / API উদাহরণ

```
ibmcloud login
ibmcloud account show
ibmcloud resource groups
ibmcloud resource group-create technova-dev
ibmcloud resource group-create technova-staging
ibmcloud resource group-create technova-prod
ibmcloud target -g technova-dev
ibmcloud resource groups --output JSON
```

প্রথমে লগইন ও অ্যাকাউন্ট যাচাই, তারপর গ্রুপ তালিকা দেখে নেওয়া, তারপর তৈরি করা — এই ক্রমটি অভ্যাস করুন। `ibmcloud target -g` দিয়ে গ্রুপ নির্ধারণ করলে পরের কমান্ডগুলো সেই গ্রুপে সম্পদ তৈরি করবে। ভুল গ্রুপে তৈরি সম্পদ সরানো কঠিন হতে পারে; অনেক ক্ষেত্রে মুছে নতুন করে তৈরি করতে হয় — তাই তৈরির আগে টার্গেট যাচাই করুন।

২.১৩ IaC উদাহরণ

```
resource "ibm_resource_group" "dev" {
  name = "technova-dev"
}
resource "ibm_resource_group" "staging" {
  name = "technova-staging"
}
resource "ibm_resource_group" "prod" {
  name = "technova-prod"
}
resource "ibm_resource_tag" "prod_tag" {
  resource_id = "crn:v1:bluemix:public:resource-controller::a/CHANGE-ME::resource-
group:CHANGE-ME"
  tags        = ["env:prod", "team:platform"]
}
```

উপরের ট্যাগ উদাহরণটি আকৃতি বোঝায়; CRN (Cloud Resource Name) আপনার পরিবেশের আসল মান দিয়ে প্রতিস্থাপন করতে হবে — কখনও অন্যের CRN বা গোপন মান কপি করবেন না। Terraform-এর বিস্তারিত প্রমাণীকরণ অধ্যায় ৪-এ।

> **WARNING:** Default রিসোর্স গ্রুপে প্রোডাকশন রাখবেন না। Default গ্রুপ সবার কাছে দৃশ্যমান হওয়ার প্রবণতা থাকে এবং নতুন সদস্যরা ভুল করে সেখানে সম্পদ তৈরি করে। প্রোডাকশনের জন্য নামকৃত গ্রুপ বাধ্যতামূলক করুন।

২.১৪ প্রোডাকশন পরিস্থিতি

TechNova তাদের তিনটি পরিবেশ আলাদা করল, কিন্তু ডেভেলপাররা অভিযোগ করলেন স্টেজিংয়ে ডেটা নেই বলে বাগ ধরা যায় না। সমাধান ছিল প্রোডাকশন ডেটা কপি করা নয়, বরং সংশ্লেষিত ডেটা জেনারেটর তৈরি করা — বাস্তব আকৃতি ও সম্পর্কসহ, কিন্তু কোনো বাস্তব ব্যক্তিগত তথ্য ছাড়া। এতে স্টেজিং পরীক্ষা বাস্তবসম্মত হলো, গোপনীয়তা ঝুঁকি তৈরি হলো না।

আরেকটি পরিস্থিতি: অর্থ দল মাসের শেষে বিল দেখে জিজ্ঞেস করল “এই খরচ কার?” — ট্যাগ থাকায় উত্তর ছিল এক মিনিটের কাজ: `env:prod` বনাম `env:dev` ভাগ করে দেখানো। ট্যাগিং শুধু প্রযুক্তি নয়, এটি সংগঠনের ভাষা।

২.১৫ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: প্রোডাকশন গ্রুপে অ্যাক্সেস সীমিত রাখুন, ডেভ গ্রুপে প্রশস্ত রাখুন; শেয়ার্ড গ্রুপে কেবল প্ল্যাটফর্ম দল। প্রতিটি গ্রুপে কে অ্যাক্সেস পেল তা নিয়মিত পর্যালোচনা করুন। খরচ: প্রতি গ্রুপে বাজেট সতর্কতা বসান; ডেভ পরিবেশ রাতে বা সপ্তাহান্তে বন্ধ করার নীতি বিবেচনা করুন (যেখানে সেবা তা সমর্থন করে); অপ্রয়োজনীয় গ্রুপের সম্পদ মুছে ফেলুন। মনে রাখুন: একটি খালি গ্রুপের বিল আসে না, কিন্তু গ্রুপের ভেতরে ভুলে থাকা একটি সার্ভারের বিল আসে।

২.১৬ সাধারণ ভুল

- সব সম্পদ Default গ্রুপে রাখা।
- পরিবেশের নাম শুধু সম্পদের নামে রাখা, গ্রুপে নয় — তখন ফিল্টার ও অ্যাক্সেস কাজ করে না।
- ট্যাগ না দেওয়া, তারপর বিলের মালিক খুঁজে না পাওয়া।
- প্রোডাকশন ডেটা স্টেজিং/ডেভে কপি করা।
- অনেক অ্যাকাউন্ট তৈরি করে কেন্দ্রীয় দৃশ্যমানতা হারানো।

২.১৭ ট্রাবলশুটিং

> **TROUBLESHOOTING:** সম্পদ খুঁজে পাচ্ছেন না? যাচাই করুন: (১) সঠিক অ্যাকাউন্টে আছেন কি? (২) সঠিক রিসোর্স গ্রুপ টার্গেট করেছেন কি? (৩) সঠিক অঞ্চলে আছেন কি? অধিকাংশ “হারানো” সম্পদ আসলে ভুল প্রসঙ্গে তৈরি সম্পদ। কনসোলে গ্রুপ ও অঞ্চল ফিল্টার বদলে খুঁজুন, CLI-তে `ibmcloud resource groups` ও `ibmcloud target` দিয়ে প্রসঙ্গ নিশ্চিত করুন।

বিল প্রত্যাশার চেয়ে বেশি? প্রথমে বিলিং ভিউতে গ্রুপ ও ট্যাগ অনুযায়ী ভাগ করুন; তারপর সবচেয়ে বড় খাতের সম্পদগুলো তালিকা করুন — প্রায়ই দেখা যায় একটি ভুলে চালু থাকা বড় সার্ভার বা পুরনো স্ন্যাপশটই কারণ। সম্পদ মুছলে বিলিং ইতিহাস মুছে যায় না; পরের বিলে পরিবর্তন দেখা যাবে।

২.১৮ সারসংক্ষেপ

অ্যাকাউন্ট হলো বিলিং ও পরিচয়ের সীমানা, Resource Group হলো সংগঠন ও অ্যাক্সেসের একক, ট্যাগ হলো খরচ ও অনুসন্ধানের লেবেল। ডেভ, স্টেজিং ও প্রোড আলাদা করা মানে নাম আলাদা করা নয় — সীমানা, ডেটা ও অ্যাক্সেস আলাদা করা। এন্টারপ্রাইজে অ্যাকাউন্ট ভাগ করা বিস্ফোরণ ব্যাসার্ধ ও বিলিংয়ের জন্য শক্তিশালী, কিন্তু জটিলতার দামে। TechNova-র নকশা — একটি অ্যাকাউন্ট, চারটি গ্রুপ (shared/dev/staging/prod), বাধ্যতামূলক ট্যাগ — ছোট দলের জন্য সঠিক শুরু।

২.১৯ পর্যালোচনা প্রশ্ন

1. Resource Group ও ট্যাগের মধ্যে পার্থক্য কী, এবং কখন কোনটি ব্যবহার করবেন?
2. একটি অ্যাকাউন্ট বনাম একাধিক অ্যাকাউন্ট — TechNova-র জন্য কখন ভাগ করা যুক্তিযুক্ত?
3. স্টেজিংয়ে প্রোডাকশন ডেটা কপি করা কেন ঝুঁকিপূর্ণ, এবং বিকল্প কী?

4. শেয়ার্ড রিসোর্স গ্রুপ কেন দরকার, এবং এতে কী রাখা উচিত নয়?
5. বিলিংয়ে খরচের মালিকানা না থাকলে সংগঠনে কী সমস্যা হয়?

২.২০ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি সংগঠন নথি লিখুন: অ্যাকাউন্ট, চারটি রিসোর্স গ্রুপ, প্রতিটি গ্রুপের উদ্দেশ্য, কে অ্যাক্সেস পাবে, এবং ট্যাগিং নিয়ম — একটি টেবিলে।
2. CLI দিয়ে তিনটি রিসোর্স গ্রুপ তৈরি করুন (বা তৈরির কমান্ড লিখে শুকনো-চালনা করুন) এবং `ibmcloud target -g` দিয়ে প্রসঙ্গ বদলে যাচাই করুন যে ভুল গ্রুপে সম্পদ তৈরি হওয়ার ঝুঁকি কীভাবে কমে।

অধ্যায় ৩ • PART I — IBM Cloud Foundations & Architecture

Chapter 3 — IBM Cloud IAM

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক পারবেন — (১) IBM Cloud IAM-এর স্থাপত্য ব্যাখ্যা করতে; (২) ব্যবহারকারী, গ্রুপ ও Access Group-এর মধ্যে পার্থক্য করতে; (৩) Service ID ও API Key দিয়ে মেশিন পরিচয় পরিচালনা করতে; (৪) ভূমিকা (role) ও নীতি (policy) দিয়ে সম্পদ-স্তরের অনুমতি দিতে; (৫) সেবা-থেকে-সেবা অনুমোদন নকশা করতে; (৬) ন্যূনতম সুযোগ (least privilege) প্রয়োগ ও পরিচয় সমস্যা সমাধান করতে। এই অধ্যায়টি এই খণ্ডের সবচেয়ে গুরুত্বপূর্ণ অধ্যায় — IAM ভুল হলে নিরাপত্তা ভুল হয়, আবার IAM খুব কঠোর হলে কাজ থেমে যায়।

- প্রমাণীকরণ (authentication — আপনি কে) ও অনুমোদন (authorization — আপনি কী করতে পারেন) আলাদা করা।
- মানুষের পরিচয় বনাম মেশিনের পরিচয়ের জন্য আলাদা পথ বেছে নেওয়া।
- একটি নীতি পড়ে বলা — কে, কী, কোন সম্পদে, কোন শর্তে করতে পারে।
- অতিরিক্ত অনুমতি খুঁজে কমানো ও ভাঙা অ্যাক্সেস মেরামত করা।

৩.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

অধ্যায় ২ থেকে মনে করুন: অ্যাকাউন্ট, রিসোর্স গ্রুপ, ট্যাগ। এবার সেই কাঠামোর ওপরে তালা বসানো হবে। একটি বাড়ির উদাহরণ নিন: অ্যাকাউন্ট হলো বাড়ি, রিসোর্স গ্রুপ হলো ঘর, সম্পদ হলো ঘরের জিনিস। IAM হলো চাবি ও তালার ব্যবস্থা — কার কাছে কোন ঘরের চাবি, কে শুধু দেখতে পারে, কে বদলাতে পারে। মনে রাখুন: পরিচয় (identity) একটি সত্তা — মানুষ, অ্যাপ্লিকেশন বা সেবা; ভূমিকা হলো সেই সত্তার ক্ষমতার সমষ্টি; নীতি হলো ভূমিকাকে সম্পদের সঙ্গে যুক্ত করার নিয়ম। একটি API Key হলো একটি গোপন চাবি — যার কাছে আছে, সে সেই পরিচয় হিসেবে কাজ করতে পারে। তাই API Key পাসওয়ার্ডের মতোই গোপন রাখতে হয়, কখনও কোডে বা চ্যাটে দেওয়া যায় না।

৩.৩ প্রথম নীতি থেকে ব্যাখ্যা: কেন IAM ছাড়া ক্লাউড চলে না?

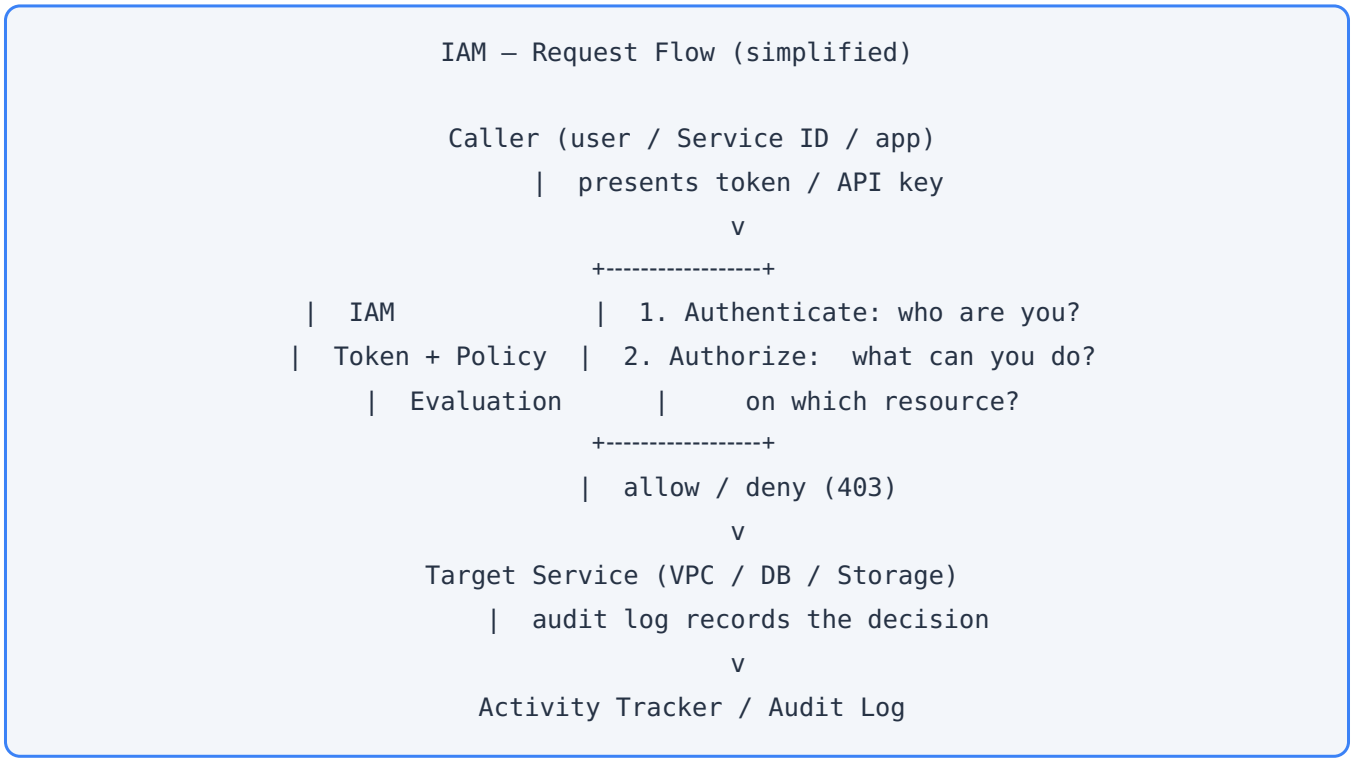
ক্লাউডে সম্পদ API দিয়ে তৈরি হয় — অর্থাৎ যে কেউ সঠিক শংসাপত্র (credential) পেলে দূর থেকে সার্ভার তৈরি, ডেটা পড়া বা মুছে ফেলা করতে পারে। ভৌত ডেটা সেন্টারে দরজায় পাহারা ছিল; ক্লাউডে দরজা হলো IAM। সমস্যাটি তিনটি: (১) কে? — মানুষ, CI/CD পাইপলাইন, নাকি অন্য একটি সেবা অনুরোধ করছে? (২) কী? — পড়া, লেখা, মুছে ফেলা, নাকি পরিচালনা? (৩) কোথায়? — কোন অ্যাকাউন্ট, কোন গ্রুপ, কোন সম্পদে? IAM এই তিন প্রশ্নের উত্তর একটি নীতি হিসেবে লিখে রাখে এবং প্রতিটি অনুরোধে যাচাই করে। নীতি ছাড়া দুটি চরম হয়: সবাই সবকিছু করতে পারে (অনিরাপদ), অথবা কেউ কিছু করতে পারে না (অচল)। ভালো IAM মানে এই দুই চরমের মধ্যে সুনির্দিষ্ট, ন্যূনতম অনুমতি।

ন্যূনতম সুযোগের নীতি (least privilege) হলো: প্রতিটি পরিচয়কে কেবল তার কাজের জন্য প্রয়োজনীয় অনুমতি দিন, তার বেশি নয়, এবং কেবল যতক্ষণ প্রয়োজন ততক্ষণ। এটি একটি নৈতিক বক্তব্য নয়, একটি ঝুঁকি গণনা: একটি শংসাপত্র ফাঁস হলে ক্ষতি ততটুকুই, যতটুকু অনুমতি তার ছিল। TechNova-র ডেভেলপারকে ডেভ গ্রুপে সম্পদ তৈরি করার অনুমতি দেওয়া যুক্তিযুক্ত; প্রোড ডেটাবেস মুছে ফেলার অনুমতি দেওয়া নয়। CI/CD পাইপলাইনকে স্টেজিংয়ে ডেপ্লয় করার অনুমতি দেওয়া যুক্তিযুক্ত; প্রোডাকশনে ডেপ্লয় করার অনুমতি কেবল অনুমোদিত পাইপলাইন ও সীমিত সময়ের জন্য।

> **CONCEPT:** প্রমাণীকরণ বলে "আপনি কে" (পাসওয়ার্ড, API Key, টোকেন দিয়ে যাচাই); অনুমোদন বলে "আপনি কী করতে পারেন" (নীতি ও ভূমিকা দিয়ে সিদ্ধান্ত)। একটি সঠিক পাসওয়ার্ড থাকলেই প্রোডাকশন মুছে ফেলার অধিকার আসে না — দ্বিতীয় ধাপটি আলাদাভাবে যাচাই হয়। এই পার্থক্য ভুলে গেলে ট্রাবলশুটিং অসম্ভব হয়।

৩.৪ IAM স্থাপত্য: অনুরোধের যাত্রা

IBM Cloud IAM-এর স্থাপত্যকে একটি গেটের মতো ভাবুন। প্রতিটি অনুরোধ প্রথমে IAM-এ যায়: টোকেন যাচাই করা হয়, পরিচয় শনাক্ত করা হয়, তারপর নীতি মূল্যায়ন করা হয় — এই পরিচয়ের এই সম্পদে এই কাজের অনুমতি আছে কি? অনুমতি থাকলে অনুরোধটি সেবার দিকে যায়; না থাকলে 403 (Forbidden) ফিরে আসে। এই মূল্যায়ন প্রতিটি কলে হয়, একবার লগইন করলেই চিরকালের অনুমতি মেলে না — টোকেনের মেয়াদ থাকে, নীতি বদলালে পরের কলেই তা কার্যকর হয়। অভ্যন্তরে নীতি হলো একটি নিয়মের সমষ্টি: বিষয় (subject — কে), ভূমিকা (role — কী করতে পারে), সম্পদ (resource — কোথায়), ও শর্ত (condition — কখন, কোথা থেকে)।



দুটি ভূমিকা পরিবার মনে রাখুন। প্ল্যাটফর্ম ভূমিকা (platform roles) — যেমন Viewer, Operator, Editor, Administrator — নিয়ন্ত্রণ করে কে সম্পদ পরিচালনা করতে পারে (তৈরি, মুছা, সেটিং বদলানো)। সেবা ভূমিকা (service roles) — যেমন Reader, Writer, Manager — নিয়ন্ত্রণ করে কে সেবার ভেতরের ডেটা ব্যবহার করতে পারে (ডেটাবেসে পড়া/লেখা, অবজেক্ট স্টোরেজে ফাইল পড়া)। একটি সাধারণ ভুল: কাউকে Editor দেওয়া মানেই তিনি ডেটা পড়তে পারবেন ভাবা। বাস্তবে প্ল্যাটফর্ম Editor সেটিং বদলাতে পারে, কিন্তু ডেটা পড়তে সেবা ভূমিকা দরকার হতে পারে। দুটি পরিবার একসঙ্গে বুঝতে হবে — নীতি লেখার সময় কোন ভূমিকা কোন স্তরে কাজ করে তা যাচাই করুন।

৩.৫ ব্যবহারকারী ও গ্রুপ, Access Group

ব্যবহারকারী (User) হলো একজন মানুষের পরিচয় — সাধারণত কোম্পানির পরিচয় প্রদানকারীর (identity provider) সঙ্গে যুক্ত। গ্রুপ ধারণাটি দুই জায়গায় আসে: পরিচয় প্রদানকারীর গ্রুপ (যেমন কোম্পানির ডিরেক্টরি গ্রুপ) ও IBM Cloud-এর Access Group। Access Group হলো IBM Cloud-এর নিজস্ব গুচ্ছ: একদল ব্যবহারকারী ও Service ID-কে একত্রে রেখে

তাদের একটিমাত্র নীতি দেওয়া যায়। উদাহরণ: `technova-platform-admins` Access Group-এ প্ল্যাটফর্ম দলের সবাই, `technova-developers`-এ ডেভেলপাররা। নতুন ডেভেলপার যোগ দিলে তাকে আলাদা নীতি না দিয়ে গ্রুপে যোগ করলেই হয় — এতে ভুল কমে, পর্যালোচনা সহজ হয়।

সরাসরি ব্যক্তিগত নীতি বনাম গ্রুপ নীতি: ব্যক্তিগত নীতি দ্রুত, কিন্তু দল বাড়লে পরিচালনা অসম্ভব হয় — কে কী পেল, কেন পেল, তার ইতিহাস হারিয়ে যায়। গ্রুপ নীতি ধীর শুরুতে, কিন্তু স্কেলে টিকে থাকে। নিয়ম: ব্যতিক্রম ছাড়া ব্যক্তিগত নীতি দেবেন না; ব্যতিক্রম হলে তার মেয়াদ ও কারণ লিখে রাখুন। Access Group-এর ব্যর্থতা: একটি গ্রুপে ভুল ব্যক্তি যোগ হলে তিনি গ্রুপের সব অনুমতি পান — তাই গ্রুপ সদস্যতা নিয়মিত পর্যালোচনা করুন, বিশেষ করে কেউ দল বা প্রকল্প ছাড়লে।

পরিচয়	কার জন্য	শংসাপত্র	কখন ব্যবহার করবেন	কখন এড়াবেন
User	মানুষ	পাসওয়ার্ড / SSO	দৈনন্দিন কাজ	অটোমেশনে নয়
Access Group	দল	— (সদস্যদের মাধ্যমে)	দলভিত্তিক নীতি	একজনের বিশেষ কাজে নয়
Service ID	অ্যাপ / পাইপলাইন	API Key	মেশিন-থেকে-মেশিন	মানুষের লগইনে নয়
API Key	পরিচয়ের চাবি	গোপন মান	CLI / SDK প্রমাণীকরণ	কোডে হার্ডকোড নয়

৩.৬ Service ID ও API Key: মেশিনের পরিচয়

Service ID হলো একটি অ্যাপ্লিকেশন বা অটোমেশনের জন্য তৈরি অ-মানব পরিচয়। এর নিজের পাসওয়ার্ড থাকে না; এর সঙ্গে একটি API Key যুক্ত থাকে, যা দিয়ে CLI, SDK বা Terraform প্রমাণীকরণ করে। কেন মানুষের API Key দিয়ে অটোমেশন চালাবেন না? কারণ মানুষ চাকরি বদলায়, পাসওয়ার্ড বদলায়, ছুটিতে যায় — আর তার Key দিয়ে চলা পাইপলাইন হঠাৎ থেমে যায়। Service ID-র Key-র মালিকানা দলের, ব্যক্তির নয়; Key ঘোরানো (rotation) ও বাতিল করা যায় মানুষের অ্যাকাউন্ট স্পর্শ না করে। TechNova-র CI/CD পাইপলাইনের জন্য একটি Service ID তৈরি করুন — যেমন `technova-cicd` — কেবল স্টেজিংয়ে ডেপ্লয় করার নীতি দিয়ে, প্রোডাকশনে নয় (প্রোডের জন্য আলাদা, অনুমোদিত Service ID)।

API Key পরিচালনার নিয়ম: (১) Key তৈরি করার সময়ই একবারই সম্পূর্ণ মান দেখা যায় — তখনই নিরাপদ ভল্টে (vault / secrets manager) সংরক্ষণ করুন; পরে আবার দেখা যায় না। (২) Key কখনও কোড রিপোজিটরি, চ্যাট বা টিকিটে রাখবেন না। (৩) নিয়মিত ঘোরান — নতুন Key তৈরি, পুরনোটি বাতিল, মধ্যবর্তী সময়ে দুটি সক্রিয় রাখা। (৪) প্রতিটি উদ্দেশ্যে আলাদা Key — একটি Key সব কাজে ব্যবহার করলে ফাঁসের ক্ষতি সর্বব্যাপী হয়। (৫) অপ্রয়োজনীয় Key মুছে ফেলুন; পর্যালোচনায় “শেষ ব্যবহৃত” সময় দেখে নিষ্ক্রিয় Key শনাক্ত করুন।

> **WARNING:** API Key হলো পাসওয়ার্ডের সমতুল্য। স্ক্রিনশট, লগ বা টার্মিনাল ইতিহাসে Key রেখে যাবেন না। একটি Key ফাঁস হয়েছে সন্দেহ হলে তর্ক করবেন না — অবিলম্বে বাতিল করে নতুন Key তৈরি করুন, তারপর তদন্ত করুন। CHANGE-ME-... এর মতো প্লেসহোল্ডার কখনও বাস্তব Key হিসেবে ব্যবহার করবেন না।

৩.৭ ভূমিকা ও নীতি, সম্পদ-স্তরের অনুমতি

নীতি (Policy) হলো একটি বাক্য: “এই পরিচয় (বা গ্রুপ) এই ভূমিকায় এই সম্পদে কাজ করতে পারে।” সম্পদ নির্দিষ্ট করা যায় অ্যাকাউন্ট-স্তরে, রিসোর্স গ্রুপ-স্তরে, সেবা-স্তরে, এমনকি একটি নির্দিষ্ট সম্পদ-স্তরে (resource-level)। নীতিমালার নকশা: শুরুতে প্রশস্ত নয়, সংকীর্ণ শুরু করুন — নির্দিষ্ট গ্রুপ, নির্দিষ্ট ভূমিকা, নির্দিষ্ট সেবা। উদাহরণ: ডেভেলপার গ্রুপকে `technova-dev` গ্রুপে Editor (প্ল্যাটফর্ম) + Reader (সেবা) দিন; প্রোড গ্রুপে কেবল Viewer দিন, পরিবর্তনের জন্য

আলাদা অনুমোদিত পথ রাখুন। সম্পদ-স্তরের অনুমতি মানে একটি নির্দিষ্ট ডেটাবেসে Writer দেওয়া, সব ডেটাবেসে নয় — এতে একটি ভুলের ক্ষতি সীমিত থাকে।

নীতি মূল্যায়নের অভ্যন্তরীণ যুক্তি: IAM সব প্রযোজ্য নীতি সংগ্রহ করে; যদি কোনো একটি নীতি অনুমতি দেয়, তবে অনুমতি দেওয়া হয় (allow-ভিত্তিক মডেল) — তবে কিছু ক্ষেত্রে explicit deny বা শর্ত (যেমন IP সীমা, সময়) সেই অনুমতিকে আটকাতে পারে। তাই "আমি তো Viewer দিয়েছি, তবু Editor-এর মতো কাজ করছে কেন?" — উত্তর খুঁজুন অন্য কোনো গ্রুপের নীতিতে; ব্যক্তিটি একাধিক গ্রুপে থাকলে তাদের অনুমতি যোগ হয়। এই যোগ হওয়ার ধারণাটি (union) মনে রাখা ট্রাবলশুটিংয়ের চাবি।

৩.৮ সেবা-থেকে-সেবা অনুমোদন

একটি সেবা যখন অন্য সেবাকে ডাকে — যেমন অ্যাপ সার্ভার ডেটাবেস পড়ে, বা লগিং সেবা অবজেক্ট স্টোরেজে লেখে — তখন মানুষের Key ব্যবহার করা উচিত নয়। IBM Cloud-এ সেবা-থেকে-সেবা অনুমোদন (service-to-service authorization) দিয়ে একটি সেবা-দৃষ্টান্তকে (service instance) অন্য সেবায় নির্দিষ্ট ভূমিকা দেওয়া যায়, Service ID-র মাধ্যমে বা সরাসরি অনুমোদন নীতিতে। উদাহরণ: TechNova-র অ্যাপের Service ID-কে ডেটাবেসে Reader/Writer দিন, কিন্তু সেই Service ID-কে VPC পরিচালনার অনুমতি দেবেন না। এতে অ্যাপের শংসাপত্র ফাঁস হলেও আক্রমণকারী কেবল ডেটাবেস পর্যন্ত যেতে পারে, পুরো অ্যাকাউন্ট নয়। সেবা-থেকে-সেবা অনুমোদনের ব্যর্থতা সাধারণত "অ্যাপ ডেটাবেসে সংযোগ পাচ্ছে না" হিসেবে দেখা যায় — তখন অ্যাপের লগ নয়, IAM নীতি ও সেবার অনুমোদন পরীক্ষা করুন।

৩.৯ ন্যূনতম সুযোগ প্রয়োগ: একটি কার্যকর পদ্ধতি

ন্যূনতম সুযোগ একবারে অর্জিত হয় না; এটি একটি চক্র: (১) কাজটি সংজ্ঞায়িত করুন — কী করতে হবে, কোন সম্পদে, কতক্ষণ; (২) সবচেয়ে সংকীর্ণ নীতি লিখুন; (৩) পরীক্ষা করুন — কাজটি হয় কি? (৪) পর্যবেক্ষণ করুন — অডিট লগে কী অনুরোধ প্রত্যাখ্যাত হচ্ছে, কী অনুমতি কখনও ব্যবহৃত হচ্ছে না; (৫) কমান — অব্যবহৃত অনুমতি বাতিল করুন। TechNova-র উদাহরণ: প্রথমে ডেভেলপারদের ডেভ গ্রুপে Editor দেওয়া হলো; এক মাস পর দেখা গেল কেউ কখনও সম্পদ মুছে ফেলেনি — তবু মুছে ফেলার অনুমতি রাখা হয়েছিল। পর্যালোচনায় মুছে ফেলার অধিকার আলাদা করা হলো — দৈনন্দিন কাজে Editor ছাড়া, মুছে ফেলার জন্য আলাদা অনুমোদন। এই চক্রটি ত্রৈমাসিক পর্যালোচনায় পরিণত করুন, একবারের কাজ নয়।

৩.১০ CLI / API উদাহরণ

```
ibmcloud login
ibmcloud iam access-group-create technova-developers
ibmcloud iam access-groups
ibmcloud iam access-group-user-add technova-developers nahid@technova.example
ibmcloud iam service-id-create technova-cicd --description "CI/CD pipeline identity"
ibmcloud iam service-api-key-create technova-cicd-key technova-cicd
ibmcloud iam access-group-policy-create technova-developers \
  --roles Editor --resource-group-name technova-dev
ibmcloud iam access-group-policies technova-developers
ibmcloud iam service-policies technova-cicd
```

উপরের কমান্ডগুলো আকৃতি বোঝায়; সঠিক ফ্ল্যাগ ও ভূমিকার নাম আপনার IBM Cloud সংস্করণ ও সেবা অনুযায়ী `ibmcloud iam --help` দিয়ে যাচাই করুন — ফ্ল্যাগ অনুমান করে চালাবেন না। নীতি তৈরি করার পর অবশ্যই তালিকা কমান্ড দিয়ে যাচাই করুন যে নীতিটি যা ভেবেছিলেন তাই হয়েছে। API Key তৈরি করার আউটপুটে সম্পূর্ণ Key একবারই দেখা যায় — তাৎক্ষণিকভাবে নিরাপদ ভল্টে সংরক্ষণ করুন, টার্মিনালে রেখে দেবেন না।

REST API-র ধারণা: CLI-র প্রতিটি কাজের পেছনে একটি REST API কল আছে — যেমন নীতি তৈরি করতে IAM Policy Management API-তে POST করা হয়। API ব্যবহার করলে Authorization হেডারে IAM টোকেন পাঠাতে হয়, যা `ibmcloud iam oauth-tokens` দিয়ে পাওয়া যায়। সরাসরি API ব্যবহারের সুবিধা: অটোমেশন ও নিজের টুল তৈরি; অসুবিধা: টোকেনের মেয়াদ ও ত্রুটি পরিচালনা নিজেকে করতে হয়। অধ্যায় ৪-এ API ও SDK বিস্তারিত।

৩.১১ IaC উদাহরণ

```
resource "ibm_iam_access_group" "developers" {
  name        = "technova-developers"
  description = "TechNova developers — dev group access only"
}
resource "ibm_iam_access_group_policy" "dev_editor" {
  access_group_id = ibm_iam_access_group.developers.id
  roles           = ["Editor"]
  resource_group_id = "CHANGE-ME-resource-group-id"
}
resource "ibm_iam_service_id" "cicd" {
  name        = "technova-cicd"
  description = "CI/CD pipeline identity"
}
```

IaC-এ IAM লেখার সুবিধা: নীতি পর্যালোচনা (code review) করা যায়, ইতিহাস থাকে, পরিবেশে পরিবেশে একই নীতি পুনরায় প্রয়োগ করা যায়। ঝুঁকি: ভুল নীতি কোডে লিখলে তা সব পরিবেশে ছড়িয়ে পড়ে — তাই IAM পরিবর্তনে আলাদা অনুমোদন রাখুন, Terraform plan মনোযোগ দিয়ে পড়ুন।

৩.১২ প্রোডাকশন পরিস্থিতি

TechNova-র প্রোডাকশন ডেপ্লয় হঠাৎ ব্যর্থ হলো — পাইপলাইন 403 পাচ্ছে। দল প্রথমে অ্যাপের কোড সন্দেহ করল, কিন্তু সমস্যা ছিল IAM-এ: প্রোডাকশনের Service ID-র নীতি স্টেজিং গ্রুপের জন্য ছিল, প্রোড গ্রুপের জন্য নয় — কপি-পেস্টের ভুল। সমাধান: প্রোডের জন্য আলাদা Service ID ও নীতি, এবং ডেপ্লয় স্ক্রিপ্টে প্রথমে `ibmcloud iam service-policies` দিয়ে নীতি যাচাই করার ধাপ যোগ করা। আরেকটি পরিস্থিতি: একজন ডেভেলপার দল ছাড়লেন, কিন্তু তার Access Group সদস্যতা থেকে গেল — ত্রৈমাসিক পর্যালোচনায় ধরা পড়ল। শিক্ষা: অফবোর্ডিং চেকলিস্টে IAM থেকে অপসারণ বাধ্যতামূলক করুন, HR প্রক্রিয়ার সঙ্গে যুক্ত করুন।

> **REAL WORLD:** একটি কোম্পানিতে সবাই Administrator ছিল “কারণ দ্রুত কাজ করতে হয়।” একদিন একটি ভুল স্ক্রিপ্ট পুরো অ্যাকাউন্টের নেটওয়ার্ক বদলে দিল। পুনরুদ্ধারে লেগেছিল দুই দিন। তারপর থেকে নীতি হলো: কেউ স্থায়ী Administrator নয়; প্রয়োজনে সাময়িক উচ্চ অনুমতি, কাজ শেষে প্রত্যাহার। গতি ও নিরাপত্তার ভারসাম্য প্রক্রিয়ায় আসে, সর্বোচ্চ অনুমতিতে নয়।

৩.১৩ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: MFA (multi-factor authentication) মানুষের অ্যাকাউন্টে বাধ্যতামূলক করুন; API Key ঘোরান; নিষ্ক্রিয় পরিচয় মুছুন; অডিট লগ পর্যবেক্ষণ করুন — কে, কখন, কী করল। খরচ: IAM-এর সরাসরি বিল সাধারণত নেই, কিন্তু ভুল অনুমতির খরচ আছে — অপ্রয়োজনীয় বড় সম্পদ তৈরি, ডেটা মুছে ফেলা, বা ফাঁস হওয়া Key দিয়ে তৈরি সম্পদের বিল। IAM তাই খরচ নিয়ন্ত্রণেরও হাতিয়ার: যার বড় সম্পদ তৈরি করার প্রয়োজন নেই, তাকে সেই অনুমতি দেবেন না।

৩.১৪ সাধারণ ভুল

- সবাইকে Administrator দেওয়া।
- মানুষের API Key দিয়ে CI/CD চালানো।
- API Key কোডে হার্ডকোড করা বা রিপোজিটরিতে কমিট করা।
- প্ল্যাটফর্ম ভূমিকা ও সেবা ভূমিকা গুলিয়ে ফেলা।
- একাধিক গ্রুপের অনুমতি যোগ হয় — এটি না জেনে নীতি লেখা।
- দল ছাড়া ব্যক্তির অ্যাক্সেস না সরানো।
- Service ID-কে প্রয়োজনের চেয়ে বেশি অনুমতি দেওয়া।

৩.১৫ ট্রাবলশুটিং: পরিচয় সমস্যা সমাধান

TROUBLESHOOTING: 403 পেলে ক্রমে যাচাই করুন: (১) টোকেন কি মেয়াদোত্তীর্ণ? আবার লগইন করুন। (২) সঠিক অ্যাকাউন্ট ও গ্রুপে আছেন কি? (৩) কোন নীতি প্রযোজ্য — `ibmcloud iam access-group-policies` ও `ibmcloud iam user-policies` দিয়ে দেখুন। (৪) প্ল্যাটফর্ম নাকি সেবা ভূমিকার অভাব? (৫) সেবা-থেকে-সেবা অনুমোদন দরকার কি? অডিট লগে প্রত্যাখ্যাত অনুরোধের বিবরণ দেখুন — কোন সম্পদ, কোন কাজ, কোন পরিচয়। অনুমান করে Administrator দিয়ে সমস্যা “ঠিক” করবেন না; তাতে মূল কারণ লুকিয়ে থাকে।

API Key কাজ করছে না? Key-টি বাতিল হয়েছে কি, Service ID মুছে গেছে কি, নাকি Key-টি ভুল অ্যাকাউন্টের? Key তৈরি করার সময় যে আউটপুট পেয়েছিলেন, তার সঙ্গে মিলিয়ে দেখুন — Key-র নাম ও Service ID। টোকেন নিয়ে API কলে 401 মানে প্রমাণীকরণ ব্যর্থ; 403 মানে প্রমাণীকরণ ঠিক, অনুমোদন নেই — এই পার্থক্য সমস্যার অর্ধেক সমাধান করে দেয়।

৩.১৬ সারসংক্ষেপ

IAM হলো ক্লাউডের দরজা: প্রমাণীকরণ বলে আপনি কে, অনুমোদন বলে আপনি কী করতে পারেন। ব্যবহারকারী মানুষের জন্য, Service ID মেশিনের জন্য, Access Group দলের জন্য। ভূমিকা ও নীতি দিয়ে সম্পদ-স্তরের অনুমতি দিন, ন্যূনতম সুযোগ মেনে চলুন, API Key গোপন রাখুন ও ঘোরান, সেবা-থেকে-সেবা অনুমোদন দিয়ে অ্যাপকে ডেটায় পৌঁছাতে দিন মানুষের Key ছাড়া। নিয়মিত পর্যালোচনা ছাড়া IAM ক্ষয়ে যায় — অনুমতি বাড়ে, মানুষ বদলায়, Key পুরনো হয়।

৩.১৭ পর্যালোচনা প্রশ্ন

1. প্রমাণীকরণ ও অনুমোদনের পার্থক্য একটি 403 ত্রুটি ব্যাখ্যায় কীভাবে সাহায্য করে?
2. CI/CD-র জন্য মানুষের API Key-র বদলে Service ID কেন ভালো?
3. প্ল্যাটফর্ম ভূমিকা ও সেবা ভূমিকা গুলিয়ে গেলে কী ধরনের সমস্যা হয়?
4. একজন ব্যবহারকারী দুটি Access Group-এ থাকলে তার অনুমতি কীভাবে নির্ধারিত হয়?
5. ন্যূনতম সুযোগ প্রয়োগের চক্রটি বর্ণনা করুন — কীভাবে অব্যবহৃত অনুমতি খুঁজবেন?

৩.১৮ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি IAM ম্যাট্রিক্স লিখুন: তিনটি ভূমিকা (ডেভেলপার, প্ল্যাটফর্ম অ্যাডমিন, অডিটর) × তিনটি পরিবেশ — প্রতিটি ঘরে প্ল্যাটফর্ম ও সেবা ভূমিকা লিখুন এবং কেন তা ন্যূনতম সুযোগ মেনে চলে তা ব্যাখ্যা করুন।

2. একটি Service ID ও API Key তৈরির ধাপ লিখুন (আসল Key ছাড়া), তারপর Key ঘোরানোর পরিকল্পনা লিখুন: নতুন Key তৈরি, পুরনোটি বাতিল, যাচাই — কোন ক্রমে করবেন যাতে পাইপলাইন না থামে?

অধ্যায় ৪ • PART I — IBM Cloud Foundations & Architecture

Chapter 4 — IBM Cloud CLI and API Automation

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক পারবেন — (১) IBM Cloud CLI ইনস্টল ও লগইন করতে; (২) রিসোর্স টার্গেটিং (অ্যাকাউন্ট, অঞ্চল, রিসোর্স গ্রুপ) নির্ভুলভাবে করতে; (৩) CLI প্লাগইন ব্যবহার করতে; (৪) REST API ও SDK দিয়ে সম্পদ পরিচালনা করতে; (৫) API Key নিরাপদে পরিচালনা করতে; (৬) অটোমেশন স্ক্রিপ্ট ও Terraform প্রমাণীকরণ লিখতে; (৭) শংসাপত্র (credential) নিরাপদে সংরক্ষণ করতে। কনসোলে ক্লিক করা শেখার জন্য ভালো; CLI ও API দলগত, পুনরুৎপাদনযোগ্য কাজের জন্য অপরিহার্য।

- কনসোল, CLI, API ও IaC — কখন কোনটি ব্যবহার করবেন তা বেছে নেওয়া।
- একটি স্ক্রিপ্ট লেখা যা ভুল টার্গেটে চললে নিজে থেমে যায়।
- API ত্রুটি কোড পড়ে সমস্যার স্তর চিহ্নিত করা।
- শংসাপত্র কখনও কোডে না রেখে অটোমেশন চালানো।

৪.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

অধ্যায় ৩ থেকে মনে করুন: Service ID, API Key, টোকেন, নীতি। এবার সেই পরিচয় দিয়ে যন্ত্রকে কাজ করানো হবে। টার্মিনালের মৌলিক ধারণা মনে করুন: পরিবেশ চলক (environment variable), পাইপ, প্রস্থান কোড (exit code — 0 মানে সফল, অন্য কিছু মানে ব্যর্থ)। একটি স্ক্রিপ্ট হলো কমান্ডের তালিকা, যা মানুষের বদলে যন্ত্র চালায় — তাই স্ক্রিপ্টে ভুল মানে ভুলটি বারবার, দ্রুত ও নীরবে ঘটে। এই কারণে অটোমেশনে যাচাই (validation) ও নিরাপদ ব্যর্থতা (fail-safe) মানুষের হাতে করা কাজের চেয়ে বেশি গুরুত্বপূর্ণ। TechNova-র প্রসঙ্গ: তাদের ডেপ্লয় স্ক্রিপ্ট রাতে চলে — কেউ দেখছে না। স্ক্রিপ্টটি যদি ভুল পরিবেশে ডেপ্লয় করে, সকালে সবাই দেখবে প্রোডাকশন বদলে গেছে।

৪.৩ প্রথম নীতি থেকে ব্যাখ্যা: কেন অটোমেশন?

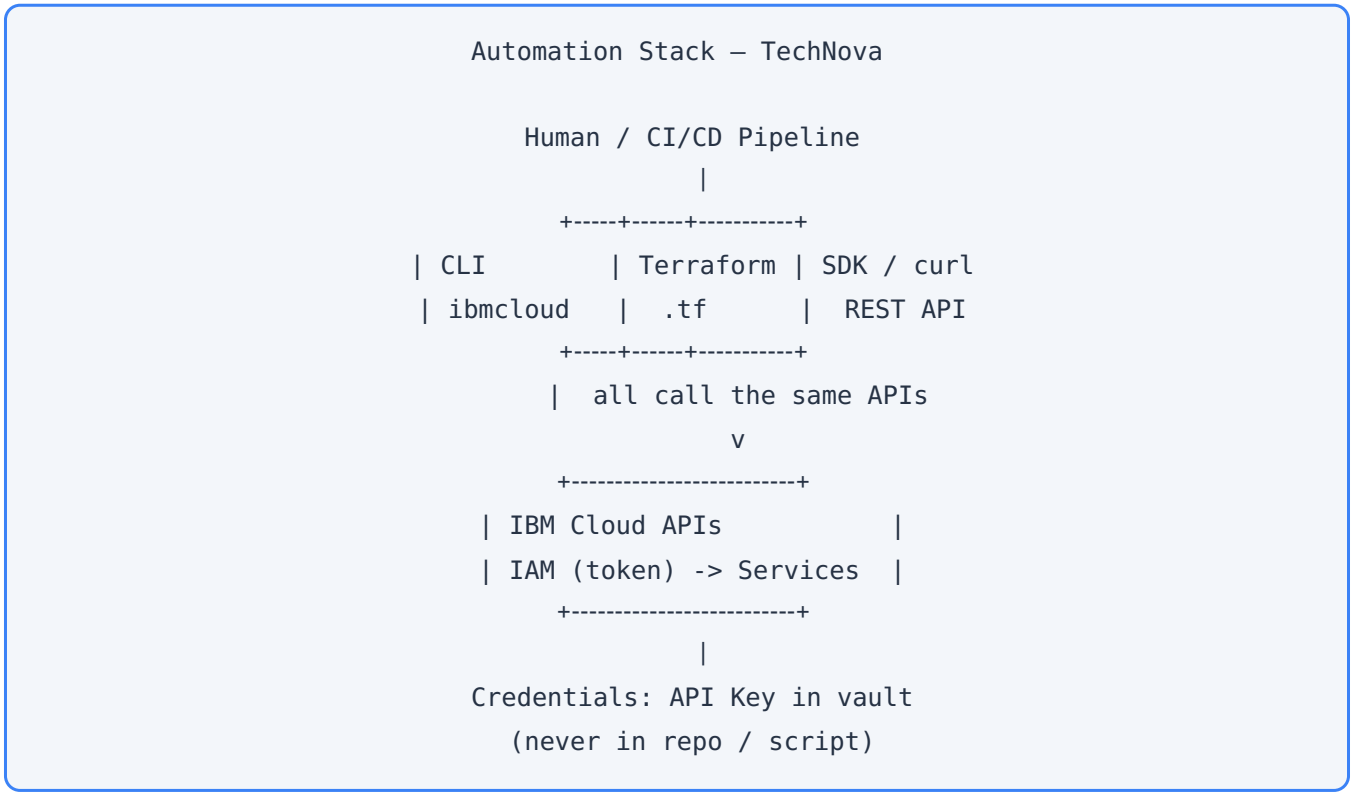
হাতে করা কাজের তিনটি সমস্যা: (১) এটি ধীর — প্রতিটি পরিবেশে একই ক্লিক বারবার; (২) এটি অসঙ্গত — দুজন মানুষ একই কাজ দুভাবে করে; (৩) এটি অদৃশ্য — কী করা হয়েছিল, তার কোনো রেকর্ড থাকে না। অটোমেশন এই তিনটির উত্তর: স্ক্রিপ্ট দ্রুত চলে, একইভাবে চলে, এবং স্ক্রিপ্ট নিজেই রেকর্ড। কিন্তু অটোমেশন একটি নতুন সমস্যা তৈরি করে: শংসাপত্রকে যন্ত্রের হাতে দিতে হয়। মানুষ পাসওয়ার্ড মনে রাখে; যন্ত্রকে Key দিতে হয় — আর সেই Key কোথায় রাখা হবে, কে দেখতে পাবে, কখন ঘোরানো হবে — তা নতুন শৃঙ্খলা দাবি করে। এই অধ্যায়ের অর্ধেক তাই CLI নয়, শংসাপত্র ব্যবস্থাপনা।

API-র ধারণাটি প্রথম নীতি থেকে বুঝুন: কনসোল একটি সুন্দর মুখোশ; তার পেছনে প্রতিটি বোতাম একটি HTTP অনুরোধ পাঠায়। CLI সেই অনুরোধগুলোকে টার্মিনাল থেকে পাঠানোর হাতিয়ার; SDK সেই অনুরোধগুলোকে প্রোগ্রামিং ভাষা থেকে পাঠানোর লাইব্রেরি; Terraform সেই অনুরোধগুলোকে ঘোষণামূলক (declarative) অবস্থা হিসেবে লেখার হাতিয়ার। চারটি একই সত্যকে চারভাবে প্রকাশ করে — কাঙ্ক্ষিত অবস্থা ও প্রকৃত অবস্থার পার্থক্য। যখন কনসোলে কিছু করতে পারছেন না, CLI-তে চেষ্টা করুন; CLI-তে না পারলে API নথি দেখুন — সেখানে সত্য থাকে।

> **CONCEPT:** CLI হলো মানুষের জন্য API-র টার্মিনাল মুখ; SDK হলো প্রোগ্রামের জন্য API-র লাইব্রেরি; Terraform হলো কাঙ্ক্ষিত অবস্থার ঘোষণা, যা API কল দিয়ে বাস্তবায়িত হয়। তিনটির পেছনে একই REST API ও একই IAM নীতি কাজ করে — তাই একটি স্তরে অনুমতি না থাকলে অন্য স্তরেও থাকবে না।

৪.৪ IBM Cloud CLI ইনস্টলেশন

IBM Cloud CLI হলো `ibmcloud` কমান্ড-লাইন হাতিয়ার। ইনস্টল করার সরকারি পথ হলো IBM Cloud-এর নথিতে দেওয়া ইনস্টলার বা প্যাকেজ ম্যানেজার ব্যবহার করা — অপারেটিং সিস্টেম অনুযায়ী (macOS, Linux, Windows) পদ্ধতি ভিন্ন। ইনস্টল করার পর `ibmcloud version` দিয়ে যাচাই করুন। একটি সাধারণ ভুল: পুরনো সংস্করণ রেখে নতুন নথির কমান্ড চালানো — ফ্ল্যাগ মেলে না। দলগত কাজে CLI সংস্করণ নথিভুক্ত রাখুন, CI/CD-তে নির্দিষ্ট সংস্করণ পিন করুন। ইনস্টলেশনের ব্যর্থতা প্রায়ই নেটওয়ার্ক, প্রক্সি বা অনুমতির কারণে হয় — ইনস্টলার বার্তাটি পড়ুন, অনুমান করে আবার চালাবেন না।



৪.৫ লগইন ও প্রমাণীকরণ

লগইন মানে IAM-কে আপনার পরিচয় প্রমাণ করে একটি টোকেন পাওয়া। মানুষের জন্য ইন্টার‍্যাক্টিভ লগইন (`ibmcloud login`) — ব্রাউজার বা SSO-এর মাধ্যমে। অটোমেশনের জন্য API Key দিয়ে লগইন — পরিবেশ চলকে Key রেখে। গুরুত্বপূর্ণ: টোকেনের মেয়াদ থাকে; দীর্ঘ স্ক্রিপ্টে টোকেন মেয়াদোত্তীর্ণ হতে পারে — তখন আবার লগইন করতে হয়। লগইনের পর `ibmcloud target` দিয়ে প্রসঙ্গ দেখুন — কোন অ্যাকাউন্ট, কোন অঞ্চল, কোন রিসোর্স গ্রুপ। এই প্রসঙ্গই নির্ধারণ করে পরের কমান্ড কোথায় কাজ করবে।

```
ibmcloud login
ibmcloud login --apikey CHANGE-ME-API-KEY -a https://cloud.ibm.com
ibmcloud target
ibmcloud target -r us-south -g technova-dev
ibmcloud iam oauth-tokens
```

দ্বিতীয় কমান্ডটি আকৃতি বোঝায় — বাস্তব Key কমান্ড লাইনে টাইপ করবেন না; টার্মিনাল ইতিহাসে থেকে যাবে। পরিবর্তে পরিবেশ চলক বা নিরাপদ ভল্ট থেকে Key পড়ুন (৪.৯ দেখুন)। `ibmcloud iam oauth-tokens` দিয়ে পাওয়া টোকেন দিয়ে সরাসরি API কল করা যায় — টোকেনও গোপন, Key-র মতোই রাখুন। লগআউট বা টোকেন বাতিল করা ভাগ করা মেশিনে কাজ শেষে অভ্যাস করুন।

৪.৬ রিসোর্স টার্গেটিং: সবচেয়ে গুরুত্বপূর্ণ অভ্যাস

টার্গেটিং হলো প্রশ্নের উত্তর: কোন অ্যাকাউন্ট, কোন অঞ্চল, কোন রিসোর্স গ্রুপ, কোন সংগঠন/স্থান (যেখানে প্রযোজ্য)? ভুল টার্গেটে সঠিক কমান্ড ভুল জায়গায় সম্পদ তৈরি করে, ভুল জায়গার সম্পদ মুছে ফেলে। প্রতিটি স্ক্রিপ্টের শুরুতে টার্গেট স্পষ্টভাবে নির্ধারণ করুন ও যাচাই করুন — এমনকি `ibmcloud target` আউটপুট পড়ে নিশ্চিত করুন। একটি নিরাপদ প্যাটার্ন: স্ক্রিপ্টের শুরুতে প্রত্যাশিত অ্যাকাউন্ট ও গ্রুপের নাম যাচাই করে না মিললে থেমে যাওয়া (fail-fast)। প্রোডাকশনে কাজ করা স্ক্রিপ্টে অতিরিক্ত নিশ্চিতকরণ (confirmation) ধাপ রাখুন — “আপনি কি প্রোডাকশনে এটি করতে চান?” — স্বয়ংক্রিয় পাইপলাইনে এই ধাপটি অনুমোদন (approval) ধাপ দিয়ে প্রতিস্থাপন করুন, বাদ দেবেন না।

৪.৭ CLI প্লাগইন

IBM Cloud CLI-র মূল কমান্ডের পাশাপাশি প্লাগইন দিয়ে অতিরিক্ত সেবার কমান্ড যোগ করা যায় — যেমন কনটেইনার, ফাংশন, বা ডেটাবেস সেবার প্লাগইন। প্লাগইন তালিকা দেখতে ও ইনস্টল করতে CLI-র প্লাগইন কমান্ড ব্যবহার করুন (`ibmcloud plugin list` ও `ibmcloud plugin install` — সঠিক নাম নথিতে যাচাই করুন)। প্লাগইনের অভ্যন্তরীণ কাজ: প্লাগইন একটি আলাদা বাইনারি, যা মূল CLI থেকে টোকেন ও প্রসঙ্গ পেয়ে সেবার API ডাকে। প্লাগইন ব্যর্থ হলে প্রথমে প্লাগইনের সংস্করণ ও মূল CLI-র সংস্করণের সামঞ্জস্য যাচাই করুন; পুরনো প্লাগইন নতুন API-তে কাজ নাও করতে পারে। দলগত পরিবেশে কোন প্লাগইন দরকার তার তালিকা নথিভুক্ত রাখুন।

```
ibmcloud plugin list
ibmcloud plugin install container-service
ibmcloud ks clusters
ibmcloud plugin update
```

তৃতীয় কমান্ডটি উদাহরণ — কনটেইনার সেবার প্লাগইন ইনস্টল থাকলে ক্লাস্টার তালিকা দেখা যায়। প্লাগইনের নাম ও কমান্ড সময়ের সঙ্গে বদলাতে পারে; `ibmcloud plugin list` ও সেবার নথিই কর্তৃপক্ষ।

৪.৮ REST API ও SDK

REST API হলো HTTP-ভিত্তিক ইন্টারফেস: একটি URL, একটি পদ্ধতি (GET/POST/PUT/DELETE), হেডার (Authorization: Bearer টোকেন), ও JSON বডি। উদাহরণ: রিসোর্স গ্রুপ তালিকা পেতে Resource Controller API-তে GET করা হয়; একটি সম্পদ তৈরি করতে POST করা হয়। API-র সুবিধা: যেকোনো ভাষা থেকে ডাকা যায়, অটোমেশনে নমনীয়। অসুবিধা: পেজিনেশন, পুনরায় চেষ্টা (retry), হার সীমা (rate limit) ও ত্রুটি পরিচালনা নিজেকে করতে হয়। SDK (যেমন Python, Go, Node.js-এর জন্য IBM Cloud SDK) সেই কাজগুলো সহজ করে — প্রমাণীকরণ, পুনরায় চেষ্টা ও মডেল ক্লাস দেয়। SDK বেছে নিন যখন নিয়মিত প্রোগ্রাম থেকে API ডাকতে হবে; সরাসরি curl/HTTP বেছে নিন যখন দ্রুত পরীক্ষা বা এককালীন কাজ।

```
export IBMCLOUD_API_KEY="CHANGE-ME-API-KEY"
ibmcloud iam oauth-tokens
# Illustrative API call shape — endpoint & version from docs
curl -X GET "https://resource-controller.cloud.ibm.com/v2/resource_groups" \
  -H "Authorization: Bearer CHANGE-ME-TOKEN" \
  -H "Content-Type: application/json"
```

উপরের curl উদাহরণটি আকৃতি বোঝায়; আসল এন্ডপয়েন্ট, সংস্করণ ও প্যারামিটার API নথিতে যাচাই করুন। টোকেন কখনও কমান্ডে সরাসরি লিখে শেয়ার করবেন না; পরিবেশ চলক থেকে পড়ুন। API ত্রুটি: 400 মানে অনুরোধ ভুল, 401 মানে

প্রমাণীকরণ ব্যর্থ, 403 মানে অনুমোদন নেই, 404 মানে সম্পদ নেই বা দেখার অধিকার নেই, 429 মানে হার সীমা, 500 মানে সেবার সমস্যা — এই পার্থক্য ট্রাবলশুটিংয়ের মানচিত্র।

৪.৯ API Key পরিচালনা ও নিরাপদ শংসাপত্র ব্যবস্থাপনা

শংসাপত্র ব্যবস্থাপনার নীতিমালা: (১) শংসাপত্র কোডে নয় — পরিবেশ চলক বা ভল্টে (যেমন IBM Cloud Secrets Manager বা CI/CD-র সিক্রেট স্টোরে)। (২) ন্যূনতম সুযোগ — অটোমেশনের Key-কে কেবল তার কাজের অনুমতি দিন (অধ্যায় ৩)। (৩) ঘোরানো — নিয়মিত Key বদলান; ঘোরানোর সময় পুরনো ও নতুন Key সাময়িকভাবে দুটিই সক্রিয় রাখুন। (৪) নিরীক্ষা — কোন Key কখন ব্যবহৃত হলো, লগে দেখুন; নিষ্ক্রিয় Key বাতিল করুন। (৫) আলাদা করা — ডেভ, স্টেজিং ও প্রোডের জন্য আলাদা Key ও আলাদা Service ID। TechNova-র CI/CD-তে Key সরাসরি পাইপলাইন সংজ্ঞায় লেখা ছিল — একদিন রিপোজিটরি ভুল করে পাবলিক হয়ে গেল। ভাগ্যক্রমে Key-টি শুধু ডেভের ছিল; তবু শিক্ষা ছিল: সিক্রেট স্টোর ছাড়া অটোমেশন নয়।

> **WARNING:** `export IBMCLOUD_API_KEY=...` টার্মিনালে টাইপ করলে তা শেল ইতিহাসে থেকে যায়। স্ক্রিপ্টে Key লিখলে তা গিট ইতিহাসে থেকে যায় — মুছে ফেললেও ইতিহাসে থাকে। Key কেবল ভল্ট বা CI/CD সিক্রেট স্টোর থেকে পড়ুন, এবং সন্দেহ হলে বাতিল করে নতুন তৈরি করুন।

৪.১০ অটোমেশন স্ক্রিপ্টিং: নিরাপদ প্যাটার্ন

একটি ভালো স্ক্রিপ্টের বৈশিষ্ট্য: শুরুতে যাচাই, মাঝে লগ, শেষে যাচাই, ব্যর্থতায় থেমে যাওয়া। Bash-এ `set -euo pipefail` দিয়ে প্রথম ত্রুটিতেই থামুন; অনির্ধারিত চলক ব্যবহারে থামুন; পাইপের ত্রুটি ধরুন। প্রতিটি ধ্বংসাত্মক কমান্ডের আগে কী মুছতে যাচ্ছেন তা লগ করুন। শুকনো-চালনা (dry-run) সমর্থন করুন যেখানে সম্ভব — প্রথমে কী হবে দেখান, তারপর করুন।

```
#!/usr/bin/env bash
set -euo pipefail
# Fail fast if target is not dev
ibmcloud target -g technova-dev
CURRENT=$(ibmcloud target --output JSON | grep -o "technova-dev" || true)
if [ -z "$CURRENT" ]; then
  echo "Wrong target — stopping"
  exit 1
fi
ibmcloud resource groups
echo "Target verified: technova-dev"
```

উপরের স্ক্রিপ্টটি ধারণা বোঝায়; JSON পার্সিংয়ের জন্য `jq` ব্যবহার করা আরও নির্ভরযোগ্য — তবে `jq` ইনস্টল আছে কি না তা যাচাই করুন। স্ক্রিপ্টে কখনও প্রোডাকশন গ্রুপের নাম হার্ডকোড করে ডেভ স্ক্রিপ্ট চালাবেন না; পরিবেশ চলক থেকে গ্রুপের নাম পড়ুন।

৪.১১ Terraform প্রমাণীকরণ

Terraform IBM Cloud প্রদানকারী (provider) দিয়ে সম্পদ ঘোষণা করে। প্রমাণীকরণের সাধারণ পথ: পরিবেশ চলকে API Key দেওয়া — `IBMCLOUD_API_KEY` — অথবা প্রদানকারী ব্লকে Key উল্লেখ না করে চলক থেকে পড়া। Key কখনও `.tf` ফাইলে সরাসরি লিখবেন না; `.tfvars`-এও নয়, যদি তা গিটে যায়। Terraform state ফাইলে সংবেদনশীল মান থাকতে পারে — state-কে দূরবর্তী ব্যাকএন্ডে (remote backend) এনক্রিপ্ট করে রাখুন, গিটে কমিট করবেন না।

`terraform plan` দিয়ে কী বদলাবে তা দেখুন, তারপর `terraform apply` করুন — plan না পড়ে apply করা হাতে চোখ বন্ধ করে সই করার সমান।

```
export IBMCLOUD_API_KEY="CHANGE-ME-API-KEY"
export IBMCLOUD_REGION="us-south"
terraform init
terraform plan -out technova.plan
terraform apply technova.plan
```

```
terraform {
  required_providers {
    ibm = {
      source = "IBM-Cloud/ibm"
    }
  }
}
provider "ibm" {
  region = "us-south"
}
resource "ibm_resource_group" "dev" {
  name = "technova-dev"
}
```

প্রদানকারীর সংস্করণ পিন করুন — নতুন সংস্করণে আচরণ বদলাতে পারে। Terraform-এর ব্যর্থতা প্রায়ই IAM অনুমতি বা ভুল অঞ্চলের কারণে হয়; plan-এর ত্রুটি বার্তা পড়ুন, state লক (lock) থাকলে কে লক করেছে দেখুন — জোর করে লক ভাঙবেন না।

৪.১২ প্রোডাকশন পরিস্থিতি

TechNova-র ডেপ্লয় পাইপলাইন একদিন থেমে গেল — Terraform 401 দিচ্ছে। কারণ: Service ID-র API Key ঘোরানো হয়েছিল, কিন্তু CI/CD সিক্রেট স্টোরে নতুন Key আপডেট করা হয়নি। সমাধান ছিল ঘোরানোর চেকলিস্ট: নতুন Key তৈরি → সিক্রেট স্টোর আপডেট → পাইপলাইন পরীক্ষা → পুরনো Key বাতিল। এই ক্রমটি লিখিত না থাকলে প্রতিবারই কেউ একটি ধাপ ভুলে যাবে। আরেকটি পরিস্থিতি: একজন ডেভেলপার স্থানীয় মেশিনে প্রোডাকশন Key রেখে ল্যাপটপ হারালেন — Key বাতিল করে নতুন Key তৈরি করা হলো, এবং নীতি হলো স্থানীয় মেশিনে প্রোড Key রাখা নিষিদ্ধ। অটোমেশন শুধু কোড নয়, প্রক্রিয়া।

> **REAL WORLD:** একটি দল প্রতিটি ডেপ্লয়ে হাতে CLI চালাত — কখনও ভুল অঞ্চল, কখনও ভুল গ্রুপ। Terraform-এ যাওয়ার পর প্রথম সপ্তাহে কাজ ধীর মনে হলো, কিন্তু দ্বিতীয় মাসে ডেপ্লয় ব্যর্থতা অর্ধেক কমে গেল — কারণ plan-এ ভুল ধরা পড়ত, প্রোডাকশনে নয়। অটোমেশনের সুফল তাৎক্ষণিক নয়, সঞ্চিত।

৪.১৩ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: Key ভল্টে রাখুন, ঘোরান, ন্যূনতম সুযোগ দিন, অডিট লগ দেখুন, ভাগ করা মেশিনে টোকেন রেখে যাবেন না। খরচ: অটোমেশন খরচ কমায় না, খরচকে দৃশ্যমান করে — স্ক্রিপ্টে ভুল লুপে শত শত সার্ভার তৈরি হতে পারে; তাই স্ক্রিপ্টে সীমা (limit) ও নিশ্চিতকরণ রাখুন, এবং বিলিং সতর্কতা বসান। একটি স্ক্রিপ্ট যা সম্পদ তৈরি করে, তার সঙ্গে সেই সম্পদ মুছে ফেলার স্ক্রিপ্টও লিখুন — তৈরি করা সহজ, পরিষ্কার করা কঠিন।

৪.১৪ সাধারণ ভুল

- Key কমান্ড লাইনে বা কোডে লেখা।
- টার্গেট যাচাই না করে স্ক্রিপ্ট চালানো।
- plan না পড়ে apply করা।
- Terraform state গিটে কমিট করা।
- মানুষের Key দিয়ে CI/CD চালানো।
- প্লাগইন সংস্করণ না মেলানো।
- হার সীমা (429) পেলে আবার দ্রুত চেষ্টা করা — ব্যাকঅফ (backoff) ছাড়া।

৪.১৫ ট্রাবলশুটিং

TROUBLESHOOTING: CLI 401 দিলে: আবার লগইন করুন, Key বাতিল হয়নি তো দেখুন, সঠিক অ্যাকাউন্টে আছেন কি দেখুন। 403 দিলে: IAM নীতি দেখুন (অধ্যায় ৩)। 404 দিলে: অঞ্চল ও গ্রুপ যাচাই করুন — সম্পদ অন্য অঞ্চলে থাকতে পারে। Terraform “provider not found” দিলে: `terraform init` আবার চালান, সংস্করণ যাচাই করুন। API 429 দিলে: থামুন, অপেক্ষা করুন, পুনরায় চেষ্টার মধ্যে ব্যবধান বাড়ান — দ্রুত পুনরায় চেষ্টা সীমা আরও বাড়ায়।

স্ক্রিপ্ট হঠাৎ থেমে গেলে প্রস্থান কোড ও শেষ লগ লাইন দেখুন; `set -x` দিয়ে স্ক্রিপ্টের প্রতিটি কমান্ড লগ করুন (তবে Key লগে না আসে সেদিকে খেয়াল রাখুন — গোপন মান `set -x` আউটপুটেও দেখা যেতে পারে)।

৪.১৬ সারসংক্ষেপ

CLI, API, SDK ও Terraform একই সত্যের চারটি মুখ — IAM-নিয়ন্ত্রিত API। লগইন ও টার্গেটিং নির্ভুলভাবে করুন, প্লাগইন সংস্করণ মেলান, API ত্রুটি কোড পড়ুন, Key ভল্টে রাখুন ও ঘোরান, Terraform-এ plan পড়ে apply করুন। অটোমেশনের মূল্য তার গতিতে নয়, তার পুনরুৎপাদনযোগ্যতা ও নিরীক্ষণযোগ্যতায়। TechNova-র পাইপলাইন এই নীতিগুলো মেনে চলে বলেই রাতের ডেপ্লয় সকালে অবাক করে না।

৪.১৭ পর্যালোচনা প্রশ্ন

1. ভুল টার্গেটে স্ক্রিপ্ট চালানোর ঝুঁকি কমাতে স্ক্রিপ্টের শুরুতে কী যাচাই করবেন?
2. মানুষের লগইন ও Service ID-ভিত্তিক লগইনের মধ্যে কখন কোনটি ব্যবহার করবেন?
3. 401, 403, 404 ও 429 — প্রতিটি কী বলে, এবং প্রতিটির পরবর্তী পদক্ষেপ কী?
4. Terraform state-এ সংবেদনশীল মান থাকতে পারে — তা কীভাবে নিরাপদে রাখবেন?
5. API Key ঘোরানোর সঠিক ক্রম কী, যাতে পাইপলাইন না থামে?

৪.১৮ ব্যবহারিক অনুশীলন

1. একটি নিরাপদ Bash স্ক্রিপ্ট লিখুন: লগইন যাচাই, টার্গেট যাচাই (technova-dev), রিসোর্স গ্রুপ তালিকা, এবং ভুল টার্গেটে থেমে যাওয়া — Key ছাড়া, পরিবেশ চলক থেকে পড়ে।

2. Terraform দিয়ে একটি রিসোর্স গ্রুপ ঘোষণা করুন: `terraform plan` চালিয়ে আউটপুট পড়ুন, কী তৈরি হবে তা ব্যাখ্যা করুন, এবং state ফাইল কোথায় রাখবেন তা লিখুন।

অধ্যায় ৫ • PART I — IBM Cloud Foundations & Architecture

Chapter 5 — Global Infrastructure, Responsibility and Billing

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক পারবেন — (১) IBM Cloud-এর গ্লোবাল স্থাপত্য ব্যাখ্যা করতে; (২) Shared Responsibility Model-এ কার দায়িত্ব কী তা নির্ধারণ করতে; (৩) নির্ভরযোগ্যতা ও উপলভ্যতা (reliability/availability) পরিমাপ ও নকশা করতে; (৪) নিরাপত্তা ও সম্মতির ধারণা প্রয়োগ করতে; (৫) মূল্য মডেল, বিলিং ও ব্যবহার পর্যবেক্ষণ করতে; (৬) খরচ বরাদ্দ, ট্যাগ ও এন্টারপ্রাইজ গভর্ন্যান্স পরিচালনা করতে। এই অধ্যায় Part I-কে একত্র করে — অবকাঠামো, দায়িত্ব ও অর্থ একই সিদ্ধান্তের তিনটি মুখ।

- SLA-র শতাংশকে ডাউনটাইম মিনিটে অনুবাদ করা।
- একটি ঘটনার দায়িত্ব প্রদানকারী নাকি গ্রাহকের — তা সীমারেখা দিয়ে বলা।
- বিলের একটি লাইন থেকে কোন সম্পদ, কোন দল দায়ী তা খুঁজে বের করা।
- খরচ কমানোর সিদ্ধান্তে নির্ভরযোগ্যতার ট্রেড-অফ ব্যাখ্যা করা।

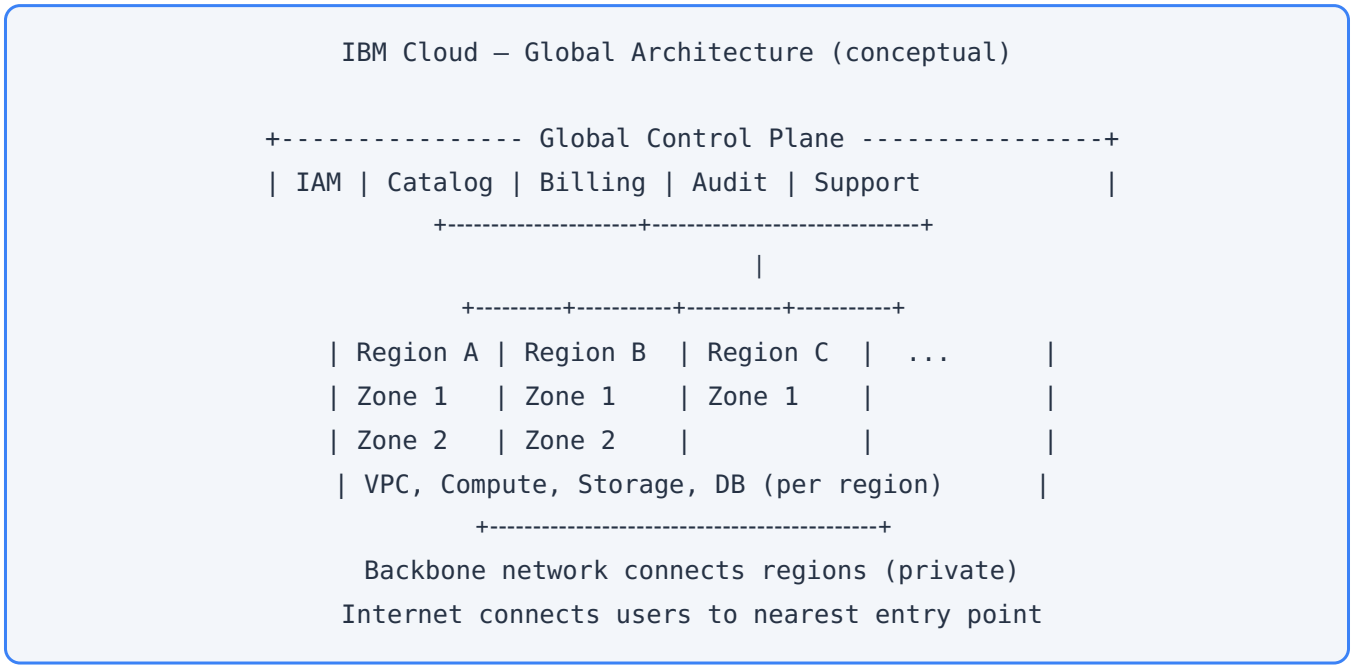
৫.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

অধ্যায় ১ থেকে রিজিয়ন/জোন, অধ্যায় ২ থেকে অ্যাকাউন্ট/গ্রুপ/ট্যাগ, অধ্যায় ৩ থেকে IAM, অধ্যায় ৪ থেকে CLI/API — সবই এই অধ্যায়ে একত্র হবে। একটি সংখ্যা মনে করুন: 99.9% উপলভ্যতা মানে মাসে প্রায় ৪৩ মিনিট ডাউনটাইমের সুযোগ; 99.99% মানে প্রায় ৪ মিনিট। শতাংশের ছোট পার্থক্য মিনিটে বড় পার্থক্য — এবং খরচে আরও বড়। আরেকটি ধারণা: MTTR (mean time to recover — পুনরুদ্ধারে গড় সময়) ও MTBF (mean time between failures — ব্যর্থতার মধ্যে গড় সময়)। উপলভ্যতা বাড়ানো মানে ব্যর্থতা কমানো (MTBF বাড়ানো) ও পুনরুদ্ধার দ্রুত করা (MTTR কমানো) — দুটিই নকশা ও প্রক্রিয়ার কাজ।

৫.৩ প্রথম নীতি থেকে ব্যাখ্যা: গ্লোবাল অবকাঠামো কেন এভাবে তৈরি?

একটি ডেটা সেন্টার যত ভালোই হোক, তার বিদ্যুৎ, কুলিং, নেটওয়ার্ক ও ভবন — প্রতিটিরই ব্যর্থতার সম্ভাবনা আছে। তাই নির্ভরযোগ্যতা আসে একটি ভবনকে নিখুঁত করার চেষ্টা থেকে নয়, একাধিক স্বাধীন ভবনে ছড়িয়ে দেওয়া থেকে। এই ধারণাই জোন: একই অঞ্চলে একাধিক স্বাধীন ডেটা সেন্টার, যাদের মধ্যে দ্রুত, নির্ভরযোগ্য সংযোগ আছে। আর অঞ্চল: ভৌগোলিকভাবে দূরে অবস্থিত জোনের সমষ্টি, যাতে একটি অঞ্চলের দুর্যোগ (বন্যা, ভূমিকম্প, বিস্তৃত বিদ্যুৎ বিভ্রাট) অন্য অঞ্চলকে স্পর্শ না করে। TechNova-র প্রোডাকশন দুটি জোনে চলে, ব্যাকআপ অন্য অঞ্চলে যায় — এই দুটি সিদ্ধান্ত দুটি ভিন্ন ঝুঁকির উত্তর।

গ্লোবাল নেটওয়ার্কের অভ্যন্তরীণ কাজ: অঞ্চলগুলোর মধ্যে IBM-এর ব্যক্তিগত ব্যাকবোন নেটওয়ার্ক থাকে, যা পাবলিক ইন্টারনেটের চেয়ে নিয়ন্ত্রিত ও অনুমানযোগ্য। আপনার VPC-র ভেতরের ট্রাফিক, অঞ্চলের মধ্যে রেপ্লিকেশন, ও ব্যাকআপ — এগুলো এই নেটওয়ার্ক দিয়ে চলে। ব্যবহারকারীর ট্রাফিক ইন্টারনেট দিয়ে এসে নিকটতম প্রবেশপথে ঢোকে, তারপর লোড ব্যালান্সারের মাধ্যমে জোনে ছড়ায়। এই পথের প্রতিটি ধাপে ব্যর্থতা সম্ভব — তাই স্বাস্থ্য পরীক্ষা, স্বয়ংক্রিয় ফেইলওভার ও পুনরায় চেষ্টা নকশায় রাখতে হয়।



CONCEPT: Shared Responsibility Model হলো দায়িত্বের সীমারেখা: প্রদানকারী ক্লাউডের নিরাপত্তা দেয় — ভৌত ডেটা সেন্টার, হাইপারভাইজার, নেটওয়ার্ক অবকাঠামো; গ্রাহক ক্লাউডে যা রাখে তার নিরাপত্তা দেয় — ডেটা, অ্যাপ কোড, IAM নীতি, নেটওয়ার্ক কনফিগারেশন, এনক্রিপশন চাবি। পরিচালিত সেবায় সীমারেখা সরে — প্রদানকারী আরও বেশি নেয়, কিন্তু ডেটা ও অ্যাক্সেসের দায়িত্ব গ্রাহকেরই থাকে।

৫.৪ Shared Responsibility Model: বিস্তারিত সীমারেখা

সীমারেখাটি সেবা অনুযায়ী বদলায় — এটাই সবচেয়ে গুরুত্বপূর্ণ বোঝাপড়া। IaaS-এ: প্রদানকারী ভৌত হার্ডওয়্যার, ভার্চুয়ালাইজেশন ও নেটওয়ার্ক অবকাঠামো দেয়; আপনি OS প্যাচ, ফায়ারওয়াল নিয়ম, অ্যাপ নিরাপত্তা ও ডেটা এনক্রিপশনের দায়িত্বে। PaaS-এ: প্রদানকারী OS, রানটাইম ও প্যাচ নেয়; আপনি অ্যাপ কোড, ডেটা, অ্যাক্সেস নীতি ও কনফিগারেশনের দায়িত্বে। SaaS-এ: প্রদানকারী প্রায় সবকিছু নেয়; আপনি কেবল ব্যবহারকারী অ্যাক্সেস, ডেটা শ্রেণিবিভাগ ও কনফিগারেশনের দায়িত্বে। TechNova-র পরিচালিত PostgreSQL-এ প্রদানকারী প্যাচ ও ব্যাকআপ চালায়, কিন্তু কে ডেটাবেসে ঢুকতে পারে, ডেটা এনক্রিপশন চাবি কে নিয়ন্ত্রণ করে, ও ব্যাকআপ কতদিন রাখা হবে — তা TechNova-র সিদ্ধান্ত।

স্তর	প্রদানকারী	গ্রাহক (TechNova)	ভুল ধারণা
ভৌত ডেটা সেন্টার	নিরাপত্তা, বিদ্যুৎ, কুলিং	—	“ক্লাউডে ডেটা নিরাপদ, আর কিছু করতে হবে না”
হাইপারভাইজার / নেটওয়ার্ক	বিচ্ছিন্নতা, প্যাচ	—	—
OS / রানটাইম (PaaS)	প্যাচ, আপডেট	কনফিগারেশন	“পরিচালিত মানেই নিরাপদ”
অ্যাপ ও ডেটা	—	কোড, ডেটা, IAM, এনক্রিপশন	“প্রদানকারী ব্যাকআপ নেয়, তাই আমাদের নিতে হবে না”
নেটওয়ার্ক নীতি	অবকাঠামো		“ডিফল্ট নেটওয়ার্কই নিরাপদ”

		সিকিউরিটি গ্রুপ, ACL, ফায়ারওয়াল	

একটি সাধারণ ভুল: ব্যাকআপের দায়িত্ব পুরোপুরি প্রদানকারীর ভাবা। পরিচালিত সেবা স্বয়ংক্রিয় ব্যাকআপ দিলেও, ব্যাকআপের ধারণ (retention), পুনরুদ্ধার পরীক্ষা, ও অন্য অঞ্চলে কপি — এগুলো গ্রাহকের নীতি। পুনরুদ্ধার পরীক্ষা না করা ব্যাকআপ, ব্যাকআপ না থাকার সমান — কারণ পুনরুদ্ধারের সময়ই জানা যায় ব্যাকআপ কাজ করে কি না।

৫.৫ নির্ভরযোগ্যতা ও উপলভ্যতা: পরিমাপ ও নকশা

উপলভ্যতা (availability) হলো সেবা ব্যবহারযোগ্য থাকার অনুপাত; নির্ভরযোগ্যতা (reliability) হলো সময়ের সঙ্গে সেই অনুপাত ধরে রাখার ক্ষমতা। SLA (Service Level Agreement) হলো প্রদানকারীর প্রতিশ্রুতি — একটি নির্দিষ্ট শতাংশ উপলভ্যতা, যা না মিললে ক্ষতিপূরণের শর্ত থাকতে পারে। SLA পড়ার সময় তিনটি বিষয় দেখুন: কোন সেবার জন্য, কোন পরিমাপে, ও কী ব্যতিক্রম আছে। একটি সেবার 99.9% SLA মানে সেই সেবার উপলভ্যতা, আপনার পুরো অ্যাপের নয় — আপনার অ্যাপের উপলভ্যতা তার সবচেয়ে দুর্বল উপাদানের ওপর নির্ভর করে। TechNova-র অ্যাপ যদি একটি জোনের ডেটাবেসের ওপর নির্ভর করে, তবে অ্যাপের উপলভ্যতা ডেটাবেসের চেয়ে বেশি হতে পারে না।

নকশার প্যাটার্ন: (১) অপ্রয়োজনীয়তা (redundancy) — প্রতিটি স্তরে অন্তত দুটি; (২) স্বাস্থ্য পরীক্ষা ও স্বয়ংক্রিয় প্রতিস্থাপন; (৩) সার্কিট ব্রেকার — ব্যর্থ সেবাকে বারবার ডেকে সময় নষ্ট না করা; (৪) পুনরায় চেষ্টা ব্যাকঅফসহ; (৫) গ্রেসফুল ডিগ্রেডেশন — একটি অংশ ব্যর্থ হলে সেবা আংশিকভাবে চলা। দুর্যোগ পুনরুদ্ধারের দুটি সংখ্যা: RPO (Recovery Point Objective — কত ডেটা হারানো চলবে) ও RTO (Recovery Time Objective — কতক্ষণে ফিরতে হবে)। TechNova-র জন্য RPO ১৫ মিনিট ও RTO ১ ঘণ্টা মানে: ১৫ মিনিটের বেশি ডেটা হারানো চলবে না, ১ ঘণ্টার মধ্যে সেবা ফিরতে হবে — এই দুটি সংখ্যাই ব্যাকআপের ঘনত্ব ও ফেইলওভার প্রক্রিয়া নির্ধারণ করে।

> **REAL WORLD:** TechNova-র স্টেজিং ডেটাবেস একবার নষ্ট হলো — কেউ ভয় পেল না, কারণ স্টেজিংয়ের RPO ছিল “দিনের শেষে পুনরায় তৈরি করা যাবে।” প্রোডাকশনের RPO ছিল ১৫ মিনিট, তাই সেখানে ক্রমাগত রেপ্লিকেশন চলত। একই প্রযুক্তি, ভিন্ন নীতি — কারণ ব্যবসায়িক ক্ষতি ভিন্ন। RPO/RTO ব্যবসার সঙ্গে ঠিক করুন, প্রযুক্তির সঙ্গে নয়।

৫.৬ নিরাপত্তা ও সম্মতি ধারণা

নিরাপত্তার তিনটি স্তম্ভ: গোপনীয়তা (confidentiality — কেবল অনুমোদিতরা দেখবে), অখণ্ডতা (integrity — ডেটা বদলানো যাবে না অনুমোদন ছাড়া), উপলভ্যতা (availability — অনুমোদিতরা প্রয়োজনে পাবে)। এনক্রিপশন গোপনীয়তা ও অখণ্ডতায় সাহায্য করে — বিশ্রামে (at rest) ও চলাচলে (in transit)। চাবি ব্যবস্থাপনা (key management) হলো এনক্রিপশনের চাবি কে তৈরি, সংরক্ষণ ও ঘোরায় — চাবি হারালে ডেটা হারায়, চাবি ফাঁস হলে ডেটা ফাঁস হয়। নেটওয়ার্ক নিরাপত্তা: VPC, সাবনেট, সিকিউরিটি গ্রুপ, নেটওয়ার্ক ACL — স্তরে স্তরে বিভাজন। অডিট ও লগ: কে, কখন, কী করল — Activity Tracker / অডিট লগে সংরক্ষণ, নিয়মিত পর্যালোচনা।

সম্মতি (compliance) মানে নিয়ম মেনে চলা — যেমন ডেটা সুরক্ষা আইন, শিল্প মান (যেমন ব্যাংকিং বা স্বাস্থ্য), ও অভ্যন্তরীণ নীতি। IBM Cloud বিভিন্ন সম্মতি কাঠামো সমর্থন করে, কিন্তু কোনো নির্দিষ্ট সার্টিফিকেশন বা সুযোগ দাবি করার আগে বর্তমান সম্মতি নথি যাচাই করতে হবে — সার্টিফিকেশন সেবা ও অঞ্চলভেদে ভিন্ন হয়। TechNova-র কর্তব্য: তাদের গ্রাহকের চুক্তি ও আইনি দাবি চিহ্নিত করা, তারপর অঞ্চল, এনক্রিপশন ও অ্যাক্সেস নীতি সেই দাবির সঙ্গে মেলানো — সম্মতি একটি চেকবক্স নয়, একটি নকশা সিদ্ধান্ত।

৫.৭ মূল্য মডেল: কীভাবে বিল হয়?

IBM Cloud-এর মূল্য মডেল সেবা অনুযায়ী ভিন্ন: কিছু সেবা ব্যবহার অনুযায়ী (pay-as-you-go — যত ব্যবহার, তত বিল), কিছু সেবা পরিকল্পনা/স্তর অনুযায়ী (নির্দিষ্ট আকারের জন্য নির্দিষ্ট মূল্য), কিছু সেবা সংরক্ষিত ক্ষমতা অনুযায়ী। এই বইয়ে কোনো নির্দিষ্ট মূল্য উল্লেখ করা হবে না — মূল্য অঞ্চল, পরিকল্পনা ও ব্যবহারের ওপর নির্ভর করে এবং পরিবর্তিত হয়। মূল্য যাচাইয়ের সঠিক পথ: IBM Cloud ক্যাটালগে সেবার পরিকল্পনা দেখুন, মূল্য ক্যালকুলেটরে আপনার ব্যবহারের অনুমান দিন, এবং বিলিং ভিউতে প্রকৃত ব্যবহার পর্যবেক্ষণ করুন। TechNova-র নিয়ম: কোনো সেবা নেওয়ার আগে ক্যালকুলেটরে অনুমান, নেওয়ার পর প্রথম বিলে যাচাই — অনুমান ও বাস্তবের পার্থক্য থেকে শেখা।

মূল্যের তিনটি লুকানো খাত মনে রাখুন। প্রথম, ডেটা স্থানান্তর (egress) — ক্লাউড থেকে বাইরে যাওয়া ডেটার খরচ, অঞ্চলের ভেতরে সাধারণত ভিন্ন। দ্বিতীয়, স্টোরেজ — শুধু ব্যবহৃত জায়গা নয়, স্ন্যাপশট, ব্যাকআপ ও অপ্রয়োজনীয় কপির খরচ। তৃতীয়, অলস সম্পদ — চলমান কিন্তু অব্যবহৃত সার্ভার, সংযুক্ত কিন্তু অব্যবহৃত ডিস্ক, বরাদ্দ কিন্তু অব্যবহৃত IP। এই তিনটি খাতই বিলে নীরবে বাড়ে, কারণ কেউ তাদের “ব্যবহার” করছে না বলে মনে হয় — কিন্তু বিল আসে বরাদ্দের ওপর।

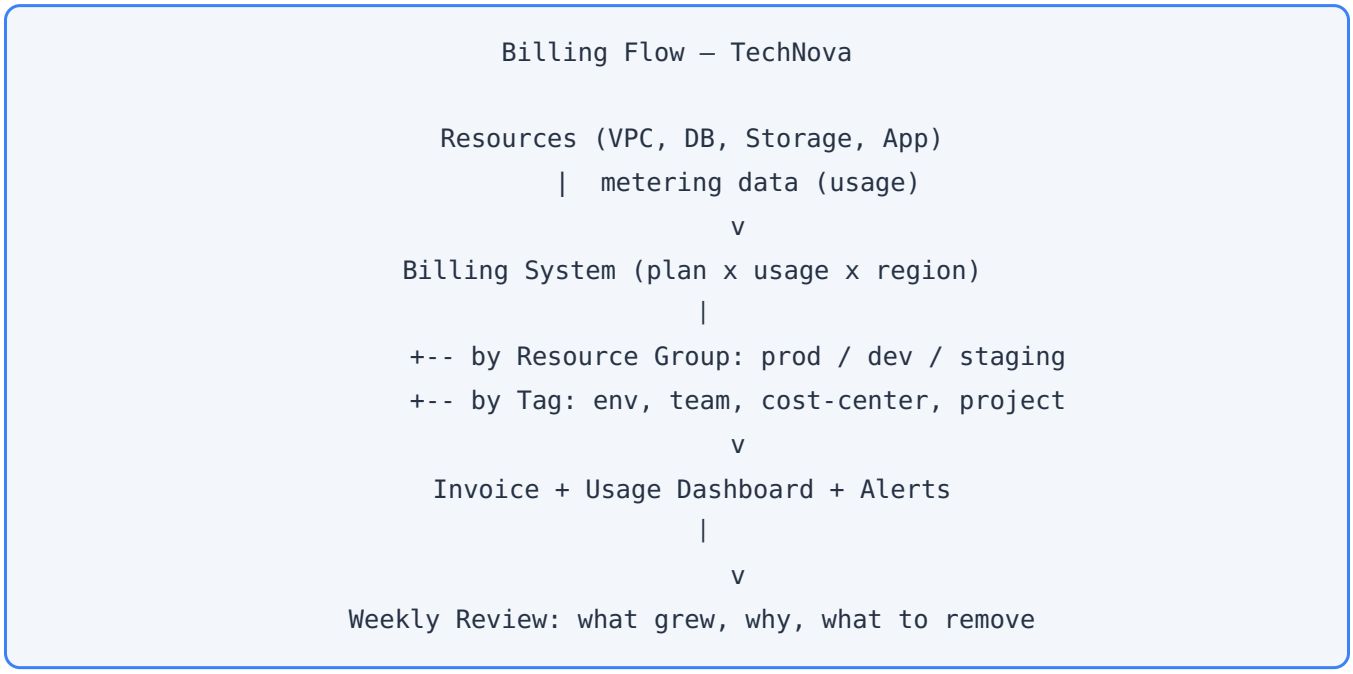
মূল্য উপাদান	কীভাবে বিল হয়	নিয়ন্ত্রণের উপায়
কম্পিউট	আকার × সময়	সঠিক আকার, অলস সময়ে বন্ধ
স্টোরেজ	ধারণক্ষমতা × সময়, শ্রেণি অনুযায়ী	পুরনো স্ন্যাপশট মুছুন, জীবনচক্র নীতি
ডেটা স্থানান্তর	বাইরে যাওয়া ডেটার পরিমাণ	ক্যাশিং, অঞ্চলের ভেতরে রাখা
পরিচালিত সেবা	পরিকল্পনা / ব্যবহার অনুযায়ী	পরিকল্পনা যাচাই, ক্যালকুলেটর
API / লেনদেন	কলের সংখ্যা অনুযায়ী (যেখানে প্রযোজ্য)	অপ্রয়োজনীয় কল কমান

৫.৮ বিলিং ও ব্যবহার পর্যবেক্ষণ

বিলিং পর্যবেক্ষণ মানে মাসের শেষে বিল দেখা নয় — সপ্তাহে বা দৈনিক ব্যবহার দেখা। IBM Cloud কনসোলে বিলিং ও ব্যবহারের ভিউতে খরচের প্রবণতা, সেবা অনুযায়ী ভাগ, ও অঞ্চল অনুযায়ী ভাগ দেখা যায়। সতর্কতা (alert) বসান: একটি সীমা অতিক্রম করলে ইমেইল বা বিজ্ঞপ্তি আসবে — তবে সতর্কতা মানে বিল থেমে যাওয়া নয়, এটি কেবল জানায়। TechNova-র পদ্ধতি: প্রতি সোমবার ১৫ মিনিটের খরচ পর্যালোচনা — গত সপ্তাহের তুলনায় কী বাড়ল, কেন বাড়ল, অপ্রয়োজনীয় কী আছে। এই অভ্যাসটি একটি বড় বিলের চেয়ে অনেক সস্তা। ব্যবহার পর্যবেক্ষণের অভ্যন্তরীণ কাজ: প্রতিটি সম্পদ মিটারিং ডেটা তৈরি করে, বিলিং ব্যবস্থা তা সংগ্রহ করে পরিকল্পনা অনুযায়ী মূল্যায়ন করে, তারপর অ্যাকাউন্টের বিলে যোগ করে।

৫.৯ খরচ বরাদ্দ, রিসোর্স ট্যাগ ও এন্টারপ্রাইজ গভর্ন্যান্স

খরচ বরাদ্দ (cost allocation) মানে প্রতিটি টাকার মালিক নির্ধারণ করা — রিসোর্স গ্রুপ, ট্যাগ (`env`, `team`, `project`, `cost-center`), ও অ্যাকাউন্ট কাঠামো দিয়ে। ট্যাগিং নীতি: প্রতিটি সম্পদে বাধ্যতামূলক ট্যাগ, নামকরণের নিয়ম, ও নিয়মিত নিরীক্ষা — ট্যাগ ছাড়া সম্পদ খুঁজে বের করে ট্যাগ করা বা মুছে ফেলা। এন্টারপ্রাইজ গভর্ন্যান্স এর ওপরে নীতি যোগ করে: কে কোন অঞ্চলে সম্পদ তৈরি করতে পারে, কোন পরিকল্পনা ব্যবহার করতে পারে, কোন সেবা নিষিদ্ধ — এই নীতিগুলো কেন্দ্রীয়ভাবে প্রয়োগ করা হয়, যাতে প্রতিটি দল আলাদা নিয়ম না বানায়। গভর্ন্যান্সের ব্যর্থতা: নীতি এত কঠোর যে দল কাজ করতে পারে না — তখন দল নীতির বাইরে গিয়ে কাজ করে (shadow IT)। ভালো গভর্ন্যান্স সুরক্ষা দেয়, গতি আটকায় না।



৫.১০ CLI / API উদাহরণ

```
ibmcloud login
ibmcloud billing account-usage
ibmcloud billing resource-usage
ibmcloud resource search "technova-prod"
ibmcloud resource tags --help
ibmcloud target -r us-south -g technova-prod
ibmcloud ks clusters --help
```

উপরের কমান্ডগুলো আকৃতি বোঝায়; বিলিং কমান্ডের সঠিক নাম ও ফ্ল্যাগ আপনার CLI সংস্করণে `ibmcloud billing --help` দিয়ে যাচাই করুন। ট্যাগিং CLI-তে বা কনসোলে করা যায় — যেভাবেই করুন, ট্যাগের নাম ও মানের নিয়ম একই রাখুন। বিলিং ডেটা API দিয়েও পাওয়া যায় — স্বয়ংক্রিয় প্রতিবেদনের জন্য API নথি দেখুন।

৫.১১ IaC উদাহরণ — ট্যাগসহ সম্পদ

```
resource "ibm_resource_group" "prod" {
  name = "technova-prod"
}
# Tagging is applied per-resource; keep tag keys consistent:
# env, team, cost-center, project — same spelling everywhere.
# Example (illustrative): attach tags at creation where the
# resource schema supports a tags argument, and audit the
# rest weekly with the billing dashboard.
```

IaC-এ ট্যাগিংয়ের সুবিধা: প্রতিটি সম্পদ তৈরির সময় ট্যাগ দেওয়া হয়, ভুলে যাওয়া যায় না। অসুবিধা: পুরনো সম্পদে ট্যাগ যোগ করতে আলাদা প্রক্রিয়া দরকার — তাই নতুন সম্পদে IaC, পুরনো সম্পদে পর্যায়ক্রমিক নিরীক্ষা।

> **WARNING:** বিলিং সতর্কতা বিল থামায় না। সতর্কতা পেয়ে যদি কেউ ব্যবস্থা না নেয়, বিল বাড়তেই থাকবে। সতর্কতার সঙ্গে একটি দায়িত্বশীল ব্যক্তি ও একটি পদক্ষেপ যুক্ত করুন — কে দেখবে, কী করবে, কতক্ষণে।

৫.১২ প্রোডাকশন পরিস্থিতি

TechNova-র এক মাসের বিল হঠাৎ দ্বিগুণ হলো। পর্যালোচনায় দেখা গেল: একটি পরীক্ষার স্ক্রিপ্ট ভুল করে প্রোডাকশনে বড় আকারের সার্ভার তৈরি করেছিল, আর কেউ মুছে ফেলেনি। কারণ ছিল দুটি: স্ক্রিপ্টে টার্গেট যাচাই ছিল না (অধ্যায় ৪), আর বিলিং সতর্কতা ছিল না। সমাধান: সতর্কতা বসানো, সাপ্তাহিক পর্যালোচনা শুরু করা, ও স্ক্রিপ্টে সীমা যোগ করা। আরেকটি পরিস্থিতি: একটি অঞ্চলে বিদ্যুৎ বিভ্রাট হলো — TechNova-র মাল্টি-জোন অ্যাপ অন্য জোনে চলতে থাকল, কিন্তু একক-জোনের একটি অভ্যন্তরীণ টুল বন্ধ হলো। শিক্ষা: প্রোডাকশন নয় এমন সেবারও উপলভ্যতা পরিকল্পনা দরকার — কোনটি বন্ধ হলে ব্যবসা থামে, কোনটি থামে না, তা আগে থেকে ঠিক করুন।

৫.১৩ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: অডিট লগ সংরক্ষণ করুন, চাবি ঘোরান, নেটওয়ার্ক বিভাজন করুন, সম্মতির দাবি নথিভুক্ত করুন। খরচ: সঠিক আকার (right-sizing) করুন — ব্যবহার না হওয়া CPU/মেমরি কমান; অলস সম্পদ বন্ধ করুন; স্টোরেজ জীবনচক্র নীতি বসান; ডেটা স্থানান্তর কমান; বাজেট সতর্কতা ও সাপ্তাহিক পর্যালোচনা করুন। একটি ট্রেড-অফ মনে রাখুন: উচ্চ উপলভ্যতা খরচ বাড়ায় — মাল্টি-জোন ও মাল্টি-অঞ্চল বিনামূল্যে নয়। প্রতিটি “নাইন” (9) যোগ করার আগে জিজ্ঞেস করুন: ব্যবসা কি সেই ডাউনটাইমের ক্ষতি বহন করতে পারে না? যদি পারে, তবে খরচ বাঁচান।

৫.১৪ সাধারণ ভুল

- মূল্য না দেখে সেবা নেওয়া, তারপর বিল দেখে অবাক হওয়া।
- ট্যাগ না দেওয়া, তারপর খরচের মালিক খুঁজে না পাওয়া।
- বিলিং সতর্কতা না বসানো।
- SLA-কে নিজের অ্যাপের উপলভ্যতা ভাবা।
- ব্যাকআপ আছে ভাবা, কিন্তু পুনরুদ্ধার পরীক্ষা না করা।
- প্রদানকারীর দায়িত্ব ভেবে নিজের IAM ও এনক্রিপশন অবহেলা করা।
- অলস সম্পদ ও পুরনো স্ন্যাপশট রেখে দেওয়া।

৫.১৫ ট্রাবলশুটিং

TROUBLESHOOTING: বিল বেশি মনে হলে: (১) বিলিং ভিউতে সেবা ও গ্রুপ অনুযায়ী ভাগ করুন; (২) সবচেয়ে বড় খাতের সম্পদ তালিকা করুন; (৩) অলস ও পুরনো সম্পদ খুঁজুন; (৪) ডেটা স্থানান্তর ও স্টোরেজ পরীক্ষা করুন। সেবা ধীর বা অনুপলভ্য মনে হলে: (১) কোন জোন/অঞ্চলে সমস্যা তা আলাদা করুন; (২) স্বাস্থ্য পরীক্ষা ও লগ দেখুন; (৩) SLA ও স্ট্যাটাস পাতা দেখুন — সমস্যাটি আপনার কনফিগারেশনে নাকি প্রদানকারীর দিকে তা জানা জরুরি। পুনরুদ্ধার পরীক্ষায় ব্যর্থ হলে: ব্যাকআপের বয়স, অঞ্চল ও অনুমতি যাচাই করুন — পুনরুদ্ধারের জন্য আলাদা অনুমতি দরকার হতে পারে।

৫.১৬ সারসংক্ষেপ

গ্লোবাল অবকাঠামো রিজিয়ন ও জোনে বিভক্ত — স্বাধীন ব্যর্থতার ডোমেইনই নির্ভরযোগ্যতার ভিত্তি। Shared Responsibility Model-এ প্রদানকারী ক্লাউডের নিরাপত্তা দেয়, গ্রাহক ক্লাউডে যা রাখে তার নিরাপত্তা দেয়; সীমারেখা সেবা অনুযায়ী বদলায়। উপলভ্যতা শতাংশকে মিনিটে অনুবাদ করুন, RPO/RTO ব্যবসার সঙ্গে ঠিক করুন, ব্যাকআপ পুনরুদ্ধার পরীক্ষা করুন। মূল্য

ক্যাটালগ ও ক্যালকুলেটরে যাচাই করুন — এই বইয়ে নির্দিষ্ট মূল্য নেই। বিলিং পর্যবেক্ষণ, ট্যাগিং ও গভর্ন্যান্স ছাড়া খরচ অদৃশ্য থাকে। Part I-এর শিক্ষা: সংগঠন, পরিচয়, অটোমেশন ও অর্থ — চারটি একসঙ্গে নকশা করুন, আলাদা নয়।

৫.১৭ পর্যালোচনা প্রশ্ন

1. Shared Responsibility Model-এ পরিচালিত ডেটাবেসে প্রদানকারী ও গ্রাহকের দায়িত্বের সীমারেখা কোথায়?
2. 99.9% ও 99.99% উপলভ্যতার মধ্যে মাসিক ডাউনটাইমের পার্থক্য কত, এবং সেই পার্থক্যের খরচ কীভাবে বিচার করবেন?
3. RPO ও RTO কীভাবে ব্যাকআপের ঘনত্ব ও ফেইলওভার নকশা নির্ধারণ করে?
4. বিল হঠাৎ বাড়লে কোন ক্রমে তদন্ত করবেন — সেবা, গ্রুপ, ট্যাগ, সম্পদ?
5. ট্যাগিং ছাড়া খরচ বরাদ্দ কেন অসম্ভব হয়, এবং বাধ্যতামূলক ট্যাগ নীতি কীভাবে প্রয়োগ করবেন?

৫.১৮ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি RPO/RTO টেবিল লিখুন: অ্যাপ, ডেটাবেস, স্ট্যাটিক সম্পদ ও অভ্যন্তরীণ টুল — প্রতিটির RPO/RTO, ব্যাকআপ কৌশল ও পুনরুদ্ধার পরীক্ষার সময়সূচি।
2. একটি খরচ পর্যালোচনা চেকলিস্ট তৈরি করুন: বিলিং ভিউতে কী দেখবেন, ট্যাগ অনুযায়ী কীভাবে ভাগ করবেন, অলস সম্পদ কীভাবে খুঁজবেন, এবং সতর্কতার সীমা কীভাবে নির্ধারণ করবেন — মূল্য উল্লেখ না করে, ক্যাটালগ ও ক্যালকুলেটর যাচাইয়ের ধাপসহ।

PART II • DEVOPS ENGINEERING MASTERY

Networking & Infrastructure

IBM Cloud VPC, Routing ↺ Security, Load Balancing ↺ DNS, Virtual Server, Hybrid Connectivity, High Availability ↺ Resilience

অধ্যায় ৬ • PART II — Networking & Infrastructure

Chapter 6 — IBM Cloud VPC Architecture

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক IBM Cloud VPC (Virtual Private Cloud) এর মৌলিক ধারণা ব্যাখ্যা করতে পারবেন; রিজিয়ন (Region) ও জোন (Zone) এর পার্থক্য এবং ব্যর্থতার ডোমেইন হিসেবে তাদের ভূমিকা বুঝতে পারবেন; সাবনেট (Subnet) ও CIDR পরিকল্পনা নিজে হিসাব করে করতে পারবেন; পাবলিক ও প্রাইভেট IP ঠিকানার ব্যবহার, সীমা ও নিরাপত্তাগত প্রভাব মূল্যায়ন করতে পারবেন; এবং TechNova-র মতো একটি কোম্পানির জন্য নিরাপদ, স্কেলযোগ্য ও ব্যয়-সচেতন VPC নকশা তৈরি করতে পারবেন। পাঠক আরও জানবেন কখন একটি VPC যথেষ্ট, কখন একাধিক VPC দরকার, এবং নকশার সিদ্ধান্ত কীভাবে পরবর্তী অধ্যায়ের রাউটিং, নিরাপত্তা ও লোড ব্যালান্সিংকে প্রভাবিত করে।

- VPC, রিজিয়ন, জোন, সাবনেট ও CIDR-এর সংজ্ঞা ও পারস্পরিক সম্পর্ক ব্যাখ্যা করা।
- একটি প্রদত্ত ঠিকানা-স্থান থেকে সাবনেট ভাগ করা, ব্যবহারযোগ্য হোস্ট গোনা ও ভবিষ্যৎ বৃদ্ধির জন্য জায়গা রাখা।
- পাবলিক বনাম প্রাইভেট সাবনেটের নকশা, IP বরাদ্দ নীতি ও NAT-এর প্রয়োজনীয়তা নির্ধারণ করা।
- TechNova-র তিন-স্তরীয় (ওয়েব, অ্যাপ, ডেটা) আর্কিটেকচারের জন্য সম্পূর্ণ VPC পরিকল্পনা লেখা।

৬.২ প্রারম্ভিক পুনরালোচনা

আগের খণ্ডে আমরা ক্লাউড কম্পিউটিংয়ের মৌলিক ধারণা, IBM Cloud-এর পরিষেবা-মডেল এবং মাল্টি-ক্লাউড কৌশল দেখেছি। এই অধ্যায়ের জন্য তিনটি ধারণা ঝালিয়ে নেওয়া দরকার। প্রথমত, IP ঠিকানা: IPv4 ঠিকানা ৩২ বিটের সংখ্যা, যেমন 203.0.113.10; CIDR নোটেশন যেমন 10.0.0.0/16 বলে দেয় প্রথম ১৬ বিট নেটওয়ার্ক অংশ, বাকি ১৬ বিট হোস্ট অংশ। দ্বিতীয়ত, OSI মডেলের নেটওয়ার্ক স্তর (Layer 3): রাউটিং সিদ্ধান্ত নেয় প্যাকেট কোন পথে যাবে; ফায়ারওয়াল নিয়ম নির্ধারণ করে কোন প্যাকেট যেতে পারবে। তৃতীয়ত, ভার্চুয়ালাইজেশন: একই ভৌত হার্ডওয়্যারে একাধিক ভার্চুয়াল সার্ভার চলে, কিন্তু নেটওয়ার্কের দৃষ্টিতে প্রতিটি সার্ভারের নিজস্ব IP ও ইন্টারফেস আছে। এই তিনটি ধারণা না বুঝলে VPC-কে শুধু একটি কনসোল-পাতা মনে হবে; বুঝলে দেখা যাবে VPC আসলে সফটওয়্যার-নিয়ন্ত্রিত একটি সম্পূর্ণ ডেটা সেন্টার নেটওয়ার্ক।

মনে রাখবেন, অন-প্রিমিসিস নেটওয়ার্কে আপনি কেবল, সুইচ ও রাউটার কিনে বসাতেন; ক্লাউডে সেই একই কাজ API কল দিয়ে হয়। পার্থক্য শুধু এটুকু নয় যে হার্ডওয়্যার অদৃশ্য — পার্থক্য হলো নেটওয়ার্ক এখন কোড: ভুল করলে যেমন দ্রুত ঠিক করা যায়, তেমনি একটি ভুল API কল পুরো পরিবেশকে ইন্টারনেটের সামনে খুলে দিতেও পারে। তাই VPC শেখা মানে শুধু বোতাম চাপা শেখা নয়, নেটওয়ার্ক ডিজাইনের দায়িত্ব শেখা।

৬.৩ প্রথম নীতি থেকে VPC: সমস্যাটি কী?

কল্পনা করুন TechNova তাদের ওয়েব অ্যাপ্লিকেশনটি IBM Cloud-এ চালাতে চায়: একটি ওয়েব সার্ভার, একটি অ্যাপ্লিকেশন সার্ভার ও একটি ডেটাবেস। সবগুলো যদি সরাসরি ইন্টারনেটে বসানো হয়, তাহলে ডেটাবেসও ইন্টারনেট থেকে পৌঁছানো যাবে — এটি নিরাপত্তার দৃষ্টিতে অগ্রহণযোগ্য। আবার সবগুলো যদি সম্পূর্ণ বিচ্ছিন্ন থাকে, তাহলে ওয়েব সার্ভার অ্যাপ সার্ভারের সঙ্গে কথা বলতে পারবে না। প্রয়োজন এমন একটি সীমানা: ভেতরে সার্ভারগুলো স্বাধীনভাবে কথা বলবে, বাইরে থেকে শুধু অনুমোদিত ট্রাফিক ঢুকবে, আর ডেটাবেস থাকবে সম্পূর্ণ আড়ালে। এই সীমানাটিই VPC।

VPC হলো IBM Cloud-এর ভেতরে আপনার নিজস্ব, লজিক্যালভাবে বিচ্ছিন্ন ভার্চুয়াল নেটওয়ার্ক। ভার্চুয়াল মানে এর সুইচ-রাউটার ভৌত যন্ত্র নয়, সফটওয়্যার-সংজ্ঞায়িত (Software-Defined Networking, SDN) নিয়ন্ত্রণ-স্তর দিয়ে বাস্তবায়িত। প্রাইভেট মানে অন্য গ্রাহকের ট্রাফিক আপনার নেটওয়ার্কে ঢুকতে পারে না, যদিও ভৌত অবকাঠামো ভাগ করা হতে পারে।

ক্লাউড মানে আপনি এই নেটওয়ার্ককে API, CLI বা Terraform দিয়ে তৈরি, পরিবর্তন ও মুছে ফেলতে পারেন — কোনো কেবল লাগানোর প্রয়োজন নেই।

> **CONCEPT:** VPC কোনো একক যন্ত্র নয়; এটি একটি নীতিমালার সমষ্টি — কোন IP পরিসর ব্যবহার হবে, কোন সাবনেট কোন জোনে থাকবে, কোন ট্রাফিক কোথায় যেতে পারবে, কোন ঠিকানা ইন্টারনেটে প্রকাশ পাবে। কনসোলে আপনি যা দেখেন তা এই নীতিমালারই প্রতিফলন।

VPC যে সমস্যা সমাধান করে তা তিনটি: বিচ্ছিন্নতা (isolation), নিয়ন্ত্রণ (control) ও পূর্বানুমেয়তা (predictability)। বিচ্ছিন্নতা ছাড়া প্রতিবেশী গ্রাহকের ভুল বা আক্রমণ আপনার সিস্টেমে পৌঁছাতে পারে। নিয়ন্ত্রণ ছাড়া আপনি বলতে পারবেন না কোন সার্ভার কোন সার্ভারের সঙ্গে কথা বলতে পারবে। পূর্বানুমেয়তা ছাড়া IP ঠিকানা, রাউট ও DNS নিয়ে প্রতিবার অনুমান করতে হবে। VPC এই তিনটি দেয় — তবে এর বিনিময়ে আপনাকে নেটওয়ার্ক পরিকল্পনার দায়িত্ব নিতে হয়; ক্লাউড প্রদানকারী আপনার হয়ে সিদ্ধান্ত নেবে না কোন সাবনেটে ডেটাবেস রাখা উচিত।

৬.৪ রিজিয়ন ও জোন: ভূগোলই ব্যর্থতার সীমানা

IBM Cloud-এ **রিজিয়ন** হলো একটি ভৌগোলিক এলাকা, যেমন একটি শহর বা দেশের অংশ, যেখানে একাধিক ডেটা সেন্টার থাকে। **জোন** (Availability Zone) হলো সেই রিজিয়নের ভেতরে একটি স্বতন্ত্র ডেটা সেন্টার বা ডেটা সেন্টার-গোষ্ঠী, যার নিজস্ব বিদ্যুৎ, শীতাতপ নিয়ন্ত্রণ ও নেটওয়ার্ক সংযোগ আছে। একটি VPC একটি নির্দিষ্ট রিজিয়নে তৈরি হয়, কিন্তু তার সাবনেটগুলো ওই রিজিয়নের বিভিন্ন জোনে ছড়িয়ে দেওয়া যায়। এটিই উচ্চ-প্রাপ্যতার (High Availability) ভিত্তি: একটি জোনে বিদ্যুৎ বিভ্রাট হলে অন্য জোনের সার্ভার চলতে থাকে।

জোনকে ব্যর্থতার ডোমেইন (failure domain) হিসেবে ভাবুন। দুটি সার্ভার একই জোনে থাকলে একটি জোন-বিভ্রাট দুটিকেই থামিয়ে দেয়; দুটি জোনে থাকলে একটি বিভ্রাটে অর্ধেক ক্ষমতা টিকে থাকে। তবে জোন-পার ট্রাফিকের বিলম্ব (latency) সাধারণত একই জোনের চেয়ে সামান্য বেশি, এবং কিছু ক্ষেত্রে জোন-পার ডেটা স্থানান্তরের বিলিং প্রযোজ্য হতে পারে — মূল্য যাচাই করে নিন। তাই নকশার নিয়ম: গুরুত্বপূর্ণ সেবা অন্তত দুটি জোনে, ডেটাবেসের রেপ্লিকা ভিন্ন জোনে, এবং একটিমাত্র জোনের ওপর নির্ভরশীল কোনো একক ব্যর্থতার বিন্দু (Single Point of Failure) রাখা যাবে না।

রিজিয়ন নির্বাচনও কৌশলগত সিদ্ধান্ত। ব্যবহারকারীর কাছাকাছি রিজিয়ন মানে কম বিলম্ব; ডেটা-সার্বভৌমত্ব (data residency) নিয়ম মানে কিছু ডেটা নির্দিষ্ট দেশের বাইরে যাওয়া যাবে না; দুর্যোগ-পুনরুদ্ধার (Disaster Recovery) মানে একটি রিজিয়ন পুরোপুরি বন্ধ হলেও অন্য রিজিয়নে সেবা চালু রাখা। TechNova-র গ্রাহক বাংলাদেশ ও দক্ষিণ এশিয়ায় থাকলে কাছাকাছি রিজিয়ন বেছে নেওয়া যৌক্তিক, তবে আইনি প্রয়োজন যাচাই না করে শুধু বিলম্ব দেখে সিদ্ধান্ত নেওয়া ভুল।

ধারণা	পরিসর	ব্যর্থ হলে কী হয়	নকশার নিয়ম
রিজিয়ন	ভৌগোলিক এলাকা	পুরো রিজিয়নের সেবা ব্যাহত হতে পারে	DR-এর জন্য দ্বিতীয় রিজিয়ন পরিকল্পনা করুন
জোন	রিজিয়নের ভেতরে ডেটা সেন্টার	ওই জোনের রিসোর্স থামে	গুরুত্বপূর্ণ সেবা ≥ ২ জোনে ছড়ান
সাবনেট	জোনের ভেতরে IP পরিসর	ওই সাবনেটের সার্ভার বিচ্ছিন্ন হয়	প্রতি জোনে প্রতি স্তরের জন্য সাবনেট রাখুন
VPC	একটি রিজিয়নে	VPC-স্তরের ভুল কনফিগারেশন সব সাবনেটকে প্রভাবিত করে	পরিবেশ (dev/prod) আলাদা VPC-তে ভাগ করুন

৬.৫ সাবনেট ও CIDR পরিকল্পনা

সাবনেট হলো VPC-র ঠিকানা-স্থানের একটি ভাগ, যা একটি নির্দিষ্ট জোনের সঙ্গে বাঁধা থাকে। প্রতিটি সাবনেটের নিজস্ব CIDR ব্লক থাকে, যেমন 10.10.1.0/24। সাবনেটের ভেতরে তৈরি ভার্চুয়াল সার্ভার সেই ব্লকের ভেতর থেকে প্রাইভেট IP পায়। সাবনেট দুই ধরনের ভূমিকা পালন করে: এটি ঠিকানা বরাদ্দের একক, আবার নিরাপত্তা ও রাউটিং নীতির প্রয়োগ-সীমানাও। ওয়েব, অ্যাপ ও ডেটা — এই তিন স্তরের জন্য আলাদা সাবনেট রাখলে আপনি বলতে পারেন ডেটা-সাবনেট থেকে ইন্টারনেটে যাওয়ার কোনো রুটই নেই।

CIDR হিসাবের মূল সূত্র: /N প্রিফিক্স মানে 32 − N বিট হোস্টের জন্য। মোট ঠিকানা = 2^(32−N)। তবে IBM Cloud VPC-তে প্রতি সাবনেটে কিছু ঠিকানা সংরক্ষিত থাকে (নেটওয়ার্ক, গেটওয়ে, DNS ইত্যাদির জন্য) — ব্যবহারযোগ্য সংখ্যা মোটের চেয়ে কম হয়। তাই পরিকল্পনায় কখনো ঠিক কাটায় কাটায় হিসাব করবেন না; অন্তত দ্বিগুণ জায়গা রাখুন।

৬.৬ কাজের উদাহরণ: CIDR ও সাবনেট হিসাব

TechNova-র জন্য ধরা যাক VPC ঠিকানা-স্থান 10.10.0.0/16। এর মোট ঠিকানা 2^16 = 65,536। আমরা প্রতি জোনে তিনটি স্তর চাই, মোট তিনটি জোন: ৯টি সাবনেট। প্রতিটি সাবনেট /24 (২৫৬ ঠিকানা) হলে মোট ব্যবহৃত হয় ৯ × ২৫৬ = ২,৩০৪ ঠিকানা — পুরো /16-এর মাত্র ৩.৫%। বাকি জায়গা ভবিষ্যৎ সম্প্রসারণ, নতুন পরিবেশ বা নতুন স্তরের জন্য থাকে। এটিই ভালো পরিকল্পনা: ছোট দিয়ে শুরু, বড় জায়গা হাতে রাখা।

```
VPC:              10.10.0.0/16   (65,536 ঠিকানা)
জোন-১ ওয়েব:     10.10.1.0/24   (256 ঠিকানা)
জোন-১ অ্যাপ:     10.10.11.0/24
জোন-১ ডেটা:      10.10.21.0/24
জোন-২ ওয়েব:     10.10.2.0/24
জোন-২ অ্যাপ:     10.10.12.0/24
জোন-২ ডেটা:      10.10.22.0/24
জোন-৩ ওয়েব:     10.10.3.0/24
জোন-৩ অ্যাপ:     10.10.13.0/24
জোন-৩ ডেটা:      10.10.23.0/24
সংরক্ষিত:        10.10.100.0/24 (ভবিষ্যৎ / ব্যবস্থাপনা)
```

/24 ব্লকের ভেতরে ব্যবহারযোগ্য হোস্ট গোনার উদাহরণ: 10.10.1.0/24-এ প্রথম ঠিকানা 10.10.1.0 নেটওয়ার্ক ঠিকানা, শেষ ঠিকানা 10.10.1.255 ব্রডকাস্ট ঠিকানা। এ ছাড়া VPC কিছু ঠিকানা সংরক্ষণ করে, তাই ব্যবহারিকভাবে প্রায় ২৫০টির কাছাকাছি সার্ভার বসানো যায় — একটি ওয়েব সাবনেটের জন্য যথেষ্ট, কিন্তু একটি /28 (১৬ ঠিকানা) ব্লক নিলে মাত্র কয়েকটি সার্ভারের পরেই জায়গা শেষ হয়ে যেত। আরেকটি হিসাব: /20 ব্লক মানে ৪,০৯৬ ঠিকানা; /22 মানে ১,০২৪; /26 মানে ৬৪। নিয়মটি মনে রাখুন: প্রিফিক্স সংখ্যা ১ কমলে ঠিকানা দ্বিগুণ হয়, ১ বাড়লে অর্ধেক হয়।

> **WARNING:** VPC তৈরি করার পর তার প্রধান CIDR পরিসর পরিবর্তন করা সাধারণত কঠিন বা অসম্ভব। ছোট CIDR (/24 VPC) দিয়ে শুরু করলে কয়েক মাস পর ঠিকানা শেষ হয়ে যাবে, আর তখন পুরো নেটওয়ার্ক নতুন করে বানাতে হতে পারে। শুরুতে /16 বা অন্তত /20 নেওয়া এবং ভেতরে ছোট ছোট সাবনেট কাটা নিরাপদ অভ্যাস।

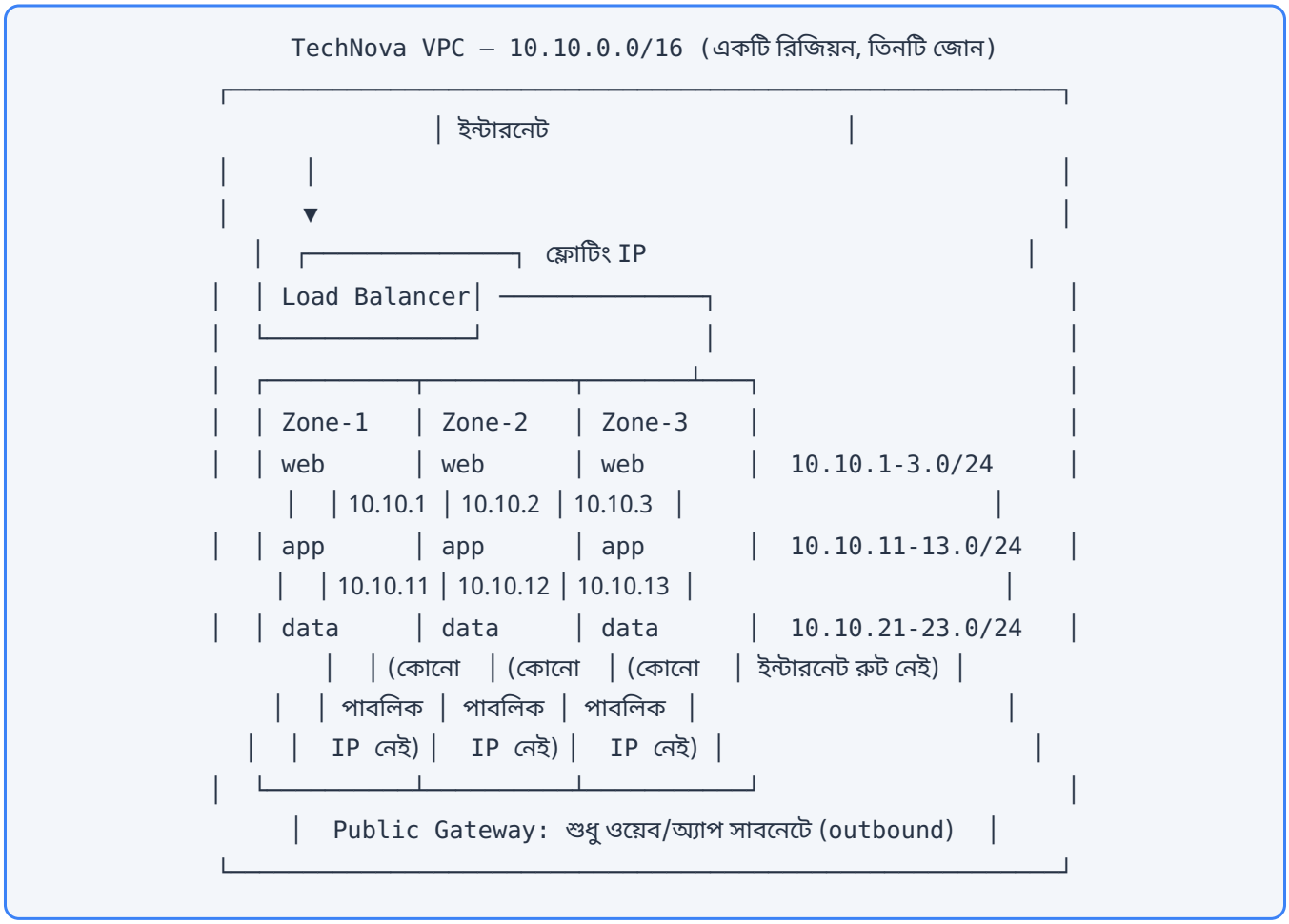
ঠিকানা-স্থান বেছে নেওয়ার সময় RFC 1918 প্রাইভেট পরিসর ব্যবহার করুন — 10.0.0.0/8, 172.16.0.0/12 বা 192.168.0.0/16। অন-প্রিমিসিস নেটওয়ার্ক বা অন্য ক্লাউডের VPC-র সঙ্গে পরে সংযোগ (VPN/Direct Link) করতে হলে ঠিকানা-পরিসর যেন না মেলে (overlap) — মিলে গেলে রাউটিং অস্পষ্ট হয়ে যায়। TechNova-র অফিস নেটওয়ার্ক যদি 192.168.1.0/24 ব্যবহার করে, তাহলে VPC-তে 10.10.0.0/16 নেওয়া সংঘাত এড়ায়। এই পরিকল্পনা নথিভুক্ত করুন: কোন ব্লক কোন উদ্দেশ্যে, কে অনুমোদন দিয়েছে, কবে পর্যালোচনা হবে।

৬.৭ পাবলিক ও প্রাইভেট IP ঠিকানা

VPC-র ভেতরে প্রতিটি ভার্চুয়াল সার্ভার একটি **প্রাইভেট IP** পায় (যেমন 10.10.1.5) — এটি শুধু VPC-র ভেতরে (ও সংযুক্ত নেটওয়ার্কে) পৌঁছানো যায়। ইন্টারনেট থেকে সার্ভারকে পৌঁছাতে হলে প্রয়োজন **পাবলিক IP** বা **ফ্লোটিং IP** (Floating IP) — একটি পাবলিক ঠিকানা যা কোনো নির্দিষ্ট সার্ভারের সঙ্গে বাঁধা যায় এবং প্রয়োজনে অন্য সার্ভারে সরিয়ে নেওয়া যায়। পার্থক্যটি গুরুত্বপূর্ণ: প্রাইভেট IP সার্ভারের পরিচয়, ফ্লোটিং IP একটি চলমান ঠিকানা যা সার্ভার বদলালেও গ্রাহকের কাছে একই থাকে।

নীতিটি সরল: পাবলিক IP শুধু সেখানেই দিন যেখানে ইন্টারনেট থেকে সরাসরি পৌঁছানো আবশ্যক — সাধারণত লোড ব্যালান্সার বা একটি বাস্টিয়ন (Bastion) হোস্ট। অ্যাপ সার্ভার ও ডেটাবেসে কখনো পাবলিক IP দেবেন না। বাইরে যাওয়ার (outbound) ইন্টারনেট প্রয়োজন — যেমন প্যাকেজ আপডেট — প্রাইভেট সাবনেটের জন্য পাবলিক গেটওয়ে (Public Gateway) দিয়ে মেটান, যা সপ্তম অধ্যায়ে বিস্তারিত আছে। পাবলিক IP-র বিলিং সম্পর্কে মনে রাখুন: বরাদ্দ করা কিন্তু ব্যবহৃত না হওয়া পাবলিক IP-ও খরচের কারণ হতে পারে; IBM Cloud কনসোলে মূল্য যাচাই করে অব্যবহৃত ঠিকানা ছেড়ে দিন।

৬.৮ VPC নকশা ও নেটওয়ার্ক আর্কিটেকচার



এই নকশায় ট্রাফিকের প্রবাহ একমুখী ও স্তরবদ্ধ: ইন্টারনেট → লোড ব্যালান্সার → ওয়েব সাবনেট → অ্যাপ সাবনেট → ডেটা সাবনেট। কোনো স্তর তার নিচের স্তর ছাড়া অন্য কারো সঙ্গে কথা বলে না। ডেটা সাবনেট থেকে ইন্টারনেটে যাওয়ার রুট নেই, তাই ডেটাবেস থেকে তথ্য বাইরে পাচার করা কঠিন। ব্যবস্থাপনার (management) জন্য আলাদা একটি ছোট সাবনেট বা বাস্টিয়ন হোস্ট রাখা যায়, যেখানে শুধু নির্দিষ্ট অফিস IP থেকে SSH অনুমোদিত।

VPC কখন ভাগ করবেন? পরিবেশ আলাদা করুন: production, staging ও development — আলাদা VPC হলে একটি ভুল কনফিগারেশন অন্যটিকে স্পর্শ করে না। দল বা ব্যবসায়িক ইউনিট আলাদা হলে, বা কঠোর নিয়ন্ত্রণ-সীমা (compliance boundary) দরকার হলে, আলাদা VPC যৌক্তিক। কিন্তু অতিরিক্ত ভাগ করলে VPC-পার সংযোগ (Transit Gateway বা VPN) পরিচালনার জটিলতা ও খরচ বাড়ে। TechNova-র মতো ছোট দলের জন্য শুরুতে দুটি VPC (prod ও non-prod) যথেষ্ট; প্রয়োজন বাড়লে ভাগ করা যায়।

ভেতরে কীভাবে কাজ করে? VPC-র নিয়ন্ত্রণ-স্তর (control plane) আপনার API অনুরোধ গ্রহণ করে, যাচাই করে এবং ডেটা-স্তরে (data plane) — অর্থাৎ ভার্চুয়াল সুইচ ও রাউটারে — নিয়ম পৌঁছে দেয়। একটি সার্ভার প্যাকেট পাঠালে ভার্চুয়াল সুইচ প্রথমে দেখে গন্তব্য একই সাবনেটে কি না; না হলে রাউটিং টেবিল দেখে; তারপর সিকিউরিটি গ্রুপ ও NACL নিয়ম প্রয়োগ করে। এই প্রক্রিয়া প্রতিটি প্যাকেটে ঘটে, তবে হার্ডওয়্যার-ত্বরিত (accelerated) পথে বলে সাধারণত বিলম্ব নগণ্য। ব্যর্থ হলে কী হয়? নিয়ন্ত্রণ-স্তরের সাময়িক সমস্যায় নতুন রিসোর্স তৈরি থামতে পারে, কিন্তু চলমান প্যাকেট-প্রবাহ সাধারণত চলতে থাকে — কারণ ডেটা-স্তর আলাদাভাবে চলে। তবু নকশায় মনে রাখুন: VPC-স্তরের একটি ভুল রুট বা ACL সব সাবনেটকে একসঙ্গে প্রভাবিত করতে পারে, তাই পরিবর্তন ধাপে ধাপে ও পরীক্ষা করে করুন।

৬.৯ CLI ও API উদাহরণ

নিচের `ibmcloud` CLI উদাহরণগুলো ধারণাগতভাবে সঠিক; বাস্তব ব্যবহারের আগে `ibmcloud is --help` দিয়ে ফ্ল্যাগ যাচাই করুন। প্রথমে টার্গেট রিজিয়ন নির্ধারণ করুন, তারপর VPC, তারপর সাবনেট তৈরি করুন — ক্রমটি গুরুত্বপূর্ণ, কারণ সাবনেট VPC ছাড়া তৈরি হয় না।

```
$ ibmcloud target -r us-south
$ ibmcloud is vpc-create technova-prod-vpc \
    --address-prefix-management auto
$ ibmcloud is subnet-create technova-web-zone1 \
    --vpc technova-prod-vpc \
    --zone us-south-1 \
    --ipv4-cidr-block 10.10.1.0/24
$ ibmcloud is subnet-create technova-app-zone1 \
    --vpc technova-prod-vpc \
    --zone us-south-1 \
    --ipv4-cidr-block 10.10.11.0/24
$ ibmcloud is vpcs
$ ibmcloud is subnets
```

API ব্যবহার করলে একই কাজ HTTP অনুরোধে হয়: VPC তৈরি একটি POST, সাবনেট তৈরি আরেকটি POST, যেখানে VPC-র ID প্যারামিটার হিসেবে যায়। API-র সুবিধা হলো অটোমেশন ও অডিট: প্রতিটি অনুরোধে কে, কখন, কী করল তার রেকর্ড থাকে। CLI দ্রুত পরীক্ষার জন্য ভালো; বারবার করা কাজের জন্য API বা Terraform ভালো।

৬.১০ IaC উদাহরণ (Terraform)

```
resource "ibm_is_vpc" "technova_prod" {
  name = "technova-prod-vpc"
}

resource "ibm_is_subnet" "web_zone1" {
  name            = "technova-web-zone1"
  vpc             = ibm_is_vpc.technova_prod.id
  zone            = "us-south-1"
  ipv4_cidr_block = "10.10.1.0/24"
}
```

```
resource "ibm_is_subnet" "data_zone1" {
  name            = "technova-data-zone1"
  vpc             = ibm_is_vpc.technova_prod.id
  zone            = "us-south-1"
  ipv4_cidr_block = "10.10.21.0/24"
}
```

Terraform-এর সুবিধা: নেটওয়ার্ক কোড হিসেবে সংস্করণ-নিয়ন্ত্রণে (Git) থাকে, পর্যালোচনা (review) করা যায়, এবং একই কোড staging ও production-এ শুধু ভেরিয়েবল বদলে চালানো যায়। সতর্কতা: Terraform state ফাইলে সংবেদনশীল তথ্য থাকতে পারে; state দূরবর্তী ব্যাকএন্ডে এনক্রিপ্ট করে রাখুন এবং কখনো Git-এ প্লেইন state কমিট করবেন না।

৬.১১ উৎপাদন পরিস্থিতি: TechNova-র প্রথম VPC

TechNova-র দল প্রথমে একটি মাত্র জোনে সবকিছু বসিয়েছিল — দ্রুত চালু হয়েছিল, কিন্তু একদিন জোন-রক্ষণাবেক্ষণের সময় পুরো সাইট বন্ধ হয়ে যায়। তারা VPC পুনর্নকশা করে: তিন জোনে ওয়েব ও অ্যাপ সাবনেট, ডেটাবেসের প্রাথমিক (primary) এক জোনে ও রেপ্লিকা আরেক জোনে, লোড ব্যালান্সার তিন জোন জুড়ে। পরের রক্ষণাবেক্ষণে একটি জোন বন্ধ থাকলেও সাইট চলেছে — গ্রাহক কিছু টের পাননি। শিক্ষা: উচ্চ-প্রাপ্যতা কোনো পরিষেবা কিনে পাওয়া যায় না; এটি নকশার ফল।

> **REAL WORLD:** একটি স্টার্টআপ /16 VPC নিয়েছিল, কিন্তু প্রতিটি সাবনেট /24 না নিয়ে /28 নিয়েছিল "ঠিকানা বাঁচাতে"। ছয় মাস পর অটোস্কেলিংয়ে নতুন সার্ভারের IP শেষ হয়ে যায় — ব্যস্ত সময়ে স্কেল-আউট ব্যর্থ হয়। তারা সাবনেট বড় করতে পারেনি; নতুন সাবনেট যোগ করে লোড ব্যালান্সার পুনর্গঠন করতে হয়েছে। ঠিকানা "বাঁচানো" প্রাইভেট পরিসরে অর্থহীন — 10.0.0.0/8-এ ১ কোটি ৬০ লাখের বেশি ঠিকানা আছে।

৬.১২ নিরাপত্তা ও ব্যয় বিবেচনা

নিরাপত্তা: ন্যূনতম সুযোগের নীতি (least privilege) — ডেটা সাবনেটে শুধু অ্যাপ সাবনেট থেকে সংযোগ; SSH শুধু বাস্টিয়ন দিয়ে ও নির্দিষ্ট উৎস IP থেকে; সব পরিবর্তন IAM ভূমিকা দিয়ে সীমিত করুন। VPC ফ্লো লগ (Flow Logs) চালু রাখলে কোন সার্ভার কোথায় সংযোগ করছে তার রেকর্ড থাকে — ঘটনা-তদন্তে অমূল্য। ব্যয়: VPC নিজে সাধারণত আলাদা চার্জের বিষয় নয়, খরচ আসে এর ভেতরের রিসোর্স থেকে — ভার্চুয়াল সার্ভার, লোড ব্যালান্সার, পাবলিক গেটওয়ে, ফ্লোটিং IP ও জোন-পার ডেটা স্থানান্তর। মূল্য যাচাই করুন IBM Cloud ক্যাটালগ ও মূল্য-ক্যালকুলেটরে; অনুমানের ওপর বাজেট করবেন না। অপ্রয়োজনীয় পাবলিক IP ও গেটওয়ে মুছে ফেলুন; non-prod পরিবেশ রাতে বন্ধ রাখার নীতি বিবেচনা করুন।

পর্যবেক্ষণ: VPC-স্তরে কী দেখবেন? সাবনেটে IP ব্যবহারের হার (৮০% ছাড়ালে সতর্কতা), ফ্লো লগে অস্বাভাবিক সংযোগ, রুট পরিবর্তনের অডিট লগ। বিলিং দৃষ্টিকোণ: VPC-র খরচ সরাসরি নয়, পরোক্ষ — ভুল নকশা (যেমন সব ট্রাফিক এক জোন থেকে অন্য জোনে ঘোরানো) ডেটা-স্থানান্তর খরচ বাড়ায়। তাই নকশার সময় ট্রাফিক-প্রবাহের চিত্র আঁকুন এবং জিজ্ঞাসা করুন: এই প্যাকেট কেন এই পথে যাচ্ছে?

৬.১৩ সাধারণ ভুল

- একটি মাত্র জোনে সবকিছু বসানো — জোন-বিভ্রাট মানেই সম্পূর্ণ বিভ্রাট।
- খুব ছোট VPC CIDR নেওয়া, পরে ঠিকানা শেষ হয়ে যাওয়া।
- অফিস বা অন্য VPC-র সঙ্গে মিলে যাওয়া (overlapping) CIDR নেওয়া।

- ডেটাবেস সাবনেটে পাবলিক IP বা ইন্টারনেট রুট রাখা।
- সব সার্ভারে পাবলিক IP দিয়ে "সহজে SSH" করা — আক্রমণের পথ খুলে দেওয়া।
- নকশা নথিভুক্ত না করা — ছয় মাস পর কেউ জানে না কেন 10.10.21.0/24 ডেটার জন্য।
- পরিবেশ (dev/prod) একই VPC-তে মেশানো — পরীক্ষার ভুল production-কে স্পর্শ করে।

৬.১৪ সমস্যা সমাধান

সার্ভার ইন্টারনেটে পৌঁছাতে পারছে না? ধাপে ধাপে যাচাই করুন: (১) সার্ভারের প্রাইভেট IP সাবনেটের CIDR-এর ভেতরে কি না; (২) সাবনেটে পাবলিক গেটওয়ে যুক্ত কি না (প্রাইভেট সাবনেটের outbound-এর জন্য); (৩) রাউটিং টেবিলে 0.0.0.0/0 রুট গেটওয়ের দিকে কি না; (৪) সিকিউরিটি গ্রুপ outbound নিয়ম অনুমোদন করে কি না। দুটি সার্ভার পরস্পরকে ping করতে পারছে না? প্রথমে দেখুন তারা একই VPC-তে কি না, তারপর সিকিউরিটি গ্রুপের inbound নিয়ম, তারপর NACL। মনে রাখুন, ping (ICMP) অনেক নিরাপত্তা-নিয়মে ডিফল্টভাবে বন্ধ থাকে — ping ব্যর্থ মানেই অ্যাপ্লিকেশন-পোর্ট বন্ধ নয়; নির্দিষ্ট পোর্ট দিয়ে পরীক্ষা করুন।

> **TROUBLESHOOTING:** নতুন সার্ভার তৈরি হচ্ছে না — "ঠিকানা নেই" ত্রুটি? সাবনেটের IP শেষ হয়ে গেছে কি না দেখুন। সমাধান: নতুন, বড় সাবনেট তৈরি করে সার্ভার সেখানে বসান; বিদ্যমান সাবনেটের CIDR সাধারণত বাড়ানো যায় না। ভবিষ্যতের জন্য IP-ব্যবহারের সতর্কতা (alert) আগেই বসান।

৬.১৫ গভীরে: ব্যর্থতা, পর্যবেক্ষণ, বিলিং ও বিকল্প

VPC-র প্রতিটি প্রধান উপাদানকে বারোটি প্রশ্নে যাচাই করুন — এই প্রশ্নগুলোই এই খণ্ডের সব অধ্যায়ে প্রতিটি পরিষেবার ক্ষেত্রে প্রযোজ্য। প্রথমে, সমস্যা: VPC ছাড়া আপনার সার্ভারগুলো একটি ভাগ করা, অনিয়ন্ত্রিত নেটওয়ার্কে থাকত — প্রতিবেশীর ভুল আপনার সিস্টেমে পৌঁছাতে পারত। দ্বিতীয়ত, প্রয়োজনীয়তা: আইনি ও নিরাপত্তা-নিয়ম প্রায়ই দাবি করে সংবেদনশীল ডেটা বিচ্ছিন্ন নেটওয়ার্কে থাকবে; VPC সেই দাবি পূরণের প্রমাণ দেয়। তৃতীয়ত, ভেতরের কাজ: নিয়ন্ত্রণ-স্তর API অনুরোধ যাচাই করে ডেটা-স্তরের ভার্চুয়াল সুইচে নিয়ম পৌঁছে দেয়; একটি প্যাকেট প্রথমে সাবনেট-পরিচয়, তারপর রুট, তারপর নিরাপত্তা-নিয়ম পার হয়। চতুর্থত, অন্যান্য পরিষেবার সঙ্গে যোগাযোগ: VPC লোড ব্যালান্সার, ভার্চুয়াল সার্ভার, ব্লক স্টোরেজ, DNS ও VPN/Direct Link — সবার ভিত্তি; VPC-র ঠিকানা-পরিসর ভুল হলে সব পরিষেবার সংযোগ ভুল হয়। পঞ্চমত, ব্যর্থতা: নিয়ন্ত্রণ-স্তরের সাময়িক সমস্যায় নতুন রিসোর্স তৈরি থামতে পারে, ডেটা-স্তর সাধারণত চলতে থাকে; কিন্তু VPC-স্তরের ভুল রুট বা CIDR-সংঘাত পুরো পরিবেশকে একসঙ্গে অচল করতে পারে। ষষ্ঠত, নিরাপত্তা: IAM দিয়ে কে VPC বদলাতে পারবে তা সীমিত করুন, ফ্লো লগ চালু রাখুন, পাবলিক IP-র তালিকা নিয়মিত পর্যালোচনা করুন। সপ্তমত, পর্যবেক্ষণ: IP-ব্যবহারের হার, সাবনেট-প্রতি সংযোগ-প্রবাহ, রুট-পরিবর্তনের অডিট লগ — তিনটি ড্যাশবোর্ডে রাখুন। অষ্টমত, বিলিং: VPC নিজে সাধারণত আলাদা চার্জ নয়, কিন্তু এর ভেতরের প্রতিটি রিসোর্স — সার্ভার, গেটওয়ে, লোড ব্যালান্সার, ডেটা স্থানান্তর — খরচের কারণ; মূল্য ক্যাটালগ ও ক্যালকুলেটরে যাচাই করুন, অনুমানে নয়। নবমত, কখন ব্যবহার করবেন: যেকোনো উৎপাদন-কাজে, যেখানে বিচ্ছিন্নতা ও নিয়ন্ত্রণ দরকার। দশমত, কখন এড়াবেন: একটি মাত্র পরীক্ষামূলক সার্ভারের জন্য পূর্ণ VPC-নকশা অপ্রয়োজনীয় — সেখানে ছোট, অস্থায়ী VPC যথেষ্ট। একাদশত, বিকল্প: ক্লাসিক ইনফ্রাস্ট্রাকচার (VPC-ছাড়া পুরোনো মডেল), অন্য ক্লাউডের VPC, বা অন-প্রিমিসিস নেটওয়ার্ক — প্রতিটির নিজস্ব সীমা আছে; IBM Cloud-এ নতুন কাজে VPC-ই মান। দ্বাদশত, আপস: বড় CIDR মানে ভবিষ্যৎ-নিরাপত্তা, কিন্তু অন্য নেটওয়ার্কের সঙ্গে মেলানোর সময় সংঘাতের ঝুঁকি; অনেক সাবনেট মানে সূক্ষ্ম নিয়ন্ত্রণ, কিন্তু পরিচালনার জটিলতা।

TechNova-র ক্ষেত্রে একটি বাস্তব সিদ্ধান্ত-উদাহরণ: দল প্রথমে একটি VPC-তে dev ও prod একসঙ্গে রেখেছিল — খরচ কমেছিল, কিন্তু একদিন dev-এর একটি পরীক্ষামূলক স্ক্রিপ্ট ভুল করে prod-ডেটাবেসের সাবনেটে ট্রাফিক পাঠায়। কোনো ডেটা হারায়নি, কিন্তু ঘটনাটি দেখিয়ে দেয় পরিবেশ-মেশানো কত বিপজ্জনক। তারা দুটি VPC-তে ভাগ করে — খরচ সামান্য

বাড়ে, কিন্তু ঝুঁকি অনেক কমে। এই আপসের হিসাব লিখিতভাবে রাখুন: কী পেলেন, কী দিলেন, কেন সিদ্ধান্ত নিলেন। ছয় মাস পর যখন কেউ জিজ্ঞাসা করবে "দুটি VPC কেন?", উত্তরটি নথিতে থাকবে।

৬.১৬ সারসংক্ষেপ

VPC হলো আপনার ক্লাউড নেটওয়ার্কের ভিত্তি: একটি রিজিয়নে একটি বিচ্ছিন্ন ঠিকানা-স্থান, জোনে ছড়ানো সাবনেট, স্তরভিত্তিক বিভাজন এবং পাবলিক-প্রাইভেট IP-র সুশৃঙ্খল ব্যবহার। ভালো VPC নকশা মানে বড় CIDR দিয়ে শুরু করা, প্রতি জোনে প্রতি স্তরের সাবনেট রাখা, ডেটাবেসকে ইন্টারনেট থেকে সম্পূর্ণ আড়ালে রাখা এবং সবকিছু কোড হিসেবে নথিভুক্ত করা। এই ভিত্তি ঠিক থাকলে পরের অধ্যায়ের রাউটিং ও নিরাপত্তা-নিয়ম সহজে বসে; ভিত্তি ভুল হলে প্রতিটি পরের সিদ্ধান্তই জোড়াতালি হয়ে যায়।

৬.১৭ পর্যালোচনা প্রশ্ন

1. VPC কী সমস্যা সমাধান করে এবং এটি অন-প্রিমিসিস নেটওয়ার্ক থেকে কীভাবে আলাদা?
2. রিজিয়ন ও জোনের পার্থক্য কী, এবং জোনকে ব্যর্থতার ডোমেইন বলা হয় কেন?
3. 10.10.0.0/16 থেকে চারটি /24 সাবনেট কাটলে মোট কত ঠিকানা ব্যবহৃত হয় ও কত অবশিষ্ট থাকে?
4. ডেটাবেস সার্ভারে পাবলিক IP দেওয়া কেন নিরাপত্তা-ঝুঁকি?
5. ফ্লোটিং IP ও প্রাইভেট IP-র পার্থক্য কী, এবং ফ্লোটিং IP কখন উপযোগী?

৬.১৮ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য 10.20.0.0/16 VPC-তে তিন জোন × তিন স্তরের সাবনেট-তালিকা কাগজে তৈরি করুন; প্রতিটি সাবনেটের CIDR, জোন, উদ্দেশ্য ও ইন্টারনেট-অ্যাক্সেস (হ্যাঁ/না) লিখুন। তারপর `ibmcloud is vpc-create` ও `subnet-create` দিয়ে পরীক্ষামূলক VPC-তে বাস্তবায়ন করুন ও মুছে ফেলুন।
2. একটি overlapping CIDR পরিস্থিতি বিশ্লেষণ করুন: অফিস 192.168.0.0/16 ব্যবহার করলে VPC-তে কোন পরিসর নিরাপদ, কোনটি সংঘাত তৈরি করবে — যুক্তিসহ লিখুন এবং Terraform-এ VPC সংজ্ঞা লিখে `terraform plan` দিয়ে যাচাই করুন।

অধ্যায় ৭ • PART II — Networking & Infrastructure

Chapter 7 — VPC Routing and Security

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক VPC-র ভেতরে প্যাকেট কীভাবে রুট হয় তা ব্যাখ্যা করতে পারবেন; রাউটিং টেবিল, সিকিউরিটি গ্রুপ (Security Group) ও নেটওয়ার্ক ACL (Network ACL) এর পার্থক্য ও একসঙ্গে ব্যবহারের নিয়ম আয়ত্ত করবেন; পাবলিক গেটওয়ে (Public Gateway) ও NAT-এর মাধ্যমে প্রাইভেট সাবনেটের নিরাপদ outbound সংযোগ নকশা করতে পারবেন; প্রাইভেট সংযোগ (private connectivity) ও নেটওয়ার্ক বিভাজন (segmentation) দিয়ে স্তরভিত্তিক প্রতিরক্ষা গড়তে পারবেন; এবং সংযোগ-ব্যর্থতার কারণ ধাপে ধাপে খুঁজে বের করতে পারবেন।

- রাউট ও রাউটিং টেবিল পড়া ও লেখা; সবচেয়ে নির্দিষ্ট (longest-prefix) মিলের নিয়ম প্রয়োগ করা।
- Stateful সিকিউরিটি গ্রুপ বনাম Stateless NACL-এর আচরণের পার্থক্য ব্যাখ্যা করা।
- প্রাইভেট সাবনেটের জন্য পাবলিক গেটওয়ে সংযুক্ত করা ও যাচাই করা।
- TechNova-র তিন স্তরের জন্য সম্পূর্ণ নিয়ম-তালিকা (rule set) তৈরি করা।

৭.২ প্রারম্ভিক পুনরালোচনা

ষষ্ঠ অধ্যায়ে আমরা VPC, সাবনেট ও CIDR শিখেছি। এখন প্রশ্ন: একটি প্যাকেট 10.10.11.5 (অ্যাপ সার্ভার) থেকে 10.10.21.5 (ডেটাবেস) যেতে চাইলে কে সিদ্ধান্ত নেয় পথটি অনুমোদিত কি না? উত্তর দুই স্তরে: রাউটিং বলে কোথায় যাবে, নিরাপত্তা-নিয়ম বলে যেতে পারবে কি না। রাউটিং ছাড়া প্যাকেট গন্তব্য খুঁজে পায় না; নিরাপত্তা ছাড়া যেকোনো প্যাকেট যেকোনো জায়গায় যেতে পারে। দুটিকে গুলিয়ে ফেললে সমস্যা সমাধান কঠিন হয় — "সংযোগ হচ্ছে না" বললে প্রথমে বুঝতে হবে রুট নেই, নাকি নিয়ম আটকাচ্ছে।

ঝালিয়ে নিন: TCP সংযোগে প্রথমে SYN, তারপর SYN-ACK, তারপর ACK — অর্থাৎ উত্তর-প্যাকেট (return traffic) আসতেই হবে। Stateful ফায়ারওয়াল এই উত্তর-প্যাকেটকে চিনে অনুমোদন দেয়; stateless ফায়ারওয়াল চেনে না, তাই উভয় দিকের নিয়ম আলাদা করে লিখতে হয়। এই পার্থক্যই সিকিউরিটি গ্রুপ ও NACL-এর মূল।

৭.৩ প্রথম নীতি থেকে রাউটিং: প্যাকেটের প্রশ্ন

প্রতিটি প্যাকেটের একটি প্রশ্ন থাকে: "আমার গন্তব্য কোথায়, আর সেখানে যেতে কোন দরজা দিয়ে বেরোতে হবে?" রাউটিং টেবিল সেই প্রশ্নের উত্তর-তালিকা: গন্তব্য-পরিসর (destination CIDR) → পরবর্তী হপ (next hop)। যেমন 10.10.0.0/16 → local (VPC-র ভেতরেই), 0.0.0.0/0 → পাবলিক গেটওয়ে। গন্তব্য একাধিক নিয়মের সঙ্গে মিললে সবচেয়ে নির্দিষ্ট নিয়মটি (দীর্ঘতম প্রিফিক্স) জেতে — 10.10.21.0/24-এর নিয়ম 10.10.0.0/16-এর নিয়মকে হারায়। একে longest-prefix match বলে।

IBM Cloud VPC-তে প্রতিটি সাবনেটের একটি রাউটিং টেবিল থাকে। ডিফল্টভাবে VPC-র ভেতরের সব সাবনেট পরস্পরকে দেখতে পায় (local রুট)। ইন্টারনেটে যেতে হলে স্পষ্ট রুট দরকার। এখানেই নকশার ক্ষমতা: ডেটা সাবনেটের টেবিলে 0.0.0.0/0 রুট না রাখলে ডেটাবেস থেকে ইন্টারনেটে যাওয়ার পথই থাকে না — কোনো ফায়ারওয়াল-নিয়মের ওপর নির্ভর করতে হয় না। রাউটিং তাই নিরাপত্তার প্রথম স্তর, ফায়ারওয়াল দ্বিতীয়।

> **CONCEPT:** রাউট বলে প্যাকেট কোথায় যাবে; নিরাপত্তা-নিয়ম বলে প্যাকেট যেতে পারবে কি না। রুট থাকা মানেই অনুমোদন নয়, আর অনুমোদন থাকা মানেই রুট আছে নয় — সংযোগের জন্য দুটোই দরকার। সমস্যা সমাধানের সময় এই দুটিকে আলাদা করে পরীক্ষা করুন।

৭.৪ সিকিউরিটি গ্রুপ: Stateful প্রতিরক্ষা

সিকিউরিটি গ্রুপ হলো ভার্চুয়াল সার্ভারের নেটওয়ার্ক ইন্টারফেসে যুক্ত একটি ভার্চুয়াল ফায়ারওয়াল। এটি *stateful*: আপনি যদি outbound সংযোগ অনুমোদন করেন, উত্তর-ট্রাফিক স্বয়ংক্রিয়ভাবে ফিরতে পারে — আলাদা inbound নিয়ম লিখতে হয় না। নিয়মগুলো অনুমোদনমূলক (allow-only); যা লেখা নেই তা ডিফল্টভাবে বন্ধ। নিয়ম লেখা যায় CIDR দিয়ে (যেমন শুধু 10.10.1.0/24 থেকে পোর্ট 443) বা অন্য সিকিউরিটি গ্রুপের উল্লেখ দিয়ে (যেমন "web-sg গ্রুপের সার্ভার থেকে পোর্ট 8080") — দ্বিতীয় পদ্ধতিটি IP বদলালেও কাজ করে, তাই বেশি টেকসই।

TechNova-র নিয়ম-তালিকার উদাহরণ: ওয়েব সার্ভারের সিকিউরিটি গ্রুপে inbound 80/443 ইন্টারনেট থেকে (0.0.0.0/0) — তবে বাস্তবে লোড ব্যালান্সারের পেছনে থাকলে শুধু লোড ব্যালান্সারের গ্রুপ থেকে; অ্যাপ গ্রুপে inbound 8080 শুধু ওয়েব গ্রুপ থেকে; ডেটা গ্রুপে inbound 5432 (PostgreSQL) শুধু অ্যাপ গ্রুপ থেকে। SSH (22) কোনো অ্যাপ-গ্রুপে নয় — শুধু বাস্টিয়ন গ্রুপ থেকে, এবং বাস্টিয়নে শুধু অফিসের IP থেকে। এইভাবে প্রতিটি নিয়মে উৎস যত সংকীর্ণ, আক্রমণের পথ তত কম।

সার্ভার	ইনবাউন্ড অনুমোদন	উৎস	আউটবাউন্ড
ওয়েব	80, 443	লোড ব্যালান্সার গ্রুপ	অ্যাপ গ্রুপে 8080; DNS/প্যাকেজ আপডেট
অ্যাপ	8080	ওয়েব গ্রুপ	ডেটা গ্রুপে 5432
ডেটা	5432	অ্যাপ গ্রুপ	কোনো ইন্টারনেট নয়
বাস্টিয়ন	22	অফিস IP (203.0.113.0/28)	VPC-র ভেতরে 22

সিকিউরিটি গ্রুপের সমস্যা কী সমাধান করে? এটি প্রতিটি সার্ভারকে নিজস্ব ফায়ারওয়াল দেয়, তাই একটি সার্ভারের ভুল অন্য সার্ভারকে উন্মুক্ত করে না। অভ্যন্তরে এটি কীভাবে কাজ করে? নিয়ন্ত্রণ-স্তর নিয়মগুলোকে হোস্ট-স্তরের ভার্চুয়াল সুইচে পৌঁছে দেয়, যেখানে প্রতিটি সংযোগের অবস্থা (state table) মনে রাখা হয়। ব্যর্থ হলে কী হয়? ভুল নিয়মে বৈধ ট্রাফিক বন্ধ হয়ে সেবা থেমে যেতে পারে — কিন্তু সাধারণত পুরো VPC থামে না। নিরাপত্তা: নিয়মে 0.0.0.0/0 দিয়ে সংবেদনশীল পোর্ট (22, 5432, 6379) খোলা সবচেয়ে সাধারণ ভুল; নিয়মিত অডিট করুন। পর্যবেক্ষণ: ফ্লো লগে প্রত্যাখ্যাত সংযোগ দেখুন। বিলিং: সিকিউরিটি গ্রুপ সাধারণত আলাদা চার্জের বিষয় নয় — খরচ আসে এর পেছনের সার্ভার থেকে। কখন ব্যবহার করবেন: সবসময়, প্রতিটি সার্ভারে। কখন এড়াবেন: সিকিউরিটি গ্রুপকে সাবনেট-স্তরের নিয়ন্ত্রণের বিকল্প ভাববেন না — দুটোই দরকার। বিকল্প: NACL ও হোস্ট-ভিত্তিক ফায়ারওয়াল (যেমন OS-স্তরের firewall) — এরা পরিপূরক, প্রতিস্থাপক নয়।

৭.৫ নেটওয়ার্ক ACL: Stateless সাবনেট-স্তর

Network ACL (NACL) সাবনেট-স্তরে কাজ করে এবং *stateless*: উত্তর-ট্রাফিকের জন্য আলাদা নিয়ম লিখতে হয়। নিয়মগুলো সংখ্যাযুক্ত (numbered) ও ক্রমানুসারে মূল্যায়িত হয় — প্রথম মিলেই সিদ্ধান্ত; তাই ক্রম গুরুত্বপূর্ণ। NACL অনুমোদন (allow) ও প্রত্যাখ্যান (deny) — দুটোই লিখতে পারে, যা সিকিউরিটি গ্রুপে নেই। ব্যবহার: সাবনেট-জোড়া বিস্তৃত নীতি, যেমন "ডেটা সাবনেট থেকে কোনো outbound ইন্টারনেট নয়" — একটি deny নিয়ম দিয়ে স্পষ্ট করা, বা নির্দিষ্ট ক্ষতিকর IP-পরিসরকে পুরো সাবনেটে আটকানো।

সিকিউরিটি গ্রুপ বনাম NACL মনে রাখার সূত্র: সিকিউরিটি গ্রুপ = সার্ভারের দরজার তালা (stateful, শুধু allow); NACL = ভবনের গেটের প্রহরী (stateless, allow ও deny, ক্রম-নির্ভর)। দুটোই থাকলে প্যাকেটকে দুটোই পেরোতে হয় — defence in depth। সাধারণ ভুল: NACL-এ inbound allow লিখে outbound উত্তর-নিয়ম ভুলে যাওয়া — তখন সংযোগ শুরু হয় কিন্তু উত্তর ফেরে না, ফলে টাইমআউট হয়। আরেকটি ভুল: NACL নিয়মের নম্বর-ক্রম না বুঝে নিচে একটি deny রেখে ওপরে বিস্তৃত allow রাখা — deny কখনো কার্যকর হয় না।

প্যাকেটের যাত্রা: ইন্টারনেট → NACL (সাবনেট) → সিকিউরিটি গ্রুপ (সার্ভার)
→ অ্যাপ → সিকিউরিটি গ্রুপ → NACL → গন্তব্য
উত্তর-প্যাকেট: সিকিউরিটি গ্রুপ (stateful: স্বয়ংক্রিয়) → NACL (নিয়ম দরকার)
রুট নেই বা নিয়ম নেই — যেকোনো একটিতে আটকালেই সংযোগ ব্যর্থ।

৭.৬ পাবলিক গেটওয়ে ও NAT ধারণা

পাবলিক গেটওয়ে একটি সাবনেটকে outbound ইন্টারনেট-অ্যাক্সেস দেয়, কিন্তু inbound সংযোগের পথ খোলে না — অর্থাৎ প্রাইভেট সার্ভার প্যাকেজ ডাউনলোড বা বাইরের API ডাকতে পারে, কিন্তু ইন্টারনেট থেকে কেউ সরাসরি সার্ভারে ঢুকতে পারে না। এর পেছনে কাজ করে **NAT** (Network Address Translation): বাইরে যাওয়ার সময় সার্ভারের প্রাইভেট IP একটি পাবলিক IP-তে বদলে যায়, উত্তর ফিরলে আবার প্রাইভেট IP-তে ফেরত আসে। এটি outbound NAT (Source NAT); inbound NAT (যেমন ফ্লোটিং IP) আলাদা ধারণা।

সমস্যা ও সমাধান: প্রাইভেট সার্ভারের সফটওয়্যার আপডেট দরকার, কিন্তু পাবলিক IP দিলে আক্রমণের ঝুঁকি। পাবলিক গেটওয়ে এই দ্বন্দ্ব মেটায়। অভ্যন্তরে: গেটওয়ে একটি পরিচালিত (managed) উপাদান — এর উচ্চ-প্রাপ্যতা IBM Cloud পরিচালনা করে, তবে এটি জোন-নির্দিষ্ট: প্রতি জোনের সাবনেটের জন্য সেই জোনের গেটওয়ে দরকার। একটি জোনের গেটওয়ে বন্ধ হলে ওই জোনের outbound থামতে পারে — তাই প্রতি জোনে আলাদা গেটওয়ে রাখুন, একটি গেটওয়ে তিন জোনে ভাগ করবেন না। বিলিং: পাবলিক গেটওয়ে ও এর মধ্য দিয়ে যাওয়া ট্রাফিকের জন্য চার্জ প্রযোজ্য হতে পারে — মূল্য যাচাই করুন ও অব্যবহৃত গেটওয়ে মুছুন। বিকল্প: নিজের NAT সার্ভার চালানো — পরিচালনার বোঝা বাড়ে, সাধারণত পরিচালিত গেটওয়েই ভালো।

৭.৭ প্রাইভেট সংযোগ ও নেটওয়ার্ক বিভাজন

প্রাইভেট সংযোগ মানে IBM Cloud-এর পরিষেবা (যেমন অবজেক্ট স্টোরেজ, ডেটাবেস পরিষেবা) বা অন্য VPC-র সঙ্গে ইন্টারনেট না ঘুরে, IBM-এর ব্যক্তিগত নেটওয়ার্ক দিয়ে সংযোগ। এর সুবিধা: বিলম্ব কম, ট্রাফিক ইন্টারনেটে প্রকাশ পায় না, এবং কিছু ক্ষেত্রে ডেটা-স্থানান্তর খরচ কমে। TechNova ডেটাবেস যদি পরিচালিত পরিষেবা হয়, তাহলে অ্যাপ সার্ভার থেকে প্রাইভেট এন্ডপয়েন্ট দিয়ে সংযোগ করাই সঠিক — পাবলিক এন্ডপয়েন্ট ব্যবহার করা মানে সংবেদনশীল ডেটা ইন্টারনেট দিয়ে যাওয়া।

নেটওয়ার্ক বিভাজন (segmentation) মানে VPC-কে এমনভাবে ভাগ করা যাতে একটি অংশে অনুপ্রবেশ ঘটলেও আক্রমণকারী সহজে অন্য অংশে যেতে না পারে। স্তরভিত্তিক সাবনেট, সিকিউরিটি গ্রুপে সংকীর্ণ উৎস, NACL-এ deny নিয়ম এবং ব্যবস্থাপনা-নেটওয়ার্ক আলাদা রাখা — সব মিলে বিভাজন। বিভাজনের বিপরীত ভুল হলো "flat network": সব সার্ভার এক সাবনেটে, সবাই সবাইকে দেখে — একবার একটি ওয়েব সার্ভার আক্রান্ত হলে ডেটাবেস পর্যন্ত পথ খোলা।

৭.৮ CLI ও API উদাহরণ

```
$ ibmcloud is security-group-create technova-web-sg \
    --vpc technova-prod-vpc
```

```
$ ibmcloud is security-group-rule-add technova-web-sg \
    --direction inbound --protocol tcp --port-min 443 --port-max 443
$ ibmcloud is public-gateway-create technova-pgw-zone1 \
    --vpc technova-prod-vpc --zone us-south-1
$ ibmcloud is subnet-update technova-web-zone1 \
    --public-gateway technova-pgw-zone1
$ ibmcloud is security-groups
$ ibmcloud is public-gateways
```

উপরের কমান্ডগুলো ধারণাগত; প্রতিটি কমান্ডের সঠিক ফ্ল্যাগ `--help` দিয়ে যাচাই করুন। নিয়ম যোগ করার আগে বিদ্যমান নিয়ম তালিকা দেখে নিন — একই নিয়ম দুবার যোগ করলে বিভ্রান্তি বাড়ে। API-তে একই কাজ: সিকিউরিটি গ্রুপ তৈরি, তারপর নিয়ম-সংযোজন — প্রতিটি ধাপ আলাদা অনুরোধ।

৭.৯ IaC উদাহরণ

```
resource "ibm_is_security_group" "web" {
  name = "technova-web-sg"
  vpc  = ibm_is_vpc.technova_prod.id
}

resource "ibm_is_security_group_rule" "web_https" {
  group     = ibm_is_security_group.web.id
  direction = "inbound"
  remote    = "0.0.0.0/0"
  tcp {
    port_min = 443
    port_max = 443
  }
}

resource "ibm_is_public_gateway" "pgw_zone1" {
  name = "technova-pgw-zone1"
  vpc  = ibm_is_vpc.technova_prod.id
  zone = "us-south-1"
}
```

Terraform-এ নিরাপত্তা-নিয়ম কোড হিসেবে থাকলে পর্যালোচনায় ধরা পড়ে কেউ ভুল করে 0.0.0.0/0 দিয়ে পোর্ট 22 খুলেছে কি না। `terraform plan` চালিয়ে দেখুন কী বদলাবে, তারপর `apply` করুন — সরাসরি কনসোলে নিয়ম বদলে পরে Terraform-এর সঙ্গে অমিল (drift) তৈরি করবেন না।

৭.১০ উৎপাদন পরিস্থিতি: হঠাৎ বন্ধ হয়ে যাওয়া ডেটাবেস সংযোগ

TechNova-র অ্যাপ সার্ভার হঠাৎ ডেটাবেসে সংযোগ করতে পারছিল না। প্রথমে সন্দেহ হয় ডেটাবেস বন্ধ; কিন্তু ডেটাবেস চলছিল। পরীক্ষায় দেখা যায় কেউ সিকিউরিটি গ্রুপে অ্যাপ-গ্রুপের উল্লেখের বদলে একটি নির্দিষ্ট IP লিখেছিল; অ্যাপ সার্ভার পুনর্নির্মাণের পর তার IP বদলে যায়, নিয়মটি অকার্যকর হয়। সমাধান: IP-ভিত্তিক নিয়মের বদলে সিকিউরিটি-গ্রুপ-ভিত্তিক নিয়ম। শিক্ষা: গতিশীল পরিবেশে IP ঠিকানা পরিচয় নয় — গ্রুপই পরিচয়।

> **REAL WORLD:** একটি দল NACL-এ "সব deny" নিয়ম সবার ওপরে রেখে দিয়েছিল, ফলে পুরো সাবনেট বিচ্ছিন্ন হয়ে যায়। তারা সিকিউরিটি গ্রুপ দেখে সময় নষ্ট করেছিল, কারণ তারা জানত না NACL প্রথমে মূল্যায়িত হয়। নিয়মের ক্রম ও স্তর (NACL বনাম সিকিউরিটি গ্রুপ) না জানলে সমস্যা সমাধান অনুমানের খেলায় পরিণত হয়।

৭.১১ নিরাপত্তা ও ব্যয় বিবেচনা

নিরাপত্তা: ন্যূনতম সুযোগ, গ্রুপ-ভিত্তিক উৎস, সংবেদনশীল পোর্টে কখনো 0.0.0.0/0 নয়, সব পরিবর্তনের অডিট, নিয়মিত নিয়ম-পর্যালোচনা (কোন নিয়ম কেউ ব্যবহার করে না তা মুছুন)। ফ্লো লগ চালু রাখুন — প্রত্যাখ্যাত সংযোগের ধারা দেখলে স্ক্যান বা ভুল কনফিগারেশন ধরা পড়ে। ব্যয়: সিকিউরিটি গ্রুপ ও NACL সাধারণত আলাদা চার্জ নয়; খরচ আসে পাবলিক গেটওয়ে ও ডেটা স্থানান্তর থেকে। প্রতি জোনে গেটওয়ে রাখা প্রাপ্যতার জন্য দরকার, কিন্তু non-prod-এ হয়তো একটি জোনেই সীমাবদ্ধ রাখা যায় — প্রাপ্যতা বনাম ব্যয়ের সচেতন আপস।

৭.১২ সাধারণ ভুল

- সব পোর্টে 0.0.0.0/0 খুলে দেওয়া, বিশেষ করে SSH ও ডেটাবেস পোর্ট।
- NACL-এ উত্তর-ট্রাফিকের নিয়ম ভুলে যাওয়া (stateless ভুলে যাওয়া)।
- IP-ভিত্তিক নিয়ম লেখা যেখানে গ্রুপ-ভিত্তিক নিয়ম টেকসই হতো।
- ডেটা সাবনেটে পাবলিক গেটওয়ে যুক্ত করা — "আপডেটের সুবিধার্থে"।
- রাউট ও নিরাপত্তা-নিয়ম গুলিয়ে ফেলা; সমস্যার সময় দুটো একসঙ্গে বদলানো।
- পরিবর্তনের আগে বর্তমান নিয়মের ব্যাকআপ না রাখা।

৭.১৩ সমস্যা সমাধান

পদ্ধতিগত পরীক্ষা-ক্রম: (১) গন্তব্য IP ও পোর্ট ঠিক কি না; (২) রাউটিং টেবিলে গন্তব্যের দিকে রুট আছে কি না; (৩) সিকিউরিটি গ্রুপের inbound (গন্তব্যে) ও outbound (উৎসে) নিয়ম; (৪) NACL-এর উভয় দিকের নিয়ম ও ক্রম; (৫) গন্তব্য সার্ভারে পরিষেবা চলছে কি না ও কোন ইন্টারফেসে শুনছে (localhost-এ বাঁধা থাকলে বাইরে থেকে পৌঁছাবে না)। প্রতিটি ধাপে একটি করে জিনিস বদলান ও পরীক্ষা করুন; একসঙ্গে অনেক কিছু বদলালে কোনটি কাজ করল বোঝা যায় না। ফ্লো লগ ও সার্ভারের লগ একসঙ্গে দেখুন — ফ্লো লগ বলে প্যাকেট পৌঁছেছে কি না, অ্যাপ-লগ বলে পরিষেবা কী করেছে।

TROUBLESHOOTING: সংযোগ "কখনো হয়, কখনো হয় না"? সম্ভবত একটি জোনের গেটওয়ে বা একটি লোড ব্যালান্সার-সদস্য খারাপ, আর ট্রাফিক এলোমেলোভাবে সেদিকে যাচ্ছে। এক জোনের সার্ভার থেকে পরীক্ষা করে দেখুন সমস্যা জোন-নির্দিষ্ট কি না; জোন-নির্দিষ্ট হলে সেই জোনের রুট/গেটওয়ে/নিয়ম আলাদা করে পরীক্ষা করুন।

WARNING: সমস্যা সমাধানের সময় সাময়িকভাবে 0.0.0.0/0 খুলে পরীক্ষা করা প্রলোভন — কিন্তু তা ভুলে গেলে স্থায়ী ছিদ্র হয়ে যায়। সাময়িক নিয়মে সময়-সীমা ও দায়িত্ব লিখে রাখুন, পরীক্ষা শেষে অবিলম্বে মুছুন, এবং পরিবর্তন-লগে নথিভুক্ত করুন।

৭.১৪ গভীরে: ব্যর্থতা, পর্যবেক্ষণ, বিলিং ও বিকল্প

রাউটিং ও নিরাপত্তার প্রতিটি উপাদানকে একই বারোটি প্রশ্নে যাচাই করুন। সিকিউরিটি গ্রুপের সমস্যা: প্রতিটি সার্ভারকে নিজস্ব ফায়ারওয়াল দেওয়া; NACL-এর সমস্যা: পুরো সাবনেটে বিস্তৃত নীতি প্রয়োগ; পাবলিক গেটওয়ের সমস্যা: প্রাইভেট সার্ভারের নিরাপদ outbound। ভেতরে: সিকিউরিটি গ্রুপের নিয়ম হোস্ট-স্তরের ভার্চুয়াল সুইচে state table সহ প্রয়োগ হয়; NACL সাবনেট-সীমানায় ক্রমানুসারে প্রয়োগ হয়; রাউটিং টেবিল প্রতিটি সাবনেটে আলাদা থাকে। অন্যান্য পরিষেবার সঙ্গে যোগাযোগ: সিকিউরিটি গ্রুপ লোড ব্যালান্সারের সঙ্গে (শুধু ব্যালান্সার থেকে ট্রাফিক), VPN-এর সঙ্গে (অফিসের CIDR সংকীর্ণভাবে), ডেটাবেস-পরিষেবার সঙ্গে (প্রাইভেট এন্ডপয়েন্ট) — প্রতিটি সংযোগে নিয়মের উৎস স্পষ্ট থাকতে হবে। ব্যর্থ

হলে: ভুল নিয়মে বৈধ ট্রাফিক বন্ধ হয়, কিন্তু সাধারণত পুরো VPC থামে না; ভুল রুটে ট্রাফিক ভুল পথে যায় বা কোথাও যায় না; NACL-এর ক্রম-ভুলে deny নিয়ম কখনো কার্যকর হয় না। নিরাপত্তা: নিয়মের নিয়মিত অডিট, অব্যবহৃত নিয়ম মুছে ফেলা, 0.0.0.0/0 দিয়ে সংবেদনশীল পোর্ট না খোলা, পরিবর্তনের আগে ব্যাকআপ। পর্যবেক্ষণ: ফ্লো লগে প্রত্যাখ্যাত সংযোগের ধারা, নিয়ম-পরিবর্তনের অডিট লগ, গেটওয়ের থ্রুপুট। বিলিং: সিকিউরিটি গ্রুপ ও NACL সাধারণত আলাদা চার্জ নয়; পাবলিক গেটওয়ে ও ডেটা স্থানান্তরের চার্জ যাচাই করুন। কখন ব্যবহার করবেন: সিকিউরিটি গ্রুপ সবসময়, NACL সাবনেট-নীতিতে, গেটওয়ে প্রাইভেট সাবনেটের outbound-এ। কখন এড়াবেন: NACL-কে সিকিউরিটি গ্রুপের বিকল্প ভাববেন না; নিজের NAT সার্ভার সাধারণত পরিচালিত গেটওয়ের চেয়ে খারাপ পছন্দ। বিকল্প: হোস্ট-ভিত্তিক ফায়ারওয়াল, WAF, বা পরিচালিত ফায়ারওয়াল পরিষেবা — এরা পরিপূরক। আপস: নিয়ম যত সংকীর্ণ, নিরাপত্তা তত ভালো, কিন্তু পরিবর্তনের সময় ভুলের ঝুঁকি তত বেশি — তাই নিয়ম কোড হিসেবে রাখুন ও পর্যালোচনা করুন।

TechNova-র একটি বাস্তব শিক্ষা: দল একবার "সাময়িকভাবে" একটি নিয়ম খুলেছিল সমস্যা সমাধানের জন্য, তারপর ভুলে গিয়েছিল। তিন মাস পর অডিটে ধরা পড়ে। তারপর থেকে নিয়ম-পরিবর্তনে দুটি নিয়ম: প্রতিটি সাময়িক নিয়মে মেয়াদ-তারিখ লেখা বাধ্যতামূলক, এবং প্রতি শুক্রবার নিয়ম-তালিকা পর্যালোচনা করা হয়। প্রক্রিয়া ছাড়া নিরাপত্তা টেকে না — প্রযুক্তি শুধু অর্ধেক।

৭.১৫ সারসংক্ষেপ

রাউটিং ঠিক করে প্যাকেটের পথ, সিকিউরিটি গ্রুপ ঠিক করে সার্ভার-স্তরের অনুমোদন, NACL ঠিক করে সাবনেট-স্তরের নীতি, আর পাবলিক গেটওয়ে প্রাইভেট সার্ভারকে নিরাপদ outbound পথ দেয়। এই চারটি একসঙ্গে defence in depth তৈরি করে: একটি স্তর ভুল হলেও অন্য স্তর আটকায়। ভালো অভ্যাস — সংকীর্ণ উৎস, গ্রুপ-ভিত্তিক নিয়ম, ডেটা সাবনেটে ইন্টারনেট-রুট না রাখা, এবং প্রতিটি পরিবর্তন কোড হিসেবে সংরক্ষণ করা।

৭.১৬ পর্যালোচনা প্রশ্ন

1. Longest-prefix match কী এবং কেন এটি রাউটিংয়ে গুরুত্বপূর্ণ?
2. Stateful ও stateless ফায়ারওয়ালের পার্থক্য কী? সিকিউরিটি গ্রুপ কোনটি?
3. NACL-এ নিয়মের ক্রম কেন গুরুত্বপূর্ণ?
4. পাবলিক গেটওয়ে কীভাবে প্রাইভেট সার্ভারকে ইন্টারনেট-অ্যাক্সেস দেয় অথচ inbound সংযোগ আটকায়?
5. ডেটা সাবনেটে 0.0.0.0/0 রুট না রাখার নিরাপত্তাগত সুবিধা কী?

৭.১৭ ব্যবহারিক অনুশীলন

1. TechNova-র তিন স্তরের জন্য সিকিউরিটি গ্রুপ ও NACL নিয়ম-তালিকা লিখুন (প্রতিটি নিয়মে দিক, প্রোটোকল, পোর্ট, উৎস)। তারপর পরীক্ষামূলক VPC-তে CLI দিয়ে বাস্তবায়ন করে ওয়েব → অ্যাপ → ডেটা সংযোগ পরীক্ষা করুন; ইচ্ছাকৃতভাবে একটি নিয়ম ভুল করে দেখুন কোন ধাপে সংযোগ থামে।
2. একটি প্রাইভেট সাবনেটে পাবলিক গেটওয়ে যুক্ত করুন, সার্ভার থেকে বাইরের প্যাকেজ-সংগ্রহস্থলে পৌঁছানো যায় কি না পরীক্ষা করুন, এবং ইন্টারনেট থেকে সার্ভারে SSH চেষ্টা করে দেখুন তা আটকায় কি না — ফলাফল নথিভুক্ত করুন।

অধ্যায় ৮ • PART II — Networking & Infrastructure

Chapter 8 — Load Balancing and DNS

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক লোড ব্যালান্সার কেন দরকার তা ব্যাখ্যা করতে পারবেন; IBM Cloud লোড ব্যালান্সারের ধরন, ব্যাকএন্ড পুল, হেলথ চেক ও ট্রাফিক-বণ্টন নীতি বুঝতে পারবেন; ফ্লোটিং IP-র ধারণা ও ব্যবহার নির্ধারণ করতে পারবেন; DNS আর্কিটেকচার, রেকর্ড-ধরন ও TLS সমাপ্তি (termination) নকশা করতে পারবেন; এবং TechNova-র ওয়েবসাইটের জন্য উচ্চ-প্রাপ্যতা ট্রাফিক-প্রবাহ তৈরি করতে পারবেন।

- একক সার্ভারের ব্যর্থতা কেন পুরো সেবা থামায় এবং লোড ব্যালান্সার কীভাবে তা দূর করে — ব্যাখ্যা করা।
- ব্যাকএন্ড পুল ও হেলথ চেক কনফিগার করা ও যাচাই করা।
- DNS রেকর্ড (A, CNAME) ও TTL-এর প্রভাব মূল্যায়ন করা।
- TLS শংসাপত্র (certificate) কোথায় শেষ হবে — লোড ব্যালান্সার না সার্ভার — সিদ্ধান্ত নেওয়া।

৮.২ প্রারম্ভিক পুনরালোচনা

ষষ্ঠ ও সপ্তম অধ্যায়ে আমরা VPC, সাবনেট, রাউটিং ও সিকিউরিটি গ্রুপ শিখেছি। এখন একটি প্রশ্ন: গ্রাহক technova.example লিখলে কোন সার্ভারে যাবে? একটি সার্ভারের IP সরাসরি দিলে সেই সার্ভার বন্ধ হলেই সাইট বন্ধ। DNS নামকে IP-তে বদলায়, কিন্তু DNS নিজে ট্রাফিক বণ্টন করে না — বণ্টনের কাজ লোড ব্যালান্সারের। আর TLS (HTTPS) ছাড়া গ্রাহকের তথ্য পথে পড়া যায় — তাই শংসাপত্র-পরিচালনাও এই অধ্যায়ের অংশ।

ঝালিয়ে নিন: HTTP অনুরোধে ক্লায়েন্ট প্রথমে DNS-এ নাম খোঁজে, তারপর IP-তে TCP সংযোগ করে, তারপর TLS হ্যান্ডশেক করে, তারপর HTTP অনুরোধ পাঠায়। এই শৃঙ্খলের যেকোনো ধাপে ব্যর্থতা মানে গ্রাহক সাইট দেখতে পাবে না — তাই প্রতিটি ধাপের পর্যবেক্ষণ দরকার।

৮.৩ প্রথম নীতি থেকে লোড ব্যালান্সিং: সমস্যাটি কী?

একটি সার্ভারে সব ট্রাফিক গেলে তিনটি সমস্যা: (১) সার্ভারটি ব্যর্থ হলে সেবা বন্ধ; (২) ট্রাফিক বাড়লে সার্ভারটি চাপে পড়ে ধীর হয়; (৩) সার্ভার রক্ষণাবেক্ষণের জন্য থামালে সেবা বন্ধ রাখতে হয়। লোড ব্যালান্সার এই তিনটি সমস্যার একটিই সমাধান দেয়: গ্রাহকের সামনে একটি স্থিতিশীল প্রবেশ-পথ (entry point) রাখা, যার পেছনে একাধিক সার্ভার (ব্যাকএন্ড পুল) থাকে; ব্যালান্সার প্রতিটি অনুরোধ সুস্থ সার্ভারে পাঠায়, অসুস্থ সার্ভারকে সরিয়ে রাখে, এবং নতুন সার্ভার যোগ করলে স্বয়ংক্রিয়ভাবে তাকে কাজে লাগায়।

লোড ব্যালান্সার দুই স্তরে কাজ করতে পারে: Layer 4 (TCP/UDP — প্যাকেটের ঠিকানা ও পোর্ট দেখে) ও Layer 7 (HTTP/HTTPS — URL, হেডার, কুকি দেখে)। Layer 7 ব্যালান্সার পথ-ভিত্তিক রাউটিং করতে পারে — যেমন /api অনুরোধ API সার্ভারে, / ছবি স্ট্যাটিক সার্ভারে। TechNova-র ওয়েব অ্যাপের জন্য Layer 7 উপযোগী; সাধারণ TCP পরিষেবার জন্য Layer 4 যথেষ্ট।

> **CONCEPT:** লোড ব্যালান্সার ট্রাফিকের "সামনের দরজা" — গ্রাহক শুধু ব্যালান্সারকে চেনে, পেছনের সার্ভারগুলোকে নয়। সার্ভার বদলালেও গ্রাহকের ঠিকানা একই থাকে। এটিই ব্যালান্সারকে রক্ষণাবেক্ষণ, স্কেলিং ও ব্যর্থতা-সামলানোর কেন্দ্র করে তোলে।

৮.৪ IBM Cloud লোড ব্যালান্সার: ভেতরে কীভাবে কাজ করে

IBM Cloud VPC লোড ব্যালান্সার একটি পরিচালিত পরিষেবা: আপনি লিসেনার (listener — যেমন পোর্ট 443), পুল (pool — ব্যাকএন্ড সার্ভারের তালিকা), হেলথ চেক ও রাউটিং-নীতি নির্ধারণ করেন; IBM Cloud ব্যালান্সারের নোডগুলো পরিচালনা করে। অনুরোধের প্রবাহ: গ্রাহক → DNS → ব্যালান্সারের IP → লিসেনার → নীতি অনুযায়ী পুল-সদস্য নির্বাচন → সার্ভার → উত্তর ব্যালান্সার দিয়ে গ্রাহকের কাছে। ব্যালান্সার সাধারণত একাধিক জোনে চলে, তাই একটি জোনের ব্যালান্সার-নোড বন্ধ হলেও অন্যটি কাজ করে।

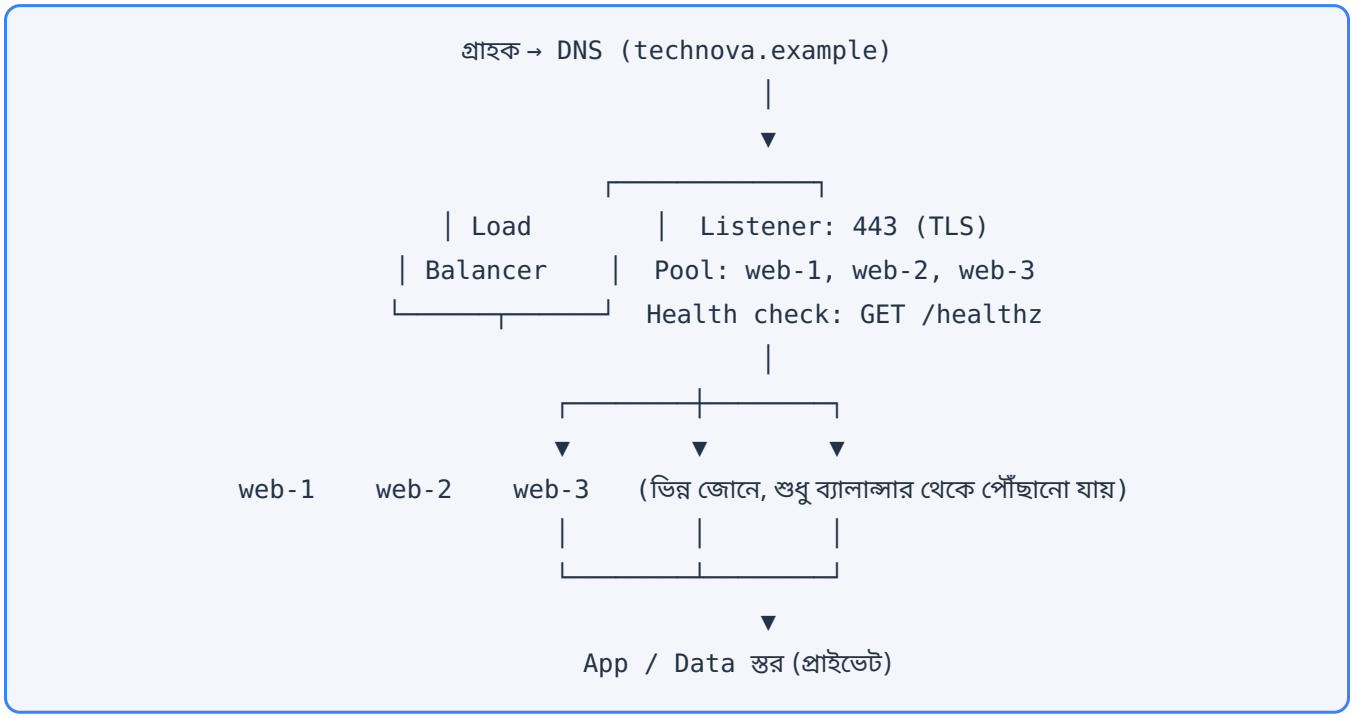
সমস্যা ও সমাধান: ব্যালান্সার নিজেই যদি একক ব্যর্থতার বিন্দু হয়, তাহলে লাভ নেই — তাই পরিচালিত ব্যালান্সার বহু-জোনে চালানো হয়। ব্যর্থ হলে কী হয়? একটি ব্যাকএন্ড সার্ভার বন্ধ হলে হেলথ চেক তা ধরে পুল থেকে সরিয়ে দেয়; ব্যালান্সার-নোড বন্ধ হলে অন্য নোড দায়িত্ব নেয়; পুরো ব্যালান্সার পরিষেবায় সমস্যা হলে — বিরল হলেও — DNS-স্তরের বিকল্প বা দ্বিতীয় ব্যালান্সারের পরিকল্পনা দরকার। নিরাপত্তা: ব্যালান্সারের সিকিউরিটি গ্রুপে শুধু 80/443 খোলা রাখুন; ব্যাকএন্ড সার্ভারের গ্রুপে শুধু ব্যালান্সার থেকে সংযোগ অনুমোদন করুন — সরাসরি ইন্টারনেট থেকে ব্যাকএন্ডে যাওয়া যাবে না। পর্যবেক্ষণ: অনুরোধ-সংখ্যা, বিলম্ব, 5xx ত্রুটির হার, সুস্থ-সদস্যের সংখ্যা — এগুলো দেখুন। বিলিং: ব্যালান্সার-ইনস্ট্যান্স ও প্রক্রিয়াকৃত ডেটার জন্য চার্জ প্রযোজ্য হতে পারে; মূল্য ক্যাটালগে যাচাই করুন। কখন ব্যবহার করবেন: একাধিক সার্ভারে ট্রাফিক বণ্টন, TLS সমাপ্তি, জোন-জোড়া প্রাপ্যতা — সব ক্ষেত্রে। কখন এড়াবেন: একটি মাত্র সার্ভারের পরীক্ষামূলক পরিবেশে ব্যালান্সার অপ্রয়োজনীয় খরচ ও জটিলতা। বিকল্প: DNS রাউন্ড-রবিন (সহজ, কিন্তু অসুস্থ সার্ভার সরায় না), নিজের HAProxy/Nginx চালানো (নিয়ন্ত্রণ বেশি, পরিচালনার বোঝাও বেশি)।

৮.৫ ব্যাকএন্ড পুল ও ট্রাফিক-বণ্টন

ব্যাকএন্ড পুল হলো সার্ভার-সদস্যদের তালিকা, প্রতিটি সদস্যের IP, পোর্ট ও ওজন (weight) সহ। বণ্টন-নীতি (algorithm) নির্ধারণ করে কোন সদস্য পরের অনুরোধ পাবে: রাউন্ড-রবিন (একে একে সবাই), least connections (যার সংযোগ কম তাকে), weighted (শক্তিশালী সার্ভারকে বেশি)। TechNova-র তিনটি সমান সার্ভারে রাউন্ড-রবিন যথেষ্ট; সার্ভারের ক্ষমতা ভিন্ন হলে ওজন ব্যবহার করুন। **সেশন পুনরাবৃত্তি** (session persistence / stickiness) — একই গ্রাহককে বারবার একই সার্ভারে পাঠানো — শুধু তখনই ব্যবহার করুন যখন অ্যাপ সেশন-অবস্থা সার্ভারে রাখে; আধুনিক অ্যাপে সেশন বাইরে (যেমন Redis বা ডেটাবেসে) রাখাই ভালো, কারণ stickiness একটি সার্ভার বন্ধ হলে সেই গ্রাহকদের সেশন হারায়।

৮.৬ হেলথ চেক: অসুস্থ সার্ভারকে সরিয়ে রাখা

হেলথ চেক হলো ব্যালান্সারের নিয়মিত পরীক্ষা: নির্দিষ্ট পথে (যেমন /healthz) HTTP অনুরোধ পাঠিয়ে দেখা সার্ভার 200 উত্তর দেয় কি না। পরামিতি: ব্যবধান (interval), টাইমআউট, সফল-প্রান্তিক (কতবার সফল হলে সুস্থ) ও ব্যর্থ-প্রান্তিক (কতবার ব্যর্থ হলে অসুস্থ)। ভুল কনফিগারেশনের পরিণতি মারাত্মক: হেলথ-পথ ভুল হলে সুস্থ সার্ভারকেও অসুস্থ মনে করে পুল খালি হয়ে যায়; ব্যবধান খুব কম হলে সার্ভারে অপ্রয়োজনীয় চাপ; টাইমআউট খুব কম হলে ধীর কিন্তু সুস্থ সার্ভার বাদ পড়ে। ভালো অভ্যাস: /healthz পথটি হালকা রাখুন — শুধু অ্যাপ চলছে কি না ও ডেটাবেসে সংযোগ আছে কি না দেখুন; ভারী কাজ (যেমন পুরো রিপোর্ট তৈরি) হেলথ চেকে রাখবেন না।



৮.৭ ফ্লোটিং IP ধারণা

ফ্লোটিং IP একটি পাবলিক IP ঠিকানা যা কোনো নির্দিষ্ট সার্ভার বা লোড ব্যালান্সারের সঙ্গে বাঁধা যায় এবং প্রয়োজনে অন্য রিসোর্সে সরিয়ে নেওয়া যায় — সার্ভার বদলালেও গ্রাহকের ঠিকানা একই থাকে। এটি উচ্চ-প্রাপ্যতার চাবি: একটি সার্ভার বন্ধ হলে ফ্লোটিং IP সুস্থ সার্ভারে সরিয়ে নিলে DNS বদলাতে হয় না। তবে মনে রাখুন: ফ্লোটিং IP নিজে কোনো সেবা নয়, এটি একটি ঠিকানা — এর পেছনে সুস্থ সার্ভার থাকতে হয়। লোড ব্যালান্সারের ক্ষেত্রে ব্যালান্সারের নিজস্ব পাবলিক ঠিকানাই সাধারণত ব্যবহৃত হয়; ফ্লোটিং IP বেশি ব্যবহৃত হয় একক সার্ভার বা বাস্টিয়নের ক্ষেত্রে। অব্যবহৃত ফ্লোটিং IP ছেড়ে দিন — বরাদ্দ করা ঠিকানা খরচের কারণ হতে পারে।

৮.৮ DNS আর্কিটেকচার

DNS (Domain Name System) নামকে IP-তে বদলায়। মূল রেকর্ড: **A রেকর্ড** — নাম → IPv4 ঠিকানা; **CNAME** — নাম → আরেকটি নাম; **AAAA** — IPv6। **TTL** (Time To Live) বলে রেকর্ড কতক্ষণ ক্যাশে থাকবে: TTL বেশি হলে পরিবর্তন ধীরে ছড়ায়, কম হলে DNS-অনুরোধ বাড়ে। TechNova-র পরিকল্পনা: technova.example-র A রেকর্ড লোড ব্যালান্সারের IP-তে; www-র CNAME মূল নামে; TTL সাধারণত 300 সেকেন্ড (পরিবর্তনের সময় কমানো যায়)। ভেতরে: DNS অনুরোধ রুট সার্ভার → TLD সার্ভার → কর্তৃত্বপূর্ণ (authoritative) সার্ভার — এই শৃঙ্খলে যায়; IBM Cloud DNS পরিষেবা কর্তৃত্বপূর্ণ সার্ভারের ভূমিকা পালন করতে পারে। ব্যর্থ হলে কী হয়? DNS বন্ধ হলে নাম খুঁজে পাওয়া যায় না — সার্ভার চললেও গ্রাহক পৌঁছাতে পারে না। তাই DNS-এর প্রাপ্যতা সাইটের প্রাপ্যতার সমান গুরুত্বপূর্ণ; দ্বিতীয় DNS প্রদানকারীর কথা বিবেচনা করুন। নিরাপত্তা: DNSSEC, জোন-পরিবর্তনে IAM নিয়ন্ত্রণ। বিলিং: DNS জোন ও অনুরোধ-সংখ্যার ভিত্তিতে চার্জ হতে পারে — যাচাই করুন।

৮.৯ TLS: শংসাপত্র কোথায় শেষ হবে

TLS (Transport Layer Security) গ্রাহক ও সার্ভারের মধ্যে তথ্য এনক্রিপ্ট করে ও সার্ভারের পরিচয় যাচাই করে। **TLS সমাপ্তি** (termination) মানে কোথায় এনক্রিপশন খোলা হবে: লোড ব্যালান্সারে সমাপ্তি করলে ব্যালান্সার শংসাপত্র (certificate) ধরে রাখে, পেছনের ট্রাফিক VPC-র ভেতরে প্লেইন HTTP-তে যায় — পরিচালনা সহজ, তবে VPC-র ভেতরের ট্রাফিক এনক্রিপ্ট করা থাকে না। End-to-end TLS-এ ব্যালান্সার শুধু পাস-থ্রু করে, শংসাপত্র প্রতিটি সার্ভারে থাকে —

নিরাপত্তা বেশি, পরিচালনা কঠিন। TechNova-র জন্য সাধারণ পছন্দ: ব্যালান্সারে সমাপ্তি + VPC-র ভেতরে প্রাইভেট নেটওয়ার্কের বিশ্বাস — তবে সংবেদনশীল ডেটার ক্ষেত্রে end-to-end বিবেচনা করুন। শংসাপত্রের মেয়াদ (expiry) পর্যবেক্ষণ করুন: মেয়াদোত্তীর্ণ শংসাপত্র মানে ব্রাউজারে সতর্কতা ও গ্রাহক-আস্থা হারানো। স্বয়ংক্রিয় নবায়ন (auto-renewal) ব্যবহার করুন।

৮.১০ CLI ও API উদাহরণ

```
$ ibmcloud is lb-create technova-web-lb \
    --vpc technova-prod-vpc \
    --subnet technova-web-zone1 \
    --subnet technova-web-zone2
$ ibmcloud is lb-listener-create technova-web-lb \
    --protocol https --port 443
$ ibmcloud is lb-pool-create technova-web-lb \
    --protocol http --port 8080 \
    --health-monitor-type http --health-monitor-url-path /healthz
$ ibmcloud is lb-pool-member-create technova-web-lb \
    --pool web-pool --address 10.10.1.5 --port 8080
$ ibmcloud is lbs
```

কমান্ডগুলো ধারণাগত; সঠিক ফ্ল্যাগ `--help` দিয়ে যাচাই করুন। পুল-সদস্য যোগ করার আগে সার্ভারের সিকিউরিটি গ্রুপে ব্যালান্সার থেকে 8080 অনুমোদন করুন — নইলে হেলথ চেক ব্যর্থ হবে।

৮.১১ IaC উদাহরণ

```
resource "ibm_is_lb" "web" {
  name    = "technova-web-lb"
  subnets = [ibm_is_subnet.web_zone1.id,
             ibm_is_subnet.web_zone2.id]
}

resource "ibm_is_lb_listener" "https" {
  lb       = ibm_is_lb.web.id
  protocol = "https"
  port     = 443
}

resource "ibm_is_lb_pool" "web_pool" {
  lb           = ibm_is_lb.web.id
  protocol     = "http"
  port         = 8080
  health_monitor {
    type     = "http"
    url_path = "/healthz"
  }
}
```

৮.১২ উৎপাদন পরিস্থিতি: হঠাৎ ট্রাফিক-বৃদ্ধি

TechNova-র একটি প্রচারণার দিন ট্রাফিক দশ গুণ বেড়ে যায়। একটি সার্ভার থাকলে সাইট বন্ধ হয়ে যেত; লোড ব্যালান্সারের পেছনে তিনটি সার্ভার থাকায় ট্রাফিক ভাগ হয়ে যায়, এবং দল আরও দুটি সার্ভার পুলে যোগ করে — DNS বা গ্রাহকের কিছু বদলাতে হয়নি। শিক্ষা: ব্যালান্সার শুধু ব্যর্থতা সামলায় না, বৃদ্ধিও সামলায়।

> **REAL WORLD:** একটি দল হেলথ চেকের পথে / লিখেছিল, যা ডেটাবেসের ভারী কুয়েরি চালাত। ট্রাফিক বাড়লে হেলথ চেকই সার্ভারকে ধীর করে দেয়, ব্যালান্সার সার্ভারকে অসুস্থ মনে করে সরিয়ে দেয়, বাকি সার্ভারে চাপ আরও বাড়ে — একটি ধ্বংসাত্মক চক্র। হেলথ-পথ সবসময় হালকা রাখুন।

৮.১৩ নিরাপত্তা ও ব্যয় বিবেচনা

নিরাপত্তা: ব্যালান্সারে TLS 1.2 বা নতুন সংস্করণ বাধ্যতামূলক করুন; পুরোনো, দুর্বল সাইফার বন্ধ করুন; ব্যাকএন্ডে শুধু ব্যালান্সার থেকে ট্রাফিক; WAF (Web Application Firewall) থাকলে বিবেচনা করুন — এটি অ্যাপ-স্তরের আক্রমণ (যেমন SQL ইনজেকশন) আটকায়। ব্যয়: ব্যালান্সার-ইনস্ট্যান্স, ডেটা প্রক্রিয়াকরণ ও জোন-পার ট্রাফিক — তিনটি মিলে খরচ; non-prod-এ ছোট ব্যালান্সার বা সরাসরি সার্ভার বিবেচনা করুন। পর্যবেক্ষণ: সুস্থ-সদস্য সংখ্যা শূন্য হলে অবিলম্বে সতর্কতা; 5xx হার বাড়লে অ্যাপ-লগ দেখুন।

৮.১৪ সাধারণ ভুল

- হেলথ চেকের পথ ভুল বা ভারী রাখা।
- ব্যাকএন্ড সার্ভারে সরাসরি পাবলিক IP রেখে ব্যালান্সারকে বাইপাস করা।
- একটি মাত্র জোনে ব্যালান্সার ও সার্ভার রাখা।
- শংসাপত্রের মেয়াদ পর্যবেক্ষণ না করা।
- TTL খুব বেশি রাখা — জরুরি পরিবর্তন ছড়াতে ঘণ্টা লেগে যাওয়া।
- সেশন-স্টিকিনেস অকারণে চালু রাখা।

৮.১৫ সমস্যা সমাধান

সাইট খুলছে না? ধাপে ধাপে: (১) DNS নাম IP-তে বদলাচ্ছে কি না (`nslookup technova.example`); (২) ব্যালান্সারের IP-তে TCP সংযোগ হচ্ছে কি না; (৩) ব্যালান্সারে সুস্থ সদস্য আছে কি না; (৪) হেলথ চেকের উত্তর কী; (৫) ব্যাকএন্ড সার্ভারে সরাসরি (বাস্টিয়ন দিয়ে) পরিষেবা চলছে কি না। 502 ত্রুটি মানে ব্যালান্সার ব্যাকএন্ডে পৌঁছাতে পারছে না — সিকিউরিটি গ্রুপ বা সার্ভার দেখুন। 503 মানে পুলে কোনো সুস্থ সদস্য নেই — হেলথ চেক দেখুন। শংসাপত্র-ত্রুটি মানে TLS কনফিগারেশন দেখুন।

> **TROUBLESHOOTING:** কিছু গ্রাহক সাইট দেখতে পাচ্ছে, কিছু পাচ্ছে না? সম্ভবত DNS ক্যাশে পুরোনো IP ধরে আছে (TTL-এর কারণে) অথবা একটি জোনের সার্ভার খারাপ। ভিন্ন নেটওয়ার্ক থেকে পরীক্ষা করুন ও TTL কমিয়ে পরিবর্তন করুন।

> **WARNING:** লোড ব্যালান্সার যুক্ত করার পর ব্যাকএন্ড সার্ভারের লগে সব অনুরোধ ব্যালান্সারের IP থেকে আসছে বলে মনে হবে — আসল গ্রাহকের IP হারিয়ে যাবে, যদি না X-Forwarded-For হেডার সংরক্ষণ ও বিশ্বাস করার ব্যবস্থা করেন। হার-সীমা (rate limiting) ও লগ-বিশ্লেষণে এটি গুরুত্বপূর্ণ।

৮.১৬ গভীরে: ব্যর্থতা, পর্যবেক্ষণ, বিলিং ও বিকল্প

লোড ব্যালান্সার, DNS ও TLS — প্রতিটিকে বারোটি প্রশ্নে যাচাই করুন। সমস্যা: একক সার্ভারের ব্যর্থতা, ট্রাফিক-বৃদ্ধি, নাম-থেকে-ঠিকানার অনিশ্চয়তা, পথে তথ্য-চুরি। ভেতরে: ব্যালান্সার অনুরোধ গ্রহণ করে লিসেনার-নীতি দেখে, পুল থেকে সুস্থ সদস্য বেছে নেয়, হেলথ চেকের ফলাফল অনুযায়ী পুল আপডেট করে; DNS শ্রেণিবদ্ধভাবে নাম মেলায়; TLS হ্যান্ডশেকে শংসাপত্র যাচাই করে এনক্রিপ্ট করা অধিবেশন (session) তৈরি করে। অন্যান্য পরিষেবার সঙ্গে যোগাযোগ: ব্যালান্সার VPC-র সিকিউরিটি গ্রুপ, সাবনেট ও ফ্লোটিং IP-র ওপর নির্ভর করে; DNS IBM Cloud DNS বা বাইরের প্রদানকারীর সঙ্গে; TLS শংসাপত্র-পরিচালকের সঙ্গে। ব্যর্থ হলে: একটি ব্যাকএন্ড বন্ধ হলে ট্রাফিক অন্যদিকে যায়; ব্যালান্সার-নোড বন্ধ হলে অন্য নোড নেয়; DNS বন্ধ হলে নাম মেলে না — সার্ভার চললেও গ্রাহক পৌঁছায় না; শংসাপত্রের মেয়াদ শেষ হলে ব্রাউজার সতর্কতা দেয়। নিরাপত্তা: TLS 1.2+, দুর্বল সাইফার বন্ধ, ব্যাকএন্ডে শুধু ব্যালান্সার থেকে ট্রাফিক, WAF বিবেচনা। পর্যবেক্ষণ: অনুরোধ-হার, বিলম্ব, 5xx হার, সুস্থ-সদস্য সংখ্যা, শংসাপত্রের মেয়াদ, DNS-উত্তরের সময়। বিলিং: ব্যালান্সার-ইনস্ট্যান্স, প্রক্রিয়াকৃত ডেটা, DNS জোন ও অনুরোধ — মূল্য ক্যাটালগে যাচাই করুন। কখন ব্যবহার করবেন: একাধিক সার্ভার, TLS সমাপ্তি, জোন-জোড়া প্রাপ্যতা। কখন এড়াবেন: একক পরীক্ষামূলক সার্ভারে ব্যালান্সার অপ্রয়োজনীয়। বিকল্প: DNS রাউন্ড-রবিন (অসুস্থ সার্ভার সরায় না), নিজের Nginx/HAProxy (নিয়ন্ত্রণ বেশি, বোঝাও বেশি)। আপস: Layer 7 ব্যালান্সার বুদ্ধিমান রাউটিং দেয় কিন্তু Layer 4-এর চেয়ে বেশি বিলম্ব ও খরচ হতে পারে; ব্যালান্সারে TLS সমাপ্তি সহজ, কিন্তু VPC-র ভেতরের ট্রাফিক প্লেইন থাকে — সংবেদনশীল কাজে end-to-end বিবেচনা করুন।

৮.১৭ সারসংক্ষেপ

লোড ব্যালান্সার একক সার্ভারের ব্যর্থতা ও সীমা দূর করে; হেলথ চেক অসুস্থ সার্ভারকে সরিয়ে রাখে; DNS নামকে স্থিতিশীল ঠিকানায় বাঁধে; TLS গ্রাহকের তথ্য রক্ষা করে। একসঙ্গে এরা TechNova-র ওয়েবসাইটকে এক সার্ভারের ভাগ্য থেকে মুক্ত করে একটি সিস্টেমে পরিণত করে — যেখানে সার্ভার বদলানো যায়, ট্রাফিক বাড়ানো যায়, আর গ্রাহক কিছু টের পায় না।

৮.১৮ পর্যালোচনা প্রশ্ন

1. Layer 4 ও Layer 7 লোড ব্যালান্সারের পার্থক্য কী?
2. হেলথ চেক কীভাবে অসুস্থ সার্ভারকে পুল থেকে সরায়?
3. ফ্লোটিং IP কেন উচ্চ-প্রাপ্যতায় সাহায্য করে?
4. TTL বেশি রাখার সুবিধা ও অসুবিধা কী?
5. TLS সমাপ্তি লোড ব্যালান্সারে করা বনাম end-to-end করার আপস (trade-off) কী?

৮.১৯ ব্যবহারিক অনুশীলন

1. দুটি ওয়েব সার্ভার ও একটি লোড ব্যালান্সার দিয়ে পরীক্ষামূলক সেটআপ তৈরি করুন; একটি সার্ভার বন্ধ করে দেখুন ট্রাফিক অন্যটিতে যায় কি না, এবং হেলথ চেকের অবস্থা পর্যবেক্ষণ করুন।
2. একটি ডোমেইনের জন্য A ও CNAME রেকর্ড পরিকল্পনা করুন, TTL 300 নির্ধারণ করুন, এবং শংসাপত্রের মেয়াদ-পর্যবেক্ষণের জন্য একটি চেকলিস্ট লিখুন।

অধ্যায় ৯ • PART II — Networking & Infrastructure

Chapter 9 — IBM Cloud Virtual Servers

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক ভার্চুয়াল সার্ভার (Virtual Server Instance, VSI) কীভাবে তৈরি ও পরিচালিত হয় তা ব্যাখ্যা করতে পারবেন; ইনস্ট্যান্স প্রোফাইল (instance profile), ইমেজ (image), বুট ও ডেটা ভলিউম এবং ব্লক স্টোরেজের পার্থক্য বুঝতে পারবেন; SSH-কী দিয়ে নিরাপদ অ্যাক্সেস স্থাপন করতে পারবেন; ইনস্ট্যান্সের জীবনচক্র (lifecycle) পরিচালনা করতে পারবেন; এবং পর্যবেক্ষণ ও সমস্যা সমাধানের মাধ্যমে উৎপাদন-সার্ভার চালাতে পারবেন।

- কাজের চাহিদা অনুযায়ী সঠিক ইনস্ট্যান্স প্রোফাইল নির্বাচন করা।
- বুট ভলিউম বনাম ডেটা ভলিউমের ব্যবহার ও ব্যাকআপ-নীতি নির্ধারণ করা।
- পাসওয়ার্ড-ভিত্তিক SSH বন্ধ করে কী-ভিত্তিক অ্যাক্সেস চালু করা।
- সার্ভারের CPU, মেমরি, ডিস্ক ও নেটওয়ার্ক পর্যবেক্ষণ করে সিদ্ধান্ত নেওয়া।

৯.২ প্রারম্ভিক পুনরালোচনা

আগের অধ্যায়গুলোতে আমরা নেটওয়ার্ক তৈরি করেছি — VPC, সাবনেট, সিকিউরিটি গ্রুপ, লোড ব্যালান্সার। এখন সেই নেটওয়ার্কে কম্পিউট বসানোর পালা। ভার্চুয়াল সার্ভার হলো ভৌত সার্ভারের ভার্চুয়াল সংস্করণ: তার নিজস্ব CPU, মেমরি, ডিস্ক ও নেটওয়ার্ক ইন্টারফেস আছে, কিন্তু সে চলে একটি হাইপারভাইজার (hypervisor) পরিচালিত ভৌত হোস্টে। ঝালিয়ে নিন: অপারেটিং সিস্টেম, SSH প্রোটোকল ও ফাইল সিস্টেমের ধারণা — এই অধ্যায়ে প্রতিদিনের কাজে এগুলো লাগবে।

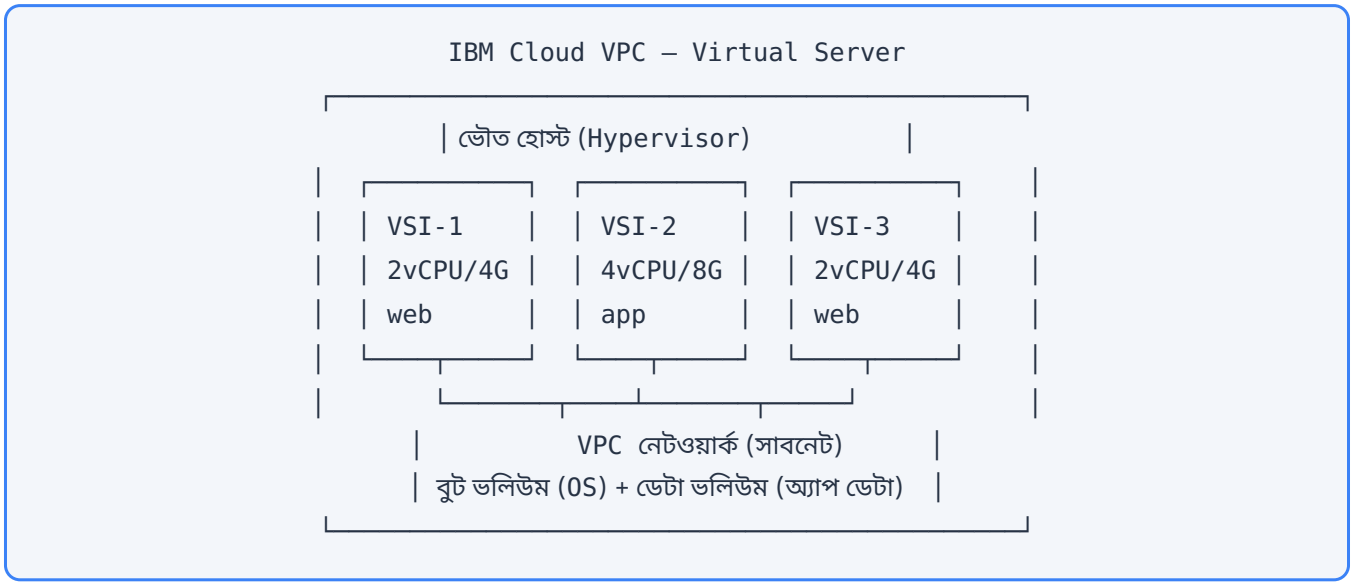
৯.৩ প্রথম নীতি থেকে ভার্চুয়াল সার্ভার: সমস্যাটি কী?

TechNova-র অ্যাপ চালাতে কম্পিউট দরকার — CPU-তে কোড চলবে, মেমরিতে ডেটা থাকবে, ডিস্কে ফাইল থাকবে। ভৌত সার্ভার কিনলে সপ্তাহ লাগে, অগ্রিম খরচ হয়, আর চাহিদা কমলে সার্ভার পড়ে থাকে। ভার্চুয়াল সার্ভার এই সমস্যা সমাধান করে: মিনিটে তৈরি, ঘণ্টা বা মাসের হিসাবে বিল, চাহিদা অনুযায়ী আকার বদলানো যায়, আর বন্ধ করলে খরচ থামে (কিছু সংরক্ষিত রিসোর্স ছাড়া)। বিনিময়ে আপনাকে মনে রাখতে হবে: ভার্চুয়াল সার্ভারও একটি কম্পিউটার — তার OS আপডেট, নিরাপত্তা-প্যাচ ও ব্যাকআপের দায়িত্ব আপনার (পরিচালিত পরিষেবার মতো নয়)।

> **CONCEPT:** ভার্চুয়াল সার্ভার = ভাড়া করা কম্পিউটার। ক্লাউড আপনাকে হার্ডওয়্যার দেয়, কিন্তু ভেতরের সফটওয়্যার, নিরাপত্তা ও ডেটার দায়িত্ব আপনার। এই দায়িত্ব-ভাগ (shared responsibility) না বুঝলে নিরাপত্তা-ছিদ্র তৈরি হয়।

৯.৪ ভার্চুয়াল সার্ভার আর্কিটেকচার ও ইনস্ট্যান্স প্রোফাইল

IBM Cloud VPC-তে ভার্চুয়াল সার্ভার চলে হাইপারভাইজারের ওপরে; প্রতিটি সার্ভারের একটি **প্রোফাইল** থাকে যা CPU, মেমরি ও নেটওয়ার্ক-ক্ষমতা নির্ধারণ করে — যেমন ২ vCPU / ৪ GB মেমরি (সাধারণ কাজ), ৪ vCPU / ৩২ GB (মেমরি-ভারী কাজ), বা GPU-যুক্ত প্রোফাইল (মেশিন লার্নিং)। প্রোফাইল নির্বাচনের নিয়ম: প্রথমে কাজের চাহিদা মাপুন — ওয়েব সার্ভারে CPU ও নেটওয়ার্ক, ডেটাবেসে মেমরি ও ডিস্ক-I/O, ব্যাচ-কাজে CPU। খুব বড় প্রোফাইল নিলে অর্থ নষ্ট; খুব ছোট নিলে সেবা ধীর হয়। শুরুতে মাঝারি নিন, পর্যবেক্ষণ দেখে বদলান।



অভ্যন্তরে: হাইপারভাইজার প্রতিটি VSI-কে CPU-সময়, মেমরি ও নেটওয়ার্ক-ব্যান্ডউইথ ভাগ করে দেয়; একটি VSI-র অতিরিক্ত চাপ প্রতিবেশী VSI-কে সাধারণত প্রভাবিত করে না (isolation), তবে ভৌত হোস্টের সমস্যায় ওই হোস্টের সব VSI প্রভাবিত হতে পারে — তাই গুরুত্বপূর্ণ সার্ভার ভিন্ন জোনে ছড়ান। ব্যর্থ হলে কী হয়? হোস্ট-বিভ্রাটে VSI বন্ধ হয়ে যেতে পারে; IBM Cloud কিছু ক্ষেত্রে স্বয়ংক্রিয়ভাবে পুনরুদ্ধার করে, কিন্তু অ্যাপ-স্তরে উচ্চ-প্রাপ্যতা (একাধিক VSI + লোড ব্যালান্সার) ছাড়া সেবা থামবে। নিরাপত্তা: হাইপারভাইজার-স্তরের বিচ্ছিন্নতা IBM Cloud-এর দায়িত্ব; OS-স্তরের নিরাপত্তা আপনার। পর্যবেক্ষণ: CPU, মেমরি, ডিস্ক, নেটওয়ার্ক — চারটি মেট্রিকই দেখুন। বিলিং: প্রোফাইল-আকার ও চলমান সময় অনুযায়ী; বন্ধ করা সার্ভারের ভলিউমের জন্য চার্জ চলতে পারে — যাচাই করুন। কখন ব্যবহার করবেন: পূর্ণ নিয়ন্ত্রণ দরকার হলে, কাস্টম সফটওয়্যার চালাতে হলে। কখন এড়াবেন: পরিচালিত পরিষেবা (যেমন পরিচালিত ডেটাবেস) দিয়ে কাজ হলে নিজে সার্ভার চালানো অপ্রয়োজনীয় বোঝা। বিকল্প: কনটেইনার (Kubernetes), সার্ভারলেস ফাংশন — এগুলো পরের খণ্ডে।

৯.৫ ইমেজ, বুট ভলিউম ও ডেটা ভলিউম

ইমেজ হলো OS ও প্রাথমিক সফটওয়্যারের টেমপ্লেট — যেমন Ubuntu, Debian বা Red Hat Enterprise Linux-এর IBM Cloud-প্রদত্ত ইমেজ, বা আপনার নিজের কাস্টম ইমেজ (সোনালি ইমেজ / golden image: OS + প্যাচ + এজেন্ট আগে থেকে বসানো)। কাস্টম ইমেজের সুবিধা: নতুন সার্ভার দ্রুত ও একইভাবে তৈরি হয়; ঝুঁকি: পুরোনো ইমেজে পুরোনো দুর্বলতা থাকতে পারে — নিয়মিত পুনর্নির্মাণ করুন।

বুট ভলিউম OS ধারণ করে; **ডেটা ভলিউম** অ্যাপের ডেটা ধারণ করে। আলাদা রাখার কারণ: OS পুনর্নির্মাণ করলে ডেটা ভলিউম অক্ষত থাকে; ডেটা ভলিউমের ব্যাকআপ-নীতি আলাদা করা যায়; ডেটা ভলিউমের আকার ও কর্মক্ষমতা (IOPS) আলাদাভাবে বাড়ানো যায়। **ব্লক স্টোরেজ** হলো নেটওয়ার্ক-সংযুক্ত ডিস্ক — VSI-র সঙ্গে যুক্ত হয়, স্ন্যাপশট (snapshot) নেওয়া যায়, এবং প্রয়োজনে অন্য VSI-তে সরানো যায় (সাধারণত বন্ধ অবস্থায়)। ব্লক স্টোরেজের কর্মক্ষমতা-স্তর (tier) আছে — বেশি IOPS মানে বেশি খরচ; ডেটাবেসে উচ্চ IOPS, লগ-সার্ভারে নিম্ন IOPS যৌক্তিক।

উপাদান	কী ধারণ করে	সার্ভার মুছলে কী হয়	ব্যাকআপ
বুট ভলিউম	OS ও সিস্টেম ফাইল	সাধারণত মুছে যায় (নীতি অনুযায়ী)	ইমেজ/স্ন্যাপশট
ডেটা ভলিউম	অ্যাপ ডেটা, ডেটাবেস ফাইল	আলাদা রাখলে টিকে থাকে	নিয়মিত স্ন্যাপশট
ইমেজ	OS টেমপ্লেট	প্রভাবিত হয় না	সংস্করণ-নিয়ন্ত্রণ

৯.৬ SSH ও নিরাপদ অ্যাক্সেস

SSH (Secure Shell) হলো সার্ভারে নিরাপদ দূরবর্তী অ্যাক্সেসের মান। সঠিক পদ্ধতি: **SSH কী-জোড়া** (public/private key) — প্রাইভেট কী আপনার কম্পিউটারে, পাবলিক কী সার্ভারে। পাসওয়ার্ড-ভিত্তিক লগইন বন্ধ করুন; পাসওয়ার্ড অনুমান (brute-force) আক্রমণের সবচেয়ে সহজ লক্ষ্য। প্রাইভেট কী কখনো শেয়ার করবেন না, কখনো Git-এ কমিট করবেন না, পাসফ্রেজ (passphrase) দিয়ে সুরক্ষিত রাখুন। বাস্টিয়ন হোস্ট দিয়ে প্রাইভেট সার্ভারে যান; সরাসরি পাবলিক IP দিয়ে প্রতিটি সার্ভারে SSH খোলা রাখবেন না।

```
$ ssh-keygen -t ed25519 -C "nahid@technova.example"
$ ibmcloud is ssh-key-create technova-key \
    --public-key @~/.ssh/id_ed25519.pub
$ ssh -i ~/.ssh/id_ed25519 root@203.0.113.10
# সার্ভারে: পাসওয়ার্ড লগইন বন্ধ আছে কি না যাচাই করুন
$ sudo grep PasswordAuthentication /etc/ssh/sshd_config
```

অ্যাক্সেস-পরিচালনা: প্রত্যেকের আলাদা কী ও ব্যবহারকারী-অ্যাকাউন্ট; দল ছাড়লে কী মুছে ফেলুন; root দিয়ে সরাসরি লগইন বন্ধ করে sudo ব্যবহার করুন; SSH লগ পর্যবেক্ষণ করুন।

৯.৭ ইনস্ট্যান্স জীবনচক্র, পর্যবেক্ষণ ও সমস্যা সমাধান

জীবনচক্র: তৈরি (create) → চালু (running) → থামানো (stop) → পুনরায় চালু (restart) → আকার বদল (resize — সাধারণত বন্ধ করে) → মুছে ফেলা (delete)। মনে রাখুন: থামানো সার্ভারের ভলিউম ও IP-র জন্য চার্জ চলতে পারে; মুছে ফেলার আগে স্ন্যাপশট ও ডেটা-ব্যাকআপ নিশ্চিত করুন; মুছে ফেলা সাধারণত ফেরানো যায় না। পরিবর্তন-পরিচালনা: production সার্ভারে সরাসরি হাতে বদল (manual change) করবেন না — ইমেজ বা IaC দিয়ে পুনর্নির্মাণ করুন (immutable infrastructure)।

পর্যবেক্ষণ: CPU ব্যবহার ৮০%-এর ওপরে দীর্ঘক্ষণ থাকলে আকার বাড়ান বা স্কেল-আউট করুন; মেমরি পূর্ণ হলে অ্যাপ ধীর বা বন্ধ হয় — মেমরি-লিক খুঁজুন; ডিস্ক ৮৫% পূর্ণ হলে সতর্কতা; নেটওয়ার্ক-ত্রুটি বাড়লে সিকিউরিটি গ্রুপ ও রুট দেখুন। IBM Cloud মনিটরিং ও লগ-পরিষেবা ব্যবহার করুন; সার্ভারের ভেতরের এজেন্ট (যেমন মেট্রিক-সংগ্রাহক) বসান।

> **WARNING:** সার্ভারে সরাসরি গুরুত্বপূর্ণ ডেটা রাখা — যার কোনো ব্যাকআপ নেই — সবচেয়ে বিপজ্জনক অভ্যাস। VSI মুছে গেলে, হোস্ট নষ্ট হলে বা ডিস্ক ক্ষতিগ্রস্ত হলে ডেটা হারিয়ে যাবে। ডেটা সবসময় ব্যাকআপ-নীতিসহ ভলিউমে বা পরিচালিত ডেটাবেসে রাখুন।

৯.৮ CLI ও API উদাহরণ

```
$ ibmcloud is instance-create technova-web-1 \
    --vpc technova-prod-vpc \
    --zone us-south-1 \
    --profile bx2-2x8 \
    --image ibm-ubuntu-22-04-minimal-amd64-1 \
    --subnet technova-web-zone1 \
    --keys technova-key
$ ibmcloud is instances
$ ibmcloud is instance technova-web-1
$ ibmcloud is instance-stop technova-web-1
$ ibmcloud is instance-start technova-web-1
```

```
$ ibmcloud is volume-create technova-data-vol \
    --profile 10iops-tier --size 100 --zone us-south-1
$ ibmcloud is instance-volume-attach technova-web-1 \
    --volume technova-data-vol
```

প্রোফাইল-নাম ও ইমেজ-নাম অঞ্চলভেদে বদলায় — তৈরি করার আগে `ibmcloud is instance-profiles` ও `ibmcloud is images` দিয়ে তালিকা দেখুন। উদাহরণের নামগুলো ধারণাগত।

৯.৯ IaC উদাহরণ

```
resource "ibm_is_instance" "web1" {
  name    = "technova-web-1"
  vpc     = ibm_is_vpc.technova_prod.id
  zone    = "us-south-1"
  profile = "bx2-2x8"
  image   = data.ibm_is_image.ubuntu.id
  keys    = [ibm_is_ssh_key.technova.id]

  primary_network_interface {
    subnet = ibm_is_subnet.web_zone1.id
  }
}

resource "ibm_is_volume" "data" {
  name    = "technova-data-vol"
  profile = "10iops-tier"
  size    = 100
  zone    = "us-south-1"
}
```

৯.১০ উৎপাদন পরিস্থিতি: ধীর হয়ে যাওয়া সার্ভার

TechNova-র অ্যাপ সার্ভার হঠাৎ ধীর হয়ে যায়। পর্যবেক্ষণে দেখা যায় CPU ৯৫%, কিন্তু অ্যাপ-লগে অস্বাভাবিক কিছু নেই। তদন্তে দেখা যায় একটি নির্ধারিত কাজ (cron job) প্রতি ঘণ্টায় পুরো ডেটাবেস স্ক্যান করছিল। সমাধান: কাজটি রাতে সরানো ও ইনডেক্স যোগ করা — সার্ভারের আকার বাড়ানো নয়। শিক্ষা: প্রথমে কারণ খুঁজুন, তারপর আকার বাড়ান; আকার বাড়ানো লক্ষণ ঢাকে, রোগ সারায় না।

> **REAL WORLD:** একটি দল production সার্ভারে হাতে প্যাচ লাগিয়ে "ঠিক" করেছিল, কিন্তু ইমেজ আপডেট করেনি। পরের বার সার্ভার পুনর্নির্মাণ হলে পুরোনো দুর্বলতা ফিরে আসে। সার্ভারকে গবাদি পশুর মতো নয়, গৃহপালিত প্রাণীর মতো নয় — বরং প্রতিস্থাপনযোগ্য হিসেবে ভাবুন: বদলালে নতুন, সঠিক সার্ভার আসবে।

৯.১১ নিরাপত্তা ও ব্যয় বিবেচনা

নিরাপত্তা: OS প্যাচ নিয়মিত, অপ্রয়োজনীয় পরিষেবা বন্ধ, ফায়ারওয়াল (OS-স্তর) চালু, SSH কী-ভিত্তিক, লগ সংগ্রহ, দুর্বলতা-স্ক্যান। ব্যয়: প্রোফাইল-আকার, চলমান সময়, ভলিউম-আকার ও IOPS-স্তর — non-prod সার্ভার রাতে বন্ধ রাখুন, অব্যবহৃত ভলিউম মুছুন, অধিক-আকারের প্রোফাইল ছোট করুন। পর্যবেক্ষণ-সতর্কতা ছাড়া ব্যয় নিয়ন্ত্রণ অসম্ভব — প্রতি সপ্তাহে বিল পর্যালোচনা করুন।

৯.১২ সাধারণ ভুল

- পাসওয়ার্ড-ভিত্তিক SSH চালু রাখা।
- প্রাইভেট কী শেয়ার করা বা Git-এ কমিট করা।
- বুট ভলিউমে ডেটা রাখা ও ব্যাকআপ না নেওয়া।
- অতিরিক্ত বড় প্রোফাইল নিয়ে অর্থ নষ্ট করা, বা অতিরিক্ত ছোট নিয়ে সেবা ধীর করা।
- production সার্ভারে হাতে বদল করা (drift তৈরি করা)।
- পর্যবেক্ষণ ছাড়া সার্ভার চালানো — সমস্যা গ্রাহক জানানোর পর ধরা পড়া।

৯.১৩ সমস্যা সমাধান

SSH হচ্ছে না? যাচাই করুন: (১) সার্ভার চলছে কি না; (২) সিকিউরিটি গ্রুপে ২২ পোর্ট আপনার IP থেকে খোলা কি না; (৩) সঠিক কী ও ব্যবহারকারী-নাম ব্যবহার করছেন কি না; (৪) সার্ভারের ডিস্ক পূর্ণ হয়ে OS আটকে গেছে কি না (কনসোল-লগ দেখুন)। সার্ভার ধীর? CPU, মেমরি, ডিস্ক I/O ও নেটওয়ার্ক — চারটি আলাদা করে দেখুন; একটিতে চাপ মানে সেই দিকের সমাধান। সার্ভার চালু হচ্ছে না? বুট-ভলিউম ও ইমেজ যাচাই করুন; প্রয়োজনে ভলিউম আলাদা করে অন্য সার্ভারে যুক্ত করে ডেটা উদ্ধার করুন।

> **TROUBLESHOOTING:** "ডিস্ক পূর্ণ" ত্রুটি দেখলে শুধু ফাইল মুছবেন না — কোন ফাইল বাড়ছে তা খুঁজুন (লগ-ঘূর্ণন / log rotation বন্ধ থাকলে লগ ফাইলই ডিস্ক ভরে ফেলে)। লগ-ঘূর্ণন চালু করুন, তারপর জায়গা পরিষ্কার করুন।

৯.১৪ গভীরে: ব্যর্থতা, পর্যবেক্ষণ, বিলিং ও বিকল্প

ভার্চুয়াল সার্ভারের প্রতিটি উপাদানকে বারোটি প্রশ্নে যাচাই করুন। সমস্যা: দ্রুত, নমনীয় কম্পিউট দরকার — ভৌত সার্ভার ধীর ও অগ্রিম-খরচের। ভেতরে: হাইপারভাইজার CPU-সময়, মেমরি ও নেটওয়ার্ক ভাগ করে; বুট ভলিউম থেকে OS ওঠে; ডেটা ভলিউম নেটওয়ার্ক-সংযুক্ত ব্লক ডিস্ক হিসেবে যুক্ত হয়। অন্যান্য পরিষেবার সঙ্গে যোগাযোগ: VSI VPC-র সাবনেট ও সিকিউরিটি গ্রুপের ভেতরে চলে, লোড ব্যালান্সারের পুল-সদস্য হয়, ব্লক স্টোরেজ ও ব্যাকআপ-পরিষেবার সঙ্গে যুক্ত থাকে। ব্যর্থ হলে: হোস্ট-বিভ্রাটে VSI বন্ধ হতে পারে; ডিস্ক-ক্ষতিতে ডেটা হারাতে পারে; OS-ত্রুটিতে সার্ভার বুট নাও হতে পারে — প্রতিটির প্রতিকার আলাদা: বহু-জোন, স্ন্যাপশট, ইমেজ-পুনর্নির্মাণ। নিরাপত্তা: OS প্যাচ, কী-ভিত্তিক SSH, অপ্রয়োজনীয় পরিষেবা বন্ধ, লগ-সংগ্রহ, দুর্বলতা-স্ক্যান। পর্যবেক্ষণ: CPU, মেমরি, ডিস্ক I/O, নেটওয়ার্ক, ডিস্ক-ব্যবহার — চারটি মেট্রিক ও সতর্কতা-সীমা। বিলিং: প্রোফাইল-আকার, চলমান সময়, ভলিউম-আকার ও IOPS-স্তর; থামানো সার্ভারের ভলিউমের চার্জ যাচাই করুন; অব্যবহৃত ভলিউম মুছুন। কখন ব্যবহার করবেন: পূর্ণ নিয়ন্ত্রণ, কাস্টম সফটওয়্যার, দীর্ঘস্থায়ী পরিষেবা। কখন এড়াবেন: পরিচালিত পরিষেবা দিয়ে কাজ হলে, বা খুব অল্পস্থায়ী কাজে সার্ভারলেস ভালো। বিকল্প: কনটেইনার/Kubernetes, সার্ভারলেস ফাংশন, পরিচালিত ডেটাবেস — প্রতিটি ভিন্ন দায়িত্ব-ভাগ দেয়। আপস: বড় প্রোফাইল মানে নিরাপত্তা-মার্জিন, কিন্তু অর্থ নষ্ট; ছোট প্রোফাইল মানে সাশ্রয়, কিন্তু চাপের সময় ধীরগতি — পর্যবেক্ষণ দেখে সিদ্ধান্ত নিন, অনুমানে নয়।

৯.১৫ সারসংক্ষেপ

ভার্চুয়াল সার্ভার হলো আপনার অ্যাপের কর্মক্ষেত্র: সঠিক প্রোফাইল, পরিষ্কার ইমেজ, আলাদা ডেটা ভলিউম, কী-ভিত্তিক SSH ও নিয়মিত পর্যবেক্ষণ — এই পাঁচটি মিলে একটি নির্ভরযোগ্য সার্ভার তৈরি করে। সার্ভারকে প্রতিস্থাপনযোগ্য ভাবুন, ডেটাকে ব্যাকআপ-সহ ভাবুন, আর প্রতিটি পরিবর্তন কোড হিসেবে রাখুন।

৯.১৬ পর্যালোচনা প্রশ্ন

1. ইনস্ট্যান্স প্রোফাইল কী নির্ধারণ করে এবং ভুল প্রোফাইল নির্বাচনের পরিণতি কী?
2. বুট ভলিউম ও ডেটা ভলিউম আলাদা রাখা কেন গুরুত্বপূর্ণ?
3. SSH কী-ভিত্তিক প্রমাণীকরণ পাসওয়ার্ডের চেয়ে নিরাপদ কেন?
4. সার্ভার ধীর হলে CPU, মেমরি, ডিস্ক ও নেটওয়ার্ক কীভাবে আলাদা করে পরীক্ষা করবেন?
5. Immutable infrastructure বলতে কী বোঝায়?

৯.১৭ ব্যবহারিক অনুশীলন

1. একটি পরীক্ষামূলক VSI তৈরি করুন (ছোট প্রোফাইল, Ubuntu ইমেজ, SSH কী), তাতে একটি সাধারণ ওয়েব সার্ভার বসান, সিকিউরিটি গ্রুপে শুধু প্রয়োজনীয় পোর্ট খুলুন, তারপর সার্ভারটি থামিয়ে আবার চালু করে IP ও ডেটা অক্ষত আছে কি না যাচাই করুন।
2. একটি ডেটা ভলিউম তৈরি করে সার্ভারে যুক্ত করুন, তাতে কিছু ফাইল রাখুন, ভলিউমের স্ন্যাপশট নিন, এবং স্ন্যাপশট থেকে ভলিউম পুনরুদ্ধারের ধাপগুলো নথিভুক্ত করুন।

অধ্যায় ১০ • PART II — Networking & Infrastructure

Chapter 10 — Hybrid Connectivity

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক হাইব্রিড সংযোগ (hybrid connectivity) কেন দরকার তা ব্যাখ্যা করতে পারবেন; সাইট-টু-সাইট VPN (Site-to-Site VPN) ও IBM Cloud Direct Link-এর ধারণা, পার্থক্য ও নির্বাচনের মানদণ্ড বুঝতে পারবেন; হাইব্রিড রাউটিং ও DNS একীকরণ নকশা করতে পারবেন; এবং সংযোগ-ব্যর্থতা নির্ণয় ও নিরাপত্তা-বিবেচনা প্রয়োগ করতে পারবেন।

- VPN বনাম Direct Link — কখন কোনটি উপযুক্ত, যুক্তিসহ নির্বাচন করা।
- অন-প্রিমিসিস ও ক্লাউডের মধ্যে রাউট-বিনিময় ও CIDR-সংঘাত এড়ানো।
- হাইব্রিড DNS নকশা করা যাতে নাম উভয় পরিবেশে সঠিকভাবে মেলে।
- সংযোগ-ব্যর্থতার কারণ ধাপে ধাপে খুঁজে বের করা।

১০.২ প্রারম্ভিক পুনরালোচনা

আগের অধ্যায়গুলোতে আমরা একটি VPC-র ভেতরের নেটওয়ার্ক শিখেছি। এখন প্রশ্ন: TechNova-র অফিসের নেটওয়ার্ক (যেখানে কর্মীরা বসেন ও পুরোনো সার্ভার আছে) ও IBM Cloud VPC কীভাবে একসঙ্গে কাজ করবে? দুটি পৃথক নেটওয়ার্ককে সংযুক্ত করাই হাইব্রিড সংযোগ। ঝালিয়ে নিন: CIDR ও রাউটিং (অধ্যায় ৬–৭), DNS (অধ্যায় ৮), এবং এনক্রিপশনের ধারণা — VPN মূলত ইন্টারনেটের ওপর দিয়ে এনক্রিপ্ট করা সুড়ঙ্গ (tunnel)।

১০.৩ প্রথম নীতি থেকে হাইব্রিড: সমস্যাটি কী?

কোনো কোম্পানিই একদিনে পুরোপুরি ক্লাউডে যায় না। TechNova-র পুরোনো হিসাব-সফটওয়্যার অফিসের সার্ভারে চলে, নতুন ওয়েব অ্যাপ ক্লাউডে। দুটিকে কথা বলতে হবে — কিন্তু অফিসের সার্ভারকে ইন্টারনেটে খুলে দিলে নিরাপত্তা-ঝুঁকি, আর ক্লাউডের ডেটাবেসকে পাবলিক করলে আরও বড় ঝুঁকি। প্রয়োজন একটি ব্যক্তিগত সেতু: দুটি নেটওয়ার্কের মধ্যে এনক্রিপ্ট করা, নিয়ন্ত্রিত পথ, যেখানে শুধু অনুমোদিত ট্রাফিক যায়। এই সেতুই হাইব্রিড সংযোগ — VPN বা Direct Link, দুটিই এই সমস্যার ভিন্ন ভিন্ন সমাধান।

> **CONCEPT:** হাইব্রিড মানে দুটি নেটওয়ার্ককে একটিতে মেলানো নয়; মানে দুটি নেটওয়ার্কের মধ্যে একটি নিয়ন্ত্রিত, পর্যবেক্ষণযোগ্য সেতু তৈরি করা। সেতুর দুই প্রান্তের ঠিকানা-পরিসর, রাউট ও নিরাপত্তা-নিয়ম — তিনটিই পরিকল্পনা করে মেলাতে হয়।

১০.৪ সাইট-টু-সাইট VPN: ইন্টারনেটের ওপর এনক্রিপ্ট করা সুড়ঙ্গ

সাইট-টু-সাইট VPN দুটি নেটওয়ার্কের মধ্যে ইন্টারনেট দিয়ে IPsec সুড়ঙ্গ তৈরি করে। IPsec প্রোটোকল ট্রাফিককে এনক্রিপ্ট করে ও প্রমাণীকরণ করে — পথে কেউ পড়তে পারে না, বদলাতে পারে না। VPN গেটওয়ে VPC-র প্রান্তে থাকে, আরেকটি গেটওয়ে অফিসের প্রান্তে (অফিসের রাউটার বা ফায়ারওয়াল)। দুই প্রান্ত "আগ্রহজনক ট্রাফিক" (interesting traffic) নির্ধারণ করে — যেমন অফিসের 192.168.0.0/16 থেকে VPC-র 10.10.0.0/16 — শুধু সেই ট্রাফিক সুড়ঙ্গে যায়।

সমস্যা ও সমাধান: দ্রুত, কম খরচে সংযোগ দরকার, কিন্তু পাবলিক ইন্টারনেট নিরাপদ নয় — VPN এনক্রিপশন দিয়ে সেই দ্বন্দ্ব মেটায়। অভ্যন্তরে: IKE (Internet Key Exchange) প্রথমে দুই প্রান্তের পরিচয় যাচাই করে ও কী বিনিময় করে, তারপর

ESP (Encapsulating Security Payload) প্রতিটি প্যাকেট এনক্রিপ্ট করে। ব্যর্থ হলে কী হয়? ইন্টারনেট-বিভ্রাটে সুড়ঙ্গ ভেঙে যায়; সাধারণত স্বয়ংক্রিয়ভাবে পুনঃসংযোগ হয়, কিন্তু সেই সময় ট্রাফিক থামে। উচ্চ-প্রাপ্যতার জন্য দুটি সুড়ঙ্গ (ভিন্ন পথে) রাখা যায়। নিরাপত্তা: প্রি-শেয়ার্ড কী (pre-shared key) শক্তিশালী ও গোপন রাখুন — এটি CHANGE-ME-... ধরনের দুর্বল মান দিয়ে কখনো চালাবেন না; কী নিয়মিত ঘোরান (rotate)। পর্যবেক্ষণ: সুড়ঙ্গের অবস্থা (up/down), থ্রুপুট, প্যাকেট-ক্ষতি। বিলিং: VPN গেটওয়ে ও ডেটা স্থানান্তরের জন্য চার্জ হতে পারে — যাচাই করুন। কখন ব্যবহার করবেন: দ্রুত সংযোগ, অস্থায়ী সংযোগ, কম-মাঝারি ট্রাফিক, ব্যাকআপ-পথ হিসেবে। কখন এড়াবেন: খুব বেশি থ্রুপুট, খুব কম বিলম্ব বা কঠোর সেবা-স্তর (SLA) দরকার হলে। বিকল্প: Direct Link, বা সম্পূর্ণ মাইগ্রেশন।

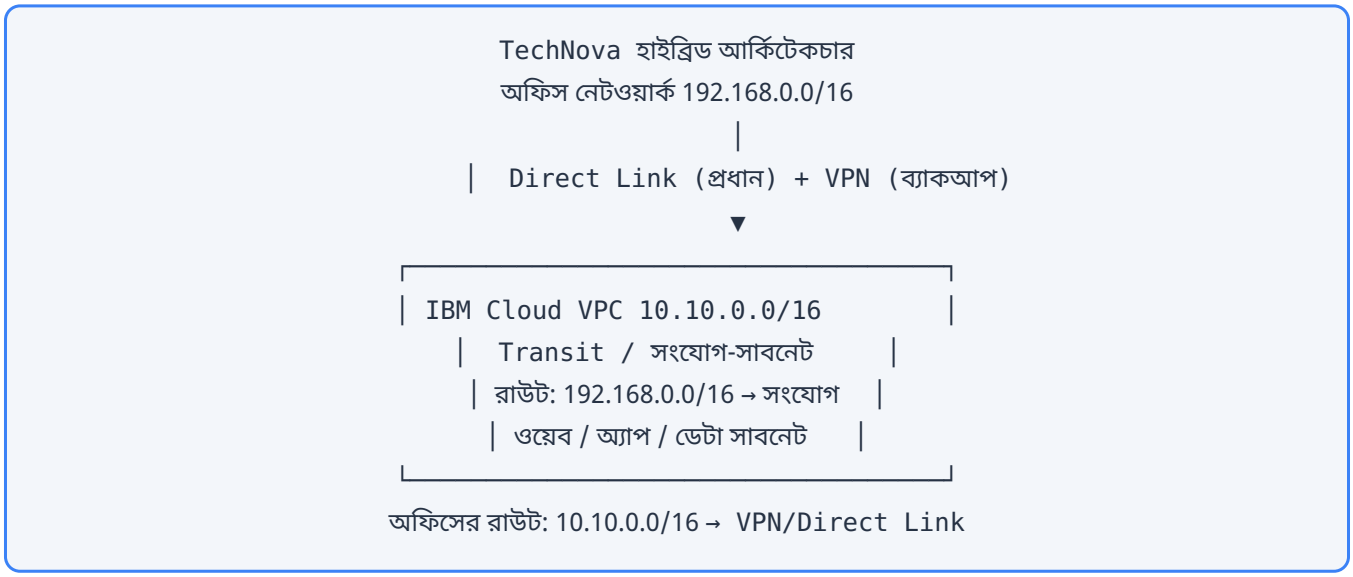
১০.৫ IBM Cloud Direct Link: নিবেদিত সংযোগ

IBM Cloud Direct Link হলো IBM Cloud-এর নিবেদিত (dedicated) সংযোগ পরিষেবা — AWS Direct Connect-এর সমতুল্য ধারণা। পাবলিক ইন্টারনেট না ঘুরে, একটি ব্যক্তিগত, নিবেদিত পথে আপনার নেটওয়ার্ক IBM Cloud-এর সঙ্গে যুক্ত হয় — সাধারণত একটি সংযোগ-প্রদানকারীর (provider) মাধ্যমে বা সরাসরি ক্রস-কানেক্ট দিয়ে। ফল: আরও স্থিতিশীল বিলম্ব, আরও বেশি থ্রুপুট, এবং ট্রাফিক ইন্টারনেটে প্রকাশ পায় না।

Direct Link-এর ধরন: নিবেদিত সংযোগ (dedicated — একটি ভৌত পোর্ট শুধু আপনার) ও ভাগ করা সংযোগ (connect — প্রদানকারীর মাধ্যমে ভাগ করা অবকাঠামো)। নিবেদিত সংযোগে স্থাপনে সময় ও খরচ বেশি, কিন্তু ক্ষমতা ও নিয়ন্ত্রণ বেশি। ব্যর্থ হলে কী হয়? ভৌত পথে সমস্যা হলে সংযোগ থামে — তাই গুরুত্বপূর্ণ কাজে দুটি ভিন্ন পথে দুটি সংযোগ (redundant Direct Link) রাখা হয়, অথবা Direct Link-এর পাশাপাশি VPN ব্যাকআপ হিসেবে। নিরাপত্তা: ব্যক্তিগত পথ মানেই এনক্রিপশন অপ্রয়োজনীয় নয় — সংবেদনশীল ডেটার জন্য Direct Link-এর ওপরেও IPsec বিবেচনা করুন। পর্যবেক্ষণ: সংযোগের অবস্থা, ব্যবহারের হার, ত্রুটি। বিলিং: পোর্ট-ক্ষমতা ও ডেটা স্থানান্তরের ভিত্তিতে চার্জ — মূল্য যাচাই করুন; Direct Link সাধারণত VPN-এর চেয়ে ব্যয়বহুল, কিন্তু বড় ট্রাফিকে প্রতি-ইউনিট খরচ কমতে পারে। কখন ব্যবহার করবেন: স্থায়ী, বেশি-ট্রাফিক, কম-বিলম্বের সংযোগ। কখন এড়াবেন: অস্থায়ী বা কম-ট্রাফিকের কাজে — সেখানে VPN যথেষ্ট।

বৈশিষ্ট্য	সাইট-টু-সাইট VPN	Direct Link
পথ	পাবলিক ইন্টারনেট (এনক্রিপ্ট করা)	নিবেদিত ব্যক্তিগত পথ
স্থাপনের সময়	ঘণ্টা–দিন	সপ্তাহ (ভৌত সংযোগ)
বিলম্ব/স্থিতি	পরিবর্তনশীল	স্থিতিশীল, কম
থ্রুপুট	মাঝারি	উচ্চ
খরচ	কম (শুরুতে)	বেশি (পোর্ট + স্থাপন)
উপযুক্ত	দ্রুত/অস্থায়ী/ব্যাকআপ	স্থায়ী/বেশি-ট্রাফিক

১০.৬ হাইব্রিড নেটওয়ার্কিং ও রাউটিং



রাউটিংয়ের নিয়ম: দুই প্রান্তেই পরস্পরের CIDR-এর দিকে রুট থাকতে হবে — একদিকে রুট থাকলেই সংযোগ হয় না, উত্তর-প্যাকেটের রুটও দরকার। CIDR যেন না মেলে — অফিস 192.168.0.0/16 ব্যবহার করলে VPC-তে 10.10.0.0/16 নিরাপদ; দুই পাশে একই পরিসর থাকলে রাউটার বুঝতে পারে না প্যাকেট কোথায় যাবে। বড় পরিবেশে একাধিক VPC থাকলে Transit Gateway ধরনের কেন্দ্রীয় সংযোগ-স্তর বিবেচনা করুন — প্রতিটি VPC-কে আলাদা করে অফিসের সঙ্গে যুক্ত করার চেয়ে একটি কেন্দ্রীয় হাব পরিচালনা সহজ।

BGP (Border Gateway Protocol) অনেক Direct Link ও উন্নত VPN সেটআপে রুট-বিনিময় স্বয়ংক্রিয় করে: এক পাশে রুট বদলালে অন্য পাশে জানানো হয়। BGP ব্যবহার করলে হাতে রুট লেখার ভুল কমে, কিন্তু BGP-র ভুল কনফিগারেশন ভুল রুট ছড়িয়ে দিতে পারে — পরিবর্তন সাবধানে করুন।

১০.৭ DNS একীকরণ

হাইব্রিড DNS-এর সমস্যা: অফিসের সার্ভারের নাম (যেমন accounts.technova.internal) ক্লাউড থেকে মেলাতে হবে, আবার ক্লাউডের সার্ভারের নাম অফিস থেকে মেলাতে হবে। সমাধান: DNS ফরওয়ার্ডিং — প্রতিটি পরিবেশের DNS সার্ভার অন্য পরিবেশের নামের অনুরোধ ফরওয়ার্ড করে। ব্যক্তিগত DNS জোন (private DNS zone) VPC-র ভেতরে নাম মেলায়; অফিসের DNS সেই জোনের দিকে ফরওয়ার্ড করে। সতর্কতা: একই নাম দুই জায়গায় ভিন্ন IP দিলে বিভ্রান্তি হয় — নাম-স্থান (namespace) পরিকল্পনা করুন: যেমন .internal অফিসের জন্য, .cloud.technova.example ক্লাউডের জন্য, অথবা একটি একীভূত ব্যক্তিগত জোন।

১০.৮ CLI ও API উদাহরণ

```
$ ibmcloud is vpn-gateway-create technova-vpn-gw \
    --vpc technova-prod-vpc \
    --subnet technova-connect-zone1
$ ibmcloud is vpn-gateway-connection-create technova-vpn-gw \
    --peer-address 203.0.113.50 \
    --preshared-key CHANGE-ME-STRONG-KEY \
    --local-cidrs 10.10.0.0/16 \
    --peer-cidrs 192.168.0.0/16
```

```
$ ibmcloud is vpn-gateways
$ ibmcloud is vpn-gateway-connections technova-vpn-gw
```

কমান্ডগুলো ধারণাগত; সঠিক ফ্ল্যাগ `--help` দিয়ে যাচাই করুন। প্রি-শেয়ার্ড কী কখনো কমান্ড-ইতিহাসে বা Git-এ রাখবেন না — নিরাপদ গোপন-ভাণ্ডার (secrets manager) ব্যবহার করুন। Direct Link স্থাপনে সাধারণত কনসোল-অনুরোধ ও প্রদানকারীর সঙ্গে সমন্বয় দরকার — শুধু CLI-তে হয় না।

১০.৯ IaC উদাহরণ

```
resource "ibm_is_vpn_gateway" "technova_vpn" {
  name   = "technova-vpn-gw"
  subnet = ibm_is_subnet.connect_zone1.id
}

resource "ibm_is_vpn_gateway_connection" "office" {
  name           = "technova-office-conn"
  vpn_gateway    = ibm_is_vpn_gateway.technova_vpn.id
  peer_address   = "203.0.113.50"
  preshared_key  = var.vpn_preshared_key
  local_cidrs    = ["10.10.0.0/16"]
  peer_cidrs     = ["192.168.0.0/16"]
}
```

`var.vpn_preshared_key` ভেরিয়েবলটি Terraform-এ প্লেইন লিখবেন না — পরিবেশ-ভেরিয়েবল বা গোপন-ভাণ্ডার থেকে নিন।

১০.১০ উৎপাদন পরিস্থিতি: মাইগ্রেশনের সময় দুই পরিবেশ

TechNova ধাপে ধাপে মাইগ্রেশন করছে: প্রথমে VPN দিয়ে অফিস ও ক্লাউড যুক্ত করা হয় — দ্রুত চালু হয়, দল পরীক্ষা করে। ট্রাফিক বাড়লে Direct Link স্থাপন করা হয়, VPN ব্যাকআপ হিসেবে থাকে। মাইগ্রেশন শেষে পুরোনো সার্ভার বন্ধ করা হয়, কিন্তু Direct Link কিছুদিন রাখা হয় — কারণ কিছু পুরোনো ক্লায়েন্ট তখনো অফিস-নেটওয়ার্ক দিয়ে আসে। শিক্ষা: হাইব্রিড সংযোগ একটি স্থায়ী কৌশল হতে পারে, আবার মাইগ্রেশনের সেতুও হতে পারে — উদ্দেশ্য স্পষ্ট করে পরিকল্পনা করুন।

> **REAL WORLD:** একটি কোম্পানি Direct Link স্থাপন করে VPN বন্ধ করে দিয়েছিল — কিন্তু Direct Link-এর রক্ষণাবেক্ষণের দিন সংযোগ থেমে যায়, কোনো ব্যাকআপ ছিল না। প্রধান ও ব্যাকআপ পথ — দুটোই রাখা ভালো অভ্যাস; ব্যাকআপ পথের খরচ, বিভ্রাটের খরচের চেয়ে কম।

১০.১১ নিরাপত্তা ও ব্যয় বিবেচনা

নিরাপত্তা: সুড়ঙ্গের দুই প্রান্তেই ফায়ারওয়াল-নিয়ম — অফিস থেকে শুধু নির্দিষ্ট সার্ভার ও পোর্টে সংযোগ; ক্লাউডের সিকিউরিটি গ্রুপে অফিসের CIDR সংকীর্ণভাবে লিখুন (পুরো অফিস-পরিসর নয়, প্রয়োজনীয় সাবনেট); কী ঘূর্ণন; সংযোগের লগ ও সতর্কতা। ব্যয়: VPN গেটওয়ে, Direct Link পোর্ট, ডেটা স্থানান্তর — তিনটি মিলে খরচ; ট্রাফিকের পরিমাণ মেপে সিদ্ধান্ত নিন। পর্যবেক্ষণ: সুড়ঙ্গ/সংযোগের অবস্থা, থ্রুপুট, বিলম্ব, প্যাকেট-ক্ষতি — কোনোটি বদলালে তদন্ত করুন।

১০.১২ সাধারণ ভুল

- CIDR মিলে যাওয়া (overlap) — সংযোগের আগেই পরিকল্পনা না করা।
- একদিকে রুট লিখে অন্যদিকে ভুলে যাওয়া।
- দুর্বল প্রি-শেয়ার্ড কী ব্যবহার করা বা কী শেয়ার করা।
- ব্যাকআপ পথ না রাখা।
- DNS একীকরণ ভুলে যাওয়া — IP দিয়ে কাজ করে, নাম দিয়ে করে না।
- ফায়ারওয়ালে পুরো অফিস-পরিসরকে সব পোর্টে অনুমোদন দেওয়া।

১০.১৩ সমস্যা সমাধান

সংযোগ হচ্ছে না? ধাপে ধাপে: (১) সুড়ঙ্গ/সংযোগের অবস্থা (up কি না); (২) দুই প্রান্তের রুট — স্থানীয় ও দূরবর্তী CIDR; (৩) ফায়ারওয়াল/সিকিউরিটি গ্রুপের নিয়ম; (৪) প্রি-শেয়ার্ড কী ও IPsec-পরামিতি মিলছে কি না; (৫) DNS নাম মেলাচ্ছে কি না। VPN-এ একটি সাধারণ সমস্যা: সুড়ঙ্গ up, কিন্তু ট্রাফিক যায় না — সাধারণত "আগ্রহজনক ট্রাফিক" (local/peer CIDR) ভুল লেখা। Direct Link-এ: ভৌত সংযোগের অবস্থা ও BGP রুট-বিনিময় দেখুন।

TROUBLESHOOTING: সংযোগ মাঝে মাঝে ভেঙে যায়? সম্ভবত ইন্টারনেট-পথের অস্থিরতা (VPN) বা একটি পথের সমস্যা (Direct Link)। দুটি সুড়ঙ্গ/পথের লগ তুলনা করুন — সমস্যা কি একটি পথে সীমাবদ্ধ, নাকি দুটোতেই? একটি পথে সীমাবদ্ধ হলে সেই পথের প্রদানকারী বা যন্ত্র দেখুন।

WARNING: হাইব্রিড সংযোগ মানে অফিসের নিরাপত্তা-দুর্বলতা ক্লাউডে পৌঁছাতে পারে — এবং উল্টোটাও। সংযোগ চালু করার আগে উভয় প্রান্তের ফায়ারওয়াল-নিয়ম পর্যালোচনা করুন; "সবকিছু অনুমোদন" দিয়ে শুরু করে পরে সংকীর্ণ করা বিপজ্জনক অভ্যাস।

১০.১৪ গভীরে: ব্যর্থতা, পর্যবেক্ষণ, বিলিং ও বিকল্প

VPN ও Direct Link — দুটিকেই বারোটি প্রশ্নে যাচাই করুন। সমস্যা: দুটি নেটওয়ার্ককে নিরাপদে যুক্ত করা — ইন্টারনেটে খুলে না দিয়ে। ভেতরে: VPN-এ IKE কী বিনিময় করে, ESP প্যাকেট এনক্রিপ্ট করে; Direct Link-এ নিবেদিত ভৌত পথে BGP রুট-বিনিময় হয়। অন্যান্য পরিষেবার সঙ্গে যোগাযোগ: হাইব্রিড সংযোগ VPC-র রাউটিং টেবিল, সিকিউরিটি গ্রুপ ও DNS ফরওয়ার্ডিংয়ের সঙ্গে যুক্ত; Transit Gateway থাকলে একাধিক VPC একই সেতু ব্যবহার করে। ব্যর্থ হলে: VPN-এ ইন্টারনেট-বিভ্রাটে সুড়ঙ্গ ভাঙে, সাধারণত স্বয়ংক্রিয়ভাবে ফেরে; Direct Link-এ ভৌত পথের সমস্যায় সংযোগ থামে — দুটি পথ না থাকলে সেবা থামে। নিরাপত্তা: শক্তিশালী প্রি-শেয়ার্ড কী ও নিয়মিত ঘূর্ণন, উভয় প্রান্তে সংকীর্ণ ফায়ারওয়াল-নিয়ম, সংবেদনশীল ডেটায় Direct Link-এর ওপরেও IPsec। পর্যবেক্ষণ: সুড়ঙ্গ/সংযোগের অবস্থা, থ্রুপুট, বিলম্ব, প্যাকেট-ক্ষতি, BGP রুট-সংখ্যা। বিলিং: VPN গেটওয়ে, Direct Link পোর্ট-ক্ষমতা ও ডেটা স্থানান্তর — মূল্য যাচাই করুন; বড় ট্রাফিকে Direct Link-এর প্রতি-ইউনিট খরচ কমতে পারে। কখন ব্যবহার করবেন: VPN দ্রুত/অস্থায়ী/ব্যাকআপে, Direct Link স্থায়ী/বেশি-ট্রাফিকে। কখন এড়াবেন: সম্পূর্ণ মাইগ্রেশন সম্ভব হলে স্থায়ী সেতু বানানো অপ্রয়োজনীয়; খুব কম ট্রাফিকে Direct Link ব্যয়বহুল। বিকল্প: সম্পূর্ণ মাইগ্রেশন, বা অ্যাপ-স্তরের API সংযোগ (নেটওয়ার্ক-স্তরের সেতু ছাড়া)। আপস: VPN সস্তা ও দ্রুত, কিন্তু বিলম্ব পরিবর্তনশীল; Direct Link স্থিতিশীল, কিন্তু স্থাপনে সপ্তাহ লাগে ও খরচ বেশি — TechNova-র পথ (VPN দিয়ে শুরু, Direct Link পরে, VPN ব্যাকআপ হিসেবে) এই আপসেরই বাস্তব উদাহরণ।

১০.১৫ সারসংক্ষেপ

VPN দ্রুত ও সস্তায় হাইব্রিড সংযোগ দেয়; Direct Link স্থিতিশীল, উচ্চ-থ্রুপুটের স্থায়ী সংযোগ দেয়। দুটোতেই সাফল্যের চাবি একই: CIDR-পরিকল্পনা, উভয় দিকের রুট, সংকীর্ণ নিরাপত্তা-নিয়ম, DNS একীকরণ ও ব্যাকআপ পথ। TechNova-র মতো দলের জন্য সাধারণ পথ: VPN দিয়ে শুরু, ট্রাফিক বাড়লে Direct Link, VPN ব্যাকআপ হিসেবে রাখা।

১০.১৬ পর্যালোচনা প্রশ্ন

1. সাইট-টু-সাইট VPN কী সমস্যা সমাধান করে এবং IPsec কী ভূমিকা পালন করে?
2. Direct Link কখন VPN-এর চেয়ে ভালো পছন্দ?
3. CIDR overlap হলে হাইব্রিড সংযোগে কী সমস্যা হয়?
4. হাইব্রিড DNS একীকরণে ফরওয়ার্ডিং কেন দরকার?
5. প্রধান সংযোগের পাশাপাশি ব্যাকআপ পথ রাখা কেন গুরুত্বপূর্ণ?

১০.১৭ ব্যবহারিক অনুশীলন

1. TechNova-র অফিস (192.168.0.0/16) ও VPC (10.10.0.0/16) এর জন্য VPN সংযোগ-পরিকল্পনা লিখুন: local/peer CIDR, রুট-তালিকা (উভয় দিকে), ফায়ারওয়াল-নিয়ম ও DNS ফরওয়ার্ডিং — তারপর Terraform-এ VPN গেটওয়ে সংজ্ঞা লিখে `terraform plan` দিয়ে যাচাই করুন।
2. একটি সিদ্ধান্ত-তালিকা তৈরি করুন: কোন পরিস্থিতিতে VPN, কোনটিতে Direct Link, কোনটিতে দুটোই — ট্রাফিকের পরিমাণ, বিলম্ব-সংবেদনশীলতা, স্থাপনের সময় ও বাজেটের ভিত্তিতে।

অধ্যায় ১১ • PART II — Networking & Infrastructure

Chapter 11 — High Availability and Resilience

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক উচ্চ-প্রাপ্যতা (High Availability, HA) ও সহনশীলতা (resilience) এর ধারণা ব্যাখ্যা করতে পারবেন; বহু-জোন নকশা, অতিরিক্ততা (redundancy), অটোস্কেলিং, স্বাস্থ্য-পর্যবেক্ষণ, ব্যর্থতার ডোমেইন, ব্যাকআপ ও ধারণক্ষমতা-পরিকল্পনা (capacity planning) প্রয়োগ করতে পারবেন; এবং TechNova-র সেবার জন্য একটি বাস্তবসম্মত নির্ভরযোগ্যতা-পরিকল্পনা লিখতে পারবেন — যেখানে প্রতিটি সিদ্ধান্তের আপস (trade-off) স্পষ্ট থাকবে।

- প্রাপ্যতা (availability) শতাংশের অর্থ বোঝা ও ডাউনটাইম-বাজেট হিসাব করা।
- একক ব্যর্থতার বিন্দু (SPOF) খুঁজে বের করে তা দূর করা।
- অটোস্কেলিং-নীতি ও স্বাস্থ্য-পরীক্ষার নকশা করা।
- ব্যাকআপ, পুনরুদ্ধার ও দুর্যোগ-পুনরুদ্ধার (DR) পরিকল্পনা লেখা।

১১.২ প্রারম্ভিক পুনরালোচনা

আগের অধ্যায়গুলোতে আমরা ভিত্তি তৈরি করেছি: VPC ও জোন (অধ্যায় ৬), রাউটিং ও নিরাপত্তা (অধ্যায় ৭), লোড ব্যালান্সার ও DNS (অধ্যায় ৮), ভার্চুয়াল সার্ভার (অধ্যায় ৯), হাইব্রিড সংযোগ (অধ্যায় ১০)। এই অধ্যায়ে প্রশ্ন বদলে যায়: এই সবকিছু মিলে যখন কিছু ভাঙবে — কারণ কিছু না কিছু ভাঙবেই — তখন সেবা কীভাবে চলতে থাকবে? ঝালিয়ে নিন: জোন, লোড ব্যালান্সার, হেলথ চেক ও ব্যাকআপের ধারণা — এখানে তারা একসঙ্গে একটি সিস্টেম হয়।

১১.৩ প্রথম নীতি থেকে সহনশীলতা: ব্যর্থতা অনিবার্য

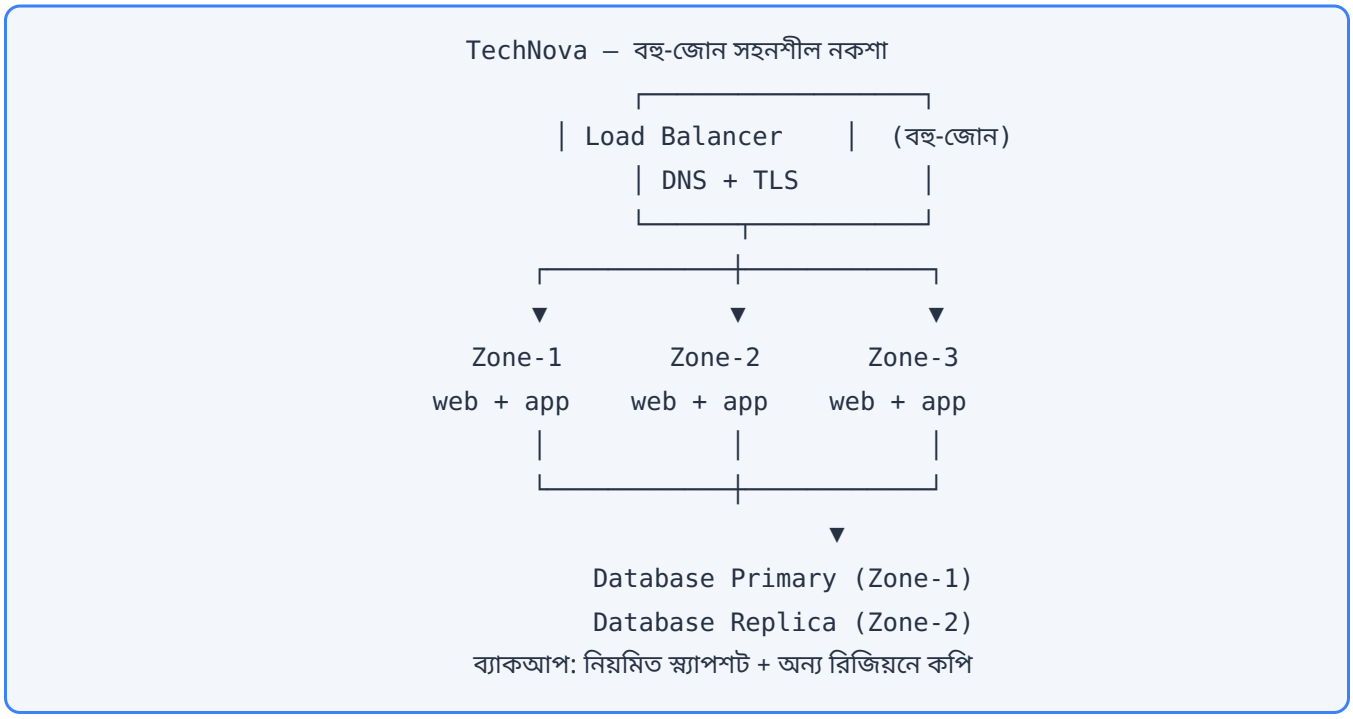
সহনশীলতার প্রথম নীতি: সবকিছু ভাঙবে — হার্ডওয়্যার নষ্ট হবে, সফটওয়্যারে বাগ থাকবে, নেটওয়ার্ক বিভ্রাট হবে, মানুষ ভুল করবে। প্রশ্ন "ভাঙবে কি না" নয়, প্রশ্ন "ভাঙলে কী হবে"। একটি সিস্টেম সহনশীল হয় যখন একটি উপাদানের ব্যর্থতা সেবার ব্যর্থতা হয় না — কারণ অন্য উপাদান দায়িত্ব নেয়, বা সেবা সীমিতভাবে চলতে থাকে (degraded mode), বা দ্রুত পুনরুদ্ধার হয়। এর বিপরীত হলো ভঙ্গুর সিস্টেম: একটি ভুল মানেই সম্পূর্ণ বন্ধ।

প্রাপ্যতা মাপা হয় শতাংশে: 99% মানে বছরে প্রায় ৩.৬৫ দিন ডাউনটাইম; 99.9% মানে প্রায় ৮.৭৬ ঘণ্টা; 99.99% মানে প্রায় ৫২.৬ মিনিট। প্রতিটি অতিরিক্ত "৯" পাওয়া ক্রমে কঠিন ও ব্যয়বহুল — কারণ তা দাবি করে আরও অতিরিক্ততা, আরও স্বয়ংক্রিয়তা, আরও পরীক্ষা। TechNova-র জন্য 99.9% একটি বাস্তবসম্মত লক্ষ্য হতে পারে; 99.99% চাইলে তার খরচ ও জটিলতা বুঝে সিদ্ধান্ত নিতে হবে। SLA (Service Level Agreement) ও SLO (Service Level Objective) — এই দুটি লক্ষ্য লিখিতভাবে ঠিক করুন, নইলে "সবসময় চালু থাকবে" একটি অর্থহীন দাবি হয়ে থাকে।

> **CONCEPT:** উচ্চ-প্রাপ্যতা মানে "কখনো ভাঙবে না" নয়; মানে "ভাঙলেও সেবা চলবে বা দ্রুত ফিরবে"। অতিরিক্ততা (redundancy) হলো এর উপায় — একই কাজের দুটি উপাদান রাখা — কিন্তু অতিরিক্ততা নিজেই জটিলতা আনে: দুটি উপাদানকে সমন্বয় করতে হয়, ডেটা মেলাতে হয়, আর ভুল সমন্বয় নিজেই ব্যর্থতার কারণ হতে পারে।

১১.৪ বহু-জোন নকশা ও ব্যর্থতার ডোমেইন

ব্যর্থতার ডোমেইন হলো এমন একটি সীমানা যার ভেতরে একটি ব্যর্থতা সবকিছুকে প্রভাবিত করে: একটি সার্ভার, একটি জোন, একটি রিজিয়ন। নকশার নিয়ম: গুরুত্বপূর্ণ সেবার উপাদানগুলোকে ভিন্ন ভিন্ন ব্যর্থতার ডোমেইনে ছড়ান। TechNova-র নকশা: ওয়েব সার্ভার তিন জোনে, অ্যাপ সার্ভার তিন জোনে, ডেটাবেসের প্রাথমিক এক জোনে ও রেপ্লিকা আরেক জোনে, লোড ব্যালান্সার বহু-জোনে। একটি জোন বন্ধ হলে দুই-তৃতীয়াংশ ক্ষমতা টিকে থাকে — সেবা ধীর হতে পারে, কিন্তু বন্ধ হয় না।



কিন্তু সতর্কতা: বহু-জোন নকশা ডেটা-স্তরে জটিলতা আনে। ডেটাবেসের রেপ্লিকা যদি পিছিয়ে থাকে (replication lag), তাহলে জোন-বদলের পর গ্রাহক পুরোনো ডেটা দেখতে পারে। স্টেটলেস অ্যাপ (অবস্থা সার্ভারে নয়, ডেটাবেস/ক্যাশে) বহু-জোনে সহজ; স্টেটফুল অ্যাপ কঠিন। তাই অ্যাপ-নকশা ও অবকাঠামো-নকশা একসঙ্গে ভাবুন।

১১.৫ অতিরিক্ততা, লোড ব্যালান্সিং ও অটোস্কেলিং

অতিরিক্ততা দুই ধরনের: সক্রিয়-সক্রিয় (active-active — সব উপাদান একসঙ্গে কাজ করে, একটি বন্ধ হলে বাকিরা ভার নেয়) ও সক্রিয়-নিষ্ক্রিয় (active-passive — একটি কাজ করে, অন্যটি অপেক্ষায় থাকে)। সক্রিয়-সক্রিয় সাধারণত ভালো — কারণ নিষ্ক্রিয় উপাদানটি আসলেই কাজ করবে কি না, তা পরীক্ষা না করলে জানা যায় না। লোড ব্যালান্সার (অধ্যায় ৮) সক্রিয়-সক্রিয়ের কেন্দ্র: সে সুস্থ সার্ভারে ট্রাফিক পাঠায়, অসুস্থকে সরায়।

অটোস্কেলিং হলো চাহিদা অনুযায়ী সার্ভার-সংখ্যা স্বয়ংক্রিয়ভাবে বাড়ানো-কমানো: CPU বা অনুরোধ-সংখ্যা একটি সীমা ছাড়ালে নতুন সার্ভার যোগ হয়, চাহিদা কমলে সার্ভার কমে। এর জন্য দরকার: (১) স্টেটলেস অ্যাপ বা বাইরে রাখা সেশন; (২) দ্রুত তৈরি হওয়া ইমেজ; (৩) হেলথ চেক যাতে নতুন সার্ভার প্রস্তুত হলেই ট্রাফিক পায়; (৪) স্কেল-ইন (কমানো) নীতি যাতে চলমান অনুরোধ শেষ না করে সার্ভার বন্ধ না হয় (graceful drain)। অটোস্কেলিংয়ের ঝুঁকি: ভুল নীতি বা আকস্মিক ট্রাফিক-বৃদ্ধিতে অতিরিক্ত সার্ভার তৈরি হয়ে খরচ বেড়ে যাওয়া; বা হেলথ চেকের ভুলে নতুন সার্ভার বারবার তৈরি ও মুছে যাওয়া (thrashing)। সীমা (min/max) নির্ধারণ করুন: সর্বনিম্ন ২ (একটি জোনের ব্যর্থতায় অন্যটি থাকে), সর্বোচ্চ বাজেট-অনুযায়ী।

১১.৬ স্বাস্থ্য-পর্যবেক্ষণ ও ব্যর্থতা-শনাক্তকরণ

স্বাস্থ্য-পর্যবেক্ষণ তিন স্তরে: অবকাঠামো (CPU, মেমরি, ডিস্ক, নেটওয়ার্ক), অ্যাপ (অনুরোধ-হার, বিলম্ব, ত্রুটি-হার), ব্যবসায়িক (সফল অর্ডার, লগইন)। শুধু CPU দেখলে অ্যাপের সমস্যা ধরা পড়ে না; শুধু অ্যাপ দেখলে ডিস্ক-পূর্ণ হওয়া ধরা পড়ে না। সতর্কতা (alert) নির্ধারণ করুন: কী ঘটলে কাকে জানানো হবে, কতক্ষণে সাড়া দিতে হবে। সতর্কতা-ক্লান্তি (alert fatigue) এড়ান — প্রতিদিন দশটি অপ্রয়োজনীয় সতর্কতা এলে আসল সতর্কতাও উপেক্ষিত হয়।

পর্যবেক্ষণের চারটি সোনালি সংকেত (golden signals): বিলম্ব (latency), ট্রাফিক (traffic), ত্রুটি (errors), সম্পৃক্ততা (saturation)। TechNova-র ড্যাশবোর্ডে এই চারটি থাকা উচিত — প্রতিটি সেবার জন্য। লগ সংগ্রহ করুন কেন্দ্রীয়ভাবে; ঘটনার সময় লগই সত্যের উৎস।

১১.৭ ব্যাকআপ, পুনরুদ্ধার ও ধারণক্ষমতা-পরিকল্পনা

ব্যাকআপ মানে ডেটার কপি — কিন্তু কপি থাকলেই পুনরুদ্ধার হয় না; পুনরুদ্ধার পরীক্ষা করতে হয়। দুটি পরিমাপ: RPO (Recovery Point Objective — কতটুকু ডেটা-ক্ষতি সহ্য করা যায়, যেমন ১ ঘণ্টা) ও RTO (Recovery Time Objective — কতক্ষণে সেবা ফিরবে, যেমন ৪ ঘণ্টা)। RPO ছোট করতে ঘন ঘন ব্যাকআপ; RTO ছোট করতে স্বয়ংক্রিয় পুনরুদ্ধার ও প্রস্তুত পরিবেশ। TechNova-র ডেটাবেসে দৈনিক স্ন্যাপশট + নিয়মিত লেনদেন-লগ ব্যাকআপ একটি বাস্তবসম্মত নীতি; গুরুত্বপূর্ণ ডেটার কপি অন্য রিজিয়নে রাখুন (রিজিয়ন-বিভ্রাটের জন্য)।

ধারণক্ষমতা-পরিকল্পনা (capacity planning) মানে ভবিষ্যৎ চাহিদার অনুমান করে আগে থেকে প্রস্তুতি: ট্রাফিক-বৃদ্ধির হার, মৌসুমি চাপ (যেমন উৎসবের সময়), নতুন ফিচারের প্রভাব। পরিকল্পনা ছাড়া বৃদ্ধি মানে ব্যস্ত সময়ে সার্ভার শেষ হয়ে যাওয়া (অধ্যায় ৬-এর /28 সাবনেটের গল্প)। প্রতি মাসে ব্যবহার পর্যালোচনা করুন: IP-ব্যবহার, CPU-প্রবণতা, ডিস্ক-বৃদ্ধি, খরচ-প্রবণতা।

পরিস্থিতি	প্রভাব	প্রতিকার	পরীক্ষা
একটি জোন বন্ধ	এক-তৃতীয়াংশ ক্ষমতা হারানো	বহু-জোন + লোড ব্যালান্সার	জোন-বিচ্ছিন্নতা মহড়া
ডেটাবেস প্রাথমিক বন্ধ	লেখা থেমে যাওয়া	রেপ্লিকায় স্বয়ংক্রিয় উত্তরণ (failover)	Failover মহড়া
ডেটা ক্ষতিগ্রস্ত	তথ্য হারানো	ব্যাকআপ + পুনরুদ্ধার-পরীক্ষা	নিয়মিত পুনরুদ্ধার-মহড়া
ট্রাফিক-বৃদ্ধি	ধীর সেবা	অটোস্কেলিং + সীমা	লোড-পরীক্ষা
পুরো রিজিয়ন বন্ধ	সেবা বন্ধ	দ্বিতীয় রিজিয়নে DR	বার্ষিক DR মহড়া

১১.৮ CLI ও API উদাহরণ

```
$ ibmcloud is instances --vpc technova-prod-vpc
$ ibmcloud is instance-groups
# অটোস্কেলিং-সদৃশ পরীক্ষা: নতুন সার্ভার তৈরি করে পুলে যোগ করুন
$ ibmcloud is instance-create technova-web-4 \
    --vpc technova-prod-vpc \
    --zone us-south-3 \
    --profile bx2-2x8 \
    --image ibm-ubuntu-22-04-minimal-amd64-1 \
    --subnet technova-web-zone3 \
    --keys technova-key
$ ibmcloud is lb-pool-member-create technova-web-lb \
    --pool web-pool --address 10.10.3.5 --port 8080
```

```
$ ibmcloud is snapshots
$ ibmcloud is snapshot-create technova-db-snap \
    --source-volume technova-data-vol
```

কমান্ডগুলো ধারণাগত; ইনস্ট্যান্স-গ্রুপ ও অটোস্কেলিংয়ের সঠিক সিনট্যাক্স ওই অঞ্চলের বর্তমান ডকুমেন্টেশনে যাচাই করুন। স্ন্যাপশট নেওয়ার আগে ডেটাবেসের সামঞ্জস্য (consistency) নিশ্চিত করুন — চলমান লেখার মাঝে স্ন্যাপশট নিলে পুনরুদ্ধারে অসামঞ্জস্য হতে পারে।

১১.৯ IaC উদাহরণ: ব্যাকআপ-নীতি ও পুনরুদ্ধার

```
resource "ibm_is_snapshot" "db_daily" {
  name          = "technova-db-daily-snap"
  source_volume = ibm_is_volume.data.id
}

# নীতি (নথি হিসেবে): প্রতিদিন স্ন্যাপশট, ৭ দিন রাখা,
# সাপ্তাহিক কপি অন্য রিজিয়নে, মাসে একবার পুনরুদ্ধার-মহড়া।
# RPO: ১ ঘণ্টা (লেনদেন-লগ ব্যাকআপ সহ)
# RTO: ৪ ঘণ্টা (পুনরুদ্ধার-মহড়ায় যাচাইকৃত)
```

IaC শুধু রিসোর্স নয়, নীতিও নথিভুক্ত করে। ব্যাকআপ-সময়সূচি, ধারণ-মেয়াদ (retention) ও পুনরুদ্ধার-মহড়ার তারিখ — সবই কোডের পাশে নথিতে লিখুন, নইলে কেউ জানে না ব্যাকআপ আসলেই নেওয়া হচ্ছে কি না।

১১.১০ উৎপাদন পরিস্থিতি: জোন-বিভ্রাটের দিন

TechNova-র এক সকালে একটি জোনে বিভ্রাট হয়। লোড ব্যালান্সার হেলথ চেকে ওই জোনের সার্ভারগুলোকে অসুস্থ দেখে ট্রাফিক সরিয়ে দেয়; ডেটাবেসের রেপ্লিকা প্রাথমিক হয়; গ্রাহকরা সামান্য ধীরগতি ছাড়া কিছু টের পান না। দল বিভ্রাট শেষে সার্ভারগুলো ফিরিয়ে আনে ও ঘটনা-পর্যালোচনা (postmortem) লেখে: কী ভেঙেছিল, কী কাজ করেছে, কী উন্নত করতে হবে। শিক্ষা: বিভ্রাটের দিন পরিকল্পনা কাজ করে কি না তা জানা যায় — পরিকল্পনা কাগজে নয়, মহড়ায় প্রমাণিত হয়।

> **REAL WORLD:** একটি দল ব্যাকআপ নিয়েছিল, কিন্তু কখনো পুনরুদ্ধার পরীক্ষা করেনি। আসল ঘটনার দিন দেখা যায় ব্যাকআপ ফাইল ক্ষতিগ্রস্ত — বছরের পর বছর ধরে নেওয়া ব্যাকআপের কোনোটিই কাজ করে না। ব্যাকআপের মূল্য তার পুনরুদ্ধারে, নেওয়ায় নয়। নিয়মিত পুনরুদ্ধার-মহড়া বাধ্যতামূলক করুন।

১১.১১ নিরাপত্তা ও ব্যয় বিবেচনা

নিরাপত্তা: Failover-এর সময় নিরাপত্তা-নিয়ম যেন শিথিল না হয় — দ্বিতীয় জোনের সার্ভারেও একই সিকিউরিটি গ্রুপ; ব্যাকআপ এনক্রিপ্ট করে রাখুন; পুনরুদ্ধার-পরিবেশেও একই অ্যাক্সেস-নিয়ন্ত্রণ। ব্যয়: অতিরিক্ততা মানে দ্বিগুণ-তিনগুণ রিসোর্স — প্রাপ্যতার প্রতিটি "৯"-এর দাম আছে। সচেতন আপস করুন: production-এ বহু-জোন, non-prod-এ এক জোন; DR পরিবেশে ছোট আকারে প্রস্তুত (pilot light) রাখা, প্রয়োজনে বড় করা। পর্যবেক্ষণ-খরচও হিসাব করুন — লগ ও মেট্রিক সংরক্ষণের খরচ বাড়তে পারে।

নির্ভরযোগ্যতা-প্রকৌশল (reliability engineering) মানে শুধু প্রযুক্তি নয়, প্রক্রিয়াও: পরিবর্তন-পর্যালোচনা, মহড়া, ঘটনা-পর্যালোচনা, অন-কল দায়িত্ব। একটি দল যে নিয়মিত মহড়া করে, সে বিভ্রাটের দিন শান্ত থাকে; যে করে না, সে অনুমান করে।

১১.১২ সাধারণ ভুল

- একটি মাত্র জোনে গুরুত্বপূর্ণ সেবা রাখা।
- ব্যাকআপ নেওয়া কিন্তু পুনরুদ্ধার কখনো পরীক্ষা না করা।
- অটোস্কেলিংয়ে সীমা (max) না বসানো — খরচের বিস্ফোরণ।
- Failover কখনো মহড়া না করা — আসল দিনে কাজ না করা।
- সতর্কতা-ক্লান্তি: এত সতর্কতা যে আসলটি উপেক্ষিত হয়।
- স্টেটফুল অ্যাপকে স্টেটলেস ভেবে স্কেল করা।
- RPO/RTO না লেখা — "দ্রুত ফিরবে" বলে আশা করা।

১১.১৩ সমস্যা সমাধান

সেবা ধীর? সোনালি সংকেত দেখুন: বিলম্ব বাড়ছে কি ট্রাফিক বাড়ছে? ত্রুটি বাড়ছে কি সম্পৃক্ততা? একটি জোনের সার্ভার খারাপ হলে লোড ব্যালান্সারের সুস্থ-সদস্য সংখ্যা দেখুন। Failover হচ্ছে না? রেপ্লিকার অবস্থা ও ল্যাগ দেখুন; স্বয়ংক্রিয় উত্তরণের শর্ত (যেমন কতক্ষণ প্রাথমিক অনুপস্থিত থাকলে) যাচাই করুন। পুনরুদ্ধার ব্যর্থ? ব্যাকআপের বয়স, সামঞ্জস্য ও ধারণ-মেয়াদ দেখুন। ঘটনার সময়: প্রথমে সেবা ফেরান (mitigate), তারপর কারণ খুঁজুন (diagnose) — উল্টো ক্রমে গ্রাহক অপেক্ষা করে।

> **TROUBLESHOOTING:** অটোস্কেলিং বারবার সার্ভার তৈরি ও মুছে ফেলছে (thrashing)? সম্ভবত স্কেল-আউট ও স্কেল-ইন সীমা খুব কাছাকাছি, বা হেলথ চেক নতুন সার্ভারকে প্রস্তুত হওয়ার আগেই অসুস্থ ঘোষণা করছে। সীমার মধ্যে ব্যবধান (hysteresis) রাখুন ও প্রস্তুতির সময় (warm-up) বাড়ান।

> **WARNING:** দুর্যোগ-পুনরুদ্ধার পরিকল্পনা কাগজে লিখে তাকে রাখবেন না। অন্তত বছরে একবার পূর্ণ মহড়া করুন — দ্বিতীয় রিজিয়নে সেবা চালু করে দেখুন আসলেই চলে কি না। মহড়া ছাড়া DR পরিকল্পনা একটি আশা, পরিকল্পনা নয়।

১১.১৪ গভীরে: ব্যর্থতা, পর্যবেক্ষণ, বিলিং ও বিকল্প

সহনশীলতার প্রতিটি উপাদানকে বারোটি প্রশ্নে যাচাই করুন। সমস্যা: ব্যর্থতা অনিবার্য — প্রশ্ন ভাঙবে কি না নয়, ভাঙলে কী হবে। ভেতরে: বহু-জোনে ট্রাফিক বণ্টন, হেলথ চেকের মাধ্যমে অসুস্থ সদস্য সরানো, ডেটাবেস-রেপ্লিকায় স্বয়ংক্রিয় উত্তরণ, অটোস্কেলিংয়ে চাহিদা-অনুযায়ী সার্ভার যোগ-বিয়োগ, ব্যাকআপ থেকে পুনরুদ্ধার। অন্যান্য পরিষেবার সঙ্গে যোগাযোগ: HA নকশা VPC, লোড ব্যালান্সার, DNS, VSI, ব্যাকআপ ও হাইব্রিড সংযোগ — সবকিছুর ওপর নির্ভর করে; একটি স্তরের দুর্বলতা পুরো সিস্টেমের দুর্বলতা। ব্যর্থ হলে: জোন-বিভ্রাটে ক্ষমতা কমে কিন্তু সেবা চলে; ডেটাবেস-উত্তরণে সামান্য বিরতি হতে পারে; রিজিয়ন-বিভ্রাটে DR ছাড়া সেবা থামে; ভুল অটোস্কেলিং-নীতিতে খরচ বিস্ফোরিত হতে পারে। নিরাপত্তা: Failover-এর সময়ও একই নিরাপত্তা-নিয়ম, ব্যাকআপ এনক্রিপ্ট করা, DR পরিবেশেও একই অ্যাক্সেস-নিয়ন্ত্রণ। পর্যবেক্ষণ: সোনালি সংকেত — বিলম্ব, ট্রাফিক, ত্রুটি, সম্পৃক্ততা — প্রতিটি সেবার জন্য; RPO/RTO-র বিরুদ্ধে পুনরুদ্ধার-মহড়ার ফলাফল। বিলিং: অতিরিক্ততা মানে দ্বিগুণ-তিনগুণ রিসোর্স; DR পরিবেশে পাইলট-লাইট (ছোট আকারে প্রস্তুত) খরচ কমায়; লগ-সংরক্ষণের খরচও হিসাব করুন। কখন ব্যবহার করবেন: উৎপাদন-সেবায় সবসময়; কখন এড়াবেন: পরীক্ষামূলক পরিবেশে পূর্ণ HA অপ্রয়োজনীয় খরচ। বিকল্প: একক-জোন + দ্রুত পুনরুদ্ধার (কম খরচ, বেশি ডাউনটাইম), পরিচালিত উচ্চ-প্রাপ্যতা পরিষেবা (দায়িত্ব প্রদানকারীর), বা মাল্টি-রিজিয়ন সক্রিয়-সক্রিয় (সবচেয়ে ব্যয়বহুল, সবচেয়ে সহনশীল)। আপস: প্রতিটি অতিরিক্ত "৯" (99.9% → 99.99%) ক্রমে ব্যয়বহুল — TechNova-র জন্য 99.9% বাস্তবসম্মত লক্ষ্য; তার চেয়ে বেশি চাইলে

খরচ ও জটিলতা লিখিতভাবে অনুমোদন করুন। সবশেষে মনে রাখুন: সহনশীলতা শুধু প্রযুক্তি নয় — মহড়া, ঘটনা-পর্যালোচনা ও অন-কল প্রক্রিয়া ছাড়া কোনো নকশাই আসল দিনে কাজ করে না।

১১.১৫ সারসংক্ষেপ

সহনশীলতা একটি নকশা-দর্শন: ব্যর্থতা অনিবার্য ধরে নিয়ে এমন সিস্টেম বানানো যেখানে একটি উপাদানের ব্যর্থতা সেবার ব্যর্থতা হয় না। বহু-জোন নকশা, লোড ব্যালান্সার, অটোস্কেলিং, স্বাস্থ্য-পর্যবেক্ষণ, ব্যাকআপ ও DR — এই সবকিছু মিলে TechNova-র সেবাকে একটি সার্ভারের ভাগ্য থেকে মুক্ত করে। এই খণ্ডের শিক্ষা এক বাক্যে: ভিত্তি (VPC) ঠিক করুন, পথ (রাউটিং) ও দরজা (নিরাপত্তা) নিয়ন্ত্রণ করুন, সামনের দরজা (লোড ব্যালান্সার) শক্ত করুন, আর সবকিছু এমনভাবে বানান যাতে একটি অংশ ভাঙলেও সেবা চলে।

১১.১৬ পর্যালোচনা প্রশ্ন

1. 99.9% প্রাপ্যতা মানে বছরে কত ডাউনটাইম? 99.99%-এর সঙ্গে তুলনা করুন।
2. ব্যর্থতার ডোমেইন কী এবং বহু-জোন নকশা কীভাবে তা ব্যবহার করে?
3. সক্রিয়-সক্রিয় ও সক্রিয়-নিষ্ক্রিয় অতিরিক্ততার পার্থক্য কী?
4. RPO ও RTO-র পার্থক্য কী?
5. অটোস্কেলিংয়ে thrashing কী এবং কীভাবে এড়ানো যায়?

১১.১৭ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি এক-পৃষ্ঠার নির্ভরযোগ্যতা-পরিকল্পনা লিখুন: SLO (যেমন 99.9%), RPO/RTO, জোন-বিন্যাস, অটোস্কেলিং সীমা (min/max), ব্যাকআপ-সময়সূচি ও মহড়ার ক্যালেন্ডার।
2. একটি ব্যর্থতা-মহড়া (game day) পরিকল্পনা করুন: একটি জোনের সার্ভার বন্ধ করে দেখুন লোড ব্যালান্সার ট্রাফিক সরায় কি না, ডেটাবেস failover হয় কি না, এবং কতক্ষণে সেবা স্বাভাবিক হয় — ফলাফল ও উন্নতির তালিকা লিখুন।

PART III • DEVOPS ENGINEERING MASTERY

Storage, Database & Application Services

IBM Cloud Object Storage, Block & File Storage, Managed Database, Kubernetes & Red Hat OpenShift, Container Registry & Application Hosting

অধ্যায় ১২ • PART III — Storage, Database & Application Services

Chapter 12 — IBM Cloud Object Storage

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক পারবেন — (১) অবজেক্ট স্টোরেজ (Object Storage) কী সমস্যা সমাধান করে তা প্রথম নীতি থেকে ব্যাখ্যা করতে; (২) বাকেট (Bucket), অবজেক্ট (Object), কী (Key) ও মেটাডেটার (Metadata) সম্পর্ক বুঝতে; (৩) স্টোরেজ ক্লাস — Standard, Vault, Cold Vault, Flex ও Smart Tier — এর মধ্যে ব্যবহার-ভিত্তিক পার্থক্য করতে; (৪) লাইফসাইকেল নীতি (Lifecycle Policy) দিয়ে ডেটার বয়স অনুযায়ী স্বয়ংক্রিয় স্থানান্তর ও মেয়াদোত্তীর্ণ ডেটা মুছে ফেলার নকশা করতে; (৫) IAM, বাকেট পলিসি ও সার্ভিস ক্রেডেনশিয়াল / HMAC কী দিয়ে নিরাপদ অ্যাক্সেস দিতে; (৬) এনক্রিপশন, রেপ্লিকেশন ও স্থিতিস্থাপকতার (Resilience) নকশা ব্যাখ্যা করতে; (৭) S3-সামঞ্জস্যপূর্ণ API ও `ibmcloud` CLI দিয়ে বাকেট পরিচালনা করতে; এবং (৮) খরচের চালক — সংরক্ষণ, অনুরোধ (Request) ও ডেটা বের করা (Egress) — চিহ্নিত করে খরচ কমানোর সিদ্ধান্ত নিতে।

১২.২ পূর্বশর্ত পুনরালোচনা

এগোনোর আগে তিনটি ধারণা ঝালিয়ে নেওয়া যাক। প্রথমত, ফাইল সিস্টেমে ডেটা থাকে একটি গাছের মতো শ্রেণিবিন্যাসে — ডিরেক্টরি, সাব-ডিরেক্টরি, ফাইল — যেখানে প্রতিটি ফাইলের অবস্থান তার পথ (Path) দিয়ে নির্ধারিত হয়। দ্বিতীয়ত, REST API-তে প্রতিটি সম্পদ (Resource) একটি URL দিয়ে চিহ্নিত হয় এবং HTTP ক্রিয়া — GET, PUT, DELETE — দিয়ে তার ওপর কাজ করা হয়। তৃতীয়ত, IBM Cloud-এ পরিচয় ও অনুমতি পরিচালিত হয় IAM (Identity and Access Management) দিয়ে: কে (Who), কী (What) করতে পারে, কোন সম্পদে (Which Resource) — এই তিনটি প্রশ্নের উত্তরই হলো অ্যাক্সেস নিয়ন্ত্রণ। এই তিনটি ধারণা একত্র করলেই অবজেক্ট স্টোরেজ বোঝা যায়: এটি একটি সমতল (Flat) নামস্থানে, HTTP API দিয়ে, IAM-নিয়ন্ত্রিতভাবে ডেটা রাখার ব্যবস্থা।

TechNova-র প্রসঙ্গ মনে রাখুন: TechNova একটি কাল্পনিক সফটওয়্যার কোম্পানি যারা ওয়েব অ্যাপ্লিকেশন, মোবাইল ব্যাকএন্ড ও ডেটা পাইপলাইন চালায়। তাদের ব্যবহারকারীরা ছবি আপলোড করে, অ্যাপ্লিকেশন লগ তৈরি করে, ব্যাকআপ ফাইল জমা হয় এবং বিশ্লেষণের জন্য বড় ডেটাসেট সংরক্ষণ করতে হয়। এই অধ্যায়ে আমরা দেখব কীভাবে এই সব ধরনের ডেটা একটিই স্টোরেজ মডেলে রাখা যায়, অথচ প্রতিটির জীবনচক্র, খরচ ও নিরাপত্তা আলাদাভাবে পরিচালনা করা যায়।

১২.৩ অবজেক্ট স্টোরেজ: প্রথম নীতি থেকে

কম্পিউটারে ডেটা সংরক্ষণের মৌলিক সমস্যাটি হলো: ডেটা বাড়ে, ডেটার ধরন বদলায়, কিন্তু ডেটা খুঁজে পাওয়ার উপায়টি যেন সহজ থাকে। ঐতিহ্যবাহী ব্লক স্টোরেজে ডেটা থাকে নির্দিষ্ট আকারের ব্লকে, ফাইল স্টোরেজে থাকে শ্রেণিবদ্ধ ফোল্ডারে। দুটিতেই একটি সীমা আছে: যখন ফাইলের সংখ্যা কোটি ছাড়ায়, যখন একই ডেটা পৃথিবীর নানা প্রান্ত থেকে একসঙ্গে পড়তে হয়, যখন ডেটার আকার কয়েক বাইট থেকে কয়েক টেরাবাইট পর্যন্ত বদলায় — তখন গাছের মতো কাঠামো ও একক সার্ভারের ওপর নির্ভরতা বোঝা হয়ে ওঠে।

অবজেক্ট স্টোরেজ এই সমস্যার উত্তর দেয় কাঠামোটি সরল করে। এখানে প্রতিটি ডেটা-খণ্ড একটি **অবজেক্ট** — একটি বাইনারি বস্তু (Blob) যার সঙ্গে থাকে একটি অনন্য **কী** (Key, অর্থাৎ নাম) ও **মেটাডেটা** (Metadata, অর্থাৎ ডেটা সম্পর্কে ডেটা: তৈরি হওয়ার সময়, বিষয়বস্তুর ধরন, মালিক, কাস্টম ট্যাগ)। অবজেক্টগুলো থাকে **বাকেটে** (Bucket) — একটি নামযুক্ত ধারক, যাকে মোটামুটিভাবে একটি সমতল কনটেইনার হিসেবে ভাবা যায়। বাকেটের ভেতরে কোনো সত্যিকারের ফোল্ডার নেই; `photos/2026/user1.jpg` নামের কী-টি আসলে একটি দীর্ঘ নাম মাত্র, যার মধ্যে স্ল্যাশ (/) অক্ষরটি আছে।

এই সরলতা-ই শক্তি: সিস্টেমকে কোনো গাছের গভীরতা অনুসরণ করতে হয় না, প্রতিটি অবজেক্টকে তার কী দিয়ে সরাসরি খুঁজে পাওয়া যায়।

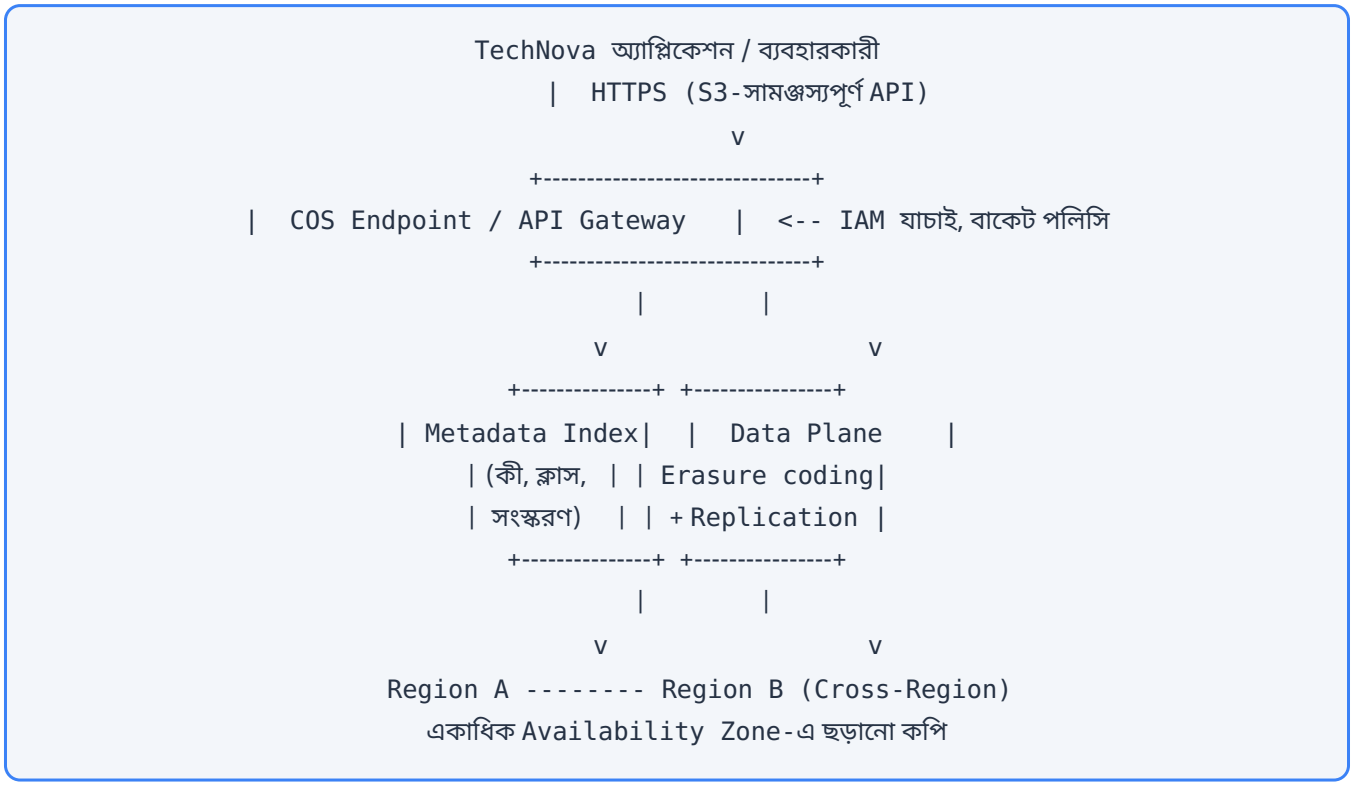
> **CONCEPT:** অবজেক্ট স্টোরেজে ডেটা ও মেটাডেটা একসঙ্গে ভ্রমণ করে। একটি ছবির অবজেক্টের সঙ্গে তার আপলোডকারী, আপলোডের সময়, কনটেন্ট-টাইপ (`image/jpeg`) ও ব্যবসায়িক ট্যাগ (যেমন `department=marketing`) সংরক্ষিত থাকে। ফলে ডেটা কোথায় আছে তা জানার পাশাপাশি ডেটাটি কী, কেন আছে, কতদিন রাখতে হবে — এসব প্রশ্নের উত্তরও একই জায়গায় পাওয়া যায়। এটাই লাইফসাইকেল ও খরচ ব্যবস্থাপনার ভিত্তি।

কোন সমস্যা এটি সমাধান করে? প্রথমত, **অসীম প্রায় স্কেল** (Near-unbounded Scale): কোটি কোটি অবজেক্ট, এক্সাবাইট-পর্যায়ের মোট আকার — বাকেটের আকার আগে থেকে নির্ধারণ করতে হয় না। দ্বিতীয়ত, **HTTP দিয়ে সর্বত্রগামী অ্যাক্সেস**: যেকোনো সার্ভার, কনটেইনার, ফাংশন বা ব্যবহারকারীর ব্রাউজার একই API দিয়ে ডেটা পড়তে ও লিখতে পারে। তৃতীয়ত, **স্থায়িত্বের নকশা** (Durability by Design): অবজেক্টটি একাধিক ভৌত স্থানে কপি করে রাখা হয়, ফলে একটি ডিস্ক বা একটি ডেটা সেন্টারের ব্যর্থতায় ডেটা হারায় না। চতুর্থত, **খরচের নমনীয়তা**: কত ঘন ঘন ডেটা পড়া হয় তার ওপর ভিত্তি করে সস্তা, ধীর শ্রেণিতে ডেটা রাখা যায়।

কখন এটি ব্যবহার করবেন না? যে ডেটা ঘন ঘন, ছোট ছোট অংশে, এলোমেলোভাবে (Random Access) বদলাতে হয় — যেমন ডেটাবেসের লাইভ ডেটা ফাইল বা ভার্চুয়াল মেশিনের ডিস্ক — তার জন্য অবজেক্ট স্টোরেজ উপযুক্ত নয়। সেখানে ব্লক স্টোরেজ দরকার (অধ্যায় ১৩ দেখুন)। একইভাবে, যে ফাইল সিস্টেম-স্তরের লকিং (File Locking) ও POSIX অনুমতি দরকার, সেখানে ফাইল স্টোরেজ দরকার। অবজেক্ট স্টোরেজের বিকল্প হলো ব্লক স্টোরেজ, ফাইল স্টোরেজ, ডেটাবেসের BLOB কলাম এবং নিজস্ব সার্ভারে ফাইল রাখা — প্রতিটির নিজস্ব বিনিময় (Trade-off) আছে, যা এই অধ্যায়ের শেষে তুলনামূলকভাবে দেখানো হয়েছে।

১২.৪ আর্কিটেকচার ও অভ্যন্তরীণ কার্যপ্রণালি

IBM Cloud Object Storage (COS) একটি বিতরণকৃত (Distributed) ব্যবস্থা। ব্যবহারকারীর দৃষ্টিতে এটি একটি endpoint — একটি HTTPS ঠিকানা — যেখানে PUT করলে অবজেক্ট জমা হয়, GET করলে ফেরত আসে। ভেতরে যা ঘটে তা অনেক স্তরে বিভক্ত। প্রথম স্তর হলো **অ্যাক্সেস স্তর**: API গেটওয়ে অনুরোধটি গ্রহণ করে, IAM টোকেন যাচাই করে, বাকেট পলিসি দেখে এবং অনুরোধটিকে বৈধ বলে মনে করলে অভ্যন্তরীণ সেবায় পাঠায়। দ্বিতীয় স্তর হলো **মেটাডেটা স্তর**: বাকেটের নাম, কী, সংস্করণ, স্টোরেজ ক্লাস, এনক্রিপশন অবস্থা — এসব তথ্য একটি দ্রুত সূচকে (Index) রাখা হয়। তৃতীয় স্তর হলো **ডেটা স্তর**: অবজেক্টের বাইটগুলোকে ভেঙে (Erasure Coding বা প্রতিলিপি — Replication — পদ্ধতিতে) একাধিক ডিস্ক ও একাধিক ভৌত অবস্থানে ছড়িয়ে দেওয়া হয়।



একটি PUT অনুরোধের অভ্যন্তরীণ যাত্রা এভাবে ভাবা যায়। ক্লায়েন্ট অবজেক্টটি পাঠায়; গেটওয়ে প্রথমে প্রমাণীকরণ (Authentication) করে — এই টোকেনটি কি বৈধ, এই পরিচয়ের কি এই বাকেটে লেখার অধিকার আছে? তারপর গেটওয়ে মেটাডেটা স্তরে কী-টি লিপিবদ্ধ করার প্রস্তুতি নেয়, ডেটা স্তরে বাইটগুলো ছড়িয়ে দেয় এবং যথেষ্ট সংখ্যক কপি নিরাপদে লেখা হয়েছে বলে নিশ্চিত হলে তবেই সাফল্যের উত্তর (HTTP 200) দেয়। এ কারণে লেখার বিলম্ব (Latency) কিছুটা বেশি হতে পারে, কিন্তু স্থায়িত্ব অনেক বেশি হয়। একটি GET অনুরোধে মেটাডেটা স্তর প্রথমে বলে দেয় অবজেক্টটি কোথায় আছে, কোন ক্লাসে আছে, কোন সংস্করণটি সর্বশেষ; তারপর ডেটা স্তর নিকটতম সুস্থ কপিটি থেকে বাইট ফেরত দেয়।

এই স্থাপত্যের একটি গুরুত্বপূর্ণ বৈশিষ্ট্য হলো **ইভেনচুয়াল কনসিস্টেন্সি বনাম স্ট্রং কনসিস্টেন্সি**-র প্রশ্ন। আধুনিক অবজেক্ট স্টোরেজে সাধারণত পড়া-লেখার ক্ষেত্রে শক্তিশালী সামঞ্জস্য (Strong Consistency) দেওয়ার চেষ্টা করা হয়: একটি PUT সফল হওয়ার পরপরই GET করলে নতুন ডেটাই ফেরত আসবে। কিন্তু বিতরণকৃত প্রতিলিপি (Replication) — বিশেষত ক্রস-রিজিয়ন প্রতিলিপি — সময় নেয়; তাই দূরবর্তী অঞ্চলে নতুন সংস্করণটি পৌঁছাতে বিলম্ব হতে পারে। প্রোডাকশন নকশায় এই বিলম্বকে ধরে নিয়েই কাজ করতে হয়: গুরুত্বপূর্ণ পড়ার কাজ যেন সেই অঞ্চল থেকেই করা হয় যেখানে লেখা হয়েছে, অথবা অ্যাপ্লিকেশন যেন পুরোনো সংস্করণ পড়লেও ভেঙে না পড়ে।

> **REAL WORLD:** TechNova-র ব্যবহারকারী প্রোফাইল ছবি আপলোড করে। ছবিটি COS-এ গেলে অ্যাপ্লিকেশন সঙ্গে সঙ্গে ব্যবহারকারীকে ছবির URL দেখায়। কিন্তু যদি ছবিটি দেখানোর অনুরোধটি অন্য অঞ্চলের ক্যাশ বা প্রতিলিপি থেকে যায়, তবে মুহূর্তের জন্য পুরোনো ছবি দেখা যেতে পারে। TechNova তাই আপলোডের পর প্রথম প্রদর্শনে লেখার অঞ্চলের endpoint ব্যবহার করে এবং ব্রাউজার-ক্যাশের মেয়াদ সংক্ষিপ্ত রাখে। ছোট এই সিদ্ধান্তটি ব্যবহারকারীর চোখে একটি বড় বিভ্রান্তি দূর করে।

অন্যান্য সেবার সঙ্গে যোগাযোগ: COS একটি কেন্দ্রীয় ডেটা হাব হিসেবে কাজ করে। Kubernetes-এর পড (অধ্যায় ১৫) COS থেকে কনফিগারেশন বা মডেল ফাইল পড়তে পারে; Code Engine-এর কাজ (অধ্যায় ১৬) COS-এ ফলাফল লিখতে পারে; IBM Cloud Databases-এর ব্যাকআপ (অধ্যায় ১৪) COS বাকেটে রাখা যায়; ডেটা পাইপলাইন COS-কে ডেটা লেক (Data Lake) হিসেবে ব্যবহার করে। প্রতিটি যোগাযোগই হয় HTTPS API দিয়ে, হয় সার্ভিস-টু-সার্ভিস IAM অনুমতি দিয়ে —

কোনো সেবারই COS-এর ভেতরের ডিস্কে সরাসরি প্রবেশাধিকার নেই। এই বিচ্ছিন্নতা (Decoupling) ইচ্ছাকৃত: একটি সেবা ব্যর্থ হলেও অন্যটি তার নিজের ডেটা থেকে কাজ চালিয়ে যেতে পারে।

ব্যর্থ হলে কী হয়? Endpoint সাময়িকভাবে অনুপলব্ধ হলে নতুন PUT/GET ব্যর্থ হয়, কিন্তু আগে লেখা ডেটা হারায় না — কারণ কপিগুলো একাধিক জায়গায় আছে। একটি Availability Zone নষ্ট হলে অন্য জোনের কপি থেকে সেবা চলতে থাকে। একটি অঞ্চল (Region) সম্পূর্ণ বিচ্ছিন্ন হলে ক্রস-রিজিয়ন বাকেটের অন্য অঞ্চলের কপি থেকে পড়া যায়, যদিও নতুন লেখা সেখানে না-ও যেতে পারে। বাকেট ভুল করে মুছে ফেললে — এটাই সবচেয়ে সাধারণ মানব-ত্রুটি — প্রতিলিপি কোনো কাজে আসে না, কারণ মুছে ফেলার নির্দেশটিও সব কপিতে প্রয়োগ হয়। তাই সংস্করণ (Versioning) ও অবজেক্ট লক (Object Lock) ব্যর্থতা-প্রতিরোধের শেষ স্তর হিসেবে অপরিহার্য।

১২.৫ বাকেট, অবজেক্ট ও নেমস্পেস

বাকেট তৈরি করার সময় তিনটি সিদ্ধান্ত নিতে হয় যা পরে বদলানো কঠিন বা অসম্ভব: **নাম, অঞ্চল ও স্থিতিস্থাপকতা (Resilience)**। বাকেটের নাম বিশ্বব্যাপী অনন্য হতে হয় এবং DNS-সামঞ্জস্যপূর্ণ হতে হয় — ছোট হাতের অক্ষর, সংখ্যা ও হাইফেন। নামে কোম্পানির নাম, পরিবেশ ও উদ্দেশ্য রাখা ভালো অভ্যাস, যেমন `technova-prod-user-photos`। অঞ্চল (Region) নির্ধারণ করে ডেটা ভৌতভাবে কোথায় থাকবে — এটি ডেটা সার্বভৌমত্ব (Data Residency) ও বিলম্বের জন্য গুরুত্বপূর্ণ। স্থিতিস্থাপকতা নির্ধারণ করে কপিগুলো কীভাবে ছড়ানো হবে: একক অঞ্চলের মধ্যে একাধিক জোনে (Regional), একটি একক ডেটা সেন্টারে (Single Data Center), নাকি একাধিক অঞ্চল জুড়ে (Cross-Region)।

স্থিতিস্থাপকতা	অর্থ	কখন ব্যবহার করবেন	বিনিময়
Regional	এক অঞ্চলের একাধিক জোনে কপি	সাধারণ প্রোডাকশন ডেটা	অঞ্চল-ব্যাপী বিপর্যয়ে ঝুঁকি থাকে
Cross-Region	একাধিক অঞ্চলে কপি	গুরুত্বপূর্ণ ডেটা, দুর্যোগ-পুনরুদ্ধার	খরচ ও প্রতিলিপি-বিলম্ব বেশি
Single Data Center	একটি ডেটা সেন্টারে কপি	অস্থায়ী, পুনর্গঠনযোগ্য ডেটা	ওই সেন্টার নষ্ট হলে ডেটা হারাতে পারে

অবজেক্টের কী নকশা করাও একটি প্রকৌশল সিদ্ধান্ত। ভালো কী অনুমানযোগ্য, তালিকাভুক্ত করা যায় (List করা যায়) এমন উপসর্গ (Prefix) দিয়ে শুরু হয়। যেমন `uploads/2026/11/09/user-1042/photo.jpg` — এই কী থেকে বোঝা যায় তারিখ, ব্যবহারকারী ও ধরন; আবার `uploads/2026/11/` উপসর্গ দিয়ে ওই মাসের সব আপলোড তালিকাভুক্ত করা যায়। খারাপ কী হলো এলোমেলো UUID মাত্র — খুঁজে পাওয়া যায়, কিন্তু তালিকা বা জীবনচক্র নীতি দিয়ে দলবদ্ধভাবে পরিচালনা করা যায় না। বড় অবজেক্ট (যেমন ভিডিও) আপলোডের জন্য মাল্টিপার্ট আপলোড (Multipart Upload) ব্যবহার করা হয়: অবজেক্টটিকে খণ্ডে ভেঙে সমান্তরালে পাঠানো হয়, কোনো খণ্ড ব্যর্থ হলে শুধু সেটি আবার পাঠানো হয়। মাল্টিপার্ট অসম্পূর্ণ রেখে গেলে অসম্পূর্ণ খণ্ডগুলো খরচ তৈরি করে — লাইফসাইকেল নীতিতে অসম্পূর্ণ আপলোড পরিষ্কার করার নিয়ম রাখা উচিত।

সংস্করণ (Versioning) চালু করলে একই কী-র প্রতিটি লেখা একটি নতুন সংস্করণ হিসেবে সংরক্ষিত হয়, পুরোনো সংস্করণ মুছে যায় না। এতে ভুল করে ওভাররাইট বা মুছে ফেলার ক্ষতি থেকে বাঁচা যায়, কিন্তু প্রতিটি সংস্করণের জন্য আলাদা সংরক্ষণ খরচ দিতে হয়। তাই সংস্করণের সঙ্গে একটি লাইফসাইকেল নীতি রাখা উচিত — যেমন পুরোনো সংস্করণ ৩০ দিন পর সস্তা ক্লাসে যাবে, ৯০ দিন পর মুছে যাবে।

১২.৬ স্টোরেজ ক্লাস ও স্মার্ট টিয়ার

সব ডেটা সমান ঘন ঘন পড়া হয় না। গতকাল আপলোড হওয়া ছবি বারবার দেখা হয়; পাঁচ বছর আগের অডিট লগ প্রায় কখনোই পড়া হয় না। স্টোরেজ ক্লাস এই বাস্তবতাকে খরচের ভাষায় অনুবাদ করে: যত কম ঘন ঘন পড়া হবে, তত সস্তা সংরক্ষণ — বিনিময়ে পড়ার খরচ বা পুনরুদ্ধারের সময় বাড়তে পারে।

স্টোরেজ ক্লাস	ব্যবহার	বৈশিষ্ট্য
Standard	ঘন ঘন পড়া হয় এমন সক্রিয় ডেটা	দ্রুত অ্যাক্সেস, ঘন ঘন পড়ার জন্য উপযুক্ত
Vault	কম ঘন ঘন পড়া হয়, তবু দ্রুত দরকার হতে পারে	সস্তা সংরক্ষণ, পড়ার খরচ বেশি হতে পারে
Cold Vault	খুব কম পড়া হয়, দীর্ঘমেয়াদি সংরক্ষণ	আরও সস্তা সংরক্ষণ, পুনরুদ্ধারে সময়/খরচ বেশি
Flex	অনিয়মিত অ্যাক্সেসের ধরণ	ব্যবহার-ভিত্তিক নমনীয় মূল্য কাঠামো
Smart Tier	অ্যাক্সেসের ধরণ আগাম জানা নেই	স্বয়ংক্রিয়ভাবে উপযুক্ত স্তরে স্থানান্তর

Smart Tier বিশেষভাবে লক্ষণীয়: এটি ডেটার অ্যাক্সেসের ধরণ পর্যবেক্ষণ করে স্বয়ংক্রিয়ভাবে সঠিক স্তরে সরিয়ে নেয়। যে দলের ডেটার ব্যবহার-ধরণ অনিয়মিত বা ঋতুভিত্তিক — যেমন বছরে একবার অডিটের সময় পড়া রিপোর্ট — তাদের জন্য এটি হাতে-কলমে ক্লাস বদলের ঝামেলা কমায়। তবে সতর্কতা আছে: Smart Tier-ও একটি পরিচালিত সিদ্ধান্ত; খরচের প্রতিবেদনে (Billing Report) কোন স্তরে কত ডেটা আছে তা নিয়মিত পরীক্ষা করা উচিত, কারণ ভুল অনুমানের চেয়ে স্বয়ংক্রিয় স্থানান্তর অনেক সময় নীরবে খরচ বাড়াতে পারে। কোনো ক্লাসেই নির্দিষ্ট দাম এই বইয়ে দেওয়া হয়নি — দাম সময় ও অঞ্চলভেদে বদলায়, তাই সিদ্ধান্ত নেওয়ার সময় IBM Cloud-এর বর্তমান মূল্য পৃষ্ঠা দেখে নিতে হবে।

> **WARNING:** সস্তা ক্লাসে ডেটা সরানো মানেই সবসময় খরচ কমে না। Vault বা Cold Vault থেকে ঘন ঘন ডেটা পড়লে প্রতি অনুরোধের খরচ ও পুনরুদ্ধারের খরচ সংরক্ষণের সাশ্রয়কে ছাড়িয়ে যেতে পারে। সিদ্ধান্ত নেওয়ার আগে প্রকৃত অ্যাক্সেস লগ দেখুন: গত ৯০ দিনে এই উপসর্গের ডেটা কতবার পড়া হয়েছে? উত্তর না জেনে ক্লাস বদলানো জুয়ার মতো।

১২.৭ লাইফসাইকেল নীতি

লাইফসাইকেল নীতি হলো বাকেটের জন্য লেখা একটি নিয়মের সেট: ডেটার বয়স বা সংস্করণের ভিত্তিতে কী হবে। সাধারণ নিয়ম তিন ধরনের: (১) **স্থানান্তর (Transition)** — যেমন ৩০ দিন পর Standard থেকে Vault-এ; (২) **মেয়াদোত্তীর্ণ (Expiration)** — যেমন ৩৬৫ দিন পর মুছে ফেলা; (৩) **অসম্পূর্ণ আপলোড পরিষ্কার** — যেমন ৭ দিনের পুরোনো অসম্পূর্ণ মাল্টিপার্ট খণ্ড মুছে ফেলা। একটি ভালো নীতি লেখার আগে ডেটার আইনি ও ব্যবসায়িক ধারণকাল (Retention) জানতে হয়: কর-সংক্রান্ত রেকর্ড কতদিন রাখতে হবে? ব্যবহারকারীর ডেটা মুছে ফেলার অনুরোধ এলে কী হবে? লগ কতদিন রাখা দরকার?

TechNova-র উদাহরণ: ব্যবহারকারীর আপলোড করা ছবি Standard-এ থাকে ৬০ দিন, তারপর Vault-এ যায়, ৩৬৫ দিন পর Cold Vault-এ যায়, আর অ্যাকাউন্ট মুছে গেলে সংশ্লিষ্ট অবজেক্ট ৩০ দিনের মধ্যে মুছে যায়। অ্যাপ্লিকেশন লগ ৩০ দিন পর Vault-এ যায়, ১৮০ দিন পর মুছে যায়। ব্যাকআপ স্ন্যাপশট ৯০ দিন পর Cold Vault-এ যায়, ৭ বছর পর মুছে যায়। প্রতিটি নিয়ম আলাদা উপসর্গে (Prefix) প্রয়োগ করা হয়, যাতে একটি নিয়ম ভুল হলে সব ডেটা ক্ষতিগ্রস্ত না হয়। লাইফসাইকেল নীতি প্রথমে একটি পরীক্ষামূলক (Non-Production) বাকেটে পরীক্ষা করা উচিত, এবং নীতির কার্যকারিতা পর্যবেক্ষণ করতে মেট্রিক দেখা উচিত — কত অবজেক্ট স্থানান্তরিত হলো, কত মুছে গেল।

১২.৮ অ্যাক্সেস নিয়ন্ত্রণ ও সার্ভিস ক্রেডেনশিয়াল

COS-এর অ্যাক্সেস নিয়ন্ত্রণ তিন স্তরে কাজ করে। প্রথম স্তর **IAM**: IBM Cloud অ্যাকাউন্টে কে কোন সেবা-ইনস্ট্যান্সে কোন ভূমিকায় (Role) — যেমন Reader, Writer, Manager — কাজ করতে পারে। দ্বিতীয় স্তর **বাকেট পলিসি**: নির্দিষ্ট বাকেটে কোন পরিচয় বা কোন শর্তে (যেমন নির্দিষ্ট IP, নির্দিষ্ট সময়) কী করতে পারবে। তৃতীয় স্তর **অবজেক্ট-স্তরের নিয়ন্ত্রণ**: প্রি-সাইন্ড URL (Presigned URL) দিয়ে সীমিত সময়ের জন্য একটি নির্দিষ্ট অবজেক্টে অ্যাক্সেস দেওয়া।

সার্ভিস ক্রেডেনশিয়াল (Service Credential) হলো অ্যাপ্লিকেশনের পরিচয়পত্র। মানুষ IAM-এ লগ ইন করে, অ্যাপ্লিকেশন করে সার্ভিস ক্রেডেনশিয়াল দিয়ে। একটি ক্রেডেনশিয়ালে থাকে API কী ও S3-সামঞ্জস্যপূর্ণ HMAC কী জোড়া (Access Key ID ও Secret Access Key)। এই চাবিগুলোকে পাসওয়ার্ডের মতো গোপন রাখতে হয়: কখনোই সোর্স কোডে, গিট রিপোজিটরিতে বা চ্যাটে রাখা যাবে না। সঠিক অভ্যাস হলো প্রতিটি অ্যাপ্লিকেশন ও প্রতিটি পরিবেশের জন্য আলাদা ক্রেডেনশিয়াল তৈরি করা, শুধুমাত্র প্রয়োজনীয় বাকেটে শুধুমাত্র প্রয়োজনীয় ভূমিকা দেওয়া (Least Privilege), এবং নিয়মিতভাবে চাবি ঘোরানো (Rotation) — নতুন চাবি তৈরি, অ্যাপ্লিকেশনে বসানো, পুরোনো চাবি মুছে ফেলা।

ব্যবহারকারীর ব্রাউজার থেকে সরাসরি আপলোডের ক্ষেত্রে সার্ভার কখনোই নিজের গোপন চাবি ব্রাউজারে দেয় না। বদলে সার্ভার একটি প্রি-সাইন্ড URL তৈরি করে দেয়, যার মেয়াদ — যেমন ১৫ মিনিট — শেষ হলে তা অকার্যকর হয়ে যায়। এতে আপলোডের বোঝা সার্ভারের ওপর না পড়ে সরাসরি COS-এ যায়, অথচ অ্যাক্সেস থাকে নিয়ন্ত্রিত। পাবলিক পড়ার জন্য (যেমন ওয়েবসাইটের ছবি) বাকেট বা অবজেক্টকে পাবলিক করা যায়, কিন্তু ডিফল্ট হওয়া উচিত ব্যক্তিগত (Private) — পাবলিক অ্যাক্সেস একটি সচেতন সিদ্ধান্ত, দুর্ঘটনা নয়।

১২.৯ এনক্রিপশন, রেপ্লিকেশন ও স্থিতিস্থাপকতা

এনক্রিপশন দুই জায়গায় হয়: **পরিবহনে (In Transit)** — TLS দিয়ে, যাতে নেটওয়ার্কে ডেটা পড়া না যায়; এবং **বিশ্রামে (At Rest)** — ডিস্কে লেখার আগে ডেটা এনক্রিপ্ট করে রাখা হয়। COS ডিফল্টভাবে সার্ভার-সাইড এনক্রিপশন দেয়, এবং উচ্চ-নিরাপত্তার কাজে গ্রাহক-পরিচালিত কী (Customer-Managed Key) ব্যবহার করা যায়, যেখানে কী-টি আলাদা কী-পরিচালনা সেবায় (যেমন IBM Cloud Hyper Protect Crypto Services বা Key Protect) থাকে। এতে চাবি ঘোরানো, চাবি প্রত্যাহার (Revoke) ও অডিট — সবই গ্রাহকের নিয়ন্ত্রণে থাকে। মনে রাখতে হবে: কী হারিয়ে গেলে ডেটা আর পড়া যাবে না; তাই কী-পরিচালনায় ব্যাকআপ ও পুনরুদ্ধারের পরিকল্পনা অপরিহার্য।

রেপ্লিকেশন দুই ধরনের। একই অঞ্চলের ভেতরে প্রতিলিপি স্বয়ংক্রিয়ভাবে হয় — এটাই Regional স্থিতিস্থাপকতার ভিত্তি। অঞ্চল জুড়ে প্রতিলিপি (Cross-Region Replication) বেছে নিলে বাকেটের একটি অনুলিপি অন্য অঞ্চলে তৈরি হয়; লেখা প্রথম অঞ্চলে হয়, তারপর অসিঙ্ক্রোনাসভাবে (Asynchronously) অন্য অঞ্চলে ছড়ায়। এই অসিঙ্ক্রোনাস প্রকৃতির অর্থ হলো দুর্যোগের মুহূর্তে সবচেয়ে সাম্প্রতিক কয়েক সেকেন্ড বা মিনিটের লেখা দূরবর্তী অঞ্চলে নাও থাকতে পারে — একে বলে RPO (Recovery Point Objective)। প্রোডাকশন নকশায় RPO ও RTO (Recovery Time Objective) লিখিতভাবে ঠিক করে নিতে হয়, মুখের কথায় নয়।

পর্যবেক্ষণ (Monitoring) ছাড়া স্থিতিস্থাপকতা শুধুই আশা। দেখতে হয়: বাকেটের মোট আকার, অবজেক্টের সংখ্যা, অনুরোধের হার ও ত্রুটির হার, প্রতিলিপির পিছিয়ে থাকা (Replication Lag), লাইফসাইকেল নীতির কার্যকারিতা, এবং অস্বাভাবিক অ্যাক্সেসের চেষ্টা। IBM Cloud-এর মনিটরিং ও অ্যাক্টিভিটি ট্র্যাকার সেবা দিয়ে এই সংকেতগুলো সংগ্রহ করে সতর্কতা (Alert) বসানো যায় — যেমন বাকেটের আকার হঠাৎ দ্বিগুণ হলে, বা এক ঘণ্টায় হাজার হাজার মুছে ফেলার অনুরোধ এলে।

বিলিং বোঝা: COS-এর খরচের মূল চালক তিনটি — **সংরক্ষিত ডেটার পরিমাণ ও ক্লাস** (GB-মাস), **অনুরোধের সংখ্যা ও ধরন** (PUT/GET/LIST), এবং **ডেটা বের করার পরিমাণ** (Egress, অর্থাৎ COS থেকে ইন্টারনেটে বা অন্য অঞ্চলে ডেটা যাওয়া)। একই অঞ্চলের ভেতরে IBM Cloud সেবাগুলোর মধ্যে ডেটা চলাচলের নিয়ম আলাদা হতে পারে, তাই নকশার সময়

ডেটা-প্রবাহের মানচিত্র আঁকা উচিত: ডেটা কোথা থেকে আসে, কোথায় যায়, কতবার পড়া হয়। সবচেয়ে সাধারণ খরচ-ভুল হলো অপ্রয়োজনীয় LIST অনুরোধ — একটি অ্যাপ্লিকেশন যদি প্রতি অনুরোধে পুরো বাকেট তালিকাভুক্ত করে, তবে অনুরোধের খরচ সংরক্ষণের খরচকে ছাড়িয়ে যেতে পারে।

১২.১০ CLI ও API উদাহরণ

নিচের উদাহরণগুলো `ibmcloud` CLI ও S3-সামঞ্জস্যপূর্ণ API-র ধারণাগত ব্যবহার দেখায়। প্রকৃত কমান্ড চালানোর আগে বর্তমান CLI ডকুমেন্টেশন ও নিজের অ্যাকাউন্টের অঞ্চল ও সম্পদ-গোষ্ঠী (Resource Group) মিলিয়ে নিন। গোপন চাবি কখনোই কমান্ড-লাইনের ইতিহাসে বা স্ক্রিনশটে রাখবেন না।

```
# লগ ইন ও লক্ষ্য (target) নির্ধারণ
ibmcloud login --sso
ibmcloud target -g technova-prod -r us-south

# COS সেবা-ইনস্ট্যান্স তৈরি (নাম ও পরিকল্পনা নিজের
# অ্যাকাউন্ট অনুযায়ী দিন)
ibmcloud resource service-instance-create \
  technova-cos cloud-object-storage standard us-south

# সার্ভিস ক্রেডেনশিয়াল তৈরি (Writer ভূমিকায়)
ibmcloud resource service-key-create \
  technova-cos-writer Writer \
  --instance-name technova-cos

# বাকেট তালিকা ও বিবরণ দেখা
ibmcloud cos buckets --region us-south
ibmcloud cos bucket-get --bucket technova-prod-user-photos
```

```
# AWS CLI (S3-সামঞ্জস্যপূর্ণ endpoint দিয়ে) — ধারণাগত
# endpoint ও চাবি পরিবেশ চলক থেকে নেওয়া হয়
export AWS_ACCESS_KEY_ID="CHANGE-ME-ACCESS-KEY"
export AWS_SECRET_ACCESS_KEY="CHANGE-ME-SECRET-KEY"

aws --endpoint-url https://s3.us-south.cloud-object-storage.appdomain.cloud \
  s3 ls s3://technova-prod-user-photos

aws --endpoint-url https://s3.us-south.cloud-object-storage.appdomain.cloud \
  s3 cp ./photo.jpg \
  s3://technova-prod-user-photos/uploads/2026/11/09/photo.jpg

aws --endpoint-url https://s3.us-south.cloud-object-storage.appdomain.cloud \
  s3api head-object \
  --bucket technova-prod-user-photos \
  --key uploads/2026/11/09/photo.jpg
```

API-স্তরে প্রতিটি কাজ একটি HTTP অনুরোধ: PUT Object, GET Object, DELETE Object, LIST Objects, HEAD Object। অ্যাপ্লিকেশন কোডে সরাসরি HTTP না লিখে IBM Cloud COS-এর জন্য উপলব্ধ SDK ব্যবহার করা উচিত, কারণ SDK স্বয়ংক্রিয়ভাবে স্বাক্ষর (Signing), পুনরায় চেষ্টা (Retry) ও মাল্টিপার্ট পরিচালনা করে। পুনরায় চেষ্টার সময় exponential backoff ব্যবহার করুন — ব্যর্থ অনুরোধে সঙ্গে সঙ্গে বারবার আঘাত করলে সেবা আরও চাপে পড়ে।

১২.১১ IaC উদাহরণ

বাকেট হাতে তৈরি করলে পরিবেশগুলোর মধ্যে পার্থক্য তৈরি হয়; Terraform দিয়ে তৈরি করলে একই সংজ্ঞা থেকে dev, staging ও production বাকেট তৈরি করা যায়। নিচে একটি সংক্ষিপ্ত, ধারণাগত Terraform উদাহরণ দেওয়া হলো — প্রকৃত প্রোভাইডার ব্লক ও সম্পদ-নাম নিজের পরিবেশ অনুযায়ী মিলিয়ে নিতে হবে।

```
resource "ibm_resource_instance" "cos" {
  name     = "technova-cos"
  service  = "cloud-object-storage"
  plan     = "standard"
  location = "global"
}

resource "ibm_cos_bucket" "photos" {
  bucket_name          = "technova-prod-user-photos"
  resource_instance_id = ibm_resource_instance.cos.id
  region_location      = "us-south"
  storage_class        = "standard"
  # versioning ও object lock-এর ব্লক
  # প্রোভাইডারের বর্তমান স্কিমা অনুযায়ী লিখুন
}

# লাইফসাইকেল: 60 দিন পর Vault, 365 দিন পর মেয়াদোত্তীর্ণ
# (নিয়মের বিস্তারিত গঠন প্রোভাইডার ডকুমেন্টেশন দেখে
#  নিশ্চিত করুন; উদাহরণটি ধারণাগত)
```

IaC-এর সঙ্গে দুটি শৃঙ্খলা জরুরি। প্রথমত, Terraform state ফাইলটি নিজেই সংবেদনশীল — তাতে ক্রেডেনশিয়ালের রেফারেন্স থাকতে পারে; state-কে দূরবর্তী (Remote) ব্যাকএন্ডে, এনক্রিপ্ট করে ও অ্যাক্সেস সীমিত রেখে সংরক্ষণ করুন। দ্বিতীয়ত, বাকেটের নাম, অঞ্চল ও স্থিতিস্থাপকতা পরিবর্তন অনেক সময় বাকেটটি ধ্বংস করে নতুন করে তৈরি করার (Recreate) কারণ হয় — তাই `terraform plan`-এর আউটপুটে ধ্বংসাত্মক পরিবর্তন দেখলে তা থামিয়ে পর্যালোচনা করুন।

১২.১২ প্রোডাকশন পরিস্থিতি: TechNova

পরিস্থিতি ১ — ব্যবহারকারীর ছবি: TechNova-র ওয়েব অ্যাপে প্রতিদিন হাজার হাজার ছবি আপলোড হয়। নকশা: অ্যাপ্লিকেশন সার্ভার প্রি-সাইন্ড URL দেয়, ব্রাউজার সরাসরি COS-এ আপলোড করে, আপলোড শেষে অ্যাপ্লিকেশন মেটাডেটা যাচাই করে। ছবির তিনটি আকার (মূল, মাঝারি, থাম্বনেইল) তৈরি করে আলাদা উপসর্গে রাখা হয়। Standard ক্লাসে ৬০ দিন, তারপর Vault। থাম্বনেইল ঘন ঘন পড়া হয় বলে সেটি Standard-এ বেশি দিন থাকে। ফল: সার্ভারের ব্যান্ডউইথ বাঁচে, আপলোড দ্রুত হয়, খরচ নিয়ন্ত্রিত থাকে।

পরিস্থিতি ২ — ডেটা লেক ও ব্যাকআপ: TechNova-র বিশ্লেষণ দল প্রতিদিনের লেনদেনের ডেটা COS-এ Parquet ফরম্যাটে রাখে। কাঁচা (Raw) ডেটা আসে এক বাকেটে, পরিষ্কার করা ডেটা যায় আরেক বাকেটে — এতে কাঁচা ডেটা ভুল করে বদলে যাওয়ার ঝুঁকি কমে। ডেটাবেস ব্যাকআপ যায় তৃতীয় একটি বাকেটে, যেখানে Object Lock চালু থাকে, ফলে র‍্যানসমওয়্যার আক্রমণে ব্যাকআপ মুছে ফেলা যায় না। প্রতিটি বাকেটে আলাদা সার্ভিস ক্রেডেনশিয়াল, যাতে বিশ্লেষণ দলের চাবি দিয়ে ব্যাকআপ বাকেটে লেখা না যায়।

পরিস্থিতি ৩ — দুর্যোগ-পুনরুদ্ধার: সবচেয়ে গুরুত্বপূর্ণ ডেটা — ব্যবহারকারীর ফাইল ও চুক্তিপত্র — Cross-Region বাকেটে রাখা হয়। বছরে দুবার পুনরুদ্ধারের মহড়া (Drill) করা হয়: দ্বিতীয় অঞ্চল থেকে ডেটা পড়ে অ্যাপ্লিকেশন চালিয়ে দেখা হয়,

RTO ও RPO মাপা হয়। মহড়ার ফল লিখিতভাবে সংরক্ষণ করা হয়, কারণ মহড়া ছাড়া পুনরুদ্ধার পরিকল্পনা শুধু কাগজের প্রতিশ্রুতি।

১২.১৩ নিরাপত্তা ও খরচ অপ্টিমাইজেশন

নিরাপত্তার চেকলিস্ট: ডিফল্টভাবে সব বাকেট Private; পাবলিক অ্যাক্সেসের প্রতিটি সিদ্ধান্ত লিখিত ও পর্যালোচিত; প্রতিটি পরিবেশ ও অ্যাপ্লিকেশনের জন্য আলাদা ক্রেডেনশিয়াল; Least Privilege ভূমিকা; চাবি ঘোরানোর সময়সূচি; এনক্রিপশন চালু, সংবেদনশীল ডেটায় গ্রাহক-পরিচালিত কী; Object Lock গুরুত্বপূর্ণ ব্যাকআপে; অ্যাক্সেস লগ ও অস্বাভাবিক কার্যকলাপে সতর্কতা; নিয়মিত অ্যাক্সেস পর্যালোচনা (Access Review) — কে এখনও কোন বাকেটে লিখতে পারে, যার আর দরকার নেই তার অনুমতি তুলে নেওয়া।

খরচ অপ্টিমাইজেশনের চেকলিস্ট: লাইফসাইকেল নীতি দিয়ে অপ্রয়োজনীয় ডেটা স্বয়ংক্রিয়ভাবে সরানো বা মুছে ফেলা; অসম্পূর্ণ মাল্টিপার্ট আপলোড পরিষ্কার করা; পুরোনো সংস্করণের ধারণকাল সীমিত করা; LIST অনুরোধ কমানো — তালিকার ফলাফল ক্যাশ করা; বড় ডেটা একই অঞ্চলের ভেতরে রাখা যাতে অপ্রয়োজনীয় Egress না হয়; প্রতি বাকেটে ট্যাগ (Tag) দিয়ে খরচের হিসাব ভাগ করা — কোন দল, কোন প্রোডাক্ট কত খরচ করছে; মাসিক বিল পর্যালোচনায় হঠাৎ বৃদ্ধির কারণ খোঁজা। একটি সাধারণ ভুল হলো লগ বাকেটে অসীম ধারণকাল রাখা — লগের মূল্য সময়ের সঙ্গে কমে, খরচ কমে না।

১২.১৪ সাধারণ ভুল ও ট্রাবলশুটিং

সাধারণ ভুল: (১) একটিমাত্র সার্ভিস ক্রেডেনশিয়াল সব অ্যাপ্লিকেশনে ব্যবহার করা — একটি চাবি ফাঁস হলে সব বাকেট ঝুঁকিতে পড়ে। (২) বাকেট পাবলিক রেখে ভাবা যে কেউ খুঁজে পাবে না — বাকেটের নাম অনুমান করা যায়, স্ক্যান করা যায়। (৩) Versioning চালু না করে প্রোডাকশন ডেটা রাখা। (৪) লাইফসাইকেল নীতি ছাড়া সংস্করণ জমতে দেওয়া। (৫) একই বাকেটে কাঁচা ডেটা, ব্যাকআপ ও ব্যবহারকারীর ফাইল মিশিয়ে রাখা — এতে নীতি ও অনুমতি আলাদা করা যায় না। (৬) Egress খরচ না ধরে অন্য অঞ্চল থেকে বারবার বড় ফাইল পড়া।

> **TROUBLESHOOTING:** 403 Forbidden পেলে প্রথমে দেখুন — টোকেনের মেয়াদ আছে কি না, IAM ভূমিকাটি কি এই বাকেটের জন্য যথেষ্ট, বাকেট পলিসি কি অনুরোধটি আটকাচ্ছে, নাকি সার্ভিস ক্রেডেনশিয়ালটি ভুল ইনস্ট্যান্সের। 404 Not Found পেলে কী-র বানান, অঞ্চল (endpoint) ও বাকেটের নাম মিলিয়ে দেখুন — ভুল অঞ্চলের endpoint-এ সঠিক বাকেটও পাওয়া যাবে না। ধীর আপলোডে মাল্টিপার্ট আকার ও সমান্তরাল থ্রেড পরীক্ষা করুন, এবং আপলোডকারী যেন লেখার অঞ্চলের নিকটবর্তী endpoint ব্যবহার করে তা নিশ্চিত করুন। হঠাৎ বিল বাড়লে প্রথমে দেখুন অসম্পূর্ণ মাল্টিপার্ট খণ্ড ও পুরোনো সংস্করণের পরিমাণ।

আরও একটি জটিল সমস্যা হলো **eventual consistency-জনিত বিভ্রান্তি**: লেখার পরপরই তালিকায় (LIST) নতুন অবজেক্ট না দেখা গেলে অ্যাপ্লিকেশন যেন সেটিকে ব্যর্থতা মনে না করে। সমাধান: লেখার পর সরাসরি কী দিয়ে GET করে যাচাই করুন, তালিকার ওপর নির্ভর করবেন না; অথবা অ্যাপ্লিকেশনে পুনরায় চেষ্টার যুক্তি (Retry with Backoff) রাখুন।

১২.১৪.১ ডেটা-প্রবাহ নকশা ও মাল্টি-ক্লাউড প্রসঙ্গ

অবজেক্ট স্টোরেজকে একা দেখলে তার খরচ ও কর্মক্ষমতা বোঝা যায় না — দেখতে হয় পুরো ডেটা-প্রবাহ (Data Flow)। TechNova-র ডেটা তিনটি পথে চলে: **ইনগ্রেস (Ingress)** — ব্যবহারকারী ও অ্যাপ্লিকেশন থেকে COS-এ লেখা; **অভ্যন্তরীণ প্রবাহ** — COS থেকে Kubernetes পড, Code Engine জব বা বিশ্লেষণ পাইপলাইনে পড়া; এবং **ইগ্রেস (Egress)** — COS থেকে ইন্টারনেটে বা অন্য ক্লাউডে ডেটা বেরিয়ে যাওয়া। প্রতিটি পথের খরচ ও বিলম্ব আলাদা। সবচেয়ে ব্যয়বহুল ভুল হলো এমন নকশা যেখানে একটি বড় ফাইল বারবার ইগ্রেস হয় — যেমন প্রতিটি API অনুরোধে COS থেকে একটি ভিডিও

পড়ে ব্যবহারকারীকে পাঠানো, কোনো ক্যাশ বা কনটেন্ট ডেলিভারি স্তর ছাড়া। সঠিক নকশায় ঘন ঘন পড়া হয় এমন ছোট অবজেক্ট ক্যাশে রাখা হয়, বড় অবজেক্টের জন্য সরাসরি ডাউনলোড URL দেওয়া হয়, এবং বিশ্লেষণের কাজ ডেটার কাছে — অর্থাৎ একই অঞ্চলে — চালানো হয়, ডেটাকে কাজের কাছে টেনে না এনে। মাল্টি-ক্লাউড প্রসঙ্গে S3-সামঞ্জস্যপূর্ণ API-র মূল্য এখানেই: একই ক্লায়েন্ট কোড, সামান্য endpoint পরিবর্তনে, অন্য ক্লাউডের অবজেক্ট স্টোরেজের সঙ্গেও কথা বলতে পারে। কিন্তু সামঞ্জস্য মানেই অভিন্নতা নয় — স্টোরেজ ক্লাসের নাম, লাইফসাইকেল নীতির গঠন, IAM মডেল ও এনক্রিপশন বিকল্প প্রতিটি ক্লাউডে আলাদা। তাই মাল্টি-ক্লাউড নকশায় একটি পাতলা বিমূর্ত স্তর (Abstraction Layer) রাখা উচিত, যেখানে বাকেটের নাম, ক্লাস ও নীতি কনফিগারেশন থেকে আসে, কোডে শক্তভাবে (Hard-coded) লেখা থাকে না। আরেকটি সত্য: ক্রস-ক্লাউড প্রতিলিপি প্রযুক্তিগতভাবে সম্ভব হলেও তার খরচ, বিলম্ব ও সামঞ্জস্যের জটিলতা অনেক বেশি — সাধারণত একটি ক্লাউডকে প্রাথমিক সত্যের উৎস রেখে, অন্যটিকে দুর্যোগ-পুনরুদ্ধারের অনুলিপি হিসেবে রাখাই বাস্তবসম্মত।

ডেটা শাসন (Data Governance) ছাড়া অবজেক্ট স্টোরেজ দ্রুত একটি ডেটা-জলাভূমি (Data Swamp) হয়ে ওঠে — যেখানে ডেটা আছে, কিন্তু কেউ জানে না কোনটি কী, কে এর মালিক, কতদিন রাখতে হবে। প্রতিটি বাকেটে তিনটি তথ্য বাধ্যতামূলক করুন: মালিক দল, ডেটার শ্রেণি (যেমন ব্যক্তিগত ডেটা, আর্থিক রেকর্ড, সাধারণ), ও ধারণকাল। মেটাডেটা ও ট্যাগ দিয়ে এই তথ্য অবজেক্ট-স্তরে নামিয়ে আনুন। প্রতি ত্রৈমাসিকে একটি পর্যালোচনা করুন — কোন বাকেটে কত ডেটা, কতদিন ধরে পড়া হয়নি, কোন নীতি কাজ করছে না। এই শৃঙ্খলাটি কোনো প্রযুক্তিগত বৈশিষ্ট্য নয়, একটি পরিচালনার অভ্যাস; কিন্তু এটি ছাড়া খরচ ও ঝুঁকি — দুটোই নীরবে বাড়ে।

১২.১৫ সারাংশ

অবজেক্ট স্টোরেজ একটি সমতল নামস্থানে, HTTP API দিয়ে, প্রায় অসীম স্কেলে ডেটা রাখার মডেল। বাকেট হলো ধারক, অবজেক্ট হলো ডেটা-খণ্ড ও তার মেটাডেটা, কী হলো ঠিকানা। স্টোরেজ ক্লাস ও লাইফসাইকেল নীতি দিয়ে ডেটার বয়স ও ব্যবহার অনুযায়ী খরচ নিয়ন্ত্রণ করা যায়; IAM, বাকেট পলিসি, সার্ভিস ক্রেডেনশিয়াল ও প্রি-সাইন্ড URL দিয়ে অ্যাক্সেস নিয়ন্ত্রণ করা হয়; এনক্রিপশন, সংস্করণ ও প্রতিলিপি দিয়ে স্থায়িত্ব ও নিরাপত্তা নিশ্চিত করা হয়। এটি ছবি, ভিডিও, লগ, ব্যাকআপ ও ডেটা লেকের জন্য উপযুক্ত; লাইভ ডেটাবেস বা ফাইল-সিস্টেমের কাজের জন্য নয়। ভালো নকশা মানে — আলাদা বাকেট, আলাদা চাবি, লিখিত ধারণকাল ও নিয়মিত মহড়া।

১২.১৬ যাচাই প্রশ্ন

1. অবজেক্ট স্টোরেজে ফোল্ডারের ধারণাটি আসলে কী? কী-তে স্ল্যাশ (/) থাকা সত্ত্বেও কেন একে সমতল (Flat) নামস্থান বলা হয়?
2. Standard, Vault ও Cold Vault-এর মধ্যে কখন কোনটি বেছে নেবেন? শুধু সংরক্ষণের দাম দেখে সিদ্ধান্ত নিলে কী ভুল হতে পারে?
3. একটি বাকেট ভুল করে মুছে ফেললে প্রতিলিপি (Replication) কেন সাহায্য করে না? কোন দুটি ব্যবস্থা এই ঝুঁকি কমায়?
4. প্রি-সাইন্ড URL কেন সরাসরি সার্ভিস ক্রেডেনশিয়াল ব্রাউজারে দেওয়ার চেয়ে নিরাপদ? এর মেয়াদ নির্ধারণে কী বিবেচনা করবেন?
5. মাসের বিল হঠাৎ বেড়ে গেলে কোন তিনটি জায়গা প্রথমে পরীক্ষা করবেন, এবং কেন?

১২.১৭ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য তিনটি বাকেটের নকশা লিখুন — ব্যবহারকারীর ছবি, অ্যাপ্লিকেশন লগ, ডেটাবেস ব্যাকআপ — প্রতিটির নাম, অঞ্চল, স্থিতিস্থাপকতা, স্টোরেজ ক্লাস, লাইফসাইকেল নীতি, সংস্করণ নীতি ও অ্যাক্সেস নীতি সহ। প্রতিটি সিদ্ধান্তের পেছনে একটি করে যুক্তি লিখুন।

2. একটি পরীক্ষামূলক বাকেটে S3-সামঞ্জস্যপূর্ণ CLI দিয়ে একটি ফাইল আপলোড করুন, তার মেটাডেটা দেখুন, একটি প্রি-সাইন্ড URL তৈরি করে মেয়াদ শেষ হওয়ার পর তা কাজ করে কি না পরীক্ষা করুন, এবং ফলাফলসহ একটি সংক্ষিপ্ত প্রতিবেদন লিখুন। কোনো বাস্তব চাবি প্রতিবেদনে রাখবেন না।

অধ্যায় ১৩ • PART III — Storage, Database & Application Services

Chapter 13 — Block and File Storage

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক পারবেন — (১) ব্লক স্টোরেজ (Block Storage) ও ফাইল স্টোরেজ (File Storage) প্রথম নীতি থেকে আলাদা করতে এবং প্রতিটি কখন দরকার তা বলতে; (২) ভলিউম (Volume), স্ন্যাপশট (Snapshot) ও ব্যাকআপের সম্পর্ক ব্যাখ্যা করতে; (৩) পারসিস্টেন্ট ভলিউম (Persistent Volume) ও Kubernetes স্টোরেজ ইন্টিগ্রেশন — StorageClass, PVC ও CSI-র ধারণা — বুঝতে; (৪) ডেটা স্থায়িত্ব (Durability), কর্মক্ষমতা (IOPS, Throughput, Latency) ও খরচের বিনিময় বিশ্লেষণ করতে; (৫) ব্যাকআপ ও পুনরুদ্ধার পরিকল্পনা (RPO/RTO) নকশা করতে; এবং (৬) `ibmcloud` CLI ও Kubernetes ম্যানিফেস্ট দিয়ে ভলিউম পরিচালনা করতে।

১৩.২ পূর্বশর্ত পুনরালোচনা

অধ্যায় ১২-এ আমরা অবজেক্ট স্টোরেজ শিখেছি: HTTP API দিয়ে, সমতল নামস্থানে, পুরো অবজেক্ট একবারে পড়া বা লেখা। এবার ভিন্ন দুটি মডেলের কথা ভাবুন। আপনার ল্যাপটপের হার্ড ডিস্কে অপারেটিং সিস্টেম কীভাবে ডেটা লেখে? ছোট ছোট নির্দিষ্ট আকারের **ব্লকে** (Block), যেকোনো ব্লক যেকোনো সময় বদলানো যায় — এটাই ব্লক স্টোরেজের মডেল। আর অফিসের শেয়ার্ড ড্রাইভে একাধিক কম্পিউটার কীভাবে একই ফোল্ডার দেখে? একটি **ফাইল সিস্টেম** নেটওয়ার্কের ওপর ভাগ করে দেওয়া হয় — এটাই ফাইল স্টোরেজের মডেল। আরও মনে রাখুন: কনটেইনারের ভেতরের ফাইল সিস্টেম অস্থায়ী (Ephemeral) — পড মুছে গেলে তার ভেতরের ডেটা মুছে যায়। তাই স্থায়ী ডেটার জন্য কনটেইনারের বাইরে একটি স্থায়ী স্টোরেজ দরকার, যাকে Kubernetes-এ পারসিস্টেন্ট ভলিউম বলা হয়।

১৩.৩ প্রথম নীতি: ব্লক বনাম ফাইল বনাম অবজেক্ট

তিনটি স্টোরেজ মডেল তিনটি ভিন্ন প্রশ্নের উত্তর দেয়। ব্লক স্টোরেজের প্রশ্ন: আমি কি ডিস্কের যেকোনো বাইট যেকোনো সময় বদলাতে পারব? ফাইল স্টোরেজের প্রশ্ন: একাধিক মেশিন কি একই ফোল্ডার কাঠামো একসঙ্গে ব্যবহার করতে পারবে? অবজেক্ট স্টোরেজের প্রশ্ন: কোটি কোটি ডেটা-খণ্ড কি *HTTP* দিয়ে, যেকোনো জায়গা থেকে, সস্তায় রাখা যাবে? কোনো মডেলই সর্বোত্তম নয় — প্রশ্নটি ভুল হলে উত্তরটিও ভুল হয়।

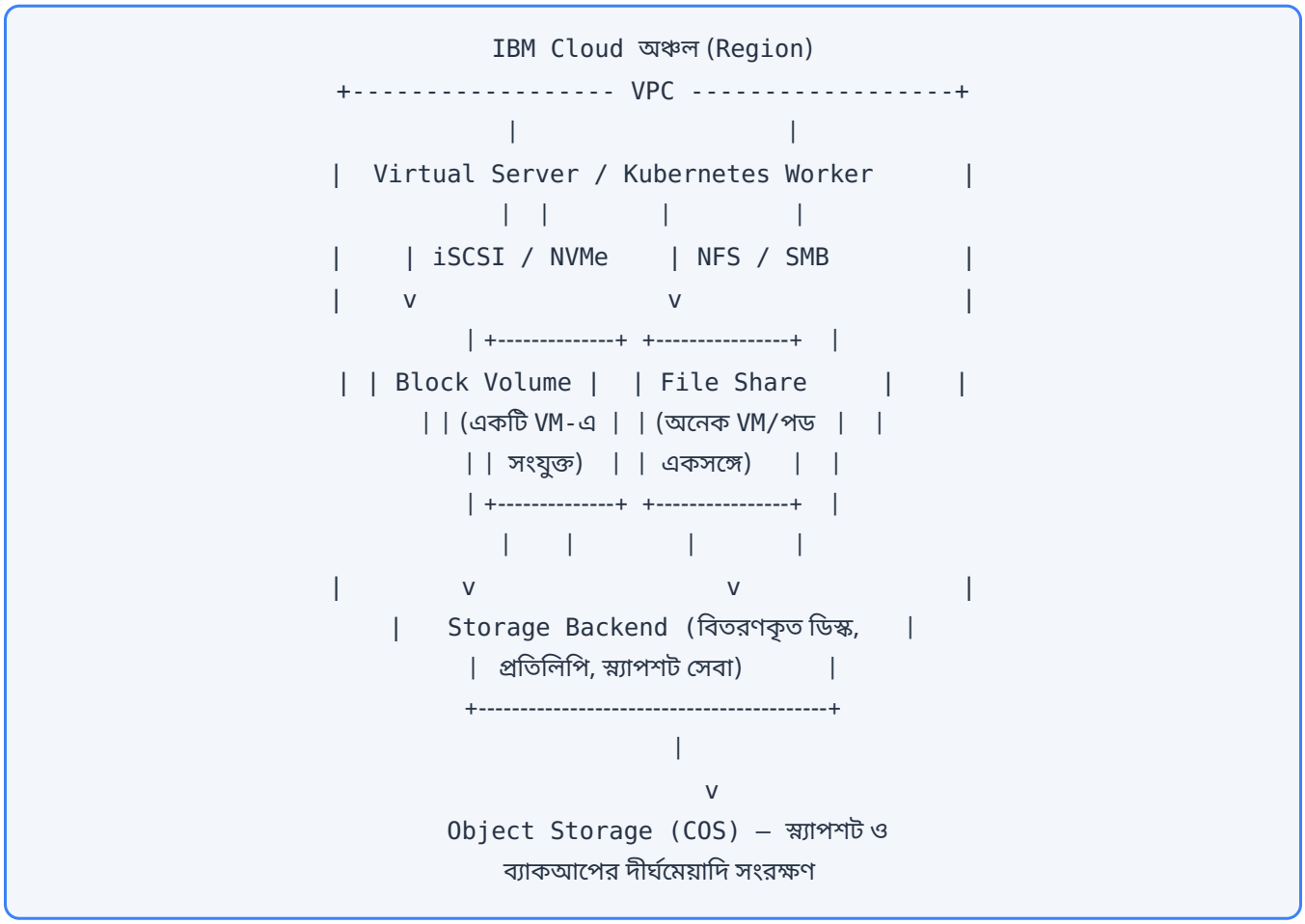
ব্লক স্টোরেজ একটি কাঁচা (Raw) ডিস্ক উপস্থাপন করে। অপারেটিং সিস্টেম তাতে নিজের ফাইল সিস্টেম তৈরি করে (যেমন ext4, XFS), যেকোনো ব্লক পড়ে ও লেখে। এটি দ্রুত, কম বিলম্বের (Low Latency) এবং ডেটাবেস, ভার্চুয়াল মেশিনের ডিস্ক ও লাইভ অ্যাপ্লিকেশনের জন্য উপযুক্ত। কিন্তু একটি ব্লক ভলিউম সাধারণত এক সময়ে একটি মাত্র মেশিনের সঙ্গে সংযুক্ত থাকে। **ফাইল স্টোরেজ** একটি সম্পূর্ণ ফাইল সিস্টেম নেটওয়ার্কের ওপর ভাগ করে দেয় (NFS বা SMB প্রোটোকলে)। একাধিক মেশিন একই ফাইল পড়তে ও লিখতে পারে, ফাইল লকিং ও অনুমতি ফাইল সিস্টেমের নিয়মে চলে। এটি শেয়ার্ড কনফিগারেশন, ব্যবহারকারীর হোম ডিরেক্টরি, মিডিয়া প্রক্রিয়াকরণের কাঁচামাল — যেখানে একাধিক কাজ একই ফাইলে হাত দেয় — সেখানে উপযুক্ত। বিনিময়ে বিলম্ব বেশি হতে পারে এবং এলোমেলো (Random) ছোট লেখায় কর্মক্ষমতা কমতে পারে।

বৈশিষ্ট্য	Block Storage	File Storage	Object Storage
অ্যাক্সেস	ডিস্ক ডিভাইস হিসেবে	NFS/SMB ফাইল সিস্টেম	HTTP API (S3-সামঞ্জস্য)
একসঙ্গে ব্যবহার	সাধারণত একটি মেশিন	একাধিক মেশিন	একাধিক ক্লায়েন্ট
আদর্শ কাজ	ডেটাবেস, VM ডিস্ক	শেয়ার্ড ফাইল, কনটেন্ট	ছবি, লগ, ব্যাকআপ, ডেটা লেক

বিলম্ব	খুব কম	মাঝারি	বেশি
স্কেল	ভলিউমের আকারে সীমিত	শেয়ারের আকারে সীমিত	প্রায় অসীম

> **CONCEPT:** স্টোরেজ বাছাইয়ের সময় প্রথম প্রশ্নটি হওয়া উচিত — ডেটাটি কীভাবে পড়া ও লেখা হবে, কোথা থেকে নয়। ডেটাবেসের ডেটা ফাইল ছোট ছোট এলোমেলো লেখায় বদলায় — তাই ব্লক। একদল কর্মী একই প্রজেক্ট ফোল্ডারে কাজ করে — তাই ফাইল। কোটি ছবি HTTP দিয়ে পরিবেশন করতে হয় — তাই অবজেক্ট। প্রশ্নটি বদলে গেলে উত্তরও বদলায়।

১৩.৪ আর্কিটেকচার ও অভ্যন্তরীণ কার্যপ্রণালি



ব্লক ভলিউমের অভ্যন্তরীণ কার্যপ্রণালি: যখন VM একটি ব্লক লেখে, স্টোরেজ ব্যাকএন্ড সেই ব্লকটিকে একাধিক ভৌত ডিস্কে প্রতিলিপি করে (বা erasure coding করে) এবং লেখা সফল বলে জানায়। পড়ার সময় নিকটতম সুস্থ কপি থেকে ব্লক ফেরত দেওয়া হয়। ফাইল শেয়ারের ক্ষেত্রে একটি ফাইল সার্ভার বা গেটওয়ে NFS/SMB প্রোটোকলের অনুরোধ গ্রহণ করে, ফাইল-স্তরের লক ও অনুমতি পরীক্ষা করে, তারপর ব্যাকএন্ডের ব্লকে ডেটা লেখে। তাই ফাইল স্টোরেজের কর্মক্ষমতা নির্ভর করে নেটওয়ার্ক, ফাইল সার্ভার ও ব্যাকএন্ড — তিনটির ওপরেই।

কর্মক্ষমতার তিনটি মাপকাঠি মনে রাখুন: **IOPS** (প্রতি সেকেন্ডে কতগুলো ইনপুট/আউটপুট অপারেশন), **Throughput** (প্রতি সেকেন্ডে কত বাইট স্থানান্তর), এবং **Latency** (একটি অনুরোধের উত্তরের জন্য কত সময় লাগে)। ডেটাবেসের ছোট এলোমেলো লেখায় IOPS ও Latency গুরুত্বপূর্ণ; বড় ফাইল স্ট্রিমিংয়ে Throughput গুরুত্বপূর্ণ। একটি ভলিউমের জন্য এই

তিনটি মানই তার **প্রোফাইল বা টিয়ার** (যেমন সাধারণ উদ্দেশ্য বনাম উচ্চ কর্মক্ষমতা) দিয়ে নির্ধারিত হয়। কর্মক্ষমতা-সংক্রান্ত সিদ্ধান্ত কখনোই অনুমানে নেবেন না — প্রকৃত কাজের ধরণ (Workload) দিয়ে পরীক্ষা (Benchmark) করুন।

অন্যান্য সেবার সঙ্গে যোগাযোগ: ব্লক ভলিউম VPC-র ভেতরে VM বা Kubernetes worker node-এর সঙ্গে সংযুক্ত হয়; ফাইল শেয়ার VPC-র ভেতরের একাধিক VM/পড থেকে মাউন্ট করা যায়; স্ন্যাপশট ও ব্যাকআপ প্রায়শই COS-এ সংরক্ষিত হয়, ফলে দীর্ঘমেয়াদি সংরক্ষণ সস্তা হয়। ডেটাবেস সেবা (অধ্যায় ১৪) তার ডেটা ফাইল রাখার জন্য ভেতরে ব্লক স্টোরেজ ব্যবহার করে — তবে সেই ব্যবস্থাপনা IBM Cloud করে, ব্যবহারকারী সরাসরি ভলিউম দেখতে পান না।

ব্যর্থ হলে কী হয়? একটি ভলিউমের ব্যাকএন্ড ডিস্ক নষ্ট হলে প্রতিলিপির কারণে সেবা চলতে থাকে, কিন্তু VM-টি নিজে নষ্ট হলে ভলিউমটি অন্য VM-এ সংযুক্ত করতে হয় — এতে সময় লাগে, তাই VM-স্তরের উচ্চ-উপলব্ধতা (High Availability) আলাদাভাবে নকশা করতে হয়। ফাইল শেয়ারের গেটওয়ে নষ্ট হলে সব ক্লায়েন্ট একসঙ্গে সংযোগ হারায়; তাই শেয়ারের উচ্চ-উপলব্ধতা পরীক্ষা করে নেওয়া উচিত। নেটওয়ার্ক বিভাজনে (Network Partition) ব্লক ভলিউমের লেখা আটকে যেতে পারে — অ্যাপ্লিকেশন যেন লেখা আটকে গেলে সঠিকভাবে ত্রুটি জানায়, নীরবে ডেটা হারায় না।

১৩.৫ স্ন্যাপশট, ব্যাকআপ ও পুনরুদ্ধার

স্ন্যাপশট হলো একটি ভলিউমের একটি নির্দিষ্ট মুহূর্তের প্রতিচ্ছবি (Point-in-Time Copy)। এটি সাধারণত কপি-অন-রাইট (Copy-on-Write) পদ্ধতিতে কাজ করে: স্ন্যাপশট নেওয়ার পর যে ব্লকগুলো বদলায়, শুধু সেগুলোর পুরোনো কপি সংরক্ষিত হয়। তাই স্ন্যাপশট দ্রুত নেওয়া যায় ও কম জায়গা নেয়, কিন্তু এটি ব্যাকআপের সম্পূর্ণ বিকল্প নয় — স্ন্যাপশটটি মূল ভলিউমের ওপর নির্ভরশীল হতে পারে। **ব্যাকআপ** হলো ডেটার একটি স্বতন্ত্র অনুলিপি, যা মূল স্টোরেজ থেকে আলাদা জায়গায় রাখা হয় এবং মূল স্টোরেজ সম্পূর্ণ নষ্ট হলেও পুনরুদ্ধার করা যায়। ভালো নীতিতে দুটিই থাকে: ঘন ঘন স্ন্যাপশট (দ্রুত পুনরুদ্ধারের জন্য) ও নিয়মিত ব্যাকআপ (দুর্যোগের জন্য)।

পুনরুদ্ধারের পরিকল্পনা লিখতে হয় RPO ও RTO দিয়ে। **RPO** বলে কতটুকু ডেটা হারানো গ্রহণযোগ্য — যেমন এক ঘণ্টার ডেটা হারানো চলবে, কিন্তু এক দিনের নয়। **RTO** বলে কতক্ষণের মধ্যে সেবা ফিরতে হবে — যেমন দুই ঘণ্টার মধ্যে। RPO নির্ধারণ করে ব্যাকআপের ঘনত্ব, RTO নির্ধারণ করে পুনরুদ্ধারের প্রক্রিয়ার গতি ও প্রস্তুতি। TechNova-র অর্ডার ডেটাবেসের RPO এক ঘণ্টা, RTO দুই ঘণ্টা — তাই প্রতি ঘণ্টায় স্ন্যাপশট ও প্রতিদিন COS-এ ব্যাকআপ। ব্যবহারকারীর প্রোফাইল ছবির RPO এক দিন, RTO এক দিন — তাই দৈনিক ব্যাকআপই যথেষ্ট। দুটি কাজের জন্য একই নীতি রাখা অপচয়।

> **WARNING:** যে ব্যাকআপ কখনো পুনরুদ্ধার করে পরীক্ষা করা হয়নি, সেটি ব্যাকআপ নয় — সেটি একটি আশা। স্ন্যাপশট থেকে নতুন ভলিউম তৈরি করে, ডেটাবেস চালিয়ে, কয়েকটি লেনদেন যাচাই করে দেখুন। এই মহড়াটি ত্রৈমাসিক হওয়া উচিত, এবং মহড়ার সময় মাপা উচিত — কারণ পুনরুদ্ধারে যদি ছয় ঘণ্টা লাগে, তবে দুই ঘণ্টার RTO লেখা অর্থহীন।

একটি গুরুত্বপূর্ণ পার্থক্য: স্ন্যাপশট মুছে ফেলার সময় সতর্ক থাকুন। স্ন্যাপশটগুলো শৃঙ্খল (Chain) আকারে থাকতে পারে — মাঝের একটি স্ন্যাপশট মুছে ফেললে পরের স্ন্যাপশটগুলোর অর্থ বদলে যেতে পারে বা জায়গা না কমে উল্টো বাড়তে পারে। নীতি লেখার সময় স্ন্যাপশট ধারণের সময়সূচি ও মুছে ফেলার ক্রম একসঙ্গে লিখুন, এবং স্বয়ংক্রিয় মুছে ফেলার আগে একটি পরীক্ষামূলক ভলিউমে যাচাই করুন।

১৩.৬ পারসিস্টেন্ট ভলিউম ও Kubernetes স্টোরেজ ইন্টিগ্রেশন

Kubernetes-এ কনটেইনারের ফাইল সিস্টেম অস্থায়ী। পড মুছে গেলে ডেটা মুছে যায়। **পারসিস্টেন্ট ভলিউম (PV)** হলো ক্লাস্টারের স্টোরেজ সম্পদ — একটি ব্লক ভলিউম বা ফাইল শেয়ার যা পডের জীবনচক্রের বাইরে থাকে। **পারসিস্টেন্ট ভলিউম ক্লেইম (PVC)** হলো অ্যাপ্লিকেশনের অনুরোধ: আমার ২০ *Gi* জায়গা চাই, এই অ্যাক্সেস মোডে। **StorageClass**

হলো নীতি: কোন ধরনের স্টোরেজ দেওয়া হবে, কোন প্রোফাইলে, কোন অঞ্চলে। **CSI (Container Storage Interface)** হলো সেই মান (Standard) যার মাধ্যমে Kubernetes বিভিন্ন স্টোরেজ প্রোভাইডারের সঙ্গে কথা বলে — IBM Cloud-এর ব্লক ও ফাইল স্টোরেজের জন্য CSI ড্রাইভার এই কাজটি করে।

অ্যাক্সেস মোড তিনটি: ReadWriteOnce (RWO) — একটি নোড থেকে পড়া-লেখা (ব্লক ভলিউমের সাধারণ মোড); ReadOnlyMany (ROX) — একাধিক নোড থেকে শুধু পড়া; ReadWriteMany (RWX) — একাধিক নোড থেকে পড়া-লেখা (ফাইল স্টোরেজের সাধারণ মোড)। ডেটাবেসের পডের জন্য RWO ব্লক ভলিউম দরকার; শেয়ার্ড মিডিয়া ফোল্ডারের জন্য RWX ফাইল শেয়ার দরকার। ভুল মোড চাইলে পড Pending অবস্থায় আটকে যাবে — এটাই সবচেয়ে সাধারণ Kubernetes স্টোরেজ ত্রুটি।

```
# StorageClass (ধারণাগত — প্রকৃত প্রোভিশনার ও
# প্যারামিটার ক্লাস্টারের ডকুমেন্টেশন দেখে নিন)
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: technova-block
provisioner: block.csi.cloud.ibm.com
parameters:
  type: "5iops-tier"
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
---
# PVC: 20 Gi, একটি নোড থেকে পড়া-লেখা
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: technova-db-data
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: technova-block
  resources:
    requests:
      storage: 20Gi
```

ডায়নামিক প্রোভিশনিং (Dynamic Provisioning) মানে PVC তৈরি হলেই স্বয়ংক্রিয়ভাবে PV ও পেছনের ভলিউম তৈরি হয় — হাতে ভলিউম তৈরি করতে হয় না। কিন্তু **Reclaim Policy** ভুলে যাবেন না: PVC মুছে গেলে PV-র ডেটা কি মুছে যাবে (Delete) নাকি থেকে যাবে (Retain)? প্রোডাকশন ডেটাবেসের জন্য Retain নিরাপদ, কারণ ভুল করে PVC মুছলেও ডেটা থেকে যায়। ভলিউমের আকার বাড়ানো (Expansion) অনেক StorageClass-এ সম্ভব, কিন্তু সংকোচন (Shrink) সাধারণত সম্ভব নয় — তাই আকার নির্ধারণে ভবিষ্যৎ বৃদ্ধি ধরে নিন।

> **REAL WORLD:** TechNova-র ডেটাবেস পড একবার ভুল StorageClass-এ RWX চেয়ে Pending হয়ে গিয়েছিল — কারণ ব্লক ভলিউম RWX দিতে পারে না। সমাধান ছিল সহজ: RWO চাওয়া, অথবা ফাইল শেয়ারের StorageClass ব্যবহার করা। এই ভুলটি প্রতিদিন ঘটে, কারণ অ্যাপ্লিকেশন ডেভেলপার স্টোরেজের ধরন না জেনে PVC লেখেন। তাই TechNova-র নিয়ম: প্রতিটি অ্যাপ্লিকেশনের README-তে লেখা থাকে কোন StorageClass ও অ্যাক্সেস মোড দরকার, এবং কেন।

১৩.৭ CLI উদাহরণ, ডেটা স্থায়িত্ব, কর্মক্ষমতা ও খরচ

```
# ভলিউম তালিকা ও বিবরণ (VPC ব্লক স্টোরেজ, ধারণাগত)
ibmcloud is volumes
ibmcloud is volume technova-db-vol-01

# স্ন্যাপশট তৈরি (নামে তারিখ ও উদ্দেশ্য রাখুন)
ibmcloud is snapshot-create technova-db-snap-2026-11-09 \
  --source-volume technova-db-vol-01

# স্ন্যাপশট থেকে নতুন ভলিউম (পুনরুদ্ধারের পরীক্ষা)
ibmcloud is volume-create technova-db-restore-test \
  --snapshot technova-db-snap-2026-11-09 \
  --profile 5iops-tier --size 20 --zone us-south-1
```

ডেটা স্থায়িত্ব (Durability) মানে ডেটা হারানোর সম্ভাবনা কত কম। ব্লক ও ফাইল স্টোরেজে স্থায়িত্ব আসে প্রতিলিপি ও erasure coding থেকে, কিন্তু এটি অবজেক্ট স্টোরেজের মতো স্বয়ংক্রিয়ভাবে বোঝানো উচিত নয় — ভলিউমের ব্যাকএন্ড, অঞ্চল ও স্ন্যাপশট নীতি দেখে নিশ্চিত হতে হয়। একটি একক জোনের ভলিউমের জন্য জোন-স্তরের ব্যর্থতার পরিকল্পনা আলাদাভাবে করতে হয়।

খরচের চালক: ভলিউমের আকার (GB), কর্মক্ষমতা প্রোফাইল (IOPS-ভিত্তিক টিয়ার), স্ন্যাপশটের সংরক্ষিত আকার, এবং ফাইল শেয়ারের ক্ষেত্রে নির্ধারিত ক্ষমতা। একটি সাধারণ অপচয় হলো প্রয়োজনের চেয়ে অনেক বড় ভলিউম তৈরি করে রাখা — কারণ খরচ নির্ধারিত আকারের ওপর, ব্যবহৃত আকারের ওপর নয়। নিয়মিতভাবে ভলিউমের ব্যবহার (Utilization) পরীক্ষা করুন: যে ভলিউম ২০% ব্যবহৃত হচ্ছে, তার আকার বা প্রোফাইল পুনর্বিবেচনা করুন। তবে ডেটাবেসের ভলিউমে আকার কমানো ঝুঁকিপূর্ণ — প্রথমে ব্যবহারের ধরণ ও বৃদ্ধির হার দেখুন।

১৩.৮ প্রোডাকশন পরিস্থিতি, নিরাপত্তা, সাধারণ ভুল ও ট্রাবলশুটিং

TechNova-র ডেটাবেস: PostgreSQL পড RWO ব্লক ভলিউমে চলে, প্রতি ঘণ্টায় স্ন্যাপশট, প্রতিদিন COS-এ ব্যাকআপ। **শেয়ার্ড মিডিয়া:** ভিডিও প্রক্রিয়াকরণের কর্মীরা (Worker) RWX ফাইল শেয়ার থেকে একই ইনপুট পড়ে, আউটপুট লেখে। **অস্থায়ী কাজ:** বিল্ড ক্যাশ ও লগের জন্য অস্থায়ী ভলিউম যথেষ্ট — সেখানে পারসিস্টেন্ট ভলিউম অপচয়।

নিরাপত্তা: ভলিউম ও শেয়ারের অ্যাক্সেস VPC-র ভেতরে সীমিত রাখুন; ফাইল শেয়ারের NFS রপ্তানি (Export) শুধুমাত্র নির্দিষ্ট সাবনেটে সীমিত করুন; স্ন্যাপশট ও ব্যাকআপ এনক্রিপ্ট করুন; কে স্ন্যাপশট মুছতে পারে তা IAM দিয়ে সীমিত করুন — কারণ ব্যাকআপ মুছে ফেলা র‍্যানসমওয়্যার আক্রমণের একটি সাধারণ ধাপ। খরচের দিক থেকে স্ন্যাপশট ধারণকাল ও ভলিউম ব্যবহার নিয়মিত পর্যালোচনা করুন।

সাধারণ ভুল: (১) ডেটাবেসের জন্য ফাইল শেয়ার ব্যবহার করা (RWX-এর লোভে) — ডেটাবেসের এলোমেলো লেখায় ফাইল স্টোরেজের বিলম্ব সমস্যা তৈরি করে। (২) Reclaim Policy Delete রেখে প্রোডাকশন PVC মোছা। (৩) স্ন্যাপশটকে ব্যাকআপ ভাবা। (৪) ভলিউমের আকার না দেখে PVC-তে অনেক বেশি জায়গা চাওয়া। (৫) একটিমাত্র জোনে সব ভলিউম রাখা।

TROUBLESHOOTING: পড Pending ও `FailedMount` দেখলে প্রথমে `kubectl describe pvc` ও `kubectl describe pod` দেখুন — StorageClass-এর নাম, অ্যাক্সেস মোড ও আকারের অনুরোধ মিলছে কি না। `FailedAttach` দেখলে ভলিউমটি অন্য নোডে সংযুক্ত আছে কি না দেখুন — ব্লক ভলিউম একসঙ্গে দুটি নোডে সংযুক্ত হতে পারে না। ফাইল শেয়ার মাউন্ট না হলে NFS পোর্ট ও সিকিউরিটি গ্রুপের নিয়ম পরীক্ষা করুন। ধীর ডেটাবেসে প্রথমে ভলিউমের IOPS ও Latency মেট্রিক দেখুন — অনেক সময় সমস্যা ডেটাবেসে নয়, ভলিউমের প্রোফাইলে।

১৩.৮.১ কর্মক্ষমতা টিউনিং ও ক্ষমতা পরিকল্পনা

ভলিউমের কর্মক্ষমতা টিউনিং শুরু হয় কাজের ধরণ (Workload Profile) মাপা দিয়ে, অনুমান দিয়ে নয়। প্রথমে জানুন — পড়া বনাম লেখার অনুপাত কত, অনুরোধগুলো ক্রমিক (Sequential) না এলোমেলো (Random), গড় অনুরোধের আকার কত, এবং শীর্ষ সময়ে (Peak) IOPS কত হয়। ডেটাবেসের লেনদেন-লগ ক্রমিক লেখা পছন্দ করে, ডেটা ফাইল এলোমেলো পড়া-লেখা করে; ভিডিও স্ট্রিমিং বড় ক্রমিক পড়া করে; বিল্ড ক্যাশ ছোট এলোমেলো লেখা করে। প্রতিটি ধরণের জন্য আলাদা প্রোফাইল উপযুক্ত। একটি ভলিউমকে প্রয়োজনের চেয়ে অনেক উঁচু প্রোফাইলে রাখা অর্থের অপচয়; প্রয়োজনের চেয়ে নিচু প্রোফাইলে রাখা মানে ব্যবহারকারীর সামনে ধীর অ্যাপ্লিকেশন। TechNova-র নিয়ম: নতুন ভলিউম প্রথমে মাঝারি প্রোফাইলে চালানো হয়, দুই সপ্তাহের মেট্রিক দেখে প্রোফাইল ঠিক করা হয়, এবং শীর্ষ সময়ের মেট্রিকের ভিত্তিতে সিদ্ধান্ত নেওয়া হয় — গড়ের ভিত্তিতে নয়। ক্ষমতা পরিকল্পনায় (Capacity Planning) শুধু বর্তমান ব্যবহার নয়, বৃদ্ধির হার দেখুন: গত ছয় মাসে মাসিক বৃদ্ধি কত শতাংশ, কোনো মৌসুমি শীর্ষ আছে কি না, ডেটা ধারণ নীতি বদলালে বৃদ্ধি কেমন হবে। ডিস্ক ৮৫% পূর্ণ হওয়ার আগেই সতর্কতা বাজা উচিত, কারণ ৯৫%-এ পৌঁছে গেলে ডেটাবেস লেখা বন্ধ করে দিতে পারে এবং জরুরি সম্প্রসারণে ভুল হওয়ার সম্ভাবনা বাড়ে। ভলিউম সম্প্রসারণের প্রক্রিয়াটি আগে পরীক্ষামূলক ভলিউমে অনুশীলন করুন — ফাইল সিস্টেম সম্প্রসারণের ধাপটি ভুলে গেলে ভলিউম বড় হলেও অপারেটিং সিস্টেম তা দেখতে পাবে না।

১৩.৮.২ মাল্টি-জোন স্থাপত্য ও স্ন্যাপশট নীতির বিস্তার

একটি ভলিউম সাধারণত একটি জোনে থাকে; তাই জোন-স্তরের ব্যর্থতা থেকে বাঁচতে অ্যাপ্লিকেশন-স্তরে নকশা করতে হয়। TechNova-র ডেটাবেসের ক্ষেত্রে প্রাইমারি এক জোনে, রেপ্লিকা আরেক জোনে — ভলিউম-স্তরে নয়, ডেটাবেস-স্তরে প্রতিলিপি দিয়ে। ফাইল শেয়ারের ক্ষেত্রে শেয়ারটি যেন একাধিক জোনের ক্লায়েন্ট থেকে পৌঁছানো যায় তা যাচাই করতে হয়, এবং শেয়ারের ব্যাকআপ COS-এ রাখা হয় যাতে জোন-ব্যাপী বিপর্যয়েও ডেটা ফেরত পাওয়া যায়। স্ন্যাপশট নীতির বিস্তার: শুধু সময়সূচি নয়, স্ন্যাপশটের নামকরণ, ধারণকাল, ও মুছে ফেলার ক্রম — সবই নীতির অংশ। নামে তারিখ, ভলিউমের নাম ও উদ্দেশ্য রাখুন, যেমন `db-vol-01-daily-2026-11-09`; এতে জরুরি মুহূর্তে ভুল স্ন্যাপশট থেকে পুনরুদ্ধারের ঝুঁকি কমে। স্ন্যাপশটের ধারণকাল ব্যবসায়িক চাহিদা দিয়ে ঠিক করুন — দৈনিক স্ন্যাপশট ৭ দিন, সাপ্তাহিক ৪ সপ্তাহ, মাসিক ১২ মাস — এবং প্রতিটি স্তরের খরচ মাসিক প্রতিবেদনে দেখুন। একটি সূক্ষ্ম ঝুঁকি: স্ন্যাপশট থেকে তৈরি ভলিউমে পুরোনো ডেটা থাকে, কিন্তু অ্যাপ্লিকেশনের কনফিগারেশন ও সিক্রেট থাকে না — পুনরুদ্ধারের পরীক্ষায় পুরো অ্যাপ্লিকেশন স্ট্যাক চালিয়ে দেখা উচিত, শুধু ভলিউম মাউন্ট করে নয়।

১৩.৯ সারাংশ

ব্লক স্টোরেজ একটি কাঁচা ডিস্ক — দ্রুত, কম বিলম্বের, একটিমাত্র মেশিনের জন্য; ফাইল স্টোরেজ একটি ভাগ করা ফাইল সিস্টেম — একাধিক মেশিনের জন্য; অবজেক্ট স্টোরেজ HTTP-ভিত্তিক — প্রায় অসীম স্কেলের জন্য। স্ন্যাপশট দ্রুত পুনরুদ্ধারের জন্য, ব্যাকআপ দুর্যোগের জন্য; দুটিরই RPO/RTO লিখিত থাকা উচিত। Kubernetes-এ PV, PVC, StorageClass ও CSI এই তিন মডেলকে পডের সঙ্গে যুক্ত করে; সঠিক অ্যাক্সেস মোড ও Reclaim Policy বেছে নেওয়া প্রোডাকশনের প্রাথমিক শৃঙ্খলা।

১৩.১০ যাচাই প্রশ্ন

1. ডেটাবেসের ডেটা ফাইলের জন্য ব্লক স্টোরেজ কেন ফাইল স্টোরেজের চেয়ে উপযুক্ত, আর শেয়ার্ড মিডিয়া ফোল্ডারের জন্য উল্টোটা কেন সত্য?
2. স্ন্যাপশট ও ব্যাকআপের মধ্যে পার্থক্য কী? শুধু স্ন্যাপশটের ওপর নির্ভর করলে কোন বিপদ থাকে?
3. RWO, ROX ও RWX-এর মধ্যে পার্থক্য কী? একটি ডেটাবেস পড RWX চাইলে কী হবে?
4. Reclaim Policy Delete বনাম Retain — প্রোডাকশন ডেটাবেসের জন্য কোনটি নিরাপদ, এবং কেন?
5. ভলিউমের খরচ নির্ধারিত আকারের ওপর হয়, ব্যবহৃত আকারের ওপর নয় — এই তথ্য থেকে কোন দুটি অপ্টিমাইজেশন বেরিয়ে আসে?

১৩.১১ ব্যবহারিক অনুশীলন

1. TechNova-র তিনটি কাজের জন্য স্টোরেজ নকশা করুন — PostgreSQL ডেটাবেস, শেয়ার্ড ভিডিও প্রক্রিয়াকরণ, দৈনিক ব্যাকআপ — প্রতিটির জন্য মডেল (ব্লক/ফাইল/অবজেক্ট), অ্যাক্সেস মোড, RPO/RTO ও স্ন্যাপশট সময়সূচি লিখুন।
2. একটি পরীক্ষামূলক Kubernetes ক্লাস্টারে StorageClass, PVC ও একটি পড তৈরি করুন যেখানে পডটি ভলিউমে একটি ফাইল লেখে; পডটি মুছে আবার তৈরি করে দেখুন ফাইলটি থাকে কি না; তারপর PVC মুছে Reclaim Policy-র আচরণ পর্যবেক্ষণ করুন।

অধ্যায় ১৪ • PART III — Storage, Database & Application Services

Chapter 14 — Managed Database Engineering

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক পারবেন — (১) পরিচালিত ডেটাবেস (Managed Database) কী সমস্যা সমাধান করে তা ব্যাখ্যা করতে; (২) IBM Cloud Databases-এর ধারণা ও সমর্থিত পরিচালিত ইঞ্জিনগুলোর (যেমন PostgreSQL, MySQL, MongoDB, Redis, Elasticsearch) ভূমিকা আলাদা করতে; (৩) সংযোগ স্থাপত্য (Connection Architecture) — প্রাইভেট ও পাবলিক endpoint, সংযোগ পুলিং — নকশা করতে; (৪) ব্যাকআপ, পুনরুদ্ধার ও উচ্চ-উপলব্ধতার (High Availability) নকশা ব্যাখ্যা করতে; (৫) ডেটাবেস নিরাপত্তা, পর্যবেক্ষণ ও প্রোডাকশন পরিচালনা (Operations) — প্যারামিটার, আপগ্রেড, স্কেলিং — পরিচালনা করতে; এবং (৬) CLI ও IaC দিয়ে ডেটাবেস সেবা তৈরি ও পরিচালনা করতে।

১৪.২ পূর্বশর্ত পুনরালোচনা

ডেটাবেস মানে সংগঠিত ডেটার ভাণ্ডার, যেখানে ডেটা টেবিল, ডকুমেন্ট বা কী-ভ্যালু আকারে থাকে এবং প্রশ্ন (Query) দিয়ে খুঁজে পাওয়া যায়। সম্পর্কযুক্ত (Relational) ডেটাবেস — যেমন PostgreSQL ও MySQL — টেবিল, সারি, কলাম ও SQL ব্যবহার করে; ডকুমেন্ট ডেটাবেস — যেমন MongoDB — JSON-সদৃশ ডকুমেন্ট রাখে; ইন-মেমরি স্টোর — যেমন Redis — দ্রুত ক্যাশ ও কিউয়ের কাজ করে; অনুসন্ধান ইঞ্জিন — যেমন Elasticsearch — পূর্ণ-পাঠ্য অনুসন্ধান করে। একটি ডেটাবেস সার্ভার চালাতে হয় অপারেটিং সিস্টেম, ডিস্ক, ব্যাকআপ, প্যাচ, প্রতিলিপি — সবই। পরিচালিত ডেটাবেস এই পরিচালনার বোঝা সেবা-প্রদানকারীর ওপর দেয়, কিন্তু ডেটা মডেল, প্রশ্ন ও অ্যাক্সেস নিয়ন্ত্রণের দায়িত্ব ব্যবহারকারীরই থাকে।

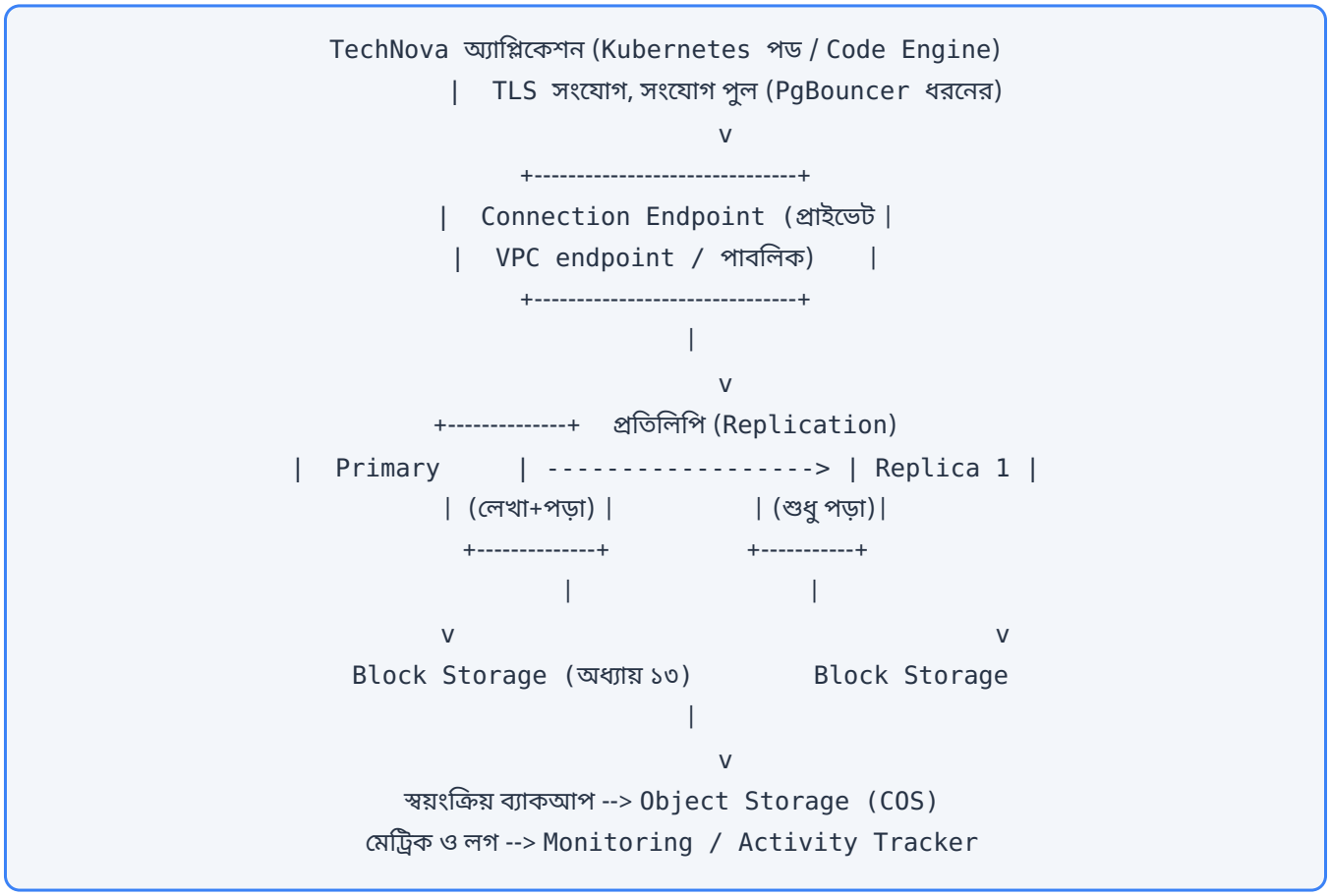
১৪.৩ প্রথম নীতি: কেন পরিচালিত ডেটাবেস

নিজে ডেটাবেস চালানো মানে শুধু সফটওয়্যার ইনস্টল করা নয়। ডিস্ক পূর্ণ হলে কী হবে? প্রাইমারি সার্ভার নষ্ট হলে রেপ্লিকা কীভাবে প্রাইমারি হবে? নিরাপত্তা প্যাচ কখন দেওয়া হবে, সেবা বন্ধ না করে? ব্যাকআপ কি সত্যিই পুনরুদ্ধার করা যায়? এই প্রশ্নগুলোর উত্তর দিতে দিতে দলটি ডেটাবেস পরিচালনায় ব্যস্ত হয়ে পড়ে, প্রোডাক্ট তৈরিতে নয়। পরিচালিত ডেটাবেস এই সমস্যার উত্তর দেয়: সেবা-প্রদানকারী স্বয়ংক্রিয়ভাবে প্রোভিশনিং, প্যাচিং, ব্যাকআপ, প্রতিলিপি ও পর্যবেক্ষণের ভিত্তি দেয়; ব্যবহারকারী মনোযোগ দেয় স্কিমা (Schema), প্রশ্নের কর্মক্ষমতা ও অ্যাক্সেস নিয়ন্ত্রণে।

কিন্তু পরিচালিত মানেই দায়িত্ব শেষ নয় — এটাই সবচেয়ে সাধারণ ভুল বোঝাবুঝি। সেবা-প্রদানকারী অবকাঠামো চালায়, কিন্তু ডেটার মালিকানা, কে কোন টেবিল পড়তে পারবে, কোন প্রশ্ন ধীর, কোন ইনডেক্স দরকার, ব্যাকআপ থেকে পুনরুদ্ধারের পরীক্ষা — এসব ব্যবহারকারীর দায়িত্ব। আরেকটি সত্য: পরিচালিত সেবায় কিছু নিয়ন্ত্রণ ছেড়ে দিতে হয় — অপারেটিং সিস্টেমে সরাসরি প্রবেশ, সব কনফিগারেশন প্যারামিটার বদলানো, অস্বাভাবিক এক্সটেনশন ইনস্টল করা — সবসময় সম্ভব নাও হতে পারে। এই বিনিময়টি বুঝে সিদ্ধান্ত নিতে হয়: দলের আকার ছোট হলে পরিচালিত সেবার সাশ্রয় অনেক বড়; দলের বিশেষ চাহিদা খুব বেশি হলে নিজে চালানোর নমনীয়তা দরকার হতে পারে।

> **CONCEPT:** পরিচালিত ডেটাবেসে **নিয়ন্ত্রণ-সমতল (Control Plane)** ও **ডেটা-সমতল (Data Plane)** আলাদা। নিয়ন্ত্রণ-সমতল সিদ্ধান্ত নেয় — সেবা তৈরি, স্কেল, ব্যাকআপ সময়সূচি, প্যাচ। ডেটা-সমতল ডেটা রাখে ও প্রশ্নের উত্তর দেয়। ব্যবহারকারী নিয়ন্ত্রণ-সমতলের API দিয়ে নির্দেশ দেয়, কিন্তু ডেটা-সমতলের ভেতরের ডিস্কে সরাসরি হাত দেয় না। এই বিভাজনই পরিচালিত সেবাকে নিরাপদ ও পুনরুৎপাদনযোগ্য করে।

১৪.৪ আর্কিটেকচার ও অভ্যন্তরীণ কার্যপ্রণালি



একটি লেখার অনুরোধের যাত্রা: অ্যাপ্লিকেশন সংযোগ পুল থেকে একটি সংযোগ নেয়, SQL পাঠায়, প্রাইমারি সেটি WAL (Write-Ahead Log) — অর্থাৎ ডিস্কে লেখার আগে একটি লগে লিপিবদ্ধ করা — পদ্ধতিতে প্রথমে লগে লেখে, তারপর ডেটা ফাইলে প্রয়োগ করে, এবং প্রতিলিপিকে (Replica) সেই লগ পাঠায়। রেপ্লিকা লগটি প্রয়োগ করে নিজের ডেটা হালনাগাদ করে। এই প্রক্রিয়াকে বলে **স্ট্রিমিং রেপ্লিকেশন** (Streaming Replication)। প্রাইমারি নষ্ট হলে একটি রেপ্লিকা নতুন প্রাইমারি হিসেবে উন্নীত (Promote) হয় — একে বলে **ফেইলওভার (Failover)**। ফেইলওভারের সময় কয়েক সেকেন্ড থেকে কয়েক মিনিট সেবা ব্যাহত হতে পারে; অ্যাপ্লিকেশনের সংযোগ পুলকে এই ব্যাঘাত সহ্য করতে হবে — অর্থাৎ সংযোগ ভেঙে গেলে পুনরায় চেষ্টা (Retry) করতে হবে, ভেঙে পড়লে চলবে না।

সংযোগ স্থাপত্যের মূল প্রশ্ন দুটি: **কোথা থেকে সংযোগ আসবে** এবং **কতগুলো সংযোগ**। প্রোডাকশনে ডেটাবেসের endpoint হওয়া উচিত প্রাইভেট — VPC-র ভেতরে — যাতে ইন্টারনেট থেকে সরাসরি সংযোগ না আসে। পাবলিক endpoint শুধুমাত্র প্রয়োজনে, IP অনুমতি-তালিকা (Allowlist) সহ চালু করা উচিত। সংযোগের সংখ্যার প্রশ্নটি গুরুত্বপূর্ণ: প্রতিটি অ্যাপ্লিকেশন ইনস্ট্যান্স যদি শত শত সংযোগ খুলে রাখে, ডেটাবেসের সংযোগ-সীমা শেষ হয়ে যাবে। সমাধান হলো **সংযোগ পুলিং (Connection Pooling)**: অ্যাপ্লিকেশন ও ডেটাবেসের মাঝে একটি পুল — যেমন PgBouncer — অল্প সংখ্যক প্রকৃত সংযোগ ভাগ করে ব্যবহার করে। পুলের আকার নির্ধারণ করতে হয় পরীক্ষা করে: খুব ছোট পুলে অনুরোধ লাইনে দাঁড়ায়, খুব বড় পুলে ডেটাবেস চাপে পড়ে।

অন্যান্য সেবার সঙ্গে যোগাযোগ: অ্যাপ্লিকেশন সার্ভার, Kubernetes পড ও Code Engine কাজ ডেটাবেসের ক্লায়েন্ট; ব্যাকআপ COS-এ যায়; মেট্রিক ও লগ মনিটরিং সেবায় যায়; সিক্রেট (পাসওয়ার্ড) পরিচালিত হয় সিক্রেট-পরিচালনা সেবা বা Kubernetes Secret-এ — কখনোই সোর্স কোডে নয়। ডেটাবেস নিজে অন্য সেবার ওপর নির্ভরশীল: ব্লক স্টোরেজের ওপর (ডেটা ফাইলের জন্য) ও নেটওয়ার্কের ওপর (প্রতিলিপির জন্য)।

ব্যর্থ হলে কী হয়? প্রাইমারি নষ্ট হলে ফেইলওভার হয়, কিন্তু ফেইলওভারের মুহূর্তে যে লেনদেনগুলো এখনও রেপ্লিকায় পৌঁছায়নি, সেগুলো হারাতে পারে — এটাই সিঙ্ক্রোনাস বনাম অ্যাসিঙ্ক্রোনাস প্রতিলিপির বিনিময়। সিঙ্ক্রোনাস প্রতিলিপিতে প্রতিটি লেখা রেপ্লিকায় নিশ্চিত না হলে সফল বলে জানানো হয় না — ডেটা নিরাপদ, কিন্তু লেখা ধীর। অ্যাসিঙ্ক্রোনাসে লেখা দ্রুত, কিন্তু ফেইলওভারে সামান্য ডেটা হারানোর ঝুঁকি থাকে। কোনটি বেছে নেবেন তা ব্যবসায়িক সিদ্ধান্ত: আর্থিক লেনদেনে সিঙ্ক্রোনাসের মূল্য দেওয়া উচিত, বিশ্লেষণমূলক ডেটায় অ্যাসিঙ্ক্রোনাস যথেষ্ট হতে পারে।

১৪.৫ সমর্থিত পরিচালিত ইঞ্জিন: কোনটি কখন

IBM Cloud Databases-এ একাধিক পরিচালিত ইঞ্জিন সমর্থিত — এই বইয়ে এগুলোকে **সমর্থিত পরিচালিত ইঞ্জিন** হিসেবে বর্ণনা করা হচ্ছে, সম্পূর্ণ তালিকা দাবি না করে। প্রতিটি ইঞ্জিন একটি ভিন্ন ডেটা মডেলের জন্য উপযুক্ত, এবং ভুল ইঞ্জিন বেছে নেওয়া পরে বদলানো খুব কঠিন।

ইঞ্জিন	ডেটা মডেল	আদর্শ ব্যবহার	এড়িয়ে চলুন যখন
PostgreSQL	সম্পর্কযুক্ত, SQL	লেনদেন, জটিল প্রশ্ন, JSON-সহ সম্পর্কযুক্ত ডেটা	শুধু কী-ভ্যালু ক্যাশ দরকার
MySQL	সম্পর্কযুক্ত, SQL	ওয়েব অ্যাপ, পরিচিত MySQL পরিবেশ	উন্নত বিশ্লেষণমূলক প্রশ্ন বেশি
MongoDB	ডকুমেন্ট (JSON-সদৃশ)	নমনীয় স্কিমা, ডকুমেন্ট-কেন্দ্রিক ডেটা	জটিল জয়েন ও লেনদেন বেশি
Redis	ইন-মেমরি কী-ভ্যালু	ক্যাশ, সেশন, কিউ, রেট-লিমিট	স্থায়ী, একমাত্র সত্যের উৎস হিসেবে
Elasticsearch	অনুসন্ধান সূচক	পূর্ণ-পাঠ্য অনুসন্ধান, লগ অনুসন্ধান	প্রাথমিক লেনদেনের ডেটাবেস হিসেবে

TechNova-র নকশা: মূল ব্যবসায়িক ডেটা — ব্যবহারকারী, অর্ডার, পেমেন্ট — PostgreSQL-এ; সেশন ও ক্যাশ Redis-এ; পণ্য অনুসন্ধান Elasticsearch-এ। Redis-কে কখনোই একমাত্র সত্যের উৎস (Source of Truth) করা হয় না — Redis মুছে গেলে ডেটা PostgreSQL থেকে আবার তৈরি করা যায়। Elasticsearch-এর সূচকও PostgreSQL থেকে পুনর্গঠনযোগ্য। এই নীতিটির নাম **পুনর্গঠনযোগ্যতা (Rebuildability)**: প্রতিটি সহায়ক স্টোর মুছে গেলেও মূল ডেটা থেকে আবার তৈরি করা যায় কি না — তা নকশার সময় যাচাই করতে হয়।

১৪.৬ ব্যাকআপ, পুনরুদ্ধার ও উচ্চ-উপলব্ধতা

পরিচালিত ডেটাবেসে ব্যাকআপ স্বয়ংক্রিয়ভাবে নেওয়া হয় — সাধারণত দৈনিক পূর্ণ ব্যাকআপ ও তার মাঝে WAL বা লগ-ভিত্তিক ব্যাকআপ, যার মাধ্যমে **পয়েন্ট-ইন-টাইম রিকভারি (PITR)** সম্ভব হয়: যেকোনো নির্দিষ্ট মুহূর্তে ডেটাবেসকে ফিরিয়ে নেওয়া যায়। কিন্তু ব্যাকআপ থাকা মানেই পুনরুদ্ধার নিশ্চিত নয়। পুনরুদ্ধারের পরীক্ষা করতে হয়: ব্যাকআপ থেকে একটি নতুন ডেটাবেস ইনস্ট্যান্স তৈরি করে, কয়েকটি গুরুত্বপূর্ণ টেবিলের সারি গুনে, একটি লেনদেন যাচাই করে দেখা। এই পরীক্ষা ত্রৈমাসিক হওয়া উচিত।

উচ্চ-উপলব্ধতার নকশা: একাধিক জোনে প্রাইমারি ও রেপ্লিকা, স্বয়ংক্রিয় ফেইলওভার, স্বাস্থ্য পরীক্ষা (Health Check) ও সতর্কতা। কিন্তু উচ্চ-উপলব্ধতা ডেটা হারানো থেকে বাঁচায় না — ভুল করে একটি টেবিল মুছে ফেললে প্রতিলিপিও সেই মুছে ফেলা প্রতিলিপি করবে। তাই উচ্চ-উপলব্ধতা ও ব্যাকআপ দুটি ভিন্ন জিনিস: প্রথমটি সেবা চালু রাখে, দ্বিতীয়টি ডেটা ফেরত দেয়। দুটিই দরকার।

> **WARNING:** `DROP TABLE` বা ভুল `DELETE` (WHERE ছাড়া) প্রতিলিপি হয়ে যায়। পরিচালিত ডেটাবেস এই ভুল থেকে বাঁচায় না — বাঁচায় PITR ও ব্যাকআপ। প্রোডাকশনে লেখার অ্যাক্সেস সীমিত রাখুন, ধ্বংসাত্মক কমান্ড চালানোর আগে ব্যাকআপ নিন, এবং সম্ভব হলে প্রথমে একটি কপিতে পরীক্ষা করুন।

১৪.৭ CLI ও IaC উদাহরণ

```
# ডেটাবেস সেবা-ইনস্ট্যান্স তৈরি (ধারণাগত —
# পরিকল্পনা ও সংস্করণ নিজের অ্যাকাউন্ট অনুযায়ী দিন)
ibmcloud resource service-instance-create \
  technova-pg databases-for-postgresql standard us-south

# সংযোগের তথ্য ও ক্রেডেনশিয়াল দেখা
ibmcloud resource service-key-create \
  technova-pg-admin Administrator \
  --instance-name technova-pg

# ব্যাকআপ তালিকা দেখা (সেবা-নির্দিষ্ট কমান্ড
# বর্তমান ডকুমেন্টেশন দেখে নিশ্চিত করুন)
ibmcloud cdb list --help
```

```
-- PostgreSQL-এ সংযোগ (psql, ধারণাগত)
-- হোস্ট, পোর্ট ও পাসওয়ার্ড সার্ভিস ক্রেডেনশিয়াল থেকে নিন
-- পাসওয়ার্ড কখনোই এখানে লিখবেন না
psql "host=CHANGE-ME-HOST port=5432 \
  dbname=ibmclouddb user=admin sslmode=verify-full"

-- সংযোগ পুলের আকার দেখা ও ধীর প্রশ্ন খোঁজা
SELECT pid, usename, application_name, state,
       now() - query_start AS duration, query
FROM pg_stat_activity
WHERE state = 'active'
ORDER BY duration DESC LIMIT 10;
```

IaC-এ ডেটাবেস তৈরি করার সময় পাসওয়ার্ড Terraform-এ লিখবেন না। সার্ভিস ক্রেডেনশিয়াল তৈরি করে সেটিকে সিক্রেট-পরিচালনা সেবায় সংরক্ষণ করুন, এবং অ্যাপ্লিকেশন সেখান থেকে পড়ুক। ডেটাবেসের সংস্করণ আপগ্রেড (যেমন PostgreSQL 15 থেকে 16) IaC-তে একটি ছোট পরিবর্তন মনে হতে পারে, কিন্তু এটি ডেটা মাইগ্রেশনের মতো গুরুত্বপূর্ণ — প্রথমে staging-এ পরীক্ষা করুন, ব্যাকআপ নিন, এবং আপগ্রেডের সময় সেবা-ব্যাঘাতের পরিকল্পনা করুন।

১৪.৮ নিরাপত্তা, পর্যবেক্ষণ ও প্রোডাকশন পরিচালনা

নিরাপত্তার স্তরগুলো: **নেটওয়ার্ক** — প্রাইভেট endpoint, IP অনুমতি-তালিকা, VPC-র ভেতরে রাখা; **প্রমাণীকরণ** — শক্তিশালী পাসওয়ার্ড, প্রতিটি অ্যাপ্লিকেশনের জন্য আলাদা ব্যবহারকারী, নিয়মিত পাসওয়ার্ড ঘোরানো; **অনুমোদন** — Least Privilege: অ্যাপ্লিকেশনের ব্যবহারকারী শুধু তার প্রয়োজনীয় টেবিলে পড়া/লেখা করতে পারে, `DROP` বা `ALTER` করতে পারে না; **এনক্রিপশন** — পরিবহনে TLS (যাচাইকৃত সার্টিফিকেট সহ), বিশ্রামে এনক্রিপশন, সংবেদনশীল কলামে অতিরিক্ত এনক্রিপশন; **অডিট** — কে কখন সংযোগ করল, কোন প্রশ্ন চালাল — তার লগ।

পর্যবেক্ষণে যা দেখতে হয়: CPU ও মেমরি ব্যবহার, ডিস্ক ব্যবহার ও বৃদ্ধির হার, সংযোগের সংখ্যা, প্রতিলিপির পিছিয়ে থাকা (Replication Lag), ধীর প্রশ্নের সংখ্যা, লক-সংক্রান্ত অপেক্ষা (Lock Waits), ব্যাকআপের সাফল্য ও বয়স। সতর্কতা বসান:

ডিস্ক ৮০% পূর্ণ হলে, প্রতিলিপি পিছিয়ে থাকলে, ব্যাকআপ ব্যর্থ হলে। একটি গুরুত্বপূর্ণ মেট্রিক হলো **ধীর প্রশ্নের লগ (Slow Query Log)** — যে প্রশ্নগুলো বারবার ধীর, সেগুলোতে ইনডেক্স বা প্রশ্নের পুনর্লেখন দরকার।

প্রোডাকশন পরিচালনার নিয়ম: স্কিমা পরিবর্তন (Migration) সবসময় পশ্চাৎ-সামঞ্জস্যপূর্ণ (Backward Compatible) রাখুন — পুরোনো অ্যাপ্লিকেশন সংস্করণ যেন নতুন স্কিমাতেও চলে; বড় মাইগ্রেশন ছোট ধাপে ভাগ করুন; ডিস্কের বৃদ্ধি পর্যবেক্ষণ করে আগেভাগে স্কেল করুন; সংস্করণ আপগ্রেডের আগে staging-এ পরীক্ষা করুন; এবং প্রতিটি পরিবর্তনের পর রোলব্যাক পরিকল্পনা লিখে রাখুন।

> **REAL WORLD:** TechNova-র একবার একটি নতুন ইনডেক্স ছাড়া প্রশ্ন প্রোডাকশনে গিয়ে পুরো টেবিল স্ক্যান করতে শুরু করে, CPU ১০০% হয়ে যায়। সমাধান ছিল প্রশ্নটি বন্ধ করে ইনডেক্স তৈরি করা — কিন্তু সতর্কতা ছিল আগে থেকে: ধীর প্রশ্নের সতর্কতা বসানো থাকলে সমস্যাটি staging-এ ধরা পড়ত। এখন TechNova-র নিয়ম: প্রতিটি নতুন প্রশ্নের `EXPLAIN` পর্যালোচনা করা হয়, এবং প্রোডাকশনে ধীর প্রশ্নের সতর্কতা বাধ্যতামূলক।

বিলিং: পরিচালিত ডেটাবেসের খরচের চালক — কম্পিউট ও মেমরি (ইনস্ট্যান্সের আকার), ডিস্কের আকার, ব্যাকআপ সংরক্ষণ, ডেটা স্থানান্তর এবং উচ্চ-উপলব্ধতার জন্য অতিরিক্ত রেপ্লিকা। খরচ কমানোর উপায়: অপ্রয়োজনীয় রেপ্লিকা না রাখা, অপ্রয়োজনীয় ইনডেক্স মুছে ফেলা (ইনডেক্স ডিস্ক ও লেখার খরচ বাড়ায়), পুরোনো ডেটা আর্কাইভ করে COS-এ সরানো, এবং dev/staging-এ ছোট ইনস্ট্যান্স ব্যবহার করা। তবে প্রোডাকশনের ডেটাবেসে খরচ বাঁচাতে গিয়ে কর্মক্ষমতা কমানো বিপজ্জনক — ধীর ডেটাবেস পুরো অ্যাপ্লিকেশনকে ধীর করে।

১৪.৯ সাধারণ ভুল ও ট্রাবলশুটিং

সাধারণ ভুল: (১) একটিমাত্র ডেটাবেস ব্যবহারকারী সব কাজে ব্যবহার করা। (২) সংযোগ পুল ছাড়া প্রতিটি অনুরোধে নতুন সংযোগ খোলা। (৩) ব্যাকআপ থেকে পুনরুদ্ধার কখনো পরীক্ষা না করা। (৪) Redis-কে একমাত্র সত্যের উৎস করা। (৫) WHERE ছাড়া DELETE/UPDATE চালানো। (৬) ইনডেক্স ছাড়া বড় টেবিলে প্রশ্ন চালানো। (৭) পাবলিক endpoint খোলা রেখে দুর্বল পাসওয়ার্ড ব্যবহার করা।

> **TROUBLESHOOTING:** সংযোগ ব্যর্থ হলে দেখুন — endpoint প্রাইভেট না পাবলিক, VPC ও সিকিউরিটি গ্রুপের নিয়ম, TLS সার্টিফিকেট যাচাই (`sslmode=verify-full`), ব্যবহারকারীর নাম ও পাসওয়ার্ড, এবং সংযোগ-সীমা পূর্ণ হয়েছে কি না (`pg_stat_activity` -তে সংযোগ গুনুন)। ধীর প্রশ্নে `EXPLAIN ANALYZE` চালান, ইনডেক্স আছে কি না দেখুন, টেবিলের পরিসংখ্যান (Statistics) হালনাগাদ আছে কি না দেখুন। প্রতিলিপি পিছিয়ে থাকলে নেটওয়ার্ক, প্রাইমারির লেখার চাপ ও রেপ্লিকার সংস্থান পরীক্ষা করুন। ডিস্ক পূর্ণ হলে প্রথমে WAL ও লগের আকার দেখুন, তারপর অপ্রয়োজনীয় ডেটা আর্কাইভ করুন — ডিস্ক পূর্ণ অবস্থায় ডেটাবেস লেখা বন্ধ করে দিতে পারে।

১৪.৯.১ স্কিমা পরিবর্তন, মাইগ্রেশন ও কর্মক্ষমতা প্রকৌশল

ডেটাবেসের সবচেয়ে ঝুঁকিপূর্ণ কাজ হলো স্কিমা পরিবর্তন (Migration) — কারণ এটি ডেটা ও কোড — দুটোকেই একসঙ্গে বদলায়। নিরাপদ মাইগ্রেশনের নীতি হলো **সম্প্রসারণ ও সংকোচন (Expand and Contract)** : প্রথমে নতুন কলাম বা টেবিল যোগ করুন (পুরোনো কোড তাতে বাধা পায় না), তারপর অ্যাপ্লিকেশনের নতুন সংস্করণ স্থাপন করুন যা নতুন কাঠামো ব্যবহার করে, তারপর পুরোনো কলাম বা কোড সরান। কখনোই এক ধাপে কলামের নাম বদলাবেন না বা কলাম মুছে ফেলবেন না — পুরোনো অ্যাপ্লিকেশন সংস্করণ তখন ভেঙে পড়বে। বড় টেবিলে ইনডেক্স তৈরি বা কলাম যোগ করার সময় লকিংয়ের কথা ভাবুন: একটি দীর্ঘ লক পুরো টেবিলের লেখা আটকে দিতে পারে। তাই বড় পরিবর্তন ট্র্যাফিক কম থাকার সময়ে, ছোট ধাপে, এবং সম্ভব হলে অনলাইন পদ্ধতিতে (Concurrent Index তৈরি করে) করুন। মাইগ্রেশনের আগে

ব্যাকআপ নিন, মাইগ্রেশনের স্ক্রিপ্টটি staging-এ প্রকৃত আকারের ডেটায় চালিয়ে সময় মাপুন, এবং রোলব্যাক স্ক্রিপ্ট লিখে রাখুন — শুধু মাইগ্রেশন স্ক্রিপ্ট নয়, তার বিপরীতটিও। কর্মক্ষমতা প্রকৌশলে (Performance Engineering) তিনটি স্তর আছে: প্রশ্ন-স্তর (সঠিক ইনডেক্স, প্রশ্নের পুনর্লেখন, `EXPLAIN` বিশ্লেষণ), স্কিমা-স্তর (সঠিক ডেটা টাইপ, অপ্রয়োজনীয় ইনডেক্স মুছে ফেলা, পার্টিশনিং), এবং অবকাঠামো-স্তর (সংযোগ পুল, রেপ্লিকা দিয়ে পড়া ভাগ করা, ক্যাশ)। বেশিরভাগ ধীর প্রশ্নের সমাধান প্রথম দুটি স্তরেই থাকে; ইনস্ট্যান্স বড় করা শেষ উপায়, প্রথম উপায় নয়। একটি কার্যকর অভ্যাস: প্রতি সপ্তাহে ধীরতম দশটি প্রশ্নের তালিকা পর্যালোচনা করা, এবং প্রতিটি প্রশ্নের জন্য — ইনডেক্স, পুনর্লেখন, না ক্যাশ — কোনটি সঠিক তা সিদ্ধান্ত নেওয়া।

১৪.৯.২ ডেটা শাসন, সম্মতি ও মাল্টি-ইঞ্জিন পরিচালনা

একাধিক ইঞ্জিন চালানো মানে একাধিক শাসন-নীতি। ব্যক্তিগত ডেটা (PII) কোন টেবিলে, কোন কলামে আছে তার একটি তালিকা (Data Catalog) রাখুন; কে সেই ডেটা দেখতে পারে তা ভূমিকা দিয়ে সীমিত করুন; ডেটা মুছে ফেলার অনুরোধ এলে PostgreSQL, Redis ও Elasticsearch — তিন জায়গা থেকেই মুছতে হবে, শুধু মূল ডেটাবেস থেকে নয়। লগে যেন পাসওয়ার্ড বা ব্যক্তিগত ডেটা না যায় তা পরীক্ষা করুন। ব্যাকআপের ক্ষেত্রেও একই নীতি প্রযোজ্য: ব্যাকআপে ব্যক্তিগত ডেটা থাকে, তাই ব্যাকআপের অ্যাক্সেসও মূল ডেটাবেসের মতোই সীমিত রাখতে হয়। সম্মতি (Compliance) কোনো এককালীন চেকলিস্ট নয় — প্রতিটি নতুন টেবিল, নতুন ইঞ্জিন, নতুন রেপ্লিকা যোগ করার সময় একই প্রশ্নগুলো আবার করতে হয়: ডেটা কোথায় থাকবে, কে দেখতে পারবে, কতদিন থাকবে, মুছে ফেলা যাবে কি না। মাল্টি-ইঞ্জিন পরিচালনায় আরেকটি শৃঙ্খলা হলো নামকরণ ও পরিবেশ-বিচ্ছিন্নতা: dev, staging ও production-এর ইনস্ট্যান্সের নাম, endpoint ও ক্রেডেনশিয়াল আলাদা ও স্পষ্ট হওয়া উচিত, যাতে ভুল করে প্রোডাকশনে পরীক্ষামূলক স্ক্রিপ্ট না চলে। TechNova-র নিয়ম: প্রোডাকশন ক্রেডেনশিয়াল শুধুমাত্র সিক্রেট-পরিচালনা সেবা থেকে আসে, ডেভেলপারের ল্যাপটপে প্রোডাকশন পাসওয়ার্ড থাকে না।

১৪.১০ সারাংশ

পরিচালিত ডেটাবেস প্রোভিশনিং, প্যাচিং, ব্যাকআপ ও প্রতিলিপির বোঝা নেয়; ব্যবহারকারীর দায়িত্ব থাকে স্কিমা, প্রশ্ন, অ্যাক্সেস নিয়ন্ত্রণ ও পুনরুদ্ধারের পরীক্ষায়। PostgreSQL লেনদেনের জন্য, MongoDB নমনীয় ডকুমেন্টের জন্য, Redis ক্যাশের জন্য, Elasticsearch অনুসন্ধানের জন্য — প্রতিটি ইঞ্জিনের নিজস্ব জায়গা আছে। প্রাইভেট endpoint, সংযোগ পুলিং, Least Privilege, নিয়মিত ব্যাকআপ পরীক্ষা ও ধীর প্রশ্নের পর্যবেক্ষণ — এই পাঁচটি শৃঙ্খলাই প্রোডাকশন ডেটাবেসকে টিকিয়ে রাখে।

১৪.১১ যাচাই প্রশ্ন

1. পরিচালিত ডেটাবেস ব্যবহার করলেও কোন দায়িত্বগুলো ব্যবহারকারীর থাকে, এবং কেন?
2. সিঙ্ক্রোনাস ও অ্যাসিঙ্ক্রোনাস প্রতিলিপির মধ্যে বিনিময় কী? আর্থিক লেনদেনের জন্য কোনটি বেছে নেবেন?
3. সংযোগ পুলিং কেন দরকার? পুল খুব ছোট বা খুব বড় হলে কী হয়?
4. উচ্চ-উপলব্ধতা ও ব্যাকআপের মধ্যে পার্থক্য কী? ভুল করে টেবিল মুছে ফেললে কোনটি সাহায্য করে?
5. Redis-কে একমাত্র সত্যের উৎস করা কেন বিপজ্জনক? পুনর্গঠনযোগ্যতার নীতি কী বলে?

১৪.১২ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি ডেটাবেস নকশা লিখুন: মূল ডেটা PostgreSQL-এ, ক্যাশ Redis-এ, অনুসন্ধান Elasticsearch-এ — প্রতিটির জন্য endpoint ধরন, ব্যবহারকারী ও ভূমিকা, ব্যাকআপ সময়সূচি, RPO/RTO ও পর্যবেক্ষণের মেট্রিক লিখুন।

2. একটি পরীক্ষামূলক PostgreSQL ডেটাবেসে ইচ্ছাকৃতভাবে একটি ধীর প্রশ্ন তৈরি করুন, `EXPLAIN ANALYZE` দিয়ে তার পরিকল্পনা পড়ুন, একটি ইনডেক্স যোগ করে উন্নতি মাপুন, এবং ফলাফলসহ একটি সংক্ষিপ্ত প্রতিবেদন লিখুন।

অধ্যায় ১৫ • PART III — Storage, Database & Application Services

Chapter 15 — Kubernetes and Red Hat OpenShift

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক পারবেন — (১) IBM Cloud Kubernetes Service (IKS) ও Red Hat OpenShift on IBM Cloud (ROKS)-এর মধ্যে পার্থক্য ব্যাখ্যা করতে; (২) Kubernetes স্থাপত্য — কন্ট্রোল প্লেন (Control Plane), ওয়ার্কার নোড (Worker Node), পড, সার্ভিস, ডেপ্লয়মেন্ট — বুঝতে; (৩) OpenShift-এর অতিরিক্ত স্তর — Operators, Routes, BuildConfig, প্রকল্প (Project) — ব্যাখ্যা করতে; (৪) Routes বনাম Ingress-এর পার্থক্য বলতে; (৫) পারসিস্টেন্ট স্টোরেজ, RBAC, ওয়ার্কলোড নিরাপত্তা ও আপগ্রেডের নকশা করতে; এবং (৬) `kubectl` ও `oc` CLI দিয়ে ক্লাস্টার পরিচালনা করতে।

১৫.২ পূর্বশর্ত পুনরালোচনা

কনটেইনার মানে একটি হালকা, বিচ্ছিন্ন পরিবেশ যেখানে অ্যাপ্লিকেশন ও তার নির্ভরতা (Dependency) একসঙ্গে প্যাকেজ করা থাকে। Docker ইমেজ হলো সেই প্যাকেজের নীলনকশা; কনটেইনার হলো তার চলমান উদাহরণ। একটি কনটেইনার চালানো সহজ, কিন্তু শত শত কনটেইনার — কোন নোডে চলবে, কে কার সঙ্গে কথা বলবে, একটি নষ্ট হলে কে আবার চালাবে, ট্র্যাফিক কীভাবে ভাগ হবে — এই প্রশ্নগুলোর উত্তর দেয় **অর্কেস্ট্রেশন (Orchestration)**। Kubernetes হলো সেই অর্কেস্ট্রেটর; OpenShift হলো Kubernetes-এর ওপরে তৈরি একটি সম্পূর্ণ প্ল্যাটফর্ম যেখানে অতিরিক্ত নিরাপত্তা, ডেভেলপার টুল ও পরিচালনার স্তর যোগ করা হয়েছে।

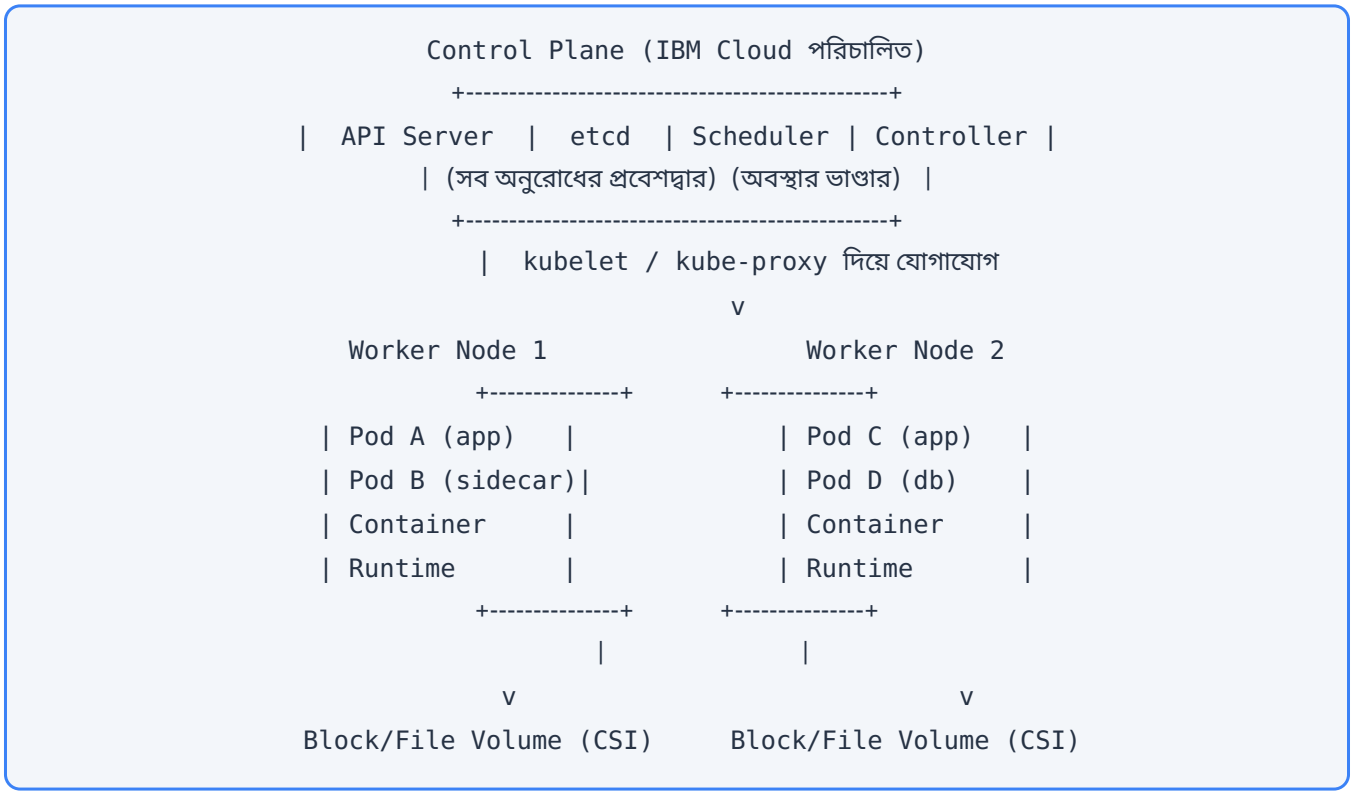
১৫.৩ প্রথম নীতি: কেন অর্কেস্ট্রেশন

একটি সার্ভারে একটি অ্যাপ্লিকেশন চালানো সহজ। দশটি সার্ভারে পঞ্চাশটি অ্যাপ্লিকেশন চালানো — প্রতিটির নিজস্ব সংস্করণ, কনফিগারেশন, সংস্থান-চাহিদা, স্বাস্থ্য — হাতে পরিচালনা করা অসম্ভবের কাছাকাছি। Kubernetes এই সমস্যার উত্তর দেয় **ঘোষণামূলক (Declarative)** মডেলে: আপনি বলেন কী চাই — যেমন তিনটি কপি (Replica) চালু থাকুক, এই ইমেজ দিয়ে, এই পোর্টে — আর Kubernetes ক্রমাগত বাস্তব অবস্থাকে সেই ঘোষিত অবস্থার সঙ্গে মিলিয়ে নেয়। একটি পড নষ্ট হলে নতুন পড তৈরি হয়; ট্র্যাফিক বাড়লে কপি বাড়ে; নতুন সংস্করণ ধীরে ধীরে (Rolling Update) পুরোনো সংস্করণের জায়গা নেয়। এই মডেলের শক্তি হলো স্বয়ংক্রিয় পুনরুদ্ধার (Self-Healing) ও পুনরুৎপাদনযোগ্যতা (Reproducibility): একই YAML ফাইল থেকে একই ক্লাস্টার আবার তৈরি করা যায়।

OpenShift কেন দরকার? Kubernetes নিজে অনেক কিছু দেয়, কিন্তু এন্টারপ্রাইজে আরও দরকার হয়: কে কী করতে পারে তার কঠোর নিয়ন্ত্রণ, ডেভেলপারদের জন্য সোর্স-থেকে-ইমেজ বিল্ড, বহিরাগত ট্র্যাফিকের জন্য সহজ রাউটিং, অপারেটর-ভিত্তিক সেবা পরিচালনা। OpenShift Kubernetes-কে বদলায় না — তার ওপরে এই স্তরগুলো যোগ করে, যাতে দলটি কম কাস্টম গ্লু কোড লিখে বেশি প্রোডাক্ট তৈরি করতে পারে। বিনিময়ে শেখার বক্ররেখা (Learning Curve) ও কিছুটা নমনীয়তা কমে — কারণ OpenShift কিছু সিদ্ধান্ত আগে থেকে নিয়ে নেয়।

> **CONCEPT:** Kubernetes-এ সবকিছু একটি **অবজেক্ট** — Pod, Service, Deployment, ConfigMap, Secret — যার একটি ঘোষিত অবস্থা (Desired State) ও একটি পর্যবেক্ষিত অবস্থা (Observed State) থাকে। কন্ট্রোল প্লেনের নিয়ন্ত্রক (Controller) দুটির পার্থক্য দেখে ব্যবস্থা নেয়। এই **নিয়ন্ত্রণ-লুপ (Control Loop)** বোঝা মানেই Kubernetes বোঝা: আপনি নির্দেশ দেন না, আপনি অবস্থা ঘোষণা করেন।

১৫.৪ আর্কিটেকচার: কন্ট্রোল প্লেন ও ওয়ার্কার নোড



কন্ট্রোল প্লেন চারটি মূল উপাদান নিয়ে গঠিত। **API Server** হলো ক্লাস্টারের প্রবেশদ্বার — সব `kubectl` অনুরোধ এখানে আসে, প্রমাণীকরণ ও অনুমোদন এখানে হয়। **etcd** হলো ক্লাস্টারের অবস্থার ভাণ্ডার — সব অবজেক্টের ঘোষিত ও পর্যবেক্ষিত অবস্থা এখানে সংরক্ষিত থাকে; etcd হারিয়ে গেলে ক্লাস্টারের স্মৃতি হারিয়ে যায়। **Scheduler** সিদ্ধান্ত নেয় কোন পড কোন নোডে চলবে — সংস্থান, নোডের অবস্থা ও সীমাবদ্ধতা (Affinity/Taint) দেখে। **Controller Manager** নিয়ন্ত্রণ-লুপগুলো চালায় — যেমন Deployment Controller নিশ্চিত করে ঘোষিত সংখ্যক কপি চলছে। IBM Cloud Kubernetes Service-এ কন্ট্রোল প্লেনটি পরিচালিত — ব্যবহারকারীকে etcd ব্যাকআপ বা API Server-এর প্যাচ নিয়ে ভাবতে হয় না, কিন্তু কন্ট্রোল প্লেনের স্বাস্থ্য ও সংস্করণ পর্যবেক্ষণ করা উচিত।

ওয়ার্কার নোড হলো সেই মেশিন যেখানে পড চলে। প্রতিটি নোডে থাকে **kubelet** — নোডের এজেন্ট, যা API Server-এর নির্দেশে পড চালায় ও তার স্বাস্থ্য জানায়; **kube-proxy** — সার্ভিসের নেটওয়ার্ক নিয়ম পরিচালনা করে; এবং **কনটেইনার রানটাইম** — ইমেজ থেকে কনটেইনার চালায়। একটি **পড (Pod)** হলো সবচেয়ে ছোট স্থাপন-একক — এক বা একাধিক কনটেইনার যারা একই নেটওয়ার্ক ও স্টোরেজ ভাগ করে। পড অস্থায়ী: নষ্ট হলে নতুন পড তৈরি হয়, নতুন IP পায় — তাই পডের IP-র ওপর নির্ভর করা যায় না, নির্ভর করতে হয় **সার্ভিস (Service)**-এর স্থির নামের ওপর।

OpenShift স্থাপত্য একই ভিত্তির ওপরে অতিরিক্ত স্তর যোগ করে। OpenShift-এ **প্রকল্প (Project)** হলো একটি নামস্থান (Namespace) যার সঙ্গে অতিরিক্ত নীতি ও কোটা যুক্ত থাকে। **Operator** হলো একটি বিশেষ নিয়ন্ত্রক যা একটি জটিল সেবা — যেমন ডেটাবেস বা মনিটরিং — পরিচালনা করে, মানুষের পরিচালকের জ্ঞান কোডে ধারণ করে। **BuildConfig** ও **ImageStream** সোর্স কোড থেকে ইমেজ তৈরির প্রক্রিয়া পরিচালনা করে। OpenShift-এর নিরাপত্তা মডেল কঠোর: ডিফল্টভাবে কনটেইনার root হিসেবে চলে না, এবং Security Context Constraints (SCC) নির্ধারণ করে কোন পড কী করতে পারে।

১৫.৫ Routes বনাম Ingress: গুরুত্বপূর্ণ পার্থক্য

Kubernetes-এ বাইরের ট্র্যাফিক ভেতরে আনার মান (Standard) উপায় হলো **Ingress** — একটি অবজেক্ট যা হোস্টনাম ও পথের নিয়ম লেখে, এবং একটি **Ingress Controller** সেই নিয়ম অনুযায়ী ট্র্যাফিক রাউট করে। OpenShift-এ সমতুল্য ধারণাটি হলো **Route** — একটি OpenShift-নির্দিষ্ট অবজেক্ট যা একটি সার্ভিসকে বাহ্যিক হোস্টনাম দেয়, TLS সমাপ্তি (TLS Termination) পরিচালনা করে এবং অন্তর্নির্মিত রাউটার (Router) দিয়ে ট্র্যাফিক পাঠায়।

বৈশিষ্ট্য	Kubernetes Ingress	OpenShift Route
উৎস	Kubernetes মান	OpenShift-নির্দিষ্ট
বহনযোগ্যতা	যেকোনো Kubernetes-এ চলে	OpenShift-এ চলে
TLS	Ingress Controller-এর ওপর নির্ভরশীল	অন্তর্নির্মিত, সহজ কনফিগারেশন
ব্যবহার	সাধারণ Kubernetes ক্লাস্টার	OpenShift ক্লাস্টার

বাস্তব পরামর্শ: যদি আপনার কাজ শুধু OpenShift-এ চলে, Route সহজ ও সরল। যদি একই অ্যাপ্লিকেশন একাধিক Kubernetes প্ল্যাটফর্মে চলতে হয়, Ingress বহনযোগ্য। দুটি একসঙ্গে ব্যবহার করলে বিভ্রান্তি তৈরি হয় — একটি পথ বেছে নিন, এবং সেটি দলিলে (Documentation) লিখে রাখুন। TLS সার্টিফিকেটের মেয়াদ ও নবায়ন — যেকোনো পথেই — স্বয়ংক্রিয় করা উচিত; মেয়াদোত্তীর্ণ সার্টিফিকেট প্রোডাকশন বিভ্রাটের একটি সাধারণ কারণ।

> **WARNING:** Ingress ও Route দুটিকেই ভুল করে একই হোস্টনামে সক্রিয় করলে ট্র্যাফিক অনিশ্চিতভাবে ভাগ হতে পারে। নতুন ক্লাস্টারে প্রথমেই ঠিক করুন কোন প্রবেশ-পথ (Ingress Path) ব্যবহার হবে, এবং পুরোনো পথটি মুছে ফেলুন। এছাড়া, কোনো সার্ভিসকে `LoadBalancer` ধরন দিয়ে সরাসরি ইন্টারনেটে খুলে দেবেন না — সেটি প্রতিটি সার্ভিসের জন্য আলাদা লোড ব্যালান্সার তৈরি করে, খরচ ও আক্রমণের পৃষ্ঠ (Attack Surface) দুটোই বাড়ায়।

১৫.৬ পারসিস্টেন্ট স্টোরেজ, RBAC ও ওয়ার্কলোড নিরাপত্তা

পারসিস্টেন্ট স্টোরেজের জন্য অধ্যায় ১৩-এর PV, PVC, StorageClass ও CSI এখানে প্রয়োগ হয়। Kubernetes-এ ডেটাবেসের মতো রাষ্ট্রযুক্ত (Stateful) কাজের জন্য **StatefulSet** ব্যবহার করা হয় — এটি প্রতিটি পডকে একটি স্থির নাম ও স্থির ভলিউম দেয়, যাতে পড আবার তৈরি হলেও একই ভলিউমে ফেরে। রাষ্ট্রহীন (Stateless) কাজের জন্য **Deployment** যথেষ্ট। ভলিউমের Reclaim Policy ও ব্যাকআপ নীতি এখানেও প্রযোজ্য।

RBAC (Role-Based Access Control) হলো Kubernetes-এর অনুমতি ব্যবস্থা: **Role** বলে কী করা যায়, **RoleBinding** বলে কে করতে পারে, **ServiceAccount** হলো পডের পরিচয়। নীতি: প্রতিটি অ্যাপ্লিকেশনের নিজস্ব ServiceAccount, শুধুমাত্র প্রয়োজনীয় অনুমতি, কখনোই `cluster-admin` অ্যাপ্লিকেশনের পডে নয়। মানুষের জন্য একই নীতি — ডেভেলপারদের শুধু তাদের প্রকল্পে (Project/Namespace) অনুমতি, ক্লাস্টার-ব্যাপী অনুমতি শুধুমাত্র পরিচালকদের।

ওয়ার্কলোড নিরাপত্তার চেকলিস্ট: ইমেজের দুর্বলতা (Vulnerability) স্ক্যান — অধ্যায় ১৬-এর রেজিস্ট্রি দেখুন; root হিসেবে না চালানো, শুধুমাত্র প্রয়োজনীয় ক্ষমতা (Capability) দেওয়া; নেটওয়ার্ক নীতি (NetworkPolicy) দিয়ে পড-থেকে-পড যোগাযোগ সীমিত করা; Secret-কে পরিবেশ চলক বা মাউন্ট করা ফাইল হিসেবে ব্যবহার করা, কখনোই ইমেজে বা YAML-এ লেখা পাসওয়ার্ড নয়; এবং সংস্থান-সীমা (Resource Requests ও Limits) বসানো — যাতে একটি পড পুরো নোডের CPU/মেমরি খেয়ে না ফেলে।

অন্যান্য সেবার সঙ্গে যোগাযোগ: Kubernetes পড COS থেকে ফাইল পড়ে (অধ্যায় ১২), পরিচালিত ডেটাবেসে সংযোগ করে (অধ্যায় ১৪), Container Registry থেকে ইমেজ টানে (অধ্যায় ১৬), এবং Code Engine-এর কাজের সঙ্গে API দিয়ে কথা বলে। প্রতিটি যোগাযোগই নেটওয়ার্ক নীতি ও IAM/ServiceAccount দিয়ে সীমিত রাখা উচিত।

ব্যর্থ হলে কী হয়? একটি নোড নষ্ট হলে তার পডগুলো অন্য নোডে আবার তৈরি হয় — যদি সংস্থান থাকে। কন্ট্রোল প্লেনের একটি উপাদান নষ্ট হলে পরিচালিত সেবা তা পুনরুদ্ধার করে, কিন্তু etcd-র ডেটা হারালে ক্লাস্টারের অবস্থা ফিরিয়ে আনা কঠিন — তাই পরিচালিত কন্ট্রোল প্লেনের ব্যাকআপ ও সংস্করণের ওপর নির্ভর করা হয়। একটি পডের ভলিউম নষ্ট হলে স্ন্যাপশট ও ব্যাকআপই ভরসা (অধ্যায় ১৩)।

১৫.৭ CLI উদাহরণ ও আপগ্রেড পরিচালনা

```
# ক্লাস্টার লক্ষ্য করা ও সংস্করণ দেখা
ibmcloud ks cluster config --cluster technova-iks-01
kubectl cluster-info
kubectl get nodes
kubectl get pods -A

# Deployment তৈরি ও স্কেল করা
kubectl create deployment technova-web \
  --image=icr.io/technova/web:v1.2.3
kubectl scale deployment technova-web --replicas=3
kubectl rollout status deployment/technova-web

# OpenShift-এ (oc CLI)
oc project technova-prod
oc get routes
oc expose service technova-web --hostname=app.technova.example
oc get pods -n technova-prod
```

```
# সার্ভিস ও Ingress/Route (ধারণাগত)
apiVersion: v1
kind: Service
metadata:
  name: technova-web
spec:
  selector:
    app: technova-web
  ports:
  - port: 80
    targetPort: 8080
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: technova-web
spec:
  rules:
  - host: app.technova.example
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
```

```
            name: technova-web
            port:
              number: 80
```

আপগ্রেডের নীতি: Kubernetes-এর সংস্করণ আপগ্রেড একটি পরিকল্পিত কাজ, হঠাৎ সিদ্ধান্ত নয়। প্রথমে staging ক্লাস্টারে আপগ্রেড করুন, অ্যাপ্লিকেশনের সামঞ্জস্য (Compatibility) যাচাই করুন — বিশেষত সরিয়ে ফেলা API সংস্করণ (Deprecated API) আছে কি না; তারপর প্রোডাকশনে আপগ্রেড করুন, নোডগুলো ধীরে ধীরে (একটি করে) আপগ্রেড করে, প্রতিটি ধাপে স্বাস্থ্য পরীক্ষা করে। আপগ্রেডের আগে etcd ও গুরুত্বপূর্ণ ডেটার ব্যাকআপ নিশ্চিত করুন, এবং রোলব্যাক পরিকল্পনা লিখে রাখুন। OpenShift-এ Operator-গুলোর আপগ্রেডও একইভাবে পরীক্ষা করে করতে হয় — একটি Operator-এর নতুন সংস্করণ পুরোনো কাস্টম রিসোর্সের সঙ্গে অসামঞ্জস্য তৈরি করতে পারে।

> **REAL WORLD:** TechNova-র একবার একটি আপগ্রেডে পুরোনো Ingress API সংস্করণ সরিয়ে ফেলা হয়, এবং একটি অ্যাপ্লিকেশনের Ingress আর কাজ করেনি। কারণ ছিল — আপগ্রেডের আগে API-র অবচয় (Deprecation) সতর্কতা পরীক্ষা করা হয়নি। এখন TechNova-র নিয়ম: প্রতিটি আপগ্রেডের আগে ক্লাস্টারের API-ব্যবহারের প্রতিবেদন দেখা হয়, এবং staging-এ সম্পূর্ণ অ্যাপ্লিকেশন স্যুট চালিয়ে দেখা হয়।

পর্যবেক্ষণ: নোডের CPU/মেমরি, পডের পুনরায় চালু হওয়ার সংখ্যা (Restart Count), Deployment-এর উপলব্ধ কপি, Ingress/Route-এর ত্রুটির হার, etcd ও API Server-এর বিলম্ব। খরচের চালক: ওয়ার্কার নোডের সংখ্যা ও আকার, লোড ব্যালান্সারের সংখ্যা, স্থায়ী ভলিউমের আকার, এবং ডেটা স্থানান্তর। খরচ কমানোর উপায়: সঠিক Requests/Limits বসিয়ে নোডের ব্যবহার বাড়ানো, অপ্রয়োজনীয় কপি কমানো, dev/staging-এ ছোট নোড ব্যবহার করা।

১৫.৮ সাধারণ ভুল ও ট্রাবলশুটিং

সাধারণ ভুল: (১) পডের IP-র ওপর নির্ভর করা। (২) Requests/Limits না বসানো। (৩) Secret YAML-এ পাসওয়ার্ড লেখা। (৪) `latest` ট্যাগের ইমেজ ব্যবহার করা — কোন সংস্করণ চলছে তা বোঝা যায় না। (৫) একটি নোডে সব কপি রাখা — নোড নষ্ট হলে সব কপি একসঙ্গে যায়। (৬) Ingress ও Route একসঙ্গে ব্যবহার করা। (৭) আপগ্রেডের আগে API অবচয় পরীক্ষা না করা।

> **TROUBLESHOOTING:** পড `CrashLoopBackOff`-এ থাকলে `kubectl logs` ও `kubectl describe pod` দেখুন — অ্যাপ্লিকেশন কি শুরু হতে পারছে না, নাকি স্বাস্থ্য পরীক্ষা (Liveness Probe) ব্যর্থ হচ্ছে? পড `Pending`-এ থাকলে সংস্থান, নোডের অবস্থা ও PVC-র অবস্থা দেখুন। সার্ভিসে সংযোগ না হলে সার্ভিসের Selector পডের লেবেলের সঙ্গে মিলছে কি না দেখুন — সবচেয়ে সাধারণ ভুল হলো লেবেলের বানান। Route/Ingress-এ 404 পেলে হোস্টনাম, পথ ও ব্যাকএন্ড সার্ভিসের পোর্ট মিলিয়ে দেখুন।

১৫.৮.১ অটোস্কেলিং, ব্যাঘাত বাজেট ও মাল্টি-ক্লাস্টার চিন্তা

স্কেলিং তিন স্তরে হয়। **Horizontal Pod Autoscaler (HPA)** মেট্রিক — যেমন CPU, মেমরি, বা কাস্টম মেট্রিক যেমন অনুরোধের হার — দেখে কপির সংখ্যা বাড়ায়-কমায়। **Vertical Pod Autoscaler (VPA)** প্রতিটি পডের সংস্থান-অনুরোধ (Requests/Limits) সমন্বয় করে। **Cluster Autoscaler** নোডের সংখ্যা বাড়ায়-কমায়। তিনটি একসঙ্গে ব্যবহার করলে সতর্ক থাকতে হয়: HPA কপি বাড়ালে Cluster Autoscaler নোড বাড়ায়, কিন্তু VPA একই সময়ে পডের আকার বদলালে পড পুনরায় তৈরি হয় — এই মিথস্ক্রিয়া পরীক্ষা না করে প্রোডাকশনে চালানো উচিত নয়। স্কেলিংয়ের মেট্রিক বেছে নেওয়াও গুরুত্বপূর্ণ: CPU-ভিত্তিক স্কেলিং সহজ, কিন্তু I/O-বদ্ধ কাজে CPU কম দেখায় — সেখানে অনুরোধের হার বা কিউয়ের দৈর্ঘ্য

ভালো সংকেত। **Pod Disruption Budget (PDB)** বলে রক্ষণাবেক্ষণের সময় একসঙ্গে কতগুলো কপি বন্ধ করা যাবে — যেমন তিনটি কপির মধ্যে অন্তত দুটি চালু থাকতে হবে। PDB ছাড়া নোড আপগ্রেডের সময় সব কপি একসঙ্গে বন্ধ হয়ে সেবা ব্যাহত হতে পারে। মাল্টি-ক্লাস্টার চিন্তায়: একটি ক্লাস্টার ব্যর্থ হলে অন্য ক্লাস্টারে ট্র্যাফিক সরানোর পরিকল্পনা, ইমেজ ও কনফিগারেশনের প্রতিলিপি, এবং DNS-স্তরে ট্র্যাফিক পরিবর্তন — এসবের মহড়া করা উচিত। তবে মাল্টি-ক্লাস্টার মানে দ্বিগুণ জটিলতা — ছোট দলের জন্য প্রথমে একক ক্লাস্টারকে সঠিকভাবে চালানো, তারপর মাল্টি-ক্লাস্টার ভাবা উচিত।

১৫.৮.২ গিটঅপস ও ক্লাস্টার পরিচালনার শৃঙ্খলা

ক্লাস্টারের অবস্থা হাতে `kubectl apply` করে বদলালে, কিছুদিন পর কেউ জানে না ক্লাস্টারে কী চলছে এবং কেন। **GitOps** এই সমস্যার উত্তর দেয়: ক্লাস্টারের ঘোষিত অবস্থা গিট রিপোজিটরিতে থাকে, একটি নিয়ন্ত্রক (যেমন Argo CD বা Flux ধরনের টুল) ক্রমাগত গিটের অবস্থা ও ক্লাস্টারের অবস্থা মিলিয়ে নেয়। পরিবর্তন মানে গিটে একটি কমিট ও পর্যালোচনা (Pull Request) — কে কী বদলাল, কখন, কেন — তার ইতিহাস গিটেই থাকে। রোলব্যাক মানে আগের কমিটে ফেরা। TechNova-র নিয়ম: প্রোডাকশন ক্লাস্টারে সরাসরি `kubectl` দিয়ে পরিবর্তন নিষিদ্ধ — জরুরি ক্ষেত্রে পরিবর্তন করা হলে ২৪ ঘণ্টার মধ্যে তা গিটে ফিরিয়ে আনতে হয়। এই শৃঙ্খলার সঙ্গে যুক্ত থাকে নীতিমালা (Policy): কোন ইমেজ ব্যবহার করা যাবে, কোন নেমস্পেসে কী সংস্থান-সীমা বাধ্যতামূলক, কোন লেবেল বাধ্যতামূলক — এসব স্বয়ংক্রিয়ভাবে পরীক্ষা করা হয়। ক্লাস্টার পরিচালনায় আরও একটি অভ্যাস: নিয়মিত ক্লাস্টার স্বাস্থ্য পর্যালোচনা — অপ্রয়োজনীয় নেমস্পেস, মেয়াদোত্তীর্ণ Secret, ব্যবহৃত না হওয়া PVC, পুরোনো API সংস্করণ — এসব পরিষ্কার করা। ক্লাস্টারও একটি জীবন্ত ব্যবস্থা; পরিষ্কার না করলে তা ধীরে ধীরে জঞ্জালে ভরে যায়।

১৫.৯ সারাংশ

Kubernetes ঘোষণামূলক অর্কেস্ট্রেশন দেয় — কন্ট্রোল প্লেন অবস্থা ঘোষণা করে, ওয়ার্কার নোড পড চালায়, সার্ভিস স্থির নাম দেয়, Deployment ও StatefulSet কপি ও ভলিউম পরিচালনা করে। OpenShift একই ভিত্তির ওপরে প্রকল্প, Operator, Route ও কঠোর নিরাপত্তা যোগ করে। বাইরের ট্র্যাফিকের জন্য Kubernetes-এ Ingress, OpenShift-এ Route — একটি পথ বেছে নিন। RBAC, নেটওয়ার্ক নীতি, সংস্থান-সীমা ও নিয়মিত আপগ্রেড পরীক্ষা ছাড়া প্রোডাকশন ক্লাস্টার টেকে না।

১৫.১০ যাচাই প্রশ্ন

1. ঘোষণামূলক (Declarative) মডেল বলতে কী বোঝায়? একটি পড নষ্ট হলে Kubernetes কীভাবে তা জানে ও কী করে?
2. IKS ও ROKS-এর মধ্যে মূল পার্থক্য কী? কখন কোনটি বেছে নেবেন?
3. Route বনাম Ingress — কখন কোনটি ব্যবহার করবেন, এবং দুটি একসঙ্গে ব্যবহার করলে কী সমস্যা হয়?
4. একটি পডের জন্য ServiceAccount ও RBAC কেন দরকার? অ্যাপ্লিকেশনের পডকে `cluster-admin` দিলে কী ঝুঁকি থাকে?
5. Kubernetes আপগ্রেডের আগে কোন দুটি জিনিস পরীক্ষা করা সবচেয়ে গুরুত্বপূর্ণ, এবং কেন?

১৫.১১ ব্যবহারিক অনুশীলন

1. TechNova-র একটি ওয়েব অ্যাপ্লিকেশনের জন্য Deployment, Service ও Ingress (অথবা Route) লিখুন — তিনটি কপি, Requests/Limits, Liveness ও Readiness Probe, এবং একটি PVC সহ। প্রতিটি ক্ষেত্রের উদ্দেশ্য এক লাইনে লিখুন।
2. একটি পরীক্ষামূলক ক্লাস্টারে ইচ্ছাকৃতভাবে একটি পড মুছে দেখুন Kubernetes কীভাবে তা আবার তৈরি করে; তারপর একটি ভুল লেবেলের সার্ভিস তৈরি করে `kubectl describe` দিয়ে সমস্যাটি খুঁজে বের করুন।

অধ্যায় ১৬ • PART III — Storage, Database & Application Services

Chapter 16 — Container Registry and Application Hosting

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে পাঠক পারবেন — (১) IBM Cloud Container Registry (ICR) কী সমস্যা সমাধান করে তা ব্যাখ্যা করতে; (২) ইমেজ পরিচালনা — ট্যাগিং, সংস্করণ, দুর্বলতা স্ক্যান, ধারণকাল — নকশা করতে; (৩) কনটেইনার স্থাপন (Deployment) ও অ্যাপ্লিকেশন হোস্টিংয়ের ধারণাগুলো আলাদা করতে; (৪) সার্ভারলেস প্ল্যাটফর্মের ইতিহাস ও বর্তমান অবস্থা ব্যাখ্যা করতে — বিশেষত Code Engine বর্তমান পছন্দ এবং Cloud Functions (Apache OpenWhisk-ভিত্তিক) ঐতিহাসিক/অবচয়িত; (৫) API ইন্টিগ্রেশন ও স্থাপনের ধরণ (Deployment Pattern) বেছে নিতে; এবং (৬) পরিচালন-পর্যবেক্ষণ (Operational Monitoring) ও খরচের নকশা করতে।

১৬.২ পূর্বশর্ত পুনরালোচনা

অধ্যায় ১৫-এ আমরা দেখেছি Kubernetes পড চালায় ইমেজ থেকে। কিন্তু ইমেজটি কোথায় থাকে? একটি ইমেজ হলো স্তরে স্তরে (Layer) সাজানো ফাইল সিস্টেমের স্ন্যাপশট, যার প্রতিটি স্তর একটি ডাইজেস্ট (Digest — বিষয়বস্তুর হ্যাশ) দিয়ে চিহ্নিত হয়। **কনটেইনার রেজিস্ট্রি** হলো সেই ভাণ্ডার যেখানে ইমেজ সংরক্ষিত থাকে, সংস্করণ দেওয়া হয়, ও সেখান থেকে ক্লাস্টার বা সার্ভারলেস প্ল্যাটফর্ম ইমেজ টেনে নেয়। অ্যাপ্লিকেশন হোস্টিং বলতে বোঝায় সেই পরিবেশ যেখানে অ্যাপ্লিকেশন চলে — ভার্চুয়াল সার্ভার, Kubernetes, বা সার্ভারলেস প্ল্যাটফর্ম। সার্ভারলেস মানে সার্ভার নেই তা নয় — মানে সার্ভার পরিচালনার দায়িত্ব প্ল্যাটফর্মের, ব্যবহারকারী শুধু কোড বা কনটেইনার দেয়।

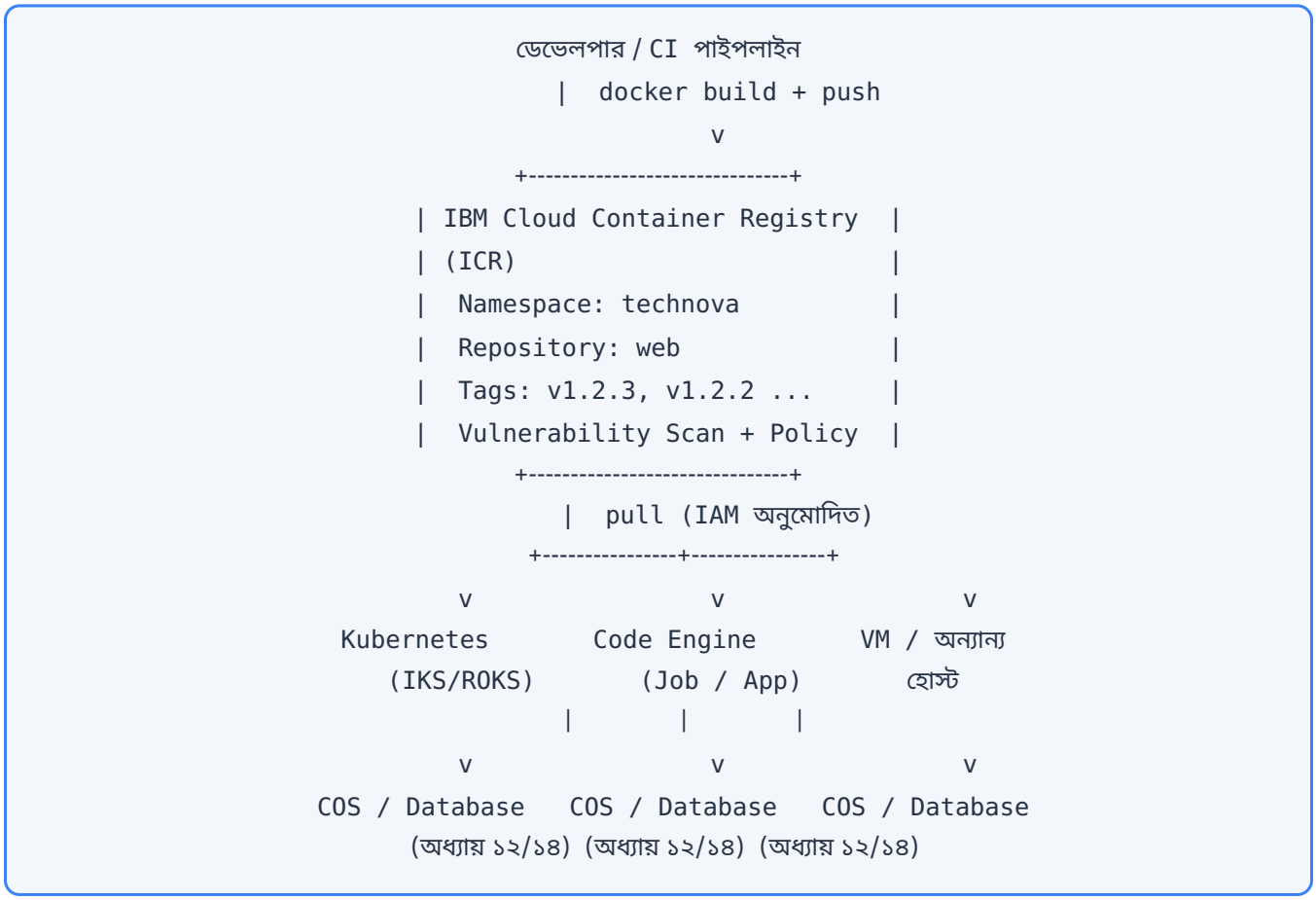
১৬.৩ প্রথম নীতি: রেজিস্ট্রি কেন দরকার

ইমেজ ছাড়া কনটেইনার চলতে পারে না, আর ইমেজ ছাড়া স্থাপন পুনরুৎপাদনযোগ্য হয় না। যদি প্রতিটি ডেভেলপার নিজের ল্যাপটপে ইমেজ তৈরি করে, তবে প্রোডাকশনে কোন ইমেজ চলছে তা কেউ নিশ্চিতভাবে বলতে পারে না। রেজিস্ট্রি এই সমস্যার উত্তর দেয়: একটি কেন্দ্রীয়, প্রমাণীকৃত ভাণ্ডার যেখানে প্রতিটি ইমেজের নাম, ট্যাগ ও ডাইজেস্ট লিপিবদ্ধ থাকে; যেখান থেকে Kubernetes, Code Engine বা VM ইমেজ টানে; যেখানে দুর্বলতা স্ক্যান করা যায়, অনুমতি নিয়ন্ত্রণ করা যায়, পুরোনো ইমেজ মুছে ফেলা যায়। রেজিস্ট্রি ছাড়া স্থাপন মানে অন্ধভাবে বিশ্বাস করা — কোনো প্রমাণ ছাড়াই যে সঠিক কোড চলছে।

অ্যাপ্লিকেশন হোস্টিংয়ের প্রশ্নটি ভিন্ন: অ্যাপ্লিকেশনটি কোথায় চলবে, কে তার জীবনচক্র পরিচালনা করবে? উত্তরগুলো একটি বর্ণালিতে (Spectrum) সাজানো যায়: এক প্রান্তে ভার্চুয়াল সার্ভার — সম্পূর্ণ নিয়ন্ত্রণ, সম্পূর্ণ দায়িত্ব; মাঝখানে Kubernetes — অর্কেস্ট্রেশন দেওয়া, কিন্তু ক্লাস্টার পরিচালনা দরকার; অন্য প্রান্তে সার্ভারলেস — প্ল্যাটফর্ম পরিচালনা করে, ব্যবহারকারী শুধু কাজের সংজ্ঞা দেয়। কোনো উত্তরই সর্বোত্তম নয় — প্রশ্নটি হলো দলের আকার, ট্র্যাফিকের ধরণ ও নিয়ন্ত্রণের চাহিদা।

> **CONCEPT:** ইমেজের **ট্যাগ (Tag)** একটি মানব-পাঠযোগ্য নাম (যেমন `v1.2.3`), আর **ডাইজেস্ট (Digest)** হলো বিষয়বস্তুর হ্যাশ — একই ডাইজেস্ট মানেই একই বিষয়বস্তু। ট্যাগ সরানো যায়, ডাইজেস্ট সরানো যায় না। তাই প্রোডাকশনে ট্যাগের ওপর নয়, ডাইজেস্টের ওপর নির্ভর করা উচিত — অথবা ট্যাগকে অপরিবর্তনীয় (Immutable) রাখার নীতি বসানো উচিত। `latest` ট্যাগটি সবচেয়ে বিপজ্জনক, কারণ তা যেকোনো সময় বদলে যেতে পারে।

১৬.৪ আর্কিটেকচার ও অভ্যন্তরীণ কার্যপ্রণালি



একটি ইমেজ পুশের যাত্রা: CI পাইপলাইন সোর্স কোড থেকে ইমেজ তৈরি করে, প্রতিটি স্তরকে ডাইজেস্ট দিয়ে চিহ্নিত করে, রেজিস্ট্রিতে পাঠায়। রেজিস্ট্রি IAM দিয়ে প্রমাণীকরণ করে — এই পরিচয়ের কি এই নেমস্পেসে পুশ করার অধিকার আছে? — তারপর স্তরগুলো সংরক্ষণ করে ও একটি ম্যানিফেস্ট (Manifest) লিপিবদ্ধ করে, যা বলে কোন স্তরগুলো মিলে এই ইমেজ। একটি পুলের সময় ক্লাস্টার একইভাবে প্রমাণীকরণ করে, ম্যানিফেস্ট পড়ে, প্রয়োজনীয় স্তরগুলো টেনে নেয়, এবং কনটেইনার চালায়। স্তরগুলো ক্যাশ করা যায় — তাই একই বেস ইমেজের ওপর তৈরি অনেক ইমেজের সাধারণ স্তর একবারই টানতে হয়।

দুর্বলতা স্ক্যান (Vulnerability Scanning) রেজিস্ট্রির একটি গুরুত্বপূর্ণ কাজ: ইমেজের প্রতিটি স্তরে পরিচিত দুর্বলতা (CVE) খোঁজা হয়, এবং ফলাফলের ভিত্তিতে নীতি বসানো যায় — যেমন গুরুতর দুর্বলতা থাকলে ইমেজটি প্রোডাকশনে স্থাপন করা যাবে না। স্ক্যানের ফলাফল একটি প্রতিবেদন, সমাধান নয় — দুর্বলতা সারাতে বেস ইমেজ হালনাগাদ করে আবার বিল্ড করতে হয়।

অন্যান্য সেবার সঙ্গে যোগাযোগ: রেজিস্ট্রি থেকে Kubernetes ও Code Engine ইমেজ টানে; CI পাইপলাইন রেজিস্ট্রিতে পুশ করে; COS-এ বিল্ড আর্টিফ্যাক্ট রাখা যায়; ডেটাবেস ও অন্যান্য সেবার সঙ্গে অ্যাপ্লিকেশন API দিয়ে কথা বলে। রেজিস্ট্রি ব্যর্থ হলে নতুন স্থাপন (Deployment) ব্যর্থ হয় — পুরোনো পডগুলো চলতে থাকে, কারণ তাদের ইমেজ ইতিমধ্যে নোডে ক্যাশ করা আছে। কিন্তু নোড পুনরায় তৈরি হলে বা নতুন পড তৈরি হলে ইমেজ টানতে হবে — তাই রেজিস্ট্রির উচ্চ-উপলব্ধতা ও অঞ্চল-ভিত্তিক প্রতিলিপি গুরুত্বপূর্ণ।

১৬.৫ ইমেজ পরিচালনা: ট্যাগিং, সংস্করণ ও ধারণকাল

ভালো ট্যাগিং নীতি: প্রতিটি বিল্ডে একটি অনন্য ট্যাগ — যেমন গিট কমিটের হ্যাশ বা সংস্করণ সংখ্যা (`v1.2.3`) — এবং একটি অর্থপূর্ণ ট্যাগ — যেমন `prod-stable` — যা একটি নির্দিষ্ট বিল্ডকে নির্দেশ করে। `latest` কখনোই প্রোডাকশনে ব্যবহার করবেন না। ট্যাগকে অপরিবর্তনীয় (Immutable) রাখার নীতি বসান: একবার পুশ করা ট্যাগ যেন আবার পুশ করে বদলে না ফেলা যায় — নইলে কোন বিল্ড চলছে তা নিশ্চিত করা যায় না।

ধারণকাল (Retention) নীতিও দরকার: প্রতিটি বিল্ডের ইমেজ চিরকাল রাখলে রেজিস্ট্রির খরচ ও বিশৃঙ্খলা বাড়ে। সাধারণ নীতি: প্রোডাকশনে স্থাপিত ইমেজ ও শেষ কয়েকটি সংস্করণ রাখা, পুরোনো dev বিল্ড নির্দিষ্ট দিন পর মুছে ফেলা। তবে মুছে ফেলার আগে নিশ্চিত করুন — কোনো চলমান পড বা রোলব্যাক পরিকল্পনা ওই ইমেজের ওপর নির্ভরশীল নয়। রোলব্যাকের জন্য আগের স্থিতিশীল ইমেজটি সবসময় রাখা উচিত।

অনুশীলন	করুন	করবেন না
ট্যাগ	অনন্য সংস্করণ ট্যাগ (`v1.2.3`)	শুধু `latest` ব্যবহার
প্রমাণীকরণ	প্রতিটি পরিবেশে আলাদা IAM অনুমতি	একটি চাবি সব জায়গায়
স্ক্যান	প্রতিটি পুশে স্ক্যান, নীতি বসানো	স্ক্যানের ফলাফল উপেক্ষা করা
ধারণকাল	পুরোনো dev ইমেজ মুছে ফেলা	প্রোডাকশন ইমেজ মুছে ফেলা

১৬.৬ সার্ভারলেস প্ল্যাটফর্ম: ইতিহাস ও বর্তমান অবস্থা

সার্ভারলেসের ইতিহাস বোঝা গুরুত্বপূর্ণ, কারণ পুরোনো নথিপত্রে পুরোনো সেবার নাম এখনও পাওয়া যায়। IBM Cloud-এ একসময় **IBM Cloud Functions** ছিল — Apache OpenWhisk-ভিত্তিক একটি ফাংশন-অ্যাজ-আ-সার্ভিস (FaaS) প্ল্যাটফর্ম, যেখানে ছোট ছোট ফাংশন ঘটনার (Event) উত্তরে চলত। এই সেবাটি এখন **ঐতিহাসিক / অবচয়িত (Legacy/Deprecated)** — নতুন কাজের জন্য এটি বেছে নেওয়া উচিত নয়। পুরোনো নথি বা টিউটোরিয়ালে এর নাম দেখলে বুঝতে হবে এটি অতীতের সেবা, বর্তমানের সুপারিশ নয়।

বর্তমান পছন্দ হলো **IBM Cloud Code Engine** — সার্ভারলেস কনটেইনার প্ল্যাটফর্ম। Code Engine-এ আপনি একটি কনটেইনার ইমেজ বা সোর্স কোড দেন, আর প্ল্যাটফর্ম তা চালায় — HTTP অনুরোধের উত্তরে (Application), নির্ধারিত সময়ে বা ঘটনার উত্তরে (Job), অথবা একবারের কাজ হিসেবে (Batch Job)। ট্র্যাফিক না থাকলে স্কেল শূন্যে (Scale to Zero) নামতে পারে, ট্র্যাফিক বাড়লে স্বয়ংক্রিয়ভাবে কপি বাড়ে। Code Engine-এর শক্তি হলো এর সরলতা: Kubernetes ক্লাস্টার পরিচালনা না করে, কনটেইনার দিয়ে সার্ভারলেস সুবিধা পাওয়া যায়।

> **WARNING:** পুরোনো ব্লগ বা ভিডিওতে IBM Cloud Functions-কে বর্তমান সেবা হিসেবে দেখানো হতে পারে — সেটি ভুল। নতুন স্থাপত্যে Code Engine ব্যবহার করুন। যদি কোনো পুরোনো সিস্টেম এখনও Cloud Functions-এ চলে, তবে তার স্থানান্তর (Migration) পরিকল্পনা করুন — Code Engine-এ ফাংশনটিকে কনটেইনার বা জব হিসেবে পুনর্লেখন করা যায় কি না, তা পরীক্ষা করে। স্থানান্তরের আগে পুরোনো ফাংশনের ট্রিগার, পরিবেশ চলক ও নির্ভরতা তালিকাভুক্ত করুন।

Code Engine কখন ব্যবহার করবেন? অস্থায়ী, ঘটনা-চালিত কাজ — যেমন ফাইল আপলোড হলে থাম্বনেইল তৈরি, নির্ধারিত সময়ে রিপোর্ট তৈরি, API-ভিত্তিক ছোট সেবা যেখানে ট্র্যাফিক অনিয়মিত। কখন এড়িয়ে চলবেন? দীর্ঘস্থায়ী, রাষ্ট্রযুক্ত, বা খুব কম বিলম্বের (Low Latency) প্রয়োজন এমন কাজে — সেখানে Kubernetes বা VM উপযুক্ত। Code

Engine-এর বিকল্প হলো Kubernetes-এ নিজে স্কেল-টু-জিরো বসানো, অথবা অন্য ক্লাউডের সার্ভারলেস সেবা — প্রতিটির নিজস্ব বিনিময় আছে, যা মাল্টি-ক্লাউড অধ্যায়গুলোতে (Volume 14-এর পরবর্তী অংশ) আলোচিত হয়েছে।

১৬.৭ CLI ও API উদাহরণ

```
# রেজিস্ট্রিতে লগ ইন ও নেমস্পেস তৈরি
ibmcloud cr login
ibmcloud cr namespace-add technova

# ইমেজ তৈরি, ট্যাগ ও পুশ (ধারণাগত)
docker build -t icr.io/technova/web:v1.2.3 .
docker push icr.io/technova/web:v1.2.3

# ইমেজ তালিকা ও দুর্বলতা স্ক্যানের ফলাফল
ibmcloud cr images --restrict technova/web
ibmcloud cr vulnerability-assessment \
  icr.io/technova/web:v1.2.3
```

```
# Code Engine: প্রকল্প তৈরি ও অ্যাপ্লিকেশন স্থাপন
# (ধারণাগত — প্রকৃত কমান্ড বর্তমান CLI ডকুমেন্টেশন
#  দেখে নিশ্চিত করুন)
ibmcloud ce project create --name technova-ce
ibmcloud ce application create --name technova-web \
  --image icr.io/technova/web:v1.2.3 \
  --port 8080 --min-scale 0 --max-scale 5

ibmcloud ce application get --name technova-web
ibmcloud ce jobrun submit --name technova-report \
  --image icr.io/technova/report:v1.0.0
```

API ইন্টিগ্রেশনের ধারণা: Code Engine অ্যাপ্লিকেশন একটি HTTP endpoint পায় — অন্য সেবা সেই endpoint-এ অনুরোধ পাঠাতে পারে। নিরাপত্তার জন্য endpoint-টি প্রমাণীকরণের পেছনে রাখা উচিত, অথবা API গেটওয়ে দিয়ে নিয়ন্ত্রিত করা উচিত। পরিবেশ চলক (Environment Variable) দিয়ে কনফিগারেশন দেওয়া হয় — ডেটাবেসের ঠিকানা, COS বাকেটের নাম — কিন্তু পাসওয়ার্ড কখনোই পরিবেশ চলকে সরাসরি লেখা উচিত নয়; সিক্রেট-পরিচালনা সেবা বা Code Engine-এর সিক্রেট ব্যবস্থা ব্যবহার করুন।

১৬.৮ স্থাপনের ধরণ, পর্যবেক্ষণ, নিরাপত্তা ও খরচ

স্থাপনের ধরণ (Deployment Pattern) তিনটি: **রোলিং (Rolling)** — ধীরে ধীরে পুরোনো কপির জায়গায় নতুন কপি আসে, সেবা বন্ধ হয় না; **ব্লু-গ্রিন (Blue-Green)** — নতুন সংস্করণ সম্পূর্ণ আলাদা পরিবেশে চালিয়ে পরীক্ষা করে, তারপর ট্র্যাফিক বদলে দেওয়া হয়; **ক্যানারি (Canary)** — প্রথমে অল্প ট্র্যাফিক (যেমন ৫%) নতুন সংস্করণে পাঠানো হয়, সমস্যা না হলে ধীরে ধীরে বাড়ানো হয়। TechNova প্রোডাকশনে ক্যানারি ব্যবহার করে — কারণ তাতে ভুল ধরা পড়লে ক্ষতি সীমিত থাকে। রোলব্যাকের পরিকল্পনা প্রতিটি ধরণেই থাকা উচিত: আগের ইমেজের ট্যাগ ও ডাইজেস্ট জানা থাকলে রোলব্যাক মানে শুধু আগের ট্যাগে ফেরা।

পর্যবেক্ষণ: অনুরোধের হার, ত্রুটির হার, বিলম্ব (Latency), কপির সংখ্যা, স্কেল-টু-জিরো থেকে জেগে ওঠার সময় (Cold Start), ইমেজ পুলের ব্যর্থতা, রেজিস্ট্রির স্ক্যান ফলাফল। সতর্কতা বসান: ত্রুটির হার হঠাৎ বাড়লে, বিলম্ব বাড়লে, স্ক্যানে গুরুতর দুর্বলতা পাওয়া গেলে। লগ সংগ্রহ করুন — Code Engine ও Kubernetes-এর লগ মনিটরিং সেবায় পাঠান, যাতে একটি অনুরোধের যাত্রা — রেজিস্ট্রি থেকে পড, পড থেকে ডেটাবেস — অনুসরণ করা যায়।

নিরাপত্তা: রেজিস্ট্রির নেমস্পেসে Least Privilege IAM; প্রাইভেট রেজিস্ট্রি ডিফল্ট, পাবলিক ইমেজ শুধু সচেতন সিদ্ধান্তে; প্রতিটি ইমেজে স্ক্যান ও নীতি; সিক্রেট কখনোই ইমেজে নয় — বিল্ডের সময় সিক্রেট যেন স্তরে (Layer) না থেকে যায় তা পরীক্ষা করুন; Code Engine-এর endpoint প্রমাণীকরণের পেছনে রাখুন। খরচের চালক: রেজিস্ট্রিতে সংরক্ষিত ইমেজের পরিমাণ, Code Engine-এ ব্যবহৃত CPU/মেমরি-সময়, অনুরোধের সংখ্যা, ডেটা স্থানান্তর। খরচ কমানোর উপায়: ছোট বেস ইমেজ ব্যবহার করা (মাল্টি-স্টেজ বিল্ড), অপ্রয়োজনীয় ইমেজ মুছে ফেলা, Code Engine-এ min-scale শূন্য রাখা যখন ট্র্যাফিক অনিয়মিত, এবং অপ্রয়োজনীয় কপি না রাখা।

> **REAL WORLD:** TechNova-র একবার একটি বিল্ডে ডেটাবেসের পাসওয়ার্ড ভুল করে ইমেজের একটি স্তরে চলে যায় — কারণ বিল্ডের সময় একটি কনফিগারেশন ফাইল কপি করা হয়েছিল। স্ক্যানে তা ধরা পড়েনি, কারণ তা দুর্বলতা নয়, গোপন তথ্য ফাঁস। সমাধান ছিল — পাসওয়ার্ড ঘোরানো, ইমেজটি মুছে ফেলা, এবং CI-তে একটি গোপন-তথ্য স্ক্যান (Secret Scanning) যোগ করা। এখন TechNova-র নিয়ম: কোনো সিক্রেট কখনোই বিল্ড প্রসঙ্গে (Build Context) আসে না।

ব্যর্থ হলে কী হয়? রেজিস্ট্রি অনুপলব্ধ হলে নতুন স্থাপন ব্যর্থ হয়, কিন্তু চলমান অ্যাপ্লিকেশন চলতে থাকে। Code Engine-এর একটি অঞ্চল ব্যর্থ হলে অন্য অঞ্চলে স্থাপন করতে হয় — তাই গুরুত্বপূর্ণ অ্যাপ্লিকেশনের জন্য মাল্টি-অঞ্চল পরিকল্পনা রাখুন। একটি খারাপ ইমেজ স্থাপিত হলে ক্যানারি ধরণে ক্ষতি সীমিত থাকে, রোলিংয়ে পুরো সেবা ক্ষতিগ্রস্ত হতে পারে — এ কারণেই স্থাপনের ধরণ একটি নিরাপত্তা সিদ্ধান্তও।

১৬.৯ সাধারণ ভুল ও ট্রাবলশুটিং

সাধারণ ভুল: (১) `latest` ট্যাগ প্রোডাকশনে ব্যবহার করা। (২) সিক্রেট ইমেজে রাখা। (৩) স্ক্যানের ফলাফল উপেক্ষা করা। (৪) Cloud Functions-কে বর্তমান সেবা ভেবে নতুন কাজ শুরু করা। (৫) Code Engine-এ দীর্ঘস্থায়ী রাষ্ট্রযুক্ত কাজ চালানো। (৬) রোলব্যাক পরিকল্পনা ছাড়া স্থাপন করা। (৭) রেজিস্ট্রির ধারণকাল নীতি না রাখা।

> **TROUBLESHOOTING:** ইমেজ পুল ব্যর্থ (`ImagePullBackOff`) হলে দেখুন — ইমেজের নাম ও ট্যাগ সঠিক কি না, রেজিস্ট্রিতে ইমেজটি আছে কি না, ক্লাস্টারের টানার অনুমতি (Image Pull Secret / IAM) সঠিক কি না, এবং নেটওয়ার্ক রেজিস্ট্রিতে পৌঁছাতে পারছে কি না। Code Engine-এ অ্যাপ্লিকেশন শুরু না হলে লগ দেখুন — পোর্ট কি সঠিক, পরিবেশ চলক কি ঠিক, স্বাস্থ্য পরীক্ষা কি ব্যর্থ হচ্ছে? ধীর কোল্ড স্টার্ট হলে ইমেজের আকার ও বেস ইমেজ পরীক্ষা করুন — ছোট ইমেজ দ্রুত শুরু হয়।

১৬.৯.১ CI/CD পাইপলাইন ও সাপ্লাই-চেইন নিরাপত্তা

রেজিস্ট্রি ও হোস্টিংয়ের মাঝখানে থাকে CI/CD পাইপলাইন — সেই পথ ধরে কোড প্রোডাকশনে যায়। একটি ভালো পাইপলাইনের ধাপগুলো: সোর্স পরীক্ষা (লিন্ট, ইউনিট পরীক্ষা), ইমেজ বিল্ড (মাল্টি-স্টেজ, ছোট বেস ইমেজ), দুর্বলতা স্ক্যান, গোপন-তথ্য স্ক্যান, স্বাক্ষর (Image Signing) — যাতে বোঝা যায় ইমেজটি বিশ্বস্ত পাইপলাইন থেকে এসেছে — তারপর staging-এ স্থাপন ও স্বয়ংক্রিয় পরীক্ষা, তারপর প্রোডাকশনে ক্যানারি। **সাপ্লাই-চেইন নিরাপত্তা** মানে এই পুরো পথটিকে সুরক্ষিত রাখা: বেস ইমেজ কোথা থেকে আসে, নির্ভরতা (Dependency) কোথা থেকে আসে, কে পাইপলাইন বদলাতে পারে, বিল্ডের সময় সিক্রেট কোথায় যায়। একটি সাধারণ আক্রমণ-পথ হলো নির্ভরতার মধ্যে দূষিত কোড ঢোকানো — তাই নির্ভরতার সংস্করণ লক (Lockfile) করা, নিয়মিত হালনাগাদ করা, ও অপ্রয়োজনীয় নির্ভরতা সরানো উচিত। পাইপলাইনের নিজের অনুমতিও Least Privilege হওয়া উচিত: CI পাইপলাইন শুধু dev/staging রেজিস্ট্রিতে পুশ করতে পারবে, প্রোডাকশনে স্থাপন করবে একটি আলাদা, পর্যালোচিত ধাপ। TechNova-র নিয়ম: প্রোডাকশন স্থাপনে মানুষের অনুমোদন

(Approval) বাধ্যতামূলক — স্বয়ংক্রিয় পাইপলাইন প্রস্তুত করে, মানুষ সিদ্ধান্ত নেয়। এই নিয়মটি ধীর মনে হতে পারে, কিন্তু এটি ভুল স্থাপনের সবচেয়ে সস্তা প্রতিরোধ।

১৬.৯.২ খরচের বাস্তবতা, কোল্ড স্টার্ট ও প্রোডাকশন প্রস্তুতি

সার্ভারলেসের খরচের প্রতিশ্রুতি — শুধু ব্যবহৃতটুকুর জন্য দাম — সত্য, কিন্তু শর্তসাপেক্ষ। Code Engine-এ খরচ আসে CPU/মেমরি-সময়, অনুরোধ ও ডেটা স্থানান্তর থেকে। যে কাজ সারাক্ষণ চলে, তার জন্য সার্ভারলেস সবসময় সস্তা নয় — একটি স্থির Kubernetes Deployment সস্তা হতে পারে। তাই সিদ্ধান্ত নেওয়ার আগে প্রকৃত ট্র্যাফিক ধরণ দেখুন: ট্র্যাফিক কি ঢেউয়ের মতো (অনিয়মিত, শীর্ষ ও খাদ), নাকি স্থির? ঢেউয়ের মতো ট্র্যাফিকে স্কেল-টু-জিরো মূল্যবান; স্থির ট্র্যাফিকে স্থির সংস্থান সস্তা হতে পারে। **কোল্ড স্টার্ট (Cold Start)** — স্কেল শূন্য থেকে জেগে ওঠার সময় — ব্যবহারকারীর অভিজ্ঞতাকে প্রভাবিত করে। ছোট ইমেজ, হালকা রানটাইম, ও উষ্ণ রাখার (Keep-Warm) নীতি — যেমন min-scale ১ রাখা — কোল্ড স্টার্ট কমায়, কিন্তু min-scale ১ রাখা মানে স্কেল-টু-জিরোর সাশ্রয় ছেড়ে দেওয়া। এই বিনিময়টি প্রতিটি অ্যাপ্লিকেশনে আলাদাভাবে ঠিক করতে হয়: ব্যবহারকারীর মুখোমুখি API-তে min-scale ১ যুক্তিসঙ্গত; রাতের ব্যাচ জবে min-scale ০ ঠিক। প্রোডাকশন প্রস্তুতির (Production Readiness) চেকলিস্ট: স্বাস্থ্য পরীক্ষা (Health Endpoint), সুন্দরভাবে বন্ধ হওয়া (Graceful Shutdown), সময়সীমা (Timeout) ও পুনরায় চেষ্টার নীতি, লগে অনুরোধের শনাক্তকারী (Request ID), সতর্কতা ও রানবুক (Runbook), রোলব্যাক পরিকল্পনা, ও খরচের সতর্কতা। একটি অ্যাপ্লিকেশন তখনই প্রোডাকশন-প্রস্তুত, যখন তা ব্যর্থ হলেও দলটি জানে কী করতে হবে — শুধু যখন তা চলছে তখন নয়।

১৬.১০ সারাংশ

Container Registry ইমেজের কেন্দ্রীয়, প্রমাণীকৃত ভাণ্ডার — ট্যাগ ও ডাইজেস্ট দিয়ে সংস্করণ, স্ক্যান দিয়ে নিরাপত্তা, ধারণকাল নীতি দিয়ে শৃঙ্খলা। অ্যাপ্লিকেশন হোস্টিংয়ের বর্ণালিতে VM, Kubernetes ও সার্ভারলেস — প্রতিটির নিজস্ব জায়গা আছে। বর্তমান সার্ভারলেস পছন্দ Code Engine; Cloud Functions (OpenWhisk-ভিত্তিক) ঐতিহাসিক, নতুন কাজের জন্য নয়। ভালো স্থাপন মানে — অনন্য ট্যাগ, স্ক্যান করা ইমেজ, ক্যানারি বা ব্লু-গ্রিন ধরণ, রোলব্যাক পরিকল্পনা, ও পর্যবেক্ষণ।

১৬.১১ যাচাই প্রশ্ন

1. ট্যাগ ও ডাইজেস্টের মধ্যে পার্থক্য কী? প্রোডাকশনে `latest` ব্যবহার করা কেন বিপজ্জনক?
2. IBM Cloud Functions ও Code Engine-এর মধ্যে সম্পর্ক কী? নতুন কাজের জন্য কোনটি বেছে নেবেন, এবং কেন?
3. রেজিস্ট্রি অনুপলব্ধ হলে চলমান অ্যাপ্লিকেশন কেন চলতে থাকে, আর নতুন স্থাপন কেন ব্যর্থ হয়?
4. রোলিং, ব্লু-গ্রিন ও ক্যানারি স্থাপনের মধ্যে পার্থক্য কী? কোন পরিস্থিতিতে ক্যানারি সবচেয়ে উপযুক্ত?
5. একটি ইমেজে গোপন তথ্য (Secret) চলে গেলে কী করতে হয়? শুধু ইমেজ মুছে ফেললেই কি যথেষ্ট?

১৬.১২ ব্যবহারিক অনুশীলন

1. TechNova-র একটি ওয়েব অ্যাপ্লিকেশনের জন্য রেজিস্ট্রি নকশা লিখুন — নেমস্পেস, ট্যাগিং নীতি, স্ক্যান নীতি, ধারণকাল নীতি ও IAM ভূমিকা সহ — এবং Code Engine-এ স্থাপনের ধাপগুলো লিখুন।
2. একটি পরীক্ষামূলক ইমেজ তৈরি করুন, তাতে ইচ্ছাকৃতভাবে একটি পুরোনো বেস ইমেজ ব্যবহার করে স্ক্যানের ফলাফল দেখুন, বেস ইমেজ হালনাগাদ করে আবার বিল্ড করুন, এবং দুটি স্ক্যানের পার্থক্যসহ একটি সংক্ষিপ্ত প্রতিবেদন লিখুন। কোনো বাস্তব চাবি ব্যবহার করবেন না।

PART IV • DEVOPS ENGINEERING MASTERY

Multi-Cloud & Platform Engineering

Multi-Cloud Architecture Fundamentals, Hybrid Cloud Networking, Multi-Cloud Kubernetes & OpenShift, Cross-Cloud Automation

অধ্যায় 17 • PART IV — Multi-Cloud & Platform Engineering

Chapter 17 — Multi-Cloud Architecture Fundamentals

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) single-cloud, hybrid cloud ও multi-cloud-এর মধ্যে ধারণাগত ও কার্যকর পার্থক্য ব্যাখ্যা করতে; (২) ব্যবসায়িক কারণে কখন multi-cloud যুক্তিযুক্ত আর কখন তা অপ্রয়োজনীয় জটিলতা, তা যুক্তি দিয়ে বিচার করতে; (৩) vendor lock-in কী, তা কোথায় তৈরি হয় এবং portability কীভাবে কমানো যায়, তা বলতে; (৪) একাধিক provider জুড়ে shared responsibility model কীভাবে ভাগ হয়, তা মানচিত্র আকারে দেখাতে; (৫) workload placement-এর সিদ্ধান্ত নিতে — কোন workload কোথায় যাবে, কেন যাবে; (৬) multi-cloud-এর প্রধান architecture pattern গুলো চিনতে ও তুলনা করতে; এবং (৭) trade-off — duplicated infrastructure, egress cost, observability ও staffing — হিসাব করে সিদ্ধান্ত নিতে।

এই অধ্যায়ের মূল সতর্কতা শুরুতেই মনে রাখুন: multi-cloud স্বয়ংক্রিয়ভাবে single-cloud-এর চেয়ে ভালো নয়। এটি একটি ব্যয়বহুল, পরিচালনাযোগ্য জটিলতা। শুধু তখনই এটি নিন, যখন কোনো নির্দিষ্ট, পরিমাপযোগ্য প্রয়োজন — যেমন নিয়ন্ত্রক বাধ্যবাধকতা, নির্দিষ্ট অঞ্চলে উপস্থিতি, কোনো provider-এর অনন্য সেবা, বা বাস্তব resilience প্রয়োজন — তা দাবি করে।

১৭.২ পূর্বশর্ত ঝালাই

এই অধ্যায় বুঝতে আপনার জানা থাকা উচিত: Volume 14-এর আগের অংশ থেকে cloud-এর মূল ধারণা — virtualization, VPC/VNet, subnet, security group, IAM, object storage, managed database; Kubernetes-এর মৌলিক ধারণা — Pod, Service, Deployment, Ingress; এবং Terraform-এর মৌলিক ধারণা — resource, provider, state। যদি এই শব্দগুলো অপরিচিত মনে হয়, তাহলে আগে অধ্যায় ১–৮ ঝালিয়ে নিন। এখানে আমরা ধরে নিচ্ছি আপনি একটি cloud-এ একটি ওয়েব অ্যাপ্লিকেশন চালাতে পারেন, এবং এখন প্রশ্ন হলো — দ্বিতীয় cloud যোগ করলে কী বদলায়?

TechNova-র প্রসঙ্গ মনে রাখুন: TechNova একটি কাল্পনিক মাঝারি আকারের সফটওয়্যার কোম্পানি। তার প্রধান পণ্য একটি SaaS অ্যাপ্লিকেশন — গ্রাহকরা ব্রাউজারে লগইন করে কাজ করেন। বর্তমানে TechNova-র workload মূলত IBM Cloud-এ চলে, কিছু analytics workload অন্য provider-এ পরীক্ষামূলকভাবে আছে। পুরো Part IV+V জুড়ে TechNova-কেই আমরা running example হিসেবে ব্যবহার করব।

১৭.৩ প্রথম নীতি থেকে: এক cloud, দুই cloud, নাকি অনেক cloud?

প্রথম নীতি থেকে শুরু করি। একটি অ্যাপ্লিকেশন চালাতে যা লাগে তা হলো: compute (কোড চালানোর জায়গা), storage (ডেটা রাখার জায়গা), network (ব্যবহারকারী ও সার্ভিসের মধ্যে যোগাযোগ) এবং identity (কে কে, কী করতে পারে)। একটি cloud provider এই চারটি জিনিস এক ছাদের নিচে, একই billing, একই IAM, একই নেটওয়ার্ক মডেলে দেয়। এটাই single-cloud-এর শক্তি: সরলতা। একজন ইঞ্জিনিয়ার একটি provider-এর IAM, নেটওয়ার্ক ও পর্যবেক্ষণ শিখলেই পুরো সিস্টেম চালাতে পারেন।

Hybrid cloud মানে হলো — on-premises ডেটা সেন্টার (নিজের বা colocation-এর সার্ভার) + এক বা একাধিক public cloud একসাথে ব্যবহার। যেমন TechNova-র পুরোনো billing ডেটা নিজের ডেটা সেন্টারে থাকে (নিয়ন্ত্রক কারণে), আর নতুন ওয়েব অ্যাপ IBM Cloud-এ চলে। Hybrid-এর মূল প্রশ্ন নেটওয়ার্ক: on-prem ও cloud-এর মধ্যে নিরাপদ, নির্ভরযোগ্য সংযোগ কীভাবে হবে? এটি অধ্যায় ১৮-এর বিষয়।

Multi-cloud মানে হলো — দুই বা ততোধিক public cloud provider একসাথে ব্যবহার, যেখানে workload-গুলো provider-দের মধ্যে ভাগ করা থাকে। যেমন TechNova-র মূল অ্যাপ IBM Cloud-এ, machine learning প্রশিক্ষণ অন্য provider-এ, আর disaster recovery তৃতীয় provider-এ। লক্ষ্য করুন: multi-cloud মানে এই নয় যে প্রতিটি workload সব provider-এ চলে। বেশিরভাগ বাস্তব multi-cloud হলো "best-of-breed + প্রয়োজনে বিভাজন" — প্রতিটি workload সেই provider-এ যায় যেখানে তার জন্য সবচেয়ে ভালো কারণ আছে।

একটি ভুল ধারণা ভাঙা জরুরি: "multi-cloud মানেই redundancy" — এটা সত্য নয়। দুটি provider-এ একই অ্যাপ চালানো মানে দুটি সম্পূর্ণ আলাদা সিস্টেম চালানো: দুটি IAM, দুটি নেটওয়ার্ক, দুটি মনিটরিং, দুটি billing, দুটি incident response পদ্ধতি। Redundancy তখনই কাজ করে যখন failover সত্যিই পরীক্ষিত হয় — ডেটা replicate হয়, DNS বদলায়, টিম জানে কী করতে হবে। পরীক্ষা না করা redundancy কাগজে সুন্দর, বাস্তবে বিপজ্জনক।

> **CONCEPT:** Vendor lock-in মানে কোনো provider-এর এমন সেবা বা API-তে আটকে যাওয়া, যেখান থেকে বের হতে সময়, খরচ বা পুনর্লিখন লাগে। Lock-in সবসময় খারাপ নয় — managed database বা serverless সেবা ব্যবহার করলে আপনি lock-in-এর বিনিময়ে পরিচালনার ভার কমান। প্রশ্নটি "lock-in আছে কি না" নয়, প্রশ্নটি "কোন স্তরে lock-in, আর তা থেকে বের হওয়ার খরচ কত"। Kubernetes, Terraform, OpenTelemetry ও S3-সামঞ্জস্যপূর্ণ API হলো portability বাড়ানোর উপায়, কিন্তু এগুলোও সম্পূর্ণ স্বাধীনতা দেয় না — প্রতিটি provider-এর IAM, নেটওয়ার্ক ও billing আলাদা থাকেই।

১৭.৪ ব্যবসায়িক কারণ: কেন multi-cloud, আর কেন নয়

Multi-cloud নেওয়ার বৈধ ব্যবসায়িক কারণগুলো সাধারণত এই কয়টির মধ্যে পড়ে। প্রথম, **নিয়ন্ত্রক ও ডেটা আবাসন (data residency)**: কোনো গ্রাহক বা নিয়ন্ত্রক চাইতে পারে ডেটা নির্দিষ্ট দেশে থাকুক, যেখানে আপনার প্রধান provider-এর region নেই। দ্বিতীয়, **নির্দিষ্ট সেবা বা অঞ্চলে উপস্থিতি**: কোনো provider-এর কোনো AI সেবা, বা কোনো অঞ্চলে কম latency, যা অন্য provider-এ নেই। তৃতীয়, **দর কষাকষি ও ঝুঁকি ছড়ানো**: এক provider-এর উপর সম্পূর্ণ নির্ভরতা কমানো, বা বড় চুক্তিতে দর কষাকষির অবস্থান শক্ত করা। চতুর্থ, **অধিগ্রহণ ও ইতিহাস**: কোম্পানি কিনে নেওয়ার পর দুটি cloud উত্তরাধিকারসূত্রে পাওয়া — এটি সবচেয়ে সাধারণ বাস্তব কারণ, যদিও এটি পরিকল্পিত সিদ্ধান্ত নয়। পঞ্চম, **resilience**: এক provider-এর বড় outage-এও সেবা চালু রাখা — তবে এটি সবচেয়ে ব্যয়বহুল কারণ, কারণ এর জন্য সত্যিকারের active-active নকশা লাগে।

অন্যদিকে, যেসব কারণে multi-cloud নেওয়া উচিত নয়: "ভবিষ্যতে যদি provider দাম বাড়ায়" — এটি অনুমান, বর্তমান খরচ নয়; "সবাই multi-cloud করছে" — এটি অনুকরণ; "আমরা vendor-neutral থাকতে চাই" — কিন্তু vendor-neutral থাকার খরচ (দুটি দল, দুটি টুলচেইন) প্রায়শই lock-in-এর খরচের চেয়ে বেশি। একটি ছোট দলের জন্য (যেমন TechNova-র ১২ জনের প্ল্যাটফর্ম দল) দুটি provider ভালোভাবে চালানো মানে প্রতিটি provider-এ অর্ধেক দক্ষতা — গভীরতার বদলে প্রস্থ।

- **Single-cloud ভালো যখন:** দল ছোট, workload সাধারণ, গতি ও সরলতা অগ্রাধিকার, বাজেট সীমিত।
- **Hybrid ভালো যখন:** on-prem-এ নিয়ন্ত্রক বা legacy বাধ্যবাধকতা আছে, কিন্তু নতুন workload cloud-এ যাচ্ছে।
- **Multi-cloud ভালো যখন:** নির্দিষ্ট, পরিমাপযোগ্য প্রয়োজন আছে — residency, অনন্য সেবা, অধিগ্রহণ, বা পরীক্ষিত resilience — এবং দল সেই জটিলতার খরচ বহন করতে পারে।

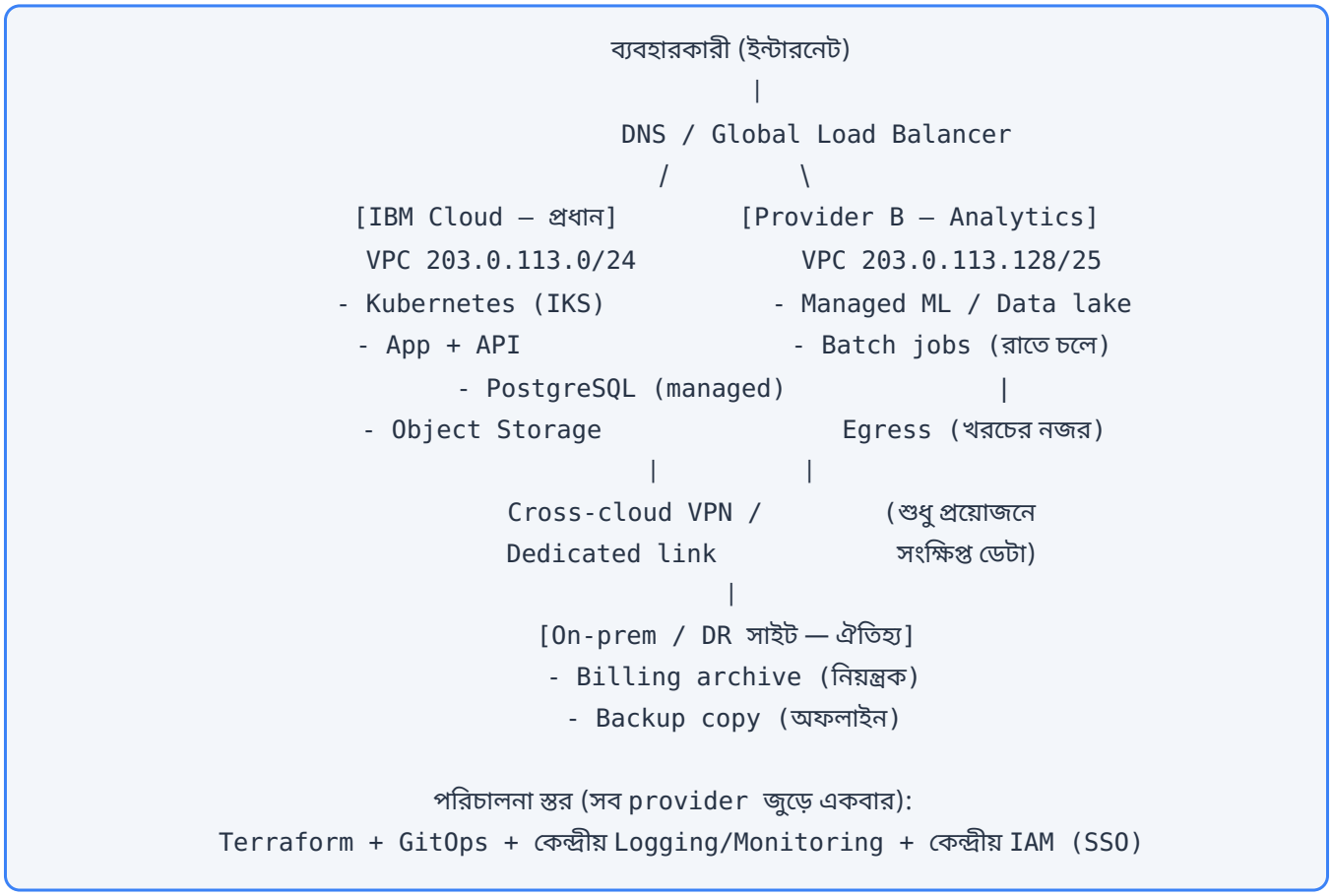
১৭.৫ Shared responsibility — provider জুড়ে দায়িত্ব কীভাবে ভাগ হয়

প্রতিটি cloud provider-এর shared responsibility model আছে: provider দায়ী physical ডেটা সেন্টার, hypervisor ও managed সেবার প্যাচের জন্য; গ্রাহক দায়ী নিজের ডেটা, IAM, নেটওয়ার্ক কনফিগারেশন ও অ্যাপ্লিকেশন কোডের জন্য। Multi-cloud-এ এই মডেলটি provider-ভেদে বদলায় — এক provider-এ যা managed (যেমন managed Kubernetes-এর control plane), অন্য provider-এ তার সীমানা ভিন্ন হতে পারে। TechNova-কে তাই প্রতিটি provider-এর জন্য আলাদা দায়িত্বের মানচিত্র রাখতে হয়: কে প্যাচ দেয়, কে ব্যাকআপ নেয়, কে encryption key রাখে, কে log রাখে।

সবচেয়ে বিপজ্জনক ফাঁক তৈরি হয় সীমানায়: provider A-র IAM-এ দেওয়া অনুমতি provider B-র ডেটা পড়তে পারে কি না; cross-cloud নেটওয়ার্কে encryption কে করে; একটি provider-এর outage হলে অন্য provider-এর ডেটা কীভাবে পুনরুদ্ধার হয়। এই সীমানাগুলোই অধ্যায় ২২-এর নিরাপত্তা আলোচনার কেন্দ্র।

দিক	Single-cloud	Hybrid	Multi-cloud
IAM মডেল	একটি	দুটি (on-prem + cloud), federation লাগে	প্রতিটি provider-এ আলাদা, federation জরুরি
নেটওয়ার্ক	একটি VPC মডেল	VPN / dedicated link	একাধিক VPC + cross-cloud সংযোগ
পর্যবেক্ষণ	একটি console	দুটি উৎস একত্র করতে হয়	কেন্দ্রীয় observability ছাড়া অন্ধকার
Billing	একটি বিল	দুটি খরচের ধারা	একাধিক বিল + egress খরচ
দক্ষতা	গভীর	মাঝারি	প্রস্থ বেশি, গভীরতা ঝুঁকিতে
Failover	Region-ভিত্তিক	On-prem ↔ cloud	Provider-ভিত্তিক (সবচেয়ে কঠিন)

১৭.৬ আর্কিটেকচার ডায়াগ্রাম: TechNova-র multi-cloud ছবি



লক্ষ্য করুন: এই ছবিতে প্রতিটি workload সব জায়গায় নেই। মূল অ্যাপ এক জায়গায়, analytics অন্য জায়গায়, archive তৃতীয় জায়গায়। পরিচালনা স্তর (Terraform, GitOps, observability, IAM) একবার লেখা হয়, সব provider-এ প্রয়োগ হয় — এটাই multi-cloud-কে টেকসই করে। যদি প্রতিটি provider-এর জন্য আলাদা পরিচালনা পদ্ধতি থাকে, তাহলে জটিলতা গুণিতক হারে বাড়ে।

১৭.৭ Architecture pattern — কোন নকশা কখন

Pattern 1 — Distributed by workload (সবচেয়ে সাধারণ): প্রতিটি workload তার সবচেয়ে উপযুক্ত provider-এ। TechNova-র মূল অ্যাপ IBM Cloud-এ, ML প্রশিক্ষণ Provider B-তে। সুবিধা: সরল, প্রতিটি workload স্বাধীন। অসুবিধা: workload-দের মধ্যে যোগাযোগে latency ও egress খরচ।

Pattern 2 — Active-passive DR: প্রধান provider-এ live, দ্বিতীয় provider-এ standby। সুবিধা: provider outage-এ বাঁচা যায়। অসুবিধা: standby-এর খরচ, ডেটা replication-এর জটিলতা, failover পরীক্ষা না করলে কাজ করে না।

Pattern 3 — Active-active: দুটি provider-এই live ট্রাফিক নেয়। সুবিধা: সর্বোচ্চ resilience ও latency। অসুবিধা: সবচেয়ে ব্যয়বহুল, ডেটা consistency-এর কঠিন সমস্যা (অধ্যায় ২৩), দুটি দলের মতো পরিচালনা। শুধু তখনই নিন যখন ব্যবসা সত্যিই মিনিটের outage-ও সহ্য করতে পারে না।

Pattern 4 — Cloud bursting: সাধারণ workload এক provider-এ, চাপ বাড়লে অতিরিক্ত capacity অন্য provider-এ। সুবিধা: নমনীয়তা। অসুবিধা: bursting-এর সময় ডেটা ও নেটওয়ার্ক প্রস্তুত রাখতে হয় — হঠাৎ করে bursting করা যায় না।

Pattern 5 — Data gravity অনুযায়ী: ডেটা যেখানে ভারী, compute সেখানে যায়। বড় ডেটা লেক এক provider-এ থাকলে analytics সেখানেই চালানো সস্তা — ডেটা টেনে অন্য provider-এ নেওয়া egress খরচে ডুবে যাবে।

> **REAL WORLD:** TechNova প্রথমে ভেবেছিল মূল অ্যাপ দুটি provider-এ active-active চালাবে। হিসাব করে দেখা গেল: ডেটাবেস replication, দুটি Kubernetes ক্লাস্টার, দুটি মনিটরিং, failover পরীক্ষা — সব মিলিয়ে খরচ প্রায় দ্বিগুণ, আর দলের সময়ের অর্ধেক যাবে দ্বিতীয় provider-এর পরিচর্যায়। তারা সিদ্ধান্ত বদলে workload-ভিত্তিক বিভাজন নিল: মূল অ্যাপ IBM Cloud-এ, analytics Provider B-তে, আর DR-এর জন্য cross-region (একই provider-এর অন্য region) — provider-ভিত্তিক DR নয়। এটাই সঠিক trade-off: resilience চাই, কিন্তু তার জন্য সবচেয়ে সস্তা যথেষ্ট উপায় বেছে নেওয়া।

১৭.৮ Workload placement — সিদ্ধান্তের কাঠামো

প্রতিটি workload-এর জন্য এই পাঁচটি প্রশ্ন করুন: (১) ডেটা কোথায় থাকে, আর ডেটা সরাতে কত খরচ (egress) ও সময় লাগবে? (২) Latency-র প্রয়োজন কত — ব্যবহারকারী কোথায়? (৩) কোনো নির্দিষ্ট সেবা কি শুধু এক provider-এ আছে? (৪) নিয়ন্ত্রক বাধ্যবাধকতা কী? (৫) দলের দক্ষতা কোথায় গভীর? এই পাঁচটির উত্তর একটি placement matrix-এ লিখুন। যে workload-এর উত্তর সব provider-এ একই, সেটি প্রধান provider-এই থাকুক — অকারণে সরাবেন না।

Portability বাড়ানোর ব্যবহারিক উপায়: container (Kubernetes) ব্যবহার করুন যাতে compute সরানো যায়; S3-সামঞ্জস্যপূর্ণ object storage API ব্যবহার করুন; managed সেবার proprietary বৈশিষ্ট্যের উপর নির্ভরতা কমান; infrastructure Terraform-এ লিখুন যাতে অন্য provider-এ পুনর্নির্মাণ করা যায়; observability-তে OpenTelemetry ব্যবহার করুন যাতে monitoring provider-নির্ভর না হয়। কিন্তু মনে রাখুন: portability-রও খরচ আছে — সর্বনিম্ন সাধারণ বিভাজক (lowest common denominator) ব্যবহার করলে আপনি প্রতিটি provider-এর সেরা সেবা থেকে বঞ্চিত হন।

১৭.৯ অভ্যন্তরীণ কার্যপ্রণালী: cross-cloud-এ একটি request কীভাবে যায়

ব্যবহারকারী technova.example-এ request পাঠালে DNS তাকে নিকটতম বা প্রধান provider-এর load balancer-এ পাঠায়। Load balancer Kubernetes Ingress-এ পাঠায়, Ingress Service-এ, Service Pod-এ। Pod-এর কোড যদি analytics ডেটা চায় যা Provider B-তে আছে, তাহলে request-টি cross-cloud VPN বা public internet দিয়ে (TLS-সহ) Provider B-র API-তে যায়, উত্তর ফিরে আসে, তারপর ব্যবহারকারীকে দেওয়া হয়। এই যাত্রায় প্রতিটি hop-এ latency, খরচ ও ব্যর্থতার সুযোগ আছে। একটি hop ব্যর্থ হলে কী হবে — timeout, retry, circuit breaker — তা আগেই ঠিক করতে হয়। এখানেই "কীভাবে যোগাযোগ করে" প্রশ্নটি গুরুত্বপূর্ণ: synchronous (সরাসরি API call) নাকি asynchronous (queue/message)? Cross-cloud-এ asynchronous প্রায়শই ভালো, কারণ এক provider-এর ধীরগতি বা outage অন্য provider-কে থামিয়ে দেয় না।

ব্যর্থতার চিত্র: Provider B-র API বন্ধ হলে TechNova-র মূল অ্যাপ কী করবে? যদি synchronous নির্ভরতা হয়, মূল অ্যাপও বন্ধ। যদি queue থাকে, মূল অ্যাপ চলবে, analytics পরে হবে। এই পার্থক্যই architecture-এর গুণমান নির্ধারণ করে।

১৭.১০ CLI/API উদাহরণ: provider-নিরপেক্ষ যাচাই

Multi-cloud-এ প্রথম কাজ হলো প্রতিটি provider-এ একই প্রশ্নের উত্তর একইভাবে পাওয়া: "কী চলছে?" নিচে Terraform-এর provider-নিরপেক্ষ ধারণা ও একটি সাধারণ যাচাই দেখানো হলো। আসল credential কখনো terminal-এ লিখবেন না; environment variable বা secret manager ব্যবহার করুন।

```
# IBM Cloud CLI — বর্তমান অ্যাকাউন্ট ও region যাচাই (উদাহরণ)
ibmcloud login --apikey CHANGE-ME-IBM-API-KEY -r us-south
ibmcloud ks cluster ls
ibmcloud resource service-instances

# একটি provider-নিরপেক্ষ ধারণা: kubectl দিয়ে যেকোনো ক্লাস্টার দেখা
kubectl config get-contexts
kubectl --context technova-iks get nodes
kubectl --context technova-provider-b get nodes

# DNS যাচাই — কোন provider-এ ট্রাফিক যাচ্ছে
dig +short technova.example
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://technova.example/health
```

এই কমান্ডগুলোর উদ্দেশ্য: provider বদলালেও যাচাইয়ের ভাষা একই রাখা। kubectl, Terraform ও curl — এই তিনটি multi-cloud ইঞ্জিনিয়ারের সবচেয়ে বেশি ব্যবহৃত “সাধারণ ভাষা”।

১৭.১১ IaC উদাহরণ: placement সিদ্ধান্ত কোডে লেখা

```
# modules/placement — কোন workload কোথায়, তা একটি জায়গায় লেখা
# এটি সিদ্ধান্তের দলিল; provider-নির্দিষ্ট resource আলাদা module-এ থাকে।
locals {
  placement = {
    web_app   = "ibm-cloud"   # কারণ: দলের দক্ষতা + প্রধান region
    analytics = "provider-b"  # কারণ: নির্দিষ্ট ML সেবা
    archive   = "on-prem"     # কারণ: নিয়ন্ত্রক বাধ্যবাধকতা
  }
}

# প্রতিটি workload-এর জন্য আলাদা module call; কোনো workload
# সব provider-এ duplicate করা হয় না — শুধু যেখানে তার কারণ আছে।
module "web_app_ibm" {
  source = "./modules/web-ibm"
  count  = local.placement["web_app"] == "ibm-cloud" ? 1 : 0
}
```

এই ছোট উদাহরণের শিক্ষা: placement সিদ্ধান্ত কোডে, মন্তব্যসহ, একটি জায়গায় রাখুন। ছয় মাস পরে কেউ জিজ্ঞেস করলে “এই workload এখানে কেন?” — উত্তর যেন git history-তে পাওয়া যায়, কারো স্মৃতিতে নয়।

১৭.১২ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — Provider outage: প্রধান provider-এর একটি region বন্ধ। TechNova-র cross-region failover (একই provider-এর অন্য region) কাজ করে, কারণ ডেটা replicate করা ছিল ও failover পরীক্ষিত ছিল। Provider-ভিত্তিক failover থাকলে আরও জটিল হতো — DNS, ডেটা, IAM সব বদলাতে হতো। শিক্ষা: region-ভিত্তিক resilience আগে, provider-ভিত্তিক পরে।

পরিস্থিতি ২ — Egress bill চমক: Analytics দল প্রতিদিন মূল ডেটাবেস থেকে বড় ডেটা Provider B-তে টানতে শুরু করল। মাসের শেষে egress বিল দেখে সবাই অবাক। সমাধান: ডেটা সরানোর বদলে analytics-কে ডেটার কাছে নেওয়া, বা রাতে batch-এ সংক্ষিপ্ত (aggregated) ডেটা পাঠানো। শিক্ষা: egress খরচ architecture-এর সিদ্ধান্ত, billing-এর নয় — ডিজাইনের সময় হিসাব করতে হয়।

পরিস্থিতি ৩ — দক্ষতার ঘাটতি: দুটি provider-এর IAM মডেল আলাদা হওয়ায় একজন নতুন ইঞ্জিনিয়ার ভুল করে Provider B-তে অতিরিক্ত অনুমতি দিয়ে ফেললেন। সমাধান: কেন্দ্রীয় SSO ও Terraform-এ IAM লেখা, হাতে console-এ অনুমতি দেওয়া বন্ধ। শিক্ষা: multi-cloud-এ মানুষের ভুলের সুযোগ দ্বিগুণ — তাই automation দ্বিগুণ জরুরি।

১৭.১৩ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: প্রতিটি provider-এ আলাদা IAM, আলাদা network, আলাদা encryption key। কেন্দ্রীয় identity federation (SSO) ছাড়া multi-cloud চালাবেন না। Cross-cloud ট্রাফিক সবসময় TLS দিয়ে encrypt করুন, এমনকি VPN-এর ভিতরেও। প্রতিটি provider-এর audit log কেন্দ্রীয় জায়গায় পাঠান — না হলে incident-এর সময় অর্ধেক ছবি অন্ধকার থাকবে। **খরচ:** খরচের চারটি ধারা — প্রতিটি provider-এর compute/storage বিল, cross-cloud egress, duplicated tooling (দুটি monitoring সেবা), এবং মানুষের সময় (দুটি provider শেখা ও চালানো)। বাজেটে চারটিই ধরুন। Cost allocation-এর জন্য প্রতিটি resource-এ tag বাধ্যতামূলক করুন: team, workload, environment — না হলে কোন workload কত খরচ করছে তা জানা যাবে না।

কখন ব্যবহার করবেন: যখন উপরের ১৭.৪-এর বৈধ কারণগুলোর অন্তত একটি পরিমাপযোগ্যভাবে আছে। কখন এড়াবেন: যখন দল ছোট, workload সাধারণ, বা কারণটি শুধু "ভবিষ্যতের ভয়"। বিকল্প: single-cloud + multi-region; hybrid (on-prem + এক cloud); বা এক provider-এর ভিতরে workload বিভাজন। Trade-off সবসময় একই: সরলতা ও গতি বনাম নমনীয়তা ও resilience।

> **WARNING:** "আমরা multi-cloud, তাই আমরা নিরাপদ" — এটি সবচেয়ে বিপজ্জনক ভুল ধারণা। পরীক্ষা না করা failover, replicate না করা ডেটা, আর দুটি provider-এর মধ্যে অজানা নির্ভরতা থাকলে outage-এর সময় আপনি আবিষ্কার করবেন যে দ্বিতীয় provider আসলে প্রথমটির উপর নির্ভরশীল ছিল (যেমন DNS বা identity এক provider-এ)। Resilience দাবি করার আগে failover পরীক্ষা করুন, আর সেই পরীক্ষার ফলাফল লিখে রাখুন।

১৭.১৪ সাধারণ ভুল

- প্রতিটি workload সব provider-এ চালানো — "সমানভাবে ভাগ" করা, কোনো কারণ ছাড়া।
- Egress খরচ হিসাব না করে ডেটা-ভারী workload আলাদা করা।
- প্রতিটি provider-এর জন্য আলাদা পরিচালনা পদ্ধতি (আলাদা CI/CD, আলাদা monitoring) রাখা।
- IAM হাতে console-এ দেওয়া — Terraform বা GitOps ছাড়া।
- Failover কখনো পরীক্ষা না করে "আমাদের DR আছে" বলা।
- Lock-in-কে শত্রু ভেবে প্রতিটি managed সেবা এড়ানো — ফলে নিজে সব পরিচালনা করতে হয়।
- Provider-এর marketing স্লাইডকে architecture প্রমাণ ভাবা।

১৭.১৫ ট্রাবলশুটিং

> **TROUBLESHOOTING:** Cross-cloud API call ধীর বা timeout হচ্ছে — প্রথমে দেখুন request-টি কোন পথে যাচ্ছে (traceroute / curl timing), egress কোথায় আটকাচ্ছে, DNS কোন IP দিচ্ছে, আর timeout/retry সেটিং কী। অনেক সময় সমস্যা provider-এ নয়, নিজের timeout (যেমন ২ সেকেন্ড) খুব কম, বা retry storm-এ। Retry-তে exponential backoff ও circuit breaker যোগ করুন, আর synchronous নির্ভরতা কমিয়ে queue ব্যবহার করুন।

সমস্যা দেখা দিলে এই ক্রমে যাচাই করুন: (১) DNS সঠিক provider-এ যাচ্ছে কি না; (২) নেটওয়ার্ক পথ (VPN/direct link) চালু আছে কি না; (৩) IAM অনুমতি প্রতিটি provider-এ সঠিক কি না; (৪) ডেটা কোথায়, আর সেটি replicate হয়েছে কি না; (৫) log কোথায় — কেন্দ্রীয় log-এ দুই provider-এর ঘটনা একই সময়রেখায় দেখা যাচ্ছে কি না। যদি log দুই জায়গায় ছড়িয়ে থাকে, incident investigation অর্ধেক সময় খুঁজতেই চলে যাবে।

১৭.১৫ক গভীরতর আলোচনা: portability-র স্তর, exit strategy ও সিদ্ধান্তের নথি

Portability-কে একটিমাত্র সুইচ ভাবলে ভুল হবে — এটি স্তরে স্তরে তৈরি হয়। সবচেয়ে নিচের স্তর হলো compute portability: container image যেকোনো Kubernetes ক্লাস্টারে চলে, কিন্তু image-এর ভিতরে যদি provider-নির্দিষ্ট agent, log forwarder বা secret-এর পথ hardcode করা থাকে, তাহলে image-টি আর portable থাকে না। তাই image বানানোর সময় provider-নির্দিষ্ট সবকিছু বাইরে রাখুন — environment variable, ConfigMap বা Secret-এর মাধ্যমে দিন, image-এর ভিতরে নয়। পরের স্তর হলো data portability: object storage-এর API যদি S3-সামঞ্জস্যপূর্ণ হয়, তাহলে ডেটা সরানো সহজ হয়, কিন্তু ডেটাবেসের ক্ষেত্রে তা সত্য নয় — প্রতিটি managed ডেটাবেসের backup ফরম্যাট, replication পদ্ধতি ও extension আলাদা। PostgreSQL-এর ক্ষেত্রে logical dump দিয়ে সরানো যায়, কিন্তু বড় ডেটাবেসে তা ঘণ্টা বা দিন লাগতে পারে, আর downtime-এর পরিকল্পনা লাগে। তাই ডেটা স্তরে portability-র প্রশ্নটি আগেই করুন: “এই ডেটাবেস থেকে বের হতে কতক্ষণ লাগবে, কত ডেটা হারাবে, আর কে তা পরীক্ষা করেছে?” পরীক্ষা না করা exit plan কাগজে সুন্দর, বাস্তবে অকেজো।

তৃতীয় স্তর হলো পরিচালনা portability: Terraform, GitOps ও OpenTelemetry — এই তিনটি টুল একসাথে থাকলে provider বদলালেও পরিচালনার ভাষা একই থাকে। কিন্তু এখানেও সীমা আছে — Terraform provider-এর নতুন সেবা আসতে সময় লাগে, আর প্রতিটি provider-এর IAM মডেল Terraform-এ লেখা হলেও তার অর্থ আলাদা। চতুর্থ স্তর হলো মানুষের portability: দলের দক্ষতা। একজন ইঞ্জিনিয়ার যদি একটি provider-এর IAM, নেটওয়ার্ক ও troubleshooting গভীরভাবে জানেন, তাহলে দ্বিতীয় provider-এ তিনি আবার শিক্ষানবিশ। এই মানবিক খরচটিই multi-cloud-এর সবচেয়ে অবহেলিত খরচ — বাজেটের শিটে তা দেখা যায় না, কিন্তু on-call-এর রাতে তা টের পাওয়া যায়। TechNova-র মতো ছোট দলের জন্য তাই নিয়মটি হলো: প্রধান provider-এ গভীরতা রাখুন, দ্বিতীয় provider-এ শুধু সেই অংশটুকু শিখুন যা সত্যিই দরকার — পুরো provider শেখার চেষ্টা করবেন না।

Exit strategy-কে একটি নথি হিসেবে লিখুন, এক পৃষ্ঠার বেশি নয়: কোন workload বের করা যাবে, কীভাবে, কতক্ষণে, কত খরচে, আর কে তা পরীক্ষা করেছে। প্রতিটি managed সেবার জন্য এই প্রশ্নগুলো করুন: ডেটা কোন ফরম্যাটে বের হয়? API কি খোলা মানের? বিকল্প সেবায় যেতে কোড কতটুকু বদলাতে হবে? যদি উত্তর হয় “জানি না”, তাহলে সেই সেবা ব্যবহার করার আগে উত্তরটি জেনে নিন — ব্যবহার শুরু করার পরে জানা অনেক দেরি হয়ে যায়। আরেকটি ব্যবহারিক কৌশল হলো abstraction-এর সঠিক স্তর বেছে নেওয়া: সবকিছুকে provider-নিরপেক্ষ করার চেষ্টা করলে আপনি প্রতিটি provider-এর সেরা সেবা থেকে বঞ্চিত হন (lowest common denominator-এর ফাঁদ), আবার সবকিছুকে provider-নির্দিষ্ট করলে lock-in সর্বোচ্চ হয়। মাঝামাঝি পথটি হলো: workload-এর মূল যুক্তি (business logic) portable রাখুন, আর provider-এর managed সেবা ব্যবহার করুন সেখানে যেখানে তার মূল্য স্পষ্ট — যেমন managed ডেটাবেসের backup ও patching — এবং সেই সিদ্ধান্তটি লিখে রাখুন।

সিদ্ধান্তের নথি (decision record) multi-cloud-এর সবচেয়ে সস্তা বীমা। প্রতিটি placement সিদ্ধান্তের জন্য একটি ছোট ADR (Architecture Decision Record) লিখুন: প্রসঙ্গ, বিবেচিত বিকল্প, সিদ্ধান্ত, পরিণতি, আর পর্যালোচনার তারিখ। ছয় মাস পরে যখন কেউ জিজ্ঞেস করবে “এই workload এখানে কেন?”, তখন উত্তরটি Git-এ থাকবে, কারো স্মৃতিতে নয়। ADR ছাড়া multi-cloud ধীরে ধীরে “কেউ জানে না কেন এটা এখানে” — এমন একটি জটিলতায় পরিণত হয়, যেখানে কেউ কিছু বদলাতে ভয় পায়। সবশেষে মনে রাখুন: multi-cloud-এর সাফল্য মাপা হয় provider-এর সংখ্যায় নয়, বরং এই প্রশ্নের উত্তরে

— “আমরা কি প্রতিটি workload কেন সেখানে আছে তা বলতে পারি, আর প্রয়োজনে তা সরাতে পারি?” যদি উত্তর হ্যাঁ হয়, তাহলে architecture সুস্থ; যদি না হয়, তাহলে provider-এর সংখ্যা বাড়ানো শুধু সমস্যাই বাড়াবে।

- **Compute portability:** image-এ provider-নির্দিষ্ট কিছু রাখবেন না; config বাইরে থেকে দিন।
- **Data portability:** ডেটাবেস থেকে বের হওয়ার সময়, ফরম্যাট ও downtime আগেই পরীক্ষা করুন।
- **পরিচালনা portability:** Terraform, GitOps ও OpenTelemetry — তিনটি একসাথে, একটি ভাষা।
- **মানুষের portability:** প্রধান provider-এ গভীরতা, দ্বিতীয়টিতে শুধু প্রয়োজনীয় অংশ।
- **Exit strategy:** এক পৃষ্ঠার নথি — কীভাবে বের হবেন, কতক্ষণে, কত খরচে, কে পরীক্ষা করেছে।

১৭.১৬ সারসংক্ষেপ

Multi-cloud মানে দুটি বা ততোধিক public cloud একসাথে ব্যবহার, প্রতিটি workload তার যুক্তিযুক্ত provider-এ। Single-cloud সরল ও দ্রুত, hybrid on-prem ও cloud-কে জোড়ে, multi-cloud নমনীয়তা দেয় কিন্তু duplicated infrastructure, egress খরচ, observability ও staffing-এর জটিলতা বাড়ায়। Vendor lock-in-কে স্তরে স্তরে বুঝুন, portability-র জন্য container, Terraform ও OpenTelemetry ব্যবহার করুন, কিন্তু সর্বনিম্ন সাধারণ বিভাজকের খরচ মনে রাখুন। Workload placement ডেটা, latency, সেবা, নিয়ন্ত্রক ও দক্ষতা দিয়ে ঠিক করুন, কোডে লিখুন, আর failover দাবি করার আগে পরীক্ষা করুন।

১৭.১৭ পর্যালোচনা প্রশ্ন

1. Single-cloud, hybrid cloud ও multi-cloud-এর মধ্যে মূল পার্থক্য কী, এবং প্রতিটি কখন উপযুক্ত?
2. Vendor lock-in কোন কোন স্তরে তৈরি হয়? Lock-in সবসময় খারাপ কেন নয়?
3. Shared responsibility model multi-cloud-এ কেন provider-ভেদে বদলায়, এবং সীমানায় কী ঝুঁকি তৈরি হয়?
4. Active-active, active-passive ও workload-ভিত্তিক বিভাজনের trade-off তুলনা করুন।
5. Egress খরচ কীভাবে architecture সিদ্ধান্তকে প্রভাবিত করে? একটি উদাহরণ দিন।

১৭.১৮ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি এক পৃষ্ঠার placement matrix তৈরি করুন: ৫টি workload (ওয়েব অ্যাপ, API, PostgreSQL, analytics, archive) — প্রতিটির জন্য ডেটা, latency, নিয়ন্ত্রক, সেবা ও দক্ষতা লিখে কোথায় রাখবেন তা যুক্তিসহ ঠিক করুন। কোনো workload অকারণে duplicate করবেন না।
2. Terraform-এ একটি `placement` locals ব্লক লিখুন যেখানে প্রতিটি workload-এর provider ও তার কারণ মন্তব্য হিসেবে আছে, এবং একটি module শুধু তখনই তৈরি হয় যখন placement মেলে। `terraform validate` চালিয়ে যাচাই করুন।

অধ্যায় 18 • PART IV — Multi-Cloud & Platform Engineering

Chapter 18 — Hybrid Cloud Networking

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) cross-cloud connectivity-র বিকল্পগুলো — public internet, Site-to-Site VPN, dedicated connectivity — তুলনা করতে; (২) hub-and-spoke architecture নকশা করতে; (৩) DNS ও IP address planning করতে যাতে CIDR overlap না হয়; (৪) routing ও segmentation দিয়ে ট্রাফিক নিয়ন্ত্রণ করতে; (৫) network observability সাজাতে; এবং (৬) connectivity failure-র পরিস্থিতিতে কী হবে তা আগেই অনুমান করে প্রস্তুতি নিতে।

১৮.২ পূর্বশর্ত ঝালাই

আপনার জানা থাকা উচিত: IP address ও CIDR notation (যেমন 203.0.113.0/24 মানে ২৫৬টি ঠিকানা), subnet, route table, security group / network ACL, DNS (A, CNAME) এবং TLS-এর মৌলিক ধারণা। VPC মানে একটি cloud-এর ভিতরে আপনার নিজের বিচ্ছিন্ন নেটওয়ার্ক; VNet/Azure বা VPC/AWS-এ ধারণাটি একই, নাম ও সীমা আলাদা। এই অধ্যায়ে আমরা ধরে নিচ্ছি আপনি একটি VPC-তে subnet বানাতে পারেন, এবং এখন প্রশ্ন হলো — দুটি VPC, বা VPC ও on-prem, কীভাবে নিরাপদে কথা বলবে?

১৮.৩ প্রথম নীতি থেকে: নেটওয়ার্ক মানে পথ, আর পথ মানে বিশ্বাস

নেটওয়ার্কিং-এর প্রথম নীতি সহজ: একটি packet-কে জানতে হয় কোথায় যেতে হবে (routing), আর গ্রহণকারীকে জানতে হয় packet-টি বিশ্বাসযোগ্য কি না (segmentation ও encryption)। Cloud-এর ভিতরে এটি সহজ, কারণ provider-ই পথ ও বিচ্ছিন্নতা দেয়। Cloud-এর বাইরে গেলেই দুটি সমস্যা: পথটি public internet দিয়ে যায় কি না, আর ঠিকানা (IP) দুই পাশে সংঘর্ষ (overlap) করে কি না।

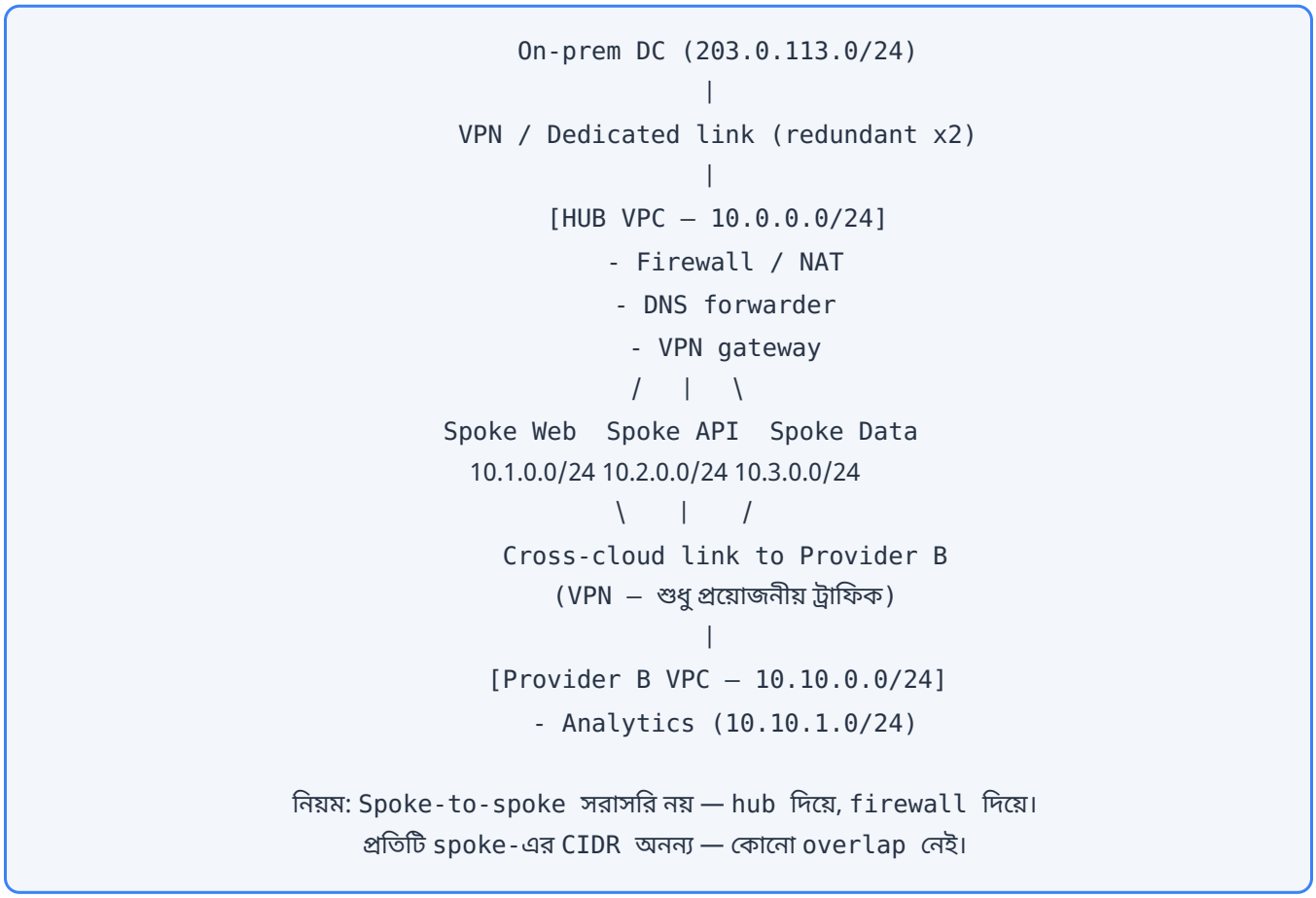
Cross-cloud connectivity-র তিনটি স্তর ভাবুন। প্রথম, **public internet** — সবচেয়ে সহজ, সবচেয়ে কম নির্ভরযোগ্য: ট্রাফিক TLS দিয়ে encrypt করা যায়, কিন্তু latency ওঠানামা করে, আর IP allowlist-এর উপর নির্ভর করতে হয়। দ্বিতীয়, **Site-to-Site VPN** — internet-এর উপর দিয়ে encrypted tunnel: সস্তা, দ্রুত বসানো যায়, কিন্তু bandwidth ও latency internet-এর উপর নির্ভরশীল। তৃতীয়, **dedicated connectivity** — provider-এর private নেটওয়ার্কে সরাসরি সংযোগ (যেমন IBM Cloud Direct Link, AWS Direct Connect, Azure ExpressRoute ধারণা): বেশি স্থিতিশীল latency, বেশি bandwidth, কিন্তু সেটআপে সময় ও খরচ বেশি, আর provider-এর পয়েন্ট অব প্রেজেন্সের উপর নির্ভর করে। কোনটি নেবেন তা নির্ভর করে ট্রাফিকের পরিমাণ, latency-র সংবেদনশীলতা ও বাজেটের উপর — “সবচেয়ে ভালো” বলে কিছু নেই।

> **CONCEPT:** Dedicated connectivity মানে আপনার নেটওয়ার্ক ও cloud provider-এর নেটওয়ার্কের মধ্যে একটি private, সরাসরি পথ — public internet এড়িয়ে। এটি internet-এর চেয়ে স্থিতিশীল, কিন্তু “dedicated” মানেই “ব্যর্থ হয় না” নয় — ফাইবার কাটা, provider-এর পোর্ট সমস্যা, BGP misconfiguration — সবই হতে পারে। তাই dedicated link-এর সাথেও redundant পথ (দ্বিতীয় link বা VPN backup) রাখা হয়। খরচের হিসাবেও সতর্ক থাকুন: dedicated link-এর port খরচের পাশাপাশি ডেটা স্থানান্তরের খরচও থাকতে পারে — provider-এর মূল্য তালিকা না দেখে অনুমান করবেন না।

১৮.৪ Hub-and-spoke architecture

Hub-and-spoke মানে: একটি কেন্দ্রীয় hub নেটওয়ার্ক (বা VPC) থাকে, আর প্রতিটি workload নেটওয়ার্ক (spoke) hub-এর সাথে যুক্ত হয়। Spoke-গুলো সরাসরি একে অপরের সাথে কথা বলে না — সব ট্রাফিক hub দিয়ে যায়, যেখানে firewall, inspection ও logging বসানো যায়। TechNova-র জন্য hub-এ থাকে: কেন্দ্রীয় firewall, DNS forwarding, VPN/dedicated link-এর termination, আর কেন্দ্রীয় observability। Spoke-এ থাকে: web, API, data — প্রতিটি আলাদা VPC বা VNet-এ।

Hub-and-spoke-এর সুবিধা: নিয়ন্ত্রণ এক জায়গায়, নতুন spoke যোগ করা সহজ, segmentation পরিষ্কার। অসুবিধা: hub-ই bottleneck ও single point of failure হতে পারে — তাই hub-কে redundant করতে হয় (দুটি availability zone, দুটি firewall instance), আর hub দিয়ে যাওয়া ট্রাফিকের latency হিসাব করতে হয়। বিকল্প হলো mesh — প্রতিটি নেটওয়ার্ক সরাসরি প্রতিটির সাথে যুক্ত — যা ছোট পরিবেশে সহজ, বড় পরিবেশে রাউটিং জটিলতায় ডুবে যায়।



১৮.৫ DNS design ও IP address planning

IP planning-এর সবচেয়ে বড় ভুল হলো পরে overlap আবিষ্কার করা। শুরুতে একটি ঠিকানা পরিকল্পনা লিখুন: প্রতিটি cloud, প্রতিটি region, প্রতিটি environment (prod/staging/dev) এবং on-prem-এর জন্য আলাদা, অ-overlapping CIDR বরাদ্দ করুন। যেমন TechNova: IBM Cloud prod 10.0.0.0/16, Provider B 10.10.0.0/16, on-prem 192.168.0.0/16, ভবিষ্যৎ বৃদ্ধির জন্য ফাঁকা জায়গা রাখুন। RFC 1918 private range (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) ব্যবহার করুন, আর প্রতিটি VPC-তে /24 বা /23 subnet দিয়ে শুরু করুন — খুব ছোট subnet পরে বাড়ানো কঠিন।

DNS design: প্রতিটি cloud-এর নিজস্ব private DNS আছে, কিন্তু cross-cloud-এ নাম কীভাবে resolve হবে? সমাধান: কেন্দ্রীয় DNS forwarding — hub-এ একটি forwarder যা প্রতিটি cloud-এর private zone জানে, বা প্রতিটি cloud-এ conditional forwarder। Public নাম (technova.example) আর private নাম (db.internal.technova.example) আলাদা zone-এ রাখুন। Split-horizon DNS-এর বিপদ: একই নাম ভিতরে ও বাইরে ভিন্ন IP দিলে debugging কঠিন হয় — নামকরণের নিয়ম লিখে রাখুন। DNS-এর TTL কম রাখুন failover-এর জন্য, কিন্তু খুব কম TTL resolver-এর উপর চাপ বাড়ায়।

১৮.৬ Routing ও segmentation — কে কার সাথে কথা বলতে পারে

Routing মানে route table-এ লেখা: “10.10.0.0/16-এর দিকে যেতে হলে VPN gateway দিয়ে যাও।” প্রতিটি VPC-তে route table যাচাই করুন — ভুল route মানে ট্রাফিক ভুল পথে, বা কোথাও যাচ্ছে না। Segmentation মানে: security group ও network ACL দিয়ে শুধু প্রয়োজনীয় port ও উৎসকে অনুমতি দেওয়া। নীতি: least privilege — web spoke শুধু API spoke-র 443 port-এ কথা বলতে পারে, data spoke-র সাথে সরাসরি নয়; data spoke শুধু API spoke থেকে 5432 port-এ সংযোগ নেয়।

Cross-cloud segmentation-এর বিশেষত্ব: provider A-র security group provider B-র IP জানে না — তাই CIDR-ভিত্তিক নিয়ম লিখতে হয়, আর CIDR বদলালে নিয়মও বদলাতে হয়। এই কারণে IP planning ও segmentation একসাথে করতে হয়, আলাদা নয়। Network policy (Kubernetes NetworkPolicy) আরও এক স্তর যোগ করে: Pod-স্তরে কে কার সাথে কথা বলতে পারে।

১৮.৭ অভ্যন্তরীণ কার্যপ্রণালী: একটি packet-এর যাত্রা ও VPN কীভাবে কাজ করে

Site-to-Site VPN-এ দুটি gateway (প্রতিটি পাশে একটি) IKE দিয়ে key বিনিময় করে, তারপর IPsec tunnel তৈরি করে। Packet hub থেকে বের হওয়ার সময় encrypt হয়ে tunnel দিয়ে যায়, অন্য পাশে decrypt হয়ে গন্তব্যে পৌঁছায়। ভিতরে কী ঘটে: encapsulation (মূল packet-এর উপর নতুন header), encryption (AES-GCM ধারণা), authentication (packet বদলানো হয়নি তা যাচাই)। Tunnel বন্ধ হলে packet যায় না — কিন্তু অ্যাপ্লিকেশন তা জানে না, শুধু timeout দেখে। তাই VPN-এর health আলাদাভাবে monitor করতে হয়: tunnel status, BGP session, latency।

BGP (Border Gateway Protocol) হলো সেই ভাষা যাতে দুটি নেটওয়ার্ক একে অপরকে route জানায়। BGP ভুল হলে ভুল route ছড়িয়ে পড়ে — ট্রাফিক ভুল জায়গায় যেতে পারে। তাই BGP-তে route filtering ও prefix limit বসানো হয়। Dedicated link-এও BGP ব্যবহৃত হয়।

১৮.৮ CLI/API উদাহরণ

```
# Route যাচাই — packet কোথায় যাবে (উদাহরণ, Linux)
ip route show
traceroute 10.10.1.10
# DNS যাচাই — private নাম resolve হচ্ছে কি না
dig +short db.internal.technova.example
nslookup analytics.internal.technova.example

# VPN tunnel status — provider CLI-তে (ধারণা; কমান্ড provider-ভেদে বদলায়)
ibmcloud is vpn-gateways
ibmcloud is vpn-gateway-connections <gateway-id>

# Kubernetes-এ NetworkPolicy আছে কি না
```

```
kubectl get networkpolicy -A
kubectl describe networkpolicy api-allow-web -n prod
```

এই কমান্ডগুলো incident-এর সময় প্রথম পাঁচ মিনিটে চালানো উচিত — "নেটওয়ার্ক কি চালু আছে?" প্রশ্নের উত্তর এগুলোই দেয়।

১৮.৯ IaC উদাহরণ: VPC, subnet ও route Terraform-এ

```
# Hub VPC + spoke VPC — CIDR overlap এড়াতে variables-এ এক জায়গায়
variable "hub_cidr"   { default = "10.0.0.0/24" }
variable "web_cidr"   { default = "10.1.0.0/24" }
variable "data_cidr"  { default = "10.3.0.0/24" }

resource "ibm_is_vpc" "hub" {
  name = "technova-hub"
}

resource "ibm_is_subnet" "hub_subnet" {
  name            = "hub-subnet"
  vpc             = ibm_is_vpc.hub.id
  zone            = "us-south-1"
  ipv4_cidr_block = var.hub_cidr
}

# Route: data spoke-এর দিকে যেতে hub firewall দিয়ে
# (আসল resource নাম provider-ভেদে বদলায় — এটি ধারণাগত উদাহরণ)
# resource "ibm_is_vpc_routing_table_route" "to_data" { ... }
```

Terraform-এ নেটওয়ার্ক লেখার সুবিধা: CIDR বদলালে সব জায়গায় একসাথে বদলায়, আর `terraform plan`-এ দেখা যায় কোন route বদলাচ্ছে — হাতে console-এ route বদলানোর চেয়ে নিরাপদ।

১৮.১০ প্রোডাকশন পরিস্থিতি ও ব্যর্থতার দৃশ্য

VPN tunnel down: এক পাশের gateway restart হলে tunnel ভেঙে যায়, cross-cloud ট্রাফিক থামে। প্রস্তুতি: redundant tunnel (দুটি), BGP-তে দ্রুত failover, অ্যাপে timeout ও retry, আর alert — tunnel down হলে ২ মিনিটের মধ্যে জানতে হবে। **CIDR overlap:** নতুন VPC-র CIDR পুরোনোটির সাথে মিলে গেল — routing অনির্ধারিত, কিছু packet ভুল জায়গায় যায়। প্রস্তুতি: IPAM (IP address management) শিট, নতুন CIDR-এর আগে পর্যালোচনা। **DNS ভুল:** private নাম public IP দিল — ডেটা internet দিয়ে ঘুরে আসে। প্রস্তুতি: DNS query log, split-horizon যাচাই। **Hub failure:** hub firewall বন্ধ — সব spoke বিচ্ছিন্ন। প্রস্তুতি: hub-কে দুটি zone-এ redundant করা, আর spoke-গুলোর মধ্যে জরুরি পথের পরিকল্পনা।

> **REAL WORLD:** TechNova একবার staging-এর CIDR prod-এর CIDR-এর সাথে overlap করে ফেলেছিল — কারণ staging-এর পরিকল্পনা আলাদা শিটে ছিল। ফলে staging-এর ডেটাবেসে যাওয়া packet prod-এ চলে গিয়েছিল। শিক্ষা: IP planning একটি মাত্র উৎসে (single source of truth) রাখুন, Terraform variable বা IPAM টুলে, আর নতুন নেটওয়ার্কের আগে সেই উৎস যাচাই বাধ্যতামূলক করুন।

১৮.১১ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: Cross-cloud ট্রাফিক সবসময় encrypt করুন — VPN-এর ভিতরেও TLS। Security group-এ 0.0.0.0/0 দিয়ে ডেটাবেস খুলবেন না। Dedicated link-ও "private" বলে encryption ছাড়বেন না — provider-এর নেটওয়ার্কে অন্য গ্রাহকও থাকে, যদিও বিচ্ছিন্ন। **খরচ:** VPN gateway-র ঘণ্টা প্রতি খরচ, dedicated link-এর port খরচ, আর cross-cloud ডেটা স্থানান্তরের খরচ — তিনটি আলাদা। ট্রাফিকের পরিমাণ না জেনে dedicated link নেবেন না; প্রথমে VPN দিয়ে পরিমাপ করুন, তারপর সিদ্ধান্ত নিন। যখন ব্যবহার করবেন: যখন নিয়মিত, latency-সংবেদনশীল cross-cloud ট্রাফিক আছে। যখন এড়াবেন: যখন ট্রাফিক মাঝে মাঝে, ছোট, আর public internet + TLS যথেষ্ট। বিকল্প: public API + mTLS, queue-ভিত্তিক asynchronous যোগাযোগ।

> **WARNING:** VPN বা dedicated link বসানো মানেই নিরাপত্তা নয়। Tunnel-এর ভিতর দিয়ে যাওয়া ট্রাফিকও যদি unauthenticated API হয়, তাহলে যে কেউ tunnel-এ ঢুকতে পারলেই সব পেয়ে যাবে। Network segmentation-এর পাশাপাশি workload identity ও mTLS (অধ্যায় ২২) লাগে। নেটওয়ার্ককে একমাত্র প্রতিরক্ষা ভাববেন না।

১৮.১২ সাধারণ ভুল

- CIDR পরিকল্পনা ছাড়া VPC বানানো, পরে overlap আবিষ্কার করা।
- একটি মাত্র VPN tunnel — redundant পথ না রাখা।
- Hub-কে single point of failure হিসেবে রেখে দেওয়া।
- DNS-কে অবহেলা করা — private ও public নাম গুলিয়ে ফেলা।
- Security group-এ অতিরিক্ত অনুমতি (0.0.0.0/0) দিয়ে "আপাতত চালানো"।
- Network observability না রাখা — flow log বন্ধ রাখা।

১৮.১৩ ট্রাবলশুটিং

> **TROUBLESHOOTING:** "এক cloud থেকে অন্য cloud-এ ping যাচ্ছে না" — ক্রম: (১) দুই পাশের CIDR overlap আছে কি না; (২) route table-এ route আছে কি না (দুই পাশেই); (৩) security group / ACL port খোলা কি না; (৪) VPN tunnel up কি না (BGP session সহ); (৫) DNS সঠিক IP দিচ্ছে কি না; (৬) অ্যাপ্লিকেশন port শুনছে কি না (telnet/curl)। সবচেয়ে সাধারণ কারণ route বা security group — provider outage সবার শেষে ভাবুন।

Flow log ও packet capture ব্যবহার করুন: কোন packet কোথায় আটকালো তা log-এ দেখা যায়। Traceroute-এর প্রতিটি hop লিখে রাখুন — incident-এর সময় এই তথ্যই provider support-কে দিতে হয়।

১৮.১৩ক গভীরতর আলোচনা: BGP, MTU, latency ও নেটওয়ার্ক পর্যবেক্ষণের বাস্তবতা

BGP-কে শুধু একটি প্রোটোকল ভাবলে তার গুরুত্ব বোঝা যাবে না — এটি আসলে দুটি সংস্থার মধ্যে বিশ্বাসের চুক্তি। আপনার নেটওয়ার্ক যখন provider-কে বলে "10.10.0.0/16 আমার দিকে আসে", তখন provider সেই কথা বিশ্বাস করে ট্রাফিক পাঠায়। ভুল prefix ঘোষণা করলে ট্রাফিক ভুল জায়গায় যেতে পারে, বা পুরো নেটওয়ার্ক বিচ্ছিন্ন হতে পারে — তাই BGP-তে prefix filtering, max-prefix limit ও AS-path যাচাই বাধ্যতামূলক। TechNova-র ক্ষেত্রে নিয়মটি হলো: শুধু নিজের CIDR-

ই ঘোষণা করা, আর provider-এর কাছ থেকে শুধু প্রয়োজনীয় route-ই নেওয়া — “সব route নেওয়া” মানে নিজের route table-কে provider-এর ভুলের উপর নির্ভরশীল করা। BGP session-এর স্বাস্থ্য আলাদাভাবে monitor করুন: session up/down, received prefix-এর সংখ্যা, আর flap (বারবার up/down) — flap মানে কোনো পাশে অস্থিরতা আছে, যা latency-র ওঠানামা হিসেবে দেখা দেবে।

MTU (Maximum Transmission Unit) একটি ছোট সংখ্যা যা বড় সমস্যা তৈরি করে। VPN tunnel-এ encapsulation-এর জন্য overhead লাগে, তাই tunnel-এর ভিতরের MTU বাইরের চেয়ে ছোট হয়। MTU ভুল হলে বড় packet ভেঙে যায় (fragmentation) বা হারিয়ে যায় — ছোট request চলে, বড় response আটকে যায়, আর debugging-এ মনে হয় “নেটওয়ার্ক মাঝে মাঝে কাজ করে”। সমাধান: tunnel-এর MTU পরীক্ষা করে সেট করা, আর Path MTU Discovery-কে firewall-এ block না করা। Latency-র ক্ষেত্রেও একই কথা: cross-cloud latency-কে একটি সংখ্যা ভাববেন না — এটি একটি বণ্টন (distribution)। গড় latency ৩০ মিলিসেকেন্ড হলেও p99 ৩০০ মিলিসেকেন্ড হতে পারে, আর সেই p99-ই ব্যবহারকারীর অভিজ্ঞতা নির্ধারণ করে। তাই latency-র SLO p95/p99 দিয়ে লিখুন, গড় দিয়ে নয়, আর latency-র বাজেট প্রতিটি hop-এ ভাগ করে দিন — DNS, TLS handshake, application, database — প্রতিটির জন্য আলাদা বাজেট।

Network observability-র তিনটি উৎস একসাথে লাগে: flow log (কে কার সাথে, কত ডেটা, কখন), packet capture (প্রয়োজনে গভীরে দেখা), আর synthetic probe (নিয়মিত ping/traceroute দিয়ে পথের স্বাস্থ্য যাচাই)। Flow log ছাড়া আপনি জানবেন না কোন workload কত egress করছে, আর synthetic probe ছাড়া জানবেন না পথটি কখন ধীর হচ্ছে — ব্যবহারকারী অভিযোগ করার আগেই probe জানিয়ে দেবে। সবশেষে, নেটওয়ার্ক পরিবর্তনের পদ্ধতি: প্রতিটি route, firewall নিয়ম বা DNS পরিবর্তন Terraform-এ, pull request-এ, আর পরিবর্তনের আগে ও পরে connectivity পরীক্ষা সহ। হাতে console-এ firewall নিয়ম বদলানো মানে পরের incident-এ কেউ জানবে না নিয়মটি কে, কেন বদলেছিল।

১৮.১৪ সারসংক্ষেপ

Hybrid networking মানে পথ ও বিশ্বাস — routing, segmentation, DNS ও IP planning একসাথে। VPN সস্তা ও দ্রুত, dedicated link স্থিতিশীল কিন্তু ব্যয়বহুল, public internet + TLS মাঝে মাঝে যথেষ্ট। Hub-and-spoke নিয়ন্ত্রণ দেয়, কিন্তু hub-কে redundant করতে হয়। CIDR overlap, DNS ভুল ও একক tunnel — এই তিনটি ভুল সবচেয়ে বেশি ক্ষতি করে। Network observability ছাড়া cross-cloud চালানো মানে অন্ধকারে গাড়ি চালানো।

১৮.১৫ পর্যালোচনা প্রশ্ন

1. Site-to-Site VPN ও dedicated connectivity-র মধ্যে কখন কোনটি বেছে নেবেন?
2. Hub-and-spoke-এর সুবিধা ও ঝুঁকি কী? Hub-কে কীভাবে redundant করবেন?
3. CIDR overlap কেন বিপজ্জনক, এবং তা প্রতিরোধে IP planning কীভাবে করবেন?
4. DNS-এ split-horizon কী, এবং এতে কী ভুল হতে পারে?
5. VPN tunnel down হলে অ্যাপ্লিকেশন ও পর্যবেক্ষণে কী দেখা যাবে, আর প্রথম পদক্ষেপ কী হবে?

১৮.১৬ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি IP পরিকল্পনা শিট তৈরি করুন: IBM Cloud prod/staging, Provider B, on-prem — প্রতিটির জন্য CIDR, subnet, আর ভবিষ্যৎ বৃদ্ধির ফাঁকা জায়গা লিখুন। কোনো overlap আছে কি না যাচাই করুন।
2. Terraform-এ hub VPC, দুটি spoke subnet ও একটি route লিখুন (ধারণাগত resource নাম ব্যবহার করুন), `terraform validate` ও `terraform plan` চালিয়ে দেখুন কোন CIDR কোথায় যাচ্ছে।

অধ্যায় 19 • PART IV — Multi-Cloud & Platform Engineering

Chapter 19 — Multi-Cloud Kubernetes and OpenShift

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Kubernetes কীভাবে portability দেয় আর কোথায় তা সীমিত, তা ব্যাখ্যা করতে; (২) EKS, AKS, GKE ও IBM Cloud Kubernetes Service (IKS)-এর তুলনা করতে; (৩) OpenShift-এর architecture ও Kubernetes-এর সাথে পার্থক্য বলতে; (৪) cluster federation ধারণা ব্যাখ্যা করতে; (৫) GitOps, Helm ও Operator দিয়ে multi-cluster পরিচালনা করতে; (৬) workload identity বুঝতে; এবং (৭) multi-cluster disaster recovery পরিকল্পনা করতে।

১৯.২ পূর্বশর্ত ঝালাই

আপনার জানা থাকা উচিত: Container ও Docker-এর ধারণা, Pod/Deployment/Service/Ingress, kubectl-এর মৌলিক ব্যবহার, এবং Git-এর মৌলিক ধারণা। Kubernetes মানে container orchestration — অনেক মেশিন জুড়ে container চালানো, স্বাস্থ্য দেখা, প্রয়োজনে পুনরায় চালু করা। Managed Kubernetes মানে provider control plane (API server, etcd) চালায়, আপনি শুধু worker node ও workload চালান। এই অধ্যায়ে আমরা ধরে নিচ্ছি আপনি একটি ক্লাস্টারে অ্যাপ deploy করতে পারেন, আর এখন প্রশ্ন হলো — দুটি বা তিনটি ক্লাস্টার, দুটি provider-এ, কীভাবে একসাথে চালাবেন?

১৯.৩ প্রথম নীতি থেকে: Kubernetes কেন portability দেয়, আর কেন সম্পূর্ণ দেয় না

Kubernetes-এর প্রতিশ্রুতি: অ্যাপ্লিকেশন YAML-এ লেখা থাকে — Deployment, Service, ConfigMap — আর যেকোনো Kubernetes ক্লাস্টারে একই YAML চলে। Container image registry থেকে আসে, Pod যেকোনো node-এ চলে। এটাই portability-র ভিত্তি। কিন্তু সীমাও আছে: StorageClass-এর নাম provider-ভেদে বদলায়, LoadBalancer-এর আচরণ বদলায়, IAM-এর সাথে সংযোগ (যেমন Pod-কে cloud storage-এর অনুমতি দেওয়া) provider-নির্দিষ্ট, আর Ingress controller-এর বৈশিষ্ট্য আলাদা। তাই “একই YAML সব জায়গায়” — এটি লক্ষ্য, বাস্তবে প্রতিটি provider-এর জন্য ছোট overlay (Kustomize বা Helm value) লাগে।

OpenShift হলো Red Hat-এর Kubernetes distribution — Kubernetes-এর উপর বাড়তি স্তর: বিল্ট-ইন CI/CD ধারণা, উন্নত security (Security Context Constraints), developer console, আর Operator-ভিত্তিক পরিচালনা। IBM Cloud-এ OpenShift (Red Hat OpenShift on IBM Cloud) একটি managed সেবা হিসেবে পাওয়া যায়। OpenShift-এর শক্তি: enterprise-এ প্রয়োজনীয় নিরাপত্তা ও পরিচালনার নিয়ম আগে থেকে দেওয়া। দুর্বলতা: Kubernetes-এর চেয়ে বেশি ধারণা শিখতে হয়, আর কিছু workload-এর জন্য বাড়তি স্তর অপ্রয়োজনীয় জটিলতা।

> **CONCEPT:** Cluster federation মানে একাধিক Kubernetes ক্লাস্টারকে একটি যুক্তিগত একক হিসেবে দেখা — একটি workload একাধিক ক্লাস্টারে ছড়িয়ে দেওয়া, বা একটি ক্লাস্টার বন্ধ হলে অন্যটি নেওয়া। ধারণাটি আকর্ষণীয়, বাস্তবায়ন কঠিন: federation-এর জন্য প্রতিটি ক্লাস্টারের API, নেটওয়ার্ক ও identity একত্র করতে হয়। বাস্তবে বেশিরভাগ দল federation-এর বদলে GitOps ব্যবহার করে — একই Git repository থেকে প্রতিটি ক্লাস্টারে একই workload deploy করা হয়, প্রতিটি ক্লাস্টার স্বাধীনভাবে চলে। Federation-এর চেয়ে GitOps সহজ, পরীক্ষিত ও পরিচালনাযোগ্য।

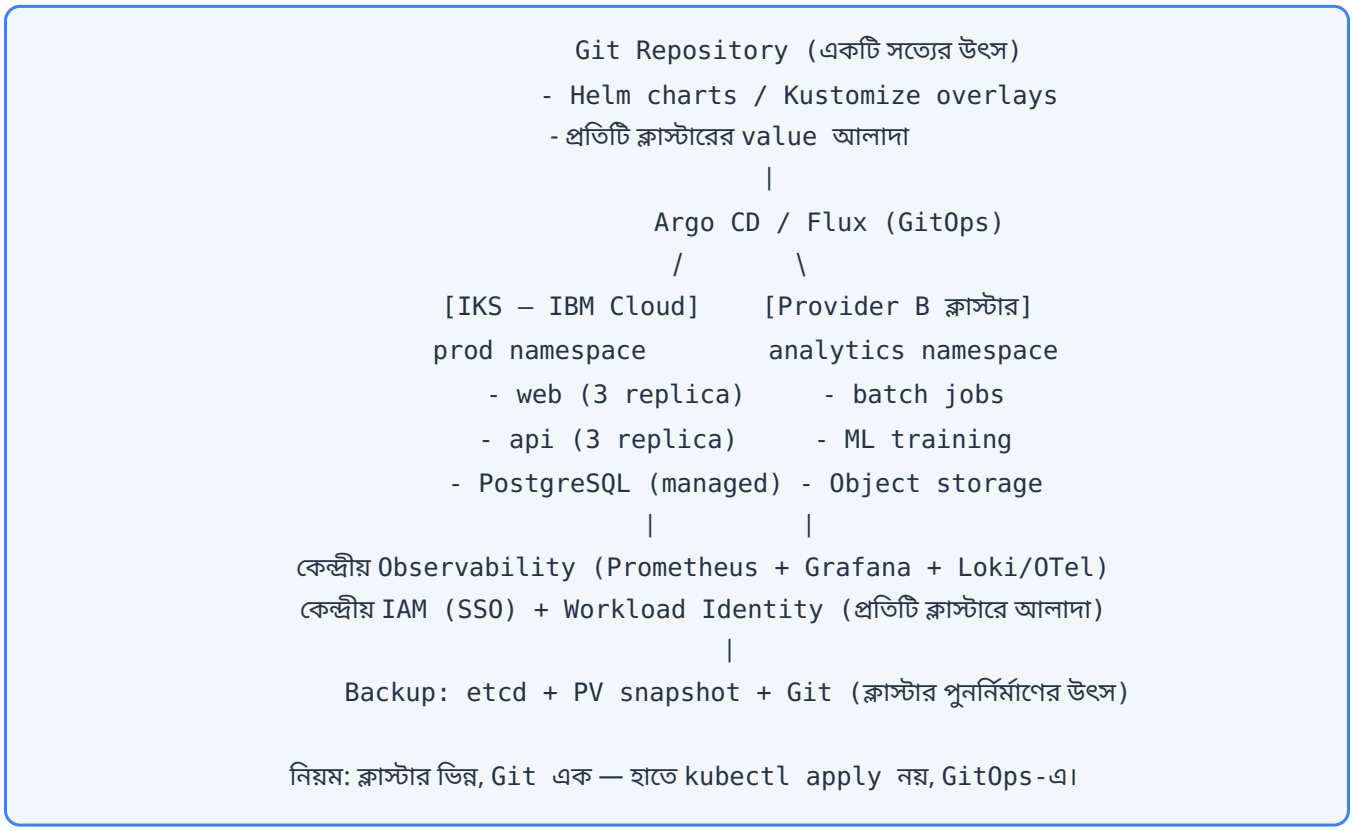
১৯.৪ Managed Kubernetes তুলনা — EKS, AKS, GKE, IKS

চারটি managed Kubernetes সেবার মূল ধারণা একই: provider control plane চালায়, আপনি node pool ও workload চালান। পার্থক্য হলো: control plane-এর পরিচালনা, node-এর ধরন, IAM-এর সাথে সংযোগ, storage ও load balancer-এর integration, আর pricing মডেল। নিচের টেবিলটি ধারণাগত তুলনা — কোনো নির্দিষ্ট মূল্য বা সীমা এখানে দেওয়া হয়নি, কারণ তা সময় ও region-ভেদে বদলায়; সিদ্ধান্তের আগে প্রতিটি provider-এর বর্তমান ডকুমেন্টেশন দেখুন।

দিক	EKS (AWS)	AKS (Azure)	GKE (Google)	IKS (IBM Cloud)
Control plane	AWS পরিচালিত	Azure পরিচালিত	Google পরিচালিত	IBM পরিচালিত
IAM সংযোগ	AWS IAM	Azure AD / Entra ID	Google IAM	IBM Cloud IAM
Node পরিচালনা	Managed node group	Node pool	Node pool / Autopilot ধারণা	Worker pool
Storage / LB	AWS-নির্দিষ্ট	Azure-নির্দিষ্ট	Google-নির্দিষ্ট	IBM-নির্দিষ্ট
OpenShift বিকল্প	ROSA ধারণা	ARO ধারণা	—	Red Hat OpenShift on IBM Cloud

তুলনার শিক্ষা: provider বদলালে YAML-এর বেশিরভাগই একই থাকে, কিন্তু IAM, storage ও load balancer-এর অংশ বদলায়। তাই multi-cloud Kubernetes-এর পরিকল্পনায় এই তিনটি অংশকে আলাদা module-এ রাখুন।

১৯.৫ আর্কিটেকচার ডায়াগ্রাম: multi-cluster TechNova



১৯.৬ অভ্যন্তরীণ কার্যপ্রণালী: Pod থেকে Service, Helm থেকে Operator

Kubernetes-এর ভিতরে: API server সব অনুরোধ নেয়, etcd অবস্থা (desired state) রাখে, scheduler Pod-কে node-এ বসায়, kubelet node-এ Pod চালায়, kube-proxy Service-এর IP-তে ট্রাফিক পাঠায়। Managed Kubernetes-এ API server ও etcd provider চালায় — আপনি তা দেখতে পান না, শুধু API ব্যবহার করেন। Helm মানে Kubernetes YAML-এর template + value — একই chart, ভিন্ন value দিয়ে prod ও staging-এ deploy করা যায়। Operator মানে Kubernetes-এর custom controller — যেমন একটি PostgreSQL Operator ডেটাবেসের lifecycle (backup, failover) নিজে পরিচালনা করে। GitOps মানে: Git-ই desired state, আর Argo CD বা Flux ক্লাস্টারকে Git-এর সাথে মিলিয়ে রাখে — হাতে বদলালে GitOps তা ফিরিয়ে দেয় (drift correction)।

Workload identity: Pod-কে cloud সেবা ব্যবহার করতে হলে তার পরিচয় লাগে — যেমন Pod-কে object storage পড়ার অনুমতি। প্রতিটি provider-এর পদ্ধতি আলাদা (IRSA, Workload Identity, IAM-এর সাথে ServiceAccount সংযোগ ধারণা)। নীতি: Pod-কে node-এর অনুমতি দেবেন না — Pod-এর নিজের, সীমিত অনুমতি দিন।

১৯.৭ CLI/API উদাহরণ

```
# ক্লাস্টার প্রসঙ্গ বদলে একই কমান্ড দুটি ক্লাস্টারে
kubectl config use-context technova-iks
kubectl get nodes
kubectl get pods -n prod

kubectl config use-context technova-provider-b
kubectl get nodes
kubectl get pods -n analytics

# Helm — একই chart, ভিন্ন value
helm upgrade --install web ./charts/web -f values-prod.yaml -n prod
helm upgrade --install web ./charts/web -f values-provider-b.yaml -n analytics

# GitOps status — Argo CD-তে sync ঠিক আছে কি না
kubectl get applications -n argocd
kubectl get pods -n argocd
```

১৯.৮ IaC উদাহরণ: Helm value ও Terraform-এ ক্লাস্টার

```
# values-prod.yaml (IBM Cloud) — storage ও ingress provider-নির্দিষ্ট
replicaCount: 3
image:
  repository: registry.example.com/technova/web
  tag: "1.4.2"
ingress:
  className: "public-iks"
  host: "app.technova.example"
storage:
  className: "ibmc-vpc-block-10iops-tier"

# values-provider-b.yaml — একই chart, ভিন্ন storage ও ingress
replicaCount: 2
ingress:
  className: "nginx-provider-b"
  host: "analytics.technova.example"
```

```
storage:
  className: "provider-b-standard"
```

```
# Terraform — IKS ক্লাস্টার (ধারণাগত; আসল field provider version-ভেদে বদলায়)
resource "ibm_container_vpc_cluster" "technova" {
  name              = "technova-prod"
  vpc_id            = ibm_is_vpc.prod.id
  flavor            = "bx2.4x16"
  worker_count      = 3
  resource_group_id = "CHANGE-ME-RESOURCE-GROUP-ID"
  zones {
    subnet_id = ibm_is_subnet.prod.id
    name      = "us-south-1"
  }
}
```

১৯.৯ প্রোডাকশন পরিস্থিতি

একটি ক্লাস্টার বন্ধ: Provider B-র ক্লাস্টার বন্ধ হলে analytics থামে, কিন্তু মূল অ্যাপ IKS-এ চলতে থাকে — কারণ workload-গুলো স্বাধীন। শিক্ষা: cross-cluster নির্ভরতা synchronous করবেন না। **Helm upgrade ব্যর্থ:** নতুন image-এ bug — GitOps-এ rollback মানে Git-এ আগের tag-এ ফেরা, তারপর Argo CD sync করে। হাতে rollback করবেন না — Git-ই সত্যের উৎস। **Node-এর resource শেষ:** Pod Pending — resource request/limit যাচাই করুন, node pool বাড়ান, বা HPA (Horizontal Pod Autoscaler) সেট করুন।

১৯.১০ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: Kubernetes-এ RBAC — কে কোন namespace-এ কী করতে পারে; Pod Security Standards; NetworkPolicy; image scanning (দুর্বলতা থাকলে deploy বন্ধ); secret-কে etcd-এ encrypt করা, আর ভালো — external secret manager ব্যবহার করা। **খরচ:** Control plane-এর খরচ, worker node-এর খরচ (চালু থাকলে, ব্যবহার না হলেও), LoadBalancer-এর খরচ, storage-এর খরচ — প্রতিটি ক্লাস্টারে আলাদা। Node-এর utilization দেখুন — ২০% ব্যবহৃত node মানে অপচয়। কখন ব্যবহার করবেন: যখন workload container-ভিত্তিক, স্কেল করতে হয়, আর portability চান। কখন এড়াবেন: যখন workload একটি ছোট, স্থির সেবা — তখন managed container বা VM যথেষ্ট হতে পারে। বিকল্প: managed container সেবা, serverless, VM-ভিত্তিক deploy।

> **WARNING:** Kubernetes-কে “একবার শিখলেই সব provider এক” ভাববেন না। IAM, storage, load balancer ও observability-র integration প্রতিটি provider-এ আলাদা, আর সেই অংশগুলোতেই বেশিরভাগ incident হয়। প্রতিটি নতুন provider-এর ক্লাস্টারে প্রথম কাজ: IAM, storage ও ingress — এই তিনটি পরীক্ষা করা।

১৯.১১ সাধারণ ভুল

- হাতে kubectl apply করা — GitOps ছাড়া, ফলে drift হয়।
- একই Helm value সব ক্লাস্টারে ব্যবহার করা — storage/ingress বদলানো হয় না।
- Pod-কে node-এর অনুমতি দেওয়া — workload identity ব্যবহার না করা।
- Resource request/limit না দেওয়া — একটি Pod সব node খেয়ে ফেলে।
- etcd ও PV backup না রাখা — ক্লাস্টার পুনর্নির্মাণের পরিকল্পনা না থাকা।

- Image-এ latest tag ব্যবহার করা — কোন version চলছে তা জানা যায় না।

১৯.১২ ট্রাবলশুটিং

TROUBLESHOOTING: Pod CrashLoopBackOff — প্রথমে `kubectl logs` ও `kubectl describe pod` দেখুন: image pull ব্যর্থ, config ভুল, নাকি অ্যাপ crash করছে? ImagePullBackOff মানে registry বা secret সমস্যা; CrashLoop মানে অ্যাপের ভিতরের সমস্যা — log-এ stack trace খুঁজুন। Pod Pending মানে scheduler জায়গা পাচ্ছে না — node capacity বা resource request দেখুন। Service-এ ট্রাফিক যাচ্ছে না — Service selector Pod label-এর সাথে মেলে কি না, আর endpoint আছে কি না যাচাই করুন।

১৯.১২ক গভীরতর আলোচনা: multi-cluster-এর দৈনন্দিন পরিচালনা, upgrade ও ব্যর্থতা

Multi-cluster পরিচালনার সবচেয়ে কঠিন অংশটি নতুন ক্লাস্টার বানানো নয় — পুরোনো ক্লাস্টারগুলোকে একই অবস্থায় রাখা। প্রতিটি provider নিয়মিত Kubernetes version-এর update দেয়, আর প্রতিটি update-এর সাথে API-র deprecation আসে — গত বছর চলা YAML এ বছর সতর্কতা দিতে পারে, পরের বছর ব্যর্থ হতে পারে। তাই upgrade-এর নিয়ম: প্রথমে staging ক্লাস্টারে, তারপর কম গুরুত্বপূর্ণ prod ক্লাস্টারে, সবশেষে প্রধান prod-এ; প্রতিটি ধাপে Helm chart-এর API version যাচাই, আর `kubectl`-এর deprecation warning পড়া। একটি ক্লাস্টারকে পিছিয়ে রাখলে (version drift) একদিন দেখা যাবে দুটি ক্লাস্টারে একই chart ভিন্নভাবে চলে — তখন debugging দ্বিগুণ কঠিন হয়। Version-এর একটি matrix রাখুন: কোন ক্লাস্টার কোন Kubernetes version-এ, কোন Helm version-এ, আর পরের upgrade কবে — এই তথ্য ছাড়া multi-cluster পরিচালনা অনুমানের উপর চলে।

Storage multi-cluster-এর দ্বিতীয় কঠিন অংশ। PersistentVolume (PV) একটি ক্লাস্টারের ভিতরে থাকে — Pod অন্য ক্লাস্টারে গেলে তার PV যায় না। তাই stateful workload-এর (যেমন ডেটাবেস) failover মানে শুধু Pod সরানো নয়, ডেটা replicate করা। Managed ডেটাবেস ব্যবহার করলে provider সেই replication করে, কিন্তু নিজে চালানো ডেটাবেস (Operator দিয়ে) হলে replication-এর দায়িত্ব আপনার। এখানেই সিদ্ধান্তটি আসে: stateful workload-কে Kubernetes-এর ভিতরে চালাবেন, নাকি provider-এর managed সেবায় রাখবেন? ছোট দলের জন্য managed সেবা প্রায়শই ভালো — কারণ ডেটাবেসের failover, backup ও patching-এর দায়িত্ব provider নেয়। Kubernetes-এ ডেটাবেস চালানো শিখতে হয়, পরীক্ষা করতে হয়, আর রাত ৩টায় তার incident সামলাতে হয়।

তৃতীয় অংশ হলো ingress ও traffic management। প্রতিটি ক্লাস্টারের নিজস্ব ingress আছে, কিন্তু ব্যবহারকারী দেখে একটি নাম — technova.example। Global load balancer বা DNS-ভিত্তিক routing সেই নামকে সঠিক ক্লাস্টারে পাঠায়, আর health check ব্যর্থ হলে ট্রাফিক সরিয়ে নেয়। কিন্তু DNS-এর TTL ও health check-এর interval-এর কারণে failover তাৎক্ষণিক নয় — মিনিট লাগতে পারে। এই বিলম্বটি RTO-তে ধরুন, আর failover-কে নিয়মিত পরীক্ষা করুন — DNS বদলানো মানে শুধু record বদলানো নয়, cache-এর মেয়াদ, resolver-এর আচরণ ও ক্লায়েন্টের retry — সব মিলিয়ে দেখতে হয়। সবশেষে, multi-cluster-এর cost: প্রতিটি ক্লাস্টারের control plane, প্রতিটি LoadBalancer, প্রতিটি storage class — সব আলাদাভাবে বিল হয়। একটি ছোট analytics ক্লাস্টারও যদি সারাক্ষণ চালু থাকে, তাহলে তার খরচ চলতেই থাকে — তাই অপ্রয়োজনীয় ক্লাস্টার বন্ধ করা, বা batch workload-এর জন্য ক্লাস্টারকে প্রয়োজনে স্কেল করা, FinOps-এর অংশ।

১৯.১৩ সারসংক্ষেপ

Kubernetes portability দেয়, কিন্তু IAM, storage ও load balancer-এর স্তরে provider-নির্দিষ্ট পার্থক্য থাকে। EKS, AKS, GKE ও IKS-এর মূল ধারণা এক, integration আলাদা। OpenShift Kubernetes-এর উপর enterprise স্তর যোগ করে। Federation-এর চেয়ে GitOps বাস্তবসম্মত — একই Git থেকে প্রতিটি ক্লাস্টার sync হয়। Helm-এ provider-নির্দিষ্ট value আলাদা রাখুন, workload identity Pod-স্তরে দিন, আর DR-এর জন্য Git + backup — দুটিই রাখুন।

১৯.১৪ পর্যালোচনা প্রশ্ন

1. Kubernetes কোন স্তরে portability দেয়, আর কোন স্তরে provider-নির্দিষ্ট পার্থক্য থাকে?
2. EKS, AKS, GKE ও IKS-এর তুলনায় কোন তিনটি দিকে সবচেয়ে বেশি পার্থক্য হয়?
3. OpenShift ও সাধারণ Kubernetes-এর মধ্যে মূল পার্থক্য কী?
4. GitOps কীভাবে cluster federation-এর চেয়ে সহজ ও নিরাপদ?
5. Workload identity কেন node-এর অনুমতির চেয়ে ভালো?

১৯.১৫ ব্যবহারিক অনুশীলন

1. একটি Helm chart-এর জন্য দুটি value ফাইল লিখুন (prod IKS ও Provider B) — replica, ingress class, storage class আলাদা করুন। `helm template` চালিয়ে দুটি output-এর পার্থক্য দেখুন।
2. একটি namespace-এ NetworkPolicy লিখুন যেখানে শুধু web Pod API Pod-এর 8080 port-এ যেতে পারে, অন্য সব Pod-এর ট্রাফিক বন্ধ। `kubectl apply --dry-run=client` দিয়ে যাচাই করুন।

অধ্যায় 20 • PART IV — Multi-Cloud & Platform Engineering

Chapter 20 — Cross-Cloud Automation

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Terraform দিয়ে multi-cloud infrastructure লিখতে; (২) Ansible দিয়ে configuration management করতে; (৩) provider-নির্দিষ্ট module ও reusable module-এর মধ্যে পার্থক্য করতে; (৪) remote state নিরাপদে রাখতে; (৫) API automation ও GitOps দিয়ে পরিবর্তন চালাতে; (৬) CI/CD-তে infrastructure পরিবর্তন পরীক্ষা করতে; (৭) drift detection করতে; এবং (৮) automation-এর নিরাপত্তা নিশ্চিত করতে।

২০.২ পূর্বশর্ত ঝালাই

আপনার জানা থাকা উচিত: Terraform-এর resource/provider/state ধারণা, YAML-এর মৌলিক গঠন, Git-এর branch ও pull request, এবং CI/CD-এর ধারণা (commit হলে পরীক্ষা ও deploy হয়)। Ansible মানে agent ছাড়া configuration management — SSH/WinRM দিয়ে সার্ভারে পৌঁছে desired state প্রয়োগ করা। এই অধ্যায়ে আমরা ধরে নিচ্ছি আপনি একটি Terraform ফাইল লিখে `apply` করতে পারেন, আর এখন প্রশ্ন হলো — একই পদ্ধতি দুটি provider-এ, দলবদ্ধভাবে, নিরাপদে কীভাবে চালাবেন?

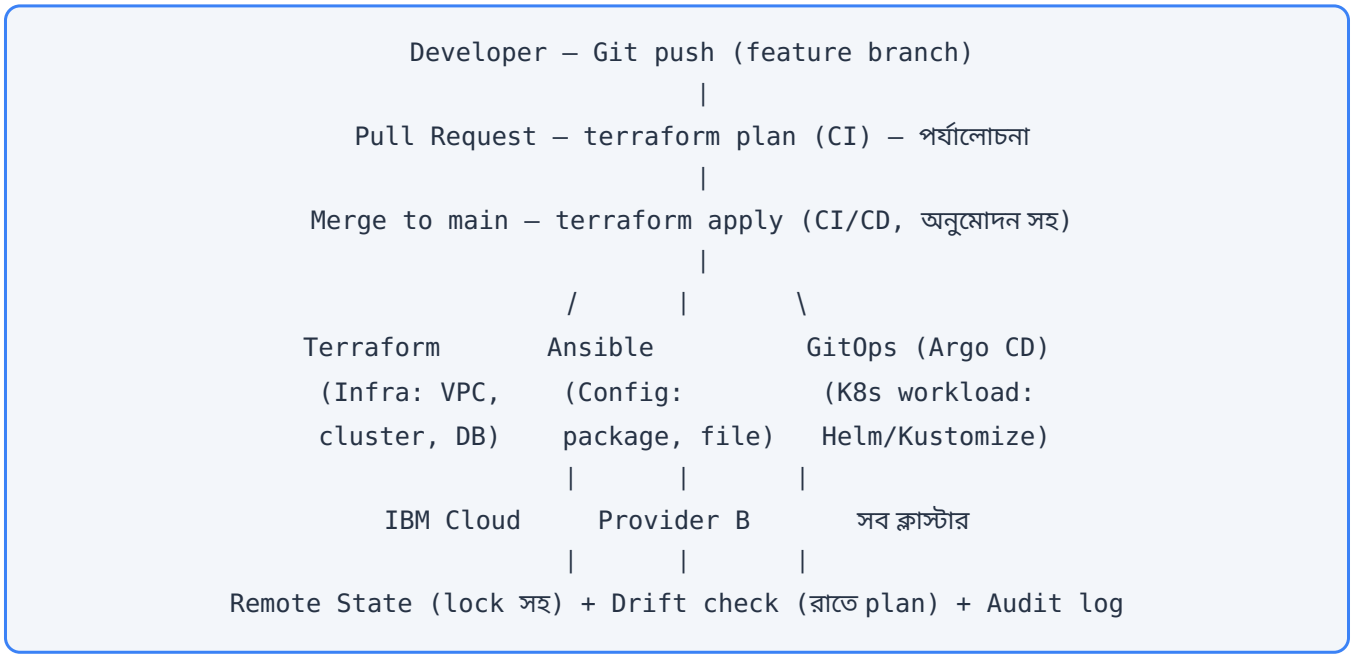
২০.৩ প্রথম নীতি থেকে: automation মানে একবার লেখা, বারবার প্রয়োগ

Automation-এর প্রথম নীতি: মানুষের হাতে করা পরিবর্তন ভুলে যায়, কোডে লেখা পরিবর্তন মনে রাখে। Infrastructure as Code (IaC) মানে infrastructure-এর বিবরণ কোডে লেখা — Git-এ version, pull request-এ পর্যালোচনা, CI-তে পরীক্ষা। Terraform declarative: আপনি বলেন “কী চাই”, Terraform বের করে “কী বদলাতে হবে”। Ansible-ও declarative ধারণায় কাজ করে, কিন্তু তার জোর configuration-এ — সার্ভারে কোন package, কোন ফাইল, কোন service চালু থাকবে।

Multi-cloud-এ automation-এর বিশেষ চ্যালেঞ্জ: প্রতিটি provider-এর API আলাদা, resource-এর নাম আলাদা, state আলাদা। সমাধান: provider-নির্দিষ্ট অংশকে module-এর ভিতরে লুকিয়ে রাখা, আর উপরের স্তরে (root module) শুধু “কী চাই” লেখা। যেমন `module "web_ibm"` ও `module "web_provider_b"` — দুটি module-এর interface একই (input: image, replica; output: URL), ভিতরের resource আলাদা।

> **CONCEPT:** Drift মানে বাস্তব infrastructure আর কোডে লেখা desired state-এর মধ্যে পার্থক্য — কেউ হাতে console-এ কিছু বদলে ফেলেছে, কিন্তু Terraform ফাইলে তা লেখা হয়নি। Drift detection মানে নিয়মিত `terraform plan` চালিয়ে দেখা — কোডের বাইরে কিছু বদলেছে কি না। Drift-কে উপেক্ষা করলে একদিন `apply` চালালে হাতে করা পরিবর্তন মুছে যাবে, বা কোডের পরিবর্তন বাস্তবে প্রয়োগ হবে না। তাই নিয়ম: console-এ হাতে পরিবর্তন নয় — পরিবর্তন মানেই pull request।

২০.৪ আর্কিটেকচার ডায়াগ্রাম: automation pipeline



এই pipeline-এর মূল কথা: কোনো পরিবর্তন সরাসরি prod-এ যায় না — plan দেখা হয়, পর্যালোচনা হয়, তারপর apply হয়। আর apply-এর পরে drift check নিশ্চিত করে বাস্তবতা কোডের সাথে মিলে আছে।

২০.৫ Terraform — reusable module ও remote state

Reusable module-এর নিয়ম: module-এর input কম রাখুন, output পরিষ্কার রাখুন, provider-নির্দিষ্ট বিবরণ ভিতরে রাখুন। একটি module-কে এক provider-এর জন্য লিখুন, অন্য provider-এর জন্য আলাদা module লিখুন — একটি module দিয়ে দুটি provider চালানোর চেষ্টা করবেন না, এতে module জটিল হয়ে যায়। Versioning করুন: module-কে Git tag দিয়ে version দিন, যাতে হঠাৎ বদলে সব environment ভেঙে না যায়।

Remote state মানে Terraform state ফাইলটি দূরে, সুরক্ষিত জায়গায় রাখা — যেমন IBM Cloud Object Storage বা অন্য provider-এর object storage, encryption ও versioning সহ। State-এ sensitive তথ্য থাকতে পারে, তাই state-কে কখনো Git-এ রাখবেন না, public করবেন না, আর state lock ব্যবহার করুন যাতে দুজন একসাথে apply না করে। প্রতিটি environment (prod/staging) ও প্রতিটি provider-এর জন্য আলাদা state ফাইল রাখুন — একটি state-এ সব রাখলে ভুলের প্রভাব বড় হয়।

২০.৬ Ansible — configuration management

Ansible-এর মূল ধারণা: playbook (YAML) — কোন host-এ, কোন task, কোন ক্রমে। Inventory — কোন সার্ভার কোন গ্রুপে। Role — পুনঃব্যবহারযোগ্য task-এর সংগ্রহ। Terraform infrastructure বানায়, Ansible তার ভিতরের configuration করে — যেমন VM-এ package install, ফাইল লেখা, service চালু করা। Multi-cloud-এ Ansible-এর সুবিধা: provider-নিরপেক্ষ — SSH দিয়ে যেকোনো VM-এ পৌঁছায়। সতর্কতা: Ansible-এর secret (password, API key) Ansible Vault-এ encrypt করে রাখুন, plain YAML-এ নয়।

২০.৭ CLI/API উদাহরণ

```
# Terraform — plan, apply, drift যাচাই
terraform init
terraform fmt -check
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
# Drift যাচাই — কোডের বাইরে কিছু বদলেছে কি না
terraform plan -detailed-exitcode

# Ansible — playbook যাচাই ও চালানো
ansible-playbook --check --diff site.yaml
ansible-playbook site.yaml --limit web
ansible-inventory --graph

# API automation — provider API-তে curl (token environment থেকে)
curl -s -H "Authorization: Bearer $CHANGE_ME_TOKEN" \
  https://api.example.com/v1/resources | head -c 500
```

২০.৮ IaC উদাহরণ: Terraform module ও Ansible playbook

```
# modules/web-ibm/main.tf — provider-নির্দিষ্ট বিবরণ ভিতরে
variable "image" {}
variable "replicas" { default = 2 }

# (আসল resource এখানে থাকবে — নাম provider version-ভেদে বদলায়)
# output "url" — root module শুধু এই URL ব্যবহার করে
output "url" {
  value = "https://app.technova.example"
}

# root main.tf — দুটি provider, দুটি module, একই interface
module "web_ibm" {
  source   = "./modules/web-ibm"
  image    = "registry.example.com/technova/web:1.4.2"
  replicas = 3
}
```

```
# ansible/site.yaml — web সার্ভারে Nginx ও অ্যাপ user
- name: Configure web servers
  hosts: web
  become: true
  vars_files:
    - vault.yaml  # secret এখানে, encrypt করা
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
    - name: Ensure nginx is running
      ansible.builtin.service:
        name: nginx
```

```
        state: started
        enabled: true
```

২০.৯ প্রোডাকশন পরিস্থিতি

CI-তে plan ব্যর্থ: Terraform plan-এ দেখা গেল একটি database মুছে যাবে — কারণ কেউ resource-এর নাম বদলে ফেলেছে। সমাধান: plan-কে pull request-এ দেখা বাধ্যতামূলক, আর destructive পরিবর্তনে আলাদা অনুমোদন। **Ansible-এ ভুল host:** playbook ভুল inventory-তে চলল — staging-এর বদলে prod-এ। সমাধান: inventory আলাদা ফাইলে, prod-এ apply-এর আগে `--check` ও অনুমোদন। **State lock:** দুজন একসাথে apply করায় lock — একজন অপেক্ষা করুন, lock জোর করে ভাঙবেন না (force-unlock শুধু তখনই যখন নিশ্চিত যে কেউ apply করছে না)।

> **REAL WORLD:** TechNova-র একবার একটি Terraform apply-তে ভুল করে staging-এর state দিয়ে prod-এর resource মুছে যাওয়ার উপক্রম হয়েছিল — কারণ দুটি environment-এর state ফাইলের নাম প্রায় একই ছিল। তারপর থেকে নিয়ম: state ফাইলের নামে environment ও provider স্পষ্টভাবে লেখা (যেমন `prod-ibm.tfstate`), আর CI-তে environment variable দিয়ে state নির্বাচন করা — হাতে টাইপ করে নয়।

২০.১০ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: Terraform ও Ansible-এর credential (API key, SSH key) কখনো কোডে লিখবেন না — CI-এর secret store বা Vault থেকে নিন, আর প্রতিটি apply-তে ন্যূনতম অনুমতির service account ব্যবহার করুন। State ফাইল encrypt করুন, access log রাখুন। `terraform plan`-এর output-এ secret দেখা গেলে তা redact করুন। **খরচ:** Automation-এর খরচ মূলত মানুষের সময় — module লেখা, pipeline বজায় রাখা। কিন্তু automation ছাড়া multi-cloud-এর খরচ আরও বেশি — হাতে করা ভুল, drift, আর দুটি provider-এর console-এ ঘোরাফেরা। যখন ব্যবহার করবেন: যখন infrastructure পুনরাবৃত্তি হয়, দলে একাধিক জন কাজ করে, বা audit trail লাগে। যখন এড়াবেন: যখন একটি একবারের পরীক্ষা — তখন হাতে console-এ করাই দ্রুত। বিকল্প: Pulumi (কোড-ভিত্তিক IaC), Crossplane (Kubernetes-ভিত্তিক), provider-এর নিজস্ব IaC টুল। Trade-off: Terraform provider-নিরপেক্ষ, কিন্তু প্রতিটি provider-এর নতুন সেবা Terraform-এ আসতে সময় লাগতে পারে।

> **WARNING:** Automation-কে “একবার লিখলেই শেষ” ভাববেন না। Provider API বদলায়, Terraform provider version বদলায়, module-এর পুরোনো version-এ দুর্বলতা থাকতে পারে। নিয়মিত provider update করুন, `terraform plan`-এ deprecation warning দেখুন, আর module-এর version pin করুন — `latest` ব্যবহার করবেন না।

২০.১১ সাধারণ ভুল

- State ফাইল Git-এ রাখা বা local laptop-এ রাখা।
- Secret Terraform variable বা Ansible YAML-এ plain লেখা।
- Module-এ provider-নির্দিষ্ট বিবরণ বাইরে ছড়িয়ে দেওয়া।
- Plan না দেখে apply করা।
- Drift check না রাখা — হাতে console-এ বদলানো।
- Prod ও staging-এর state গুলিয়ে ফেলা।

২০.১২ ট্রাবলশুটিং

TROUBLESHOOTING: `terraform apply` আটকে গেছে বা ব্যর্থ হচ্ছে — প্রথমে state lock আছে কি না দেখুন, তারপর plan-এর output পড়ুন: কোন resource-এ সমস্যা, provider API কী error দিচ্ছে? API rate limit হলে অপেক্ষা করুন, credential মেয়াদোত্তীর্ণ হলে renew করুন। State-এর সাথে বাস্তবতা না মিললে `terraform refresh` বা `import` বিবেচনা করুন — কিন্তু state ফাইল হাতে সম্পাদনা করবেন না, backup নিয়ে করুন। Ansible-এ task ব্যর্থ হলে `-vvv` দিয়ে verbose output দেখুন, আর কোন host-এ ব্যর্থ হলো তা চিহ্নিত করুন।

২০.১২ক গভীরতর আলোচনা: pipeline-এর নিরাপত্তা, পরীক্ষা ও জরুরি পরিবর্তন

Automation pipeline নিজেই একটি আক্রমণের লক্ষ্য — কারণ pipeline-এর credential দিয়ে পুরো infrastructure বদলানো যায়। তাই pipeline-এর নিরাপত্তা infrastructure-এর নিরাপত্তার মতোই গুরুত্বপূর্ণ। নিয়মগুলো: CI-এর secret-কে শুধু প্রয়োজনীয় job-এ দিন, প্রতিটি job-কে ন্যূনতম অনুমতি দিন, আর prod-এ apply করার জন্য মানুষের অনুমোদন বাধ্যতামূলক করুন — এমনকি pipeline স্বয়ংক্রিয় হলেও। Pull request-এ `terraform plan`-এর output দেখা বাধ্যতামূলক করুন, আর plan-এ যদি কোনো resource মুছে যাওয়ার কথা থাকে, তাহলে আলাদা সতর্কতা দিন। একটি সাধারণ কিন্তু কার্যকর নিয়ম: prod-এর Terraform apply শুধু main branch থেকে, শুধু অনুমোদিত reviewer-এর অনুমোদনের পরে, আর apply-এর log সংরক্ষণ করা।

পরীক্ষার স্তরগুলো আলাদা করুন: `terraform validate` ও `fmt` — সিনট্যাক্স; `plan` — কী বদলাবে; policy check (OPA/Conftest) — নিয়ম মানা হচ্ছে কি না; আর staging-এ apply — বাস্তবে কাজ করছে কি না। Staging-কে prod-এর ছোট প্রতিরূপ হিসেবে রাখুন — staging যদি prod-এর চেয়ে অনেক ছোট হয়, তাহলে staging-এ পাস করা মানে prod-এ পাস করা নয়। জরুরি পরিবর্তনের (break-glass) পদ্ধতিও আগে থেকে লিখে রাখুন: incident-এর সময় কে, কীভাবে, কোন অনুমোদনে হাতে পরিবর্তন করতে পারে, আর তারপর কতক্ষণের মধ্যে সেই পরিবর্তনকে Git-এ ফিরিয়ে আনতে হবে। Break-glass ছাড়া incident-এর সময় দল আটকে যায়, আর break-glass-এর পরে Git-এ না ফেরালে drift স্থায়ী হয়ে যায় — দুটি ঝুঁকিই একসাথে সামলাতে হয়।

Module-এর জীবনচক্রও পরিকল্পনা করুন: module-এর version pin করুন, নতুন version-কে প্রথমে একটি environment-এ পরীক্ষা করুন, তারপর সব জায়গায় ছড়ান। একটি module-এর breaking change হলে তার major version বাড়ান, আর পুরোনো version ব্যবহার করা environment-গুলোকে ধাপে ধাপে সরান। Ansible-এর ক্ষেত্রেও একই কথা: role-এর version, inventory-র পর্যালোচনা, আর `--check` মোডে প্রথমে দেখা — কী বদলাবে। সবশেষে, automation-এর ব্যর্থতার জন্য প্রস্তুতি: state ফাইলের backup, pipeline-এর বিকল্প পথ (CI বন্ধ হলে হাতে apply করার অনুমোদিত পদ্ধতি), আর প্রতিটি automation-এর runbook। Automation যখন কাজ করে, তখন সবাই খুশি; যখন কাজ করে না, তখন runbook-ই ভরসা।

২০.১৩ সারসংক্ষেপ

Cross-cloud automation মানে: Terraform-এ infrastructure, Ansible-এ configuration, GitOps-এ workload — তিনটি স্তর, একটি Git সত্যের উৎস। Reusable module-এ provider-নির্দিষ্ট বিবরণ লুকান, remote state-কে encrypt ও lock করুন, drift নিয়মিত যাচাই করুন, আর কোনো পরিবর্তন plan ও পর্যালোচনা ছাড়া prod-এ যাবে না। Automation-এর নিরাপত্তা হলো credential-এর নিরাপত্তা — কোডে secret নয়, Vault ও ন্যূনতম অনুমতি।

২০.১৪ পর্যালোচনা প্রশ্ন

1. Terraform ও Ansible-এর মধ্যে দায়িত্বের ভাগ কী? কখন কোনটি ব্যবহার করবেন?
2. Remote state কেন Git-এ রাখা যায় না, এবং state lock কেন জরুরি?
3. Drift কী, এবং তা কীভাবে ধরা যায়?
4. Reusable module লেখার তিনটি নিয়ম কী?
5. Automation-এর credential কীভাবে নিরাপদে রাখবেন?

২০.১৫ ব্যবহারিক অনুশীলন

1. একটি Terraform module লিখুন যার input শুধু `image` ও `replicas`, output শুধু `url` — ভিতরে provider-নির্দিষ্ট resource ধারণাগতভাবে লিখুন। Root module থেকে দুটি ভিন্ন value দিয়ে call করুন।
2. একটি Ansible playbook লিখুন যা একটি VM-এ একটি package install করে ও একটি service চালু করে, secret-কে Vault ফাইলে রাখার ধারণা দেখিয়ে। `ansible-playbook --check --diff` চালিয়ে যাচাই করুন।

PART V • DEVOPS ENGINEERING MASTERY

Enterprise Multi-Cloud Operations

Multi-Cloud Observability, Security, Disaster Recovery, Reliability ↻ FinOps, Enterprise Multi-Cloud Architecture ↻ Capstone I

অধ্যায় 21 • PART V — Enterprise Multi-Cloud Operations

Chapter 21 — Multi-Cloud Observability

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) metrics, logs ও traces-এর মধ্যে পার্থক্য ও সম্পর্ক ব্যাখ্যা করতে; (২) কেন্দ্রীয় monitoring নকশা করতে; (৩) OpenTelemetry দিয়ে provider-নিরপেক্ষ instrumentation করতে; (৪) Prometheus ও Grafana দিয়ে metrics সংগ্রহ ও দেখাতে; (৫) log aggregation ও distributed tracing সাজাতে; (৬) correlation দিয়ে একটি request-কে সব provider জুড়ে অনুসরণ করতে; এবং (৭) alerting ও incident investigation-এর পদ্ধতি ঠিক করতে।

২১.২ পূর্বশর্ত ঝালাই

আপনার জানা থাকা উচিত: HTTP status code (200, 500), latency ও error rate-এর ধারণা, Kubernetes Pod ও Service, এবং log-এর ধারণা (একটি ঘটনার লাইন)। Observability মানে সিস্টেমের ভিতরে কী ঘটছে তা বাইরে থেকে বোঝা — শুধু “চালু আছে কি না” নয়, “কেন ধীর, কোথায় আটকাচ্ছে” তা বোঝা। এই অধ্যায়ে আমরা ধরে নিচ্ছি আপনি একটি অ্যাপের log পড়তে পারেন, আর এখন প্রশ্ন হলো — দুটি provider-এর শত শত সার্ভিসের মধ্যে একটি ধীর request-কে কীভাবে খুঁজবেন?

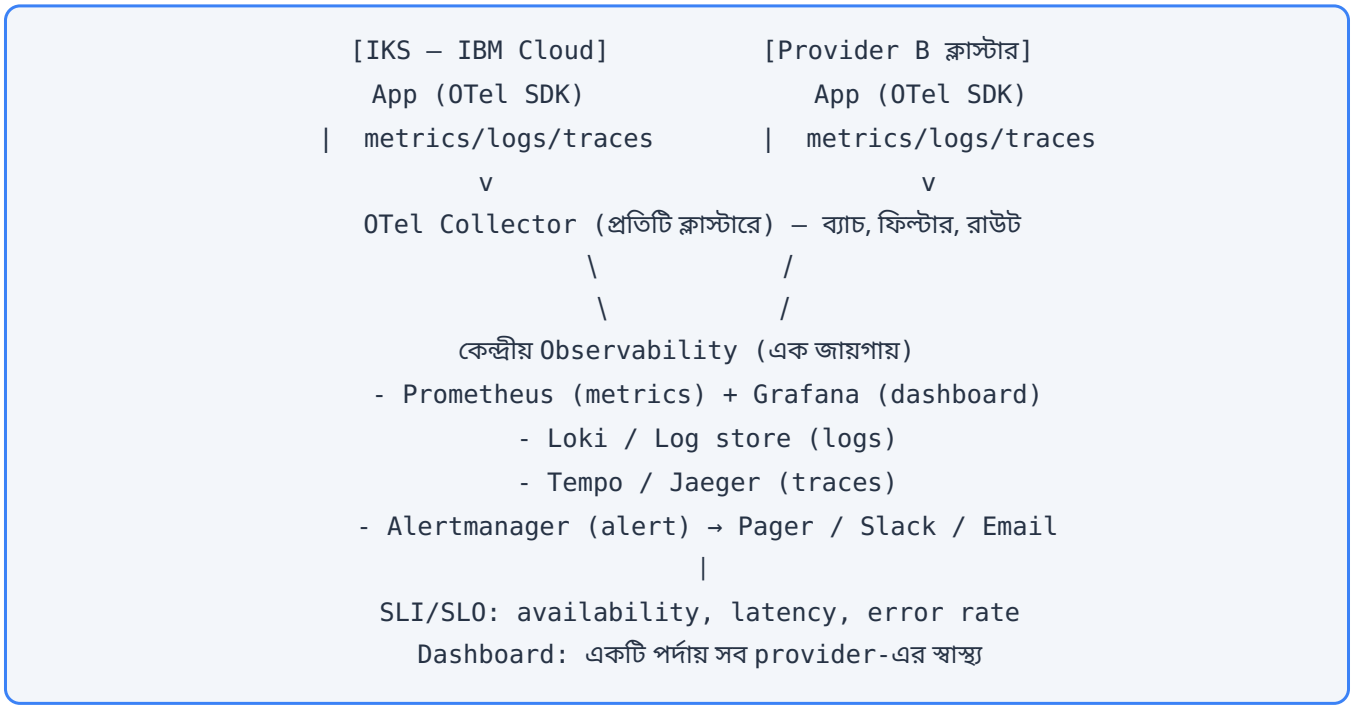
২১.৩ প্রথম নীতি থেকে: তিনটি সংকেত — metrics, logs, traces

Observability-র তিনটি স্তম্ভ: **Metrics** — সংখ্যা, সময়ের সাথে (যেমন প্রতি মিনিটে request সংখ্যা, গড় latency, error rate)। Metrics বলে “কী” ঘটছে, কিন্তু “কেন” বলে না। **Logs** — ঘটনার বিবরণ, লাইন ধরে (যেমন “2026-11-09 10:00:01 user 123 login failed”)। Logs বলে “কী ঘটেছিল” বিস্তারিতভাবে। **Traces** — একটি request-এর যাত্রা, service থেকে service-এ (যেমন web → API → database, প্রতিটি ধাপে কত সময়)। Traces বলে “কোথায় সময় গেল”। তিনটি একসাথে লাগে: metrics-এ সমস্যা দেখা যায়, trace-এ কোথায় তা খুঁজে পাওয়া যায়, log-এ কেন তা বোঝা যায়।

Multi-cloud-এ সমস্যা: প্রতিটি provider-এর নিজস্ব monitoring আছে, কিন্তু সেগুলো একে অপরকে দেখে না। Provider A-র dashboard-এ Provider B-র সমস্যা দেখা যায় না। তাই কেন্দ্রীয় observability লাগে — সব provider-এর metrics, logs ও traces একটি জায়গায় আসবে, একই সময়রেখায়, একই trace ID দিয়ে যুক্ত হবে।

> **CONCEPT:** OpenTelemetry (OTel) মানে instrumentation-এর খোলা মান — অ্যাপ্লিকেশনে একবার OTel SDK বসালে metrics, logs ও traces একই ফরম্যাটে বের হয়, আর backend (যেমন Grafana, Jaeger, বা provider-এর সেবা) বদলালেও অ্যাপ বদলাতে হয় না। OTel-এর মূল ধারণা: trace ID — একটি request-এর জন্য একটি অনন্য ID, যা প্রতিটি service-এ ছড়িয়ে যায় (propagation)। এই ID ছাড়া cross-cloud correlation অসম্ভব — আপনি জানবেন না Provider A-র log আর Provider B-র log একই request-এর কি না।

২১.৪ আর্কিটেকচার ডায়াগ্রাম: কেন্দ্রীয় observability



OTel Collector-এর ভূমিকা লক্ষ্য করুন: অ্যাপ সরাসরি কেন্দ্রীয় সার্ভারে পাঠায় না — Collector ব্যাচ করে, অপ্রয়োজনীয় ডেটা ফিল্টার করে, আর কেন্দ্রীয় সার্ভার বন্ধ থাকলেও সাময়িকভাবে ধরে রাখে। এতে অ্যাপের উপর চাপ কমে, আর খরচও কমে।

২১.৫ Prometheus ও Grafana — ভিতরে কীভাবে কাজ করে

Prometheus pull মডেলে কাজ করে: নির্দিষ্ট interval-এ (যেমন ১৫ সেকেন্ড) target-এর `/metrics` endpoint থেকে metrics টেনে নেয়, time-series ডেটাবেসে রাখে। PromQL দিয়ে জিজ্ঞাসা করা যায় — যেমন "গত ৫ মিনিটে error rate কত?" Grafana সেই জিজ্ঞাসার উত্তর dashboard-এ দেখায় — গ্রাফ, টেবিল, alert। Prometheus-এর সীমা: এটি metrics-এর জন্য, log-এর জন্য নয়; আর দীর্ঘমেয়াদি retention-এর জন্য remote storage লাগতে পারে। Multi-cloud-এ প্রতিটি ক্লাস্টারে Prometheus থাকতে পারে, আর Grafana কেন্দ্রীয়ভাবে সব Prometheus-কে জিজ্ঞাসা করে — এটিকে federation বা remote read ধারণা বলে।

Log aggregation: প্রতিটি Pod-এর log-কে একটি agent (যেমন Fluent Bit বা Promtail) সংগ্রহ করে কেন্দ্রীয় log store-এ পাঠায়। Log-এ অবশ্যই trace ID, service নাম, environment ও provider নাম থাকতে হবে — না হলে খুঁজে পাওয়া যাবে না। Distributed tracing: প্রতিটি service OTel-এর মাধ্যমে span তৈরি করে — একটি span মানে একটি কাজের অংশ (যেমন "database query, ১২০ মিলিসেকেন্ড")। সব span একই trace ID-তে যুক্ত হয়ে একটি trace হয়।

২১.৬ CLI/API উদাহরণ

```
# Prometheus-এ জিজ্ঞাসা (PromQL — ধারণা)
# গত ৫ মিনিটে error rate
# rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m])

# kubectl দিয়ে log দেখা — trace ID দিয়ে খোঁজা
kubectl logs -n prod deployment/web --tail=100 | grep "trace_id=abc123"
kubectl logs -n prod deployment/api --tail=100 | grep "trace_id=abc123"
```

```
# Grafana / Prometheus স্বাস্থ্য
kubectl get pods -n observability
curl -s http://localhost:9090/-/healthy

# OTel Collector config যাচাই
kubectl get configmap otel-collector -n observability -o yaml | head -n 60
```

২১.৭ IaC উদাহরণ: Prometheus ও alert নিয়ম

```
# Prometheus alert নিয়ম (YAML — ধারণা)
groups:
  - name: technova-web
    rules:
      - alert: HighErrorRate
        expr: |
          rate(http_requests_total{status=~"5.."}[5m])
          / rate(http_requests_total[5m]) > 0.05
        for: 5m
        labels:
          severity: page
        annotations:
          summary: "Error rate 5% ছাড়িয়েছে ({{ $labels.service }})"
          runbook: "https://runbooks.technova.example/high-error-rate"

      - alert: HighLatency
        expr: |
          histogram_quantile(0.95,
            rate(http_request_duration_seconds_bucket[5m])) > 0.5
        for: 10m
        labels:
          severity: ticket
        annotations:
          summary: "p95 latency 500ms ছাড়িয়েছে"
```

Alert-এর নিয়ম: শুধু তখনই alert করবেন যখন মানুষের কাজ লাগে। “CPU ৮০%” alert নয় — “error rate ৫%” alert। Alert-এর সাথে runbook link দিন — রাত ৩টায় কেউ যেন জানে প্রথম পদক্ষেপ কী।

২১.৮ প্রোডাকশন পরিস্থিতি ও incident investigation

পরিস্থিতি — হঠাৎ latency: Grafana-তে দেখা গেল p95 latency ২০০ মিলিসেকেন্ড থেকে ২ সেকেন্ডে উঠেছে। Trace-এ দেখা গেল API → Provider B-র analytics call-এ সময় যাচ্ছে। Log-এ দেখা গেল Provider B-র API timeout হচ্ছে। কারণ: cross-cloud link ধীর। সমাধান: timeout কমিয়ে circuit breaker, আর analytics-কে asynchronous করা। **Incident investigation-এর ক্রম:** (১) metrics-এ কখন শুরু হলো; (২) trace-এ কোন service-এ সময় গেল; (৩) log-এ সেই সময়ে কী error হলো; (৪) পরিবর্তন কী হয়েছিল (deploy, config) — GitOps history দেখুন; (৫) provider status দেখুন। এই ক্রমটি লিখে রাখুন — incident-এর সময় অনুমান নয়, ক্রম অনুসরণ করুন।

২১.৯ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: Log-এ password, token বা ব্যক্তিগত ডেটা (PII) যাবে না — OTel-এ sensitive field redact করুন। Observability সার্ভারের access সীমিত করুন — log-এ অনেক sensitive তথ্য থাকে। **খরচ:** Observability-র খরচ

তিনটি: সংগ্রহ (agent ও collector), সংরক্ষণ (metrics/log/traces retention), আর query। সব log চিরকাল রাখবেন না — error log বেশিদিন, debug log কমদিন। High-cardinality label (যেমন প্রতিটি user ID-কে label করা) Prometheus-এর খরচ ও কর্মক্ষমতা ডুবিয়ে দেয় — label কম রাখুন। কখন ব্যবহার করবেন: যখন একাধিক service বা provider আছে, আর "কেন ধীর" জানতে হয়। কখন এড়াবেন: যখন একটি ছোট, একক সেবা — তখন provider-এর বিল্ট-ইন monitoring যথেষ্ট। বিকল্প: provider-এর managed observability, বা SaaS observability সেবা। Trade-off: নিজে চালালে নিয়ন্ত্রণ বেশি, খরচ ও পরিচর্যা বেশি; SaaS নিলে পরিচর্যা কম, কিন্তু ডেটা বাইরে যায় ও খরচ ব্যবহারের উপর বাড়ে।

> **WARNING:** Observability ছাড়া multi-cloud চালানো মানে অন্ধকারে গাড়ি চালানো — কিন্তু অতিরিক্ত observability-ও বিপদ: প্রতিটি request-এর সব log রাখলে খরচ আকাশ ছোঁবে, আর log-এ sensitive ডেটা গেলে নিরাপত্তা ঝুঁকি। Sampling করুন (যেমন সব error trace রাখুন, সফল request-এর ১% রাখুন), retention নীতি লিখুন, আর নিয়মিত খরচ দেখুন।

২১.১০ সাধারণ ভুল

- শুধু metrics রাখা, trace ও log না রাখা — "কেন" জানা যায় না।
- Trace ID ছাড়া log লেখা — cross-cloud correlation অসম্ভব।
- প্রতিটি provider-এর আলাদা dashboard — কেন্দ্রীয় ছবি নেই।
- অতিরিক্ত alert — alert fatigue-এ আসল alert উপেক্ষিত হয়।
- Log-এ secret বা PII লেখা।
- High-cardinality label দিয়ে Prometheus ভারী করা।

২১.১১ ট্রাবলশুটিং

> **TROUBLESHOOTING:** "Grafana-তে ডেটা দেখা যাচ্ছে না" — ক্রম: (১) Prometheus target up কি না (`/targets` দেখুন); (২) OTel Collector চলছে কি না, তার log-এ error আছে কি না; (৩) অ্যাপ `/metrics` দিচ্ছে কি না (curl করে দেখুন); (৪) নেটওয়ার্ক — Collector থেকে কেন্দ্রীয় সার্ভারে পৌঁছাচ্ছে কি না; (৫) সময় — সব সার্ভারের ঘড়ি NTP-তে মিলছে কি না (সময় না মিললে trace ভেঙে যায়)।

২১.১১ক গভীরতর আলোচনা: SLI/SLO, cardinality ও incident-এর সংস্কৃতি

SLI (Service Level Indicator) মানে পরিমাপ — যেমন "সফল request-এর অনুপাত" বা "p95 latency"। SLO (Service Level Objective) মানে লক্ষ্য — যেমন "৩০ দিনে ৯৯.৯% request সফল হবে"। Error budget মানে SLO থেকে কতটুকু বিচ্যুতি সহ্য করা যায় — এই budget শেষ হয়ে গেলে নতুন feature-এর বদলে reliability-র কাজ অগ্রাধিকার পায়। Multi-cloud-এ SLI-কে provider-ভিত্তিক ভাগ করবেন না — ব্যবহারকারীর দৃষ্টিকোণ থেকে SLI লিখুন: "ব্যবহারকারী technova.example-এ লগইন করতে পারছে কি না" — provider A বা B-র ভিতরের সংখ্যা নয়। প্রতিটি SLI-র জন্য একটি dashboard, একটি alert, আর একটি runbook থাকতে হবে — SLI ছাড়া alert মানে শুধু শব্দ, আর alert ছাড়া SLI মানে শুধু গ্রাফ।

Cardinality-র সমস্যাটি বোঝা জরুরি: Prometheus-এ প্রতিটি label-এর প্রতিটি মান একটি নতুন time series তৈরি করে। User ID বা request ID-কে label করলে series-এর সংখ্যা লাখে পৌঁছে যায় — Prometheus ধীর হয়ে যায়, খরচ বাড়ে,

আর query ব্যর্থ হয়। নিয়ম: label-এ শুধু সীমিত মানের জিনিস রাখুন — service, environment, status code, provider — আর উচ্চ-কার্ডিনালিটি জিনিস (user ID, trace ID) log বা trace-এ রাখুন, metric-এ নয়। Log-এর ক্ষেত্রেও একই শৃঙ্খলা: structured log (JSON) লিখুন, প্রতিটি লাইনে timestamp, level, service, trace ID, আর বার্তা — যাতে খোঁজা যায়। Unstructured log-কে খুঁজে পাওয়া কঠিন, আর incident-এর সময় খোঁজার সময়ই সবচেয়ে দামি।

Incident-এর সংস্কৃতি observability-র মতোই গুরুত্বপূর্ণ: blameless post-mortem (দোষ নয়, কারণ খোঁজা), runbook-এর নিয়মিত হালনাগাদ, আর alert-এর নিয়মিত পর্যালোচনা (কোন alert কাজে এসেছে, কোনটি শুধু শব্দ করেছে)। প্রতিটি incident-এর পরে একটি প্রশ্ন করুন: “পরের বার এই সমস্যাটি কি dashboard-এ আগেই দেখা যেত?” যদি উত্তর হ্যাঁ হয়, তাহলে সেই dashboard বা alert বানান — না হলে একই incident আবার হবে। Observability-র বিনিয়োগের ফল মাপা হয় incident-এর সময় কমে কি না, তা দিয়ে — dashboard-এর সংখ্যা দিয়ে নয়।

২১.১২ সারসংক্ষেপ

Observability মানে metrics, logs ও traces — তিনটি একসাথে, একই trace ID দিয়ে যুক্ত, কেন্দ্রীয় জায়গায়। OpenTelemetry instrumentation-কে provider-নিরপেক্ষ করে, Prometheus metrics সংগ্রহ করে, Grafana দেখায়, আর alert তখনই যখন মানুষের কাজ লাগে। Multi-cloud-এ কেন্দ্রীয় observability ছাড়া incident investigation অর্ধেক অন্ধকারে হয়। Sampling ও retention দিয়ে খরচ নিয়ন্ত্রণ করুন, log-এ secret রাখবেন না।

২১.১৩ পর্যালোচনা প্রশ্ন

1. Metrics, logs ও traces-এর মধ্যে পার্থক্য কী, এবং প্রতিটি কখন কাজে আসে?
2. Trace ID কেন cross-cloud correlation-এর জন্য জরুরি?
3. Prometheus pull মডেল কীভাবে কাজ করে, আর এর সীমা কী?
4. Alert fatigue কী, এবং তা কীভাবে এড়াবেন?
5. Observability-র খরচ কীভাবে নিয়ন্ত্রণ করবেন?

২১.১৪ ব্যবহারিক অনুশীলন

1. একটি Prometheus alert নিয়ম লিখুন: error rate ৫% ছাড়ালে ৫ মিনিট পরে alert, সাথে runbook link। `promtool check rules` দিয়ে যাচাই করুন (যদি উপলব্ধ থাকে)।
2. একটি কাল্পনিক incident-এর timeline লিখুন: Grafana-তে latency বেড়েছে → trace-এ কোন service → log-এ কী error → কারণ ও সমাধান। প্রতিটি ধাপে কোন টুল ব্যবহার করবেন তা লিখুন।

অধ্যায় 22 • PART V — Enterprise Multi-Cloud Operations

Chapter 22 — Multi-Cloud Security

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) identity federation দিয়ে একাধিক provider-এর IAM একত্র করতে; (২) IAM-এর provider-ভেদে পার্থক্য ব্যাখ্যা করতে; (৩) workload identity দিতে; (৪) Zero Trust ধারণা প্রয়োগ করতে; (৫) secrets management ও KMS দিয়ে encryption সাজাতে; (৬) নেটওয়ার্ক, container ও policy governance সাজাতে; এবং (৭) security auditing ও incident response-এর পরিকল্পনা করতে।

২২.২ পূর্বশর্ত ঝালাই

আপনার জানা থাকা উচিত: IAM-এর ধারণা (user, role, policy), TLS, encryption-এর ধারণা (at rest ও in transit), Kubernetes RBAC ও NetworkPolicy, এবং secret মানে password/API key/token। এই অধ্যায়ে আমরা ধরে নিচ্ছি আপনি একটি provider-এ IAM role দিতে পারেন, আর এখন প্রশ্ন হলো — দুটি provider-এ, একজন মানুষ ও একটি Pod, কীভাবে নিরাপদে পরিচয় প্রমাণ করবে?

২২.৩ প্রথম নীতি থেকে: পরিচয়, অনুমতি, আর বিশ্বাসের সীমানা

নিরাপত্তার প্রথম নীতি: কেউ কে, তা প্রমাণ করতে হবে (authentication), তারপর সে কী করতে পারে তা ঠিক করতে হবে (authorization), আর সবকিছু লিখে রাখতে হবে (auditing)। Multi-cloud-এ সমস্যা: প্রতিটি provider-এর নিজস্ব identity আছে — IBM Cloud IAM, AWS IAM, Azure Entra ID — আর সেগুলো একে অপরকে চেনে না। সমাধান: **identity federation** — একটি কেন্দ্রীয় identity provider (IdP, যেমন কোম্পানির SSO) থেকে token নিয়ে প্রতিটি cloud-এ প্রবেশ করা। ব্যবহারকারী একবার লগইন করে, তারপর federation-এর মাধ্যমে প্রতিটি provider-এ সীমিত, সময়-সীমাবদ্ধ অনুমতি পায়।

Zero Trust মানে: “ভিতরে থাকলেই বিশ্বাসযোগ্য” — এই ধারণা বাতিল। প্রতিটি request — ভিতর থেকে হোক বা বাইরে থেকে — পরিচয়, ডিভাইস ও প্রসঙ্গ যাচাই করে অনুমতি পায়। নেটওয়ার্কের অবস্থান (VPC-র ভিতরে) আর বিশ্বাসের প্রমাণ নয়। বাস্তবে Zero Trust মানে: mTLS (service-to-service পরিচয় ও encryption), least-privilege IAM, device যাচাই, আর সব access log করা। এটি একদিনে হয় না — ধাপে ধাপে: প্রথমে SSO ও MFA, তারপর workload identity, তারপর mTLS, তারপর policy।

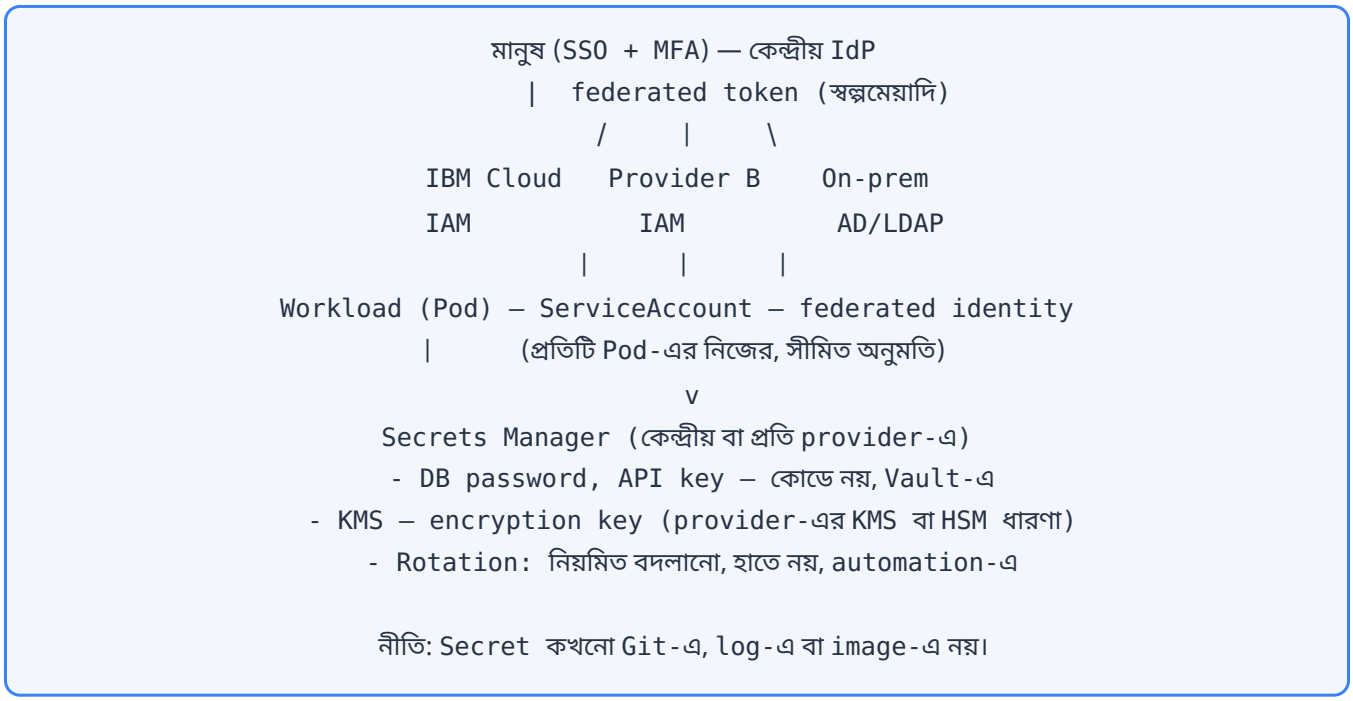
> **CONCEPT:** Workload identity মানে একটি Pod বা VM-এর নিজস্ব পরিচয় — মানুষের username/password নয়, workload-এর জন্য স্বল্পমেয়াদি credential। যেমন একটি Pod-কে object storage পড়ার জন্য তার ServiceAccount-এর সাথে যুক্ত একটি federated token দেওয়া হয়, যা কয়েক মিনিট বা ঘণ্টা পরে মেয়াদোত্তীর্ণ হয়। এতে দীর্ঘমেয়াদি API key চুরি যাওয়ার ঝুঁকি কমে, আর প্রতিটি workload-এর অনুমতি আলাদাভাবে audit করা যায়। Node-এর অনুমতি Pod-কে দেবেন না — এতে একটি Pod আপস হলে পুরো node-এর অনুমতি চলে যায়।

২২.৪ IAM-এর পার্থক্য ও federation নকশা

দিক	সাধারণ নীতি	Multi-cloud-এ সতর্কতা

Identity	কেন্দ্রীয় IdP + SSO	প্রতিটি provider-এ আলাদা user বানাবেন না
Role	Least privilege	এক provider-এর role অন্য provider-এ একই নামে ভিন্ন অর্থ হতে পারে
Policy	কোডে লেখা (Terraform)	হাতে console-এ policy দেবেন না — drift হয়
MFA	সবার জন্য বাধ্যতামূলক	Service account-এ MFA হয় না — তার বদলে স্বল্পমেয়াদি token
Audit	কেন্দ্রীয় log	প্রতিটি provider-এর audit log কেন্দ্রীয় জায়গায় পাঠান

২২.৫ আর্কিটেকচার ডায়াগ্রাম: identity ও secrets প্রবাহ



২২.৬ Secrets, KMS ও encryption

Secrets management মানে: password, API key ও token-কে একটি সুরক্ষিত store-এ রাখা (যেমন IBM Cloud Secrets Manager, HashiCorp Vault, বা provider-এর secret সেবা), অ্যাপ সেখান থেকে runtime-এ নেয়, কোডে বা environment variable-এ hardcode করা হয় না। Rotation মানে নিয়মিত secret বদলানো — হাতে নয়, automation-এ, যাতে পুরোনো secret চুরি গেলেও বেশিদিন কাজ না করে। KMS (Key Management Service) মানে encryption key-এর পরিচালনা — ডেটা encrypt করার key কে রাখে, কে ব্যবহার করতে পারে, কখন rotate হয়। নীতি: provider-এর managed key দিয়ে শুরু করুন, সংবেদনশীল ডেটার জন্য customer-managed key বিবেচনা করুন, আর key-এর access log রাখুন। Encryption at rest (ডিস্কে) ও in transit (TLS) — দুটিই বাধ্যতামূলক, একটি দিয়ে অন্যটি পূরণ হয় না।

২২.৭ নেটওয়ার্ক, container ও policy governance

নেটওয়ার্ক নিরাপত্তা: Security group ও firewall-এ least privilege, public-এ শুধু 443, ডেটাবেস private subnet-এ, cross-cloud ট্রাফিক TLS + mTLS। **Container নিরাপত্তা:** Image scanning (deploy-এর আগে দুর্বলতা খুঁজুন), signed image (কে বানিয়েছে তা যাচাই), Pod Security Standards (root হিসেবে চালাবেন না, read-only filesystem

যেখানে সম্ভব), runtime monitoring (অস্বাভাবিক আচরণ ধরা)। **Policy governance:** OPA (Open Policy Agent) বা provider-এর policy সেবা দিয়ে নিয়ম লেখা — যেমন "prod namespace-এ latest tag-এর image deploy করা যাবে না", "সব resource-এ team tag থাকতে হবে"। Policy-কে CI-তে পরীক্ষা করুন, শুধু prod-এ নয় — ভুল policy-ই deploy থামিয়ে দিতে পারে।

২২.৮ CLI/API উদাহরণ

```
# IAM — কে কী করতে পারে যাচাই (ধারণা; কমান্ড provider-ভেদে বদলায়)
ibmcloud iam access-policies
kubectl auth can-i create pods -n prod --as=developer@example.com

# Secret — Vault থেকে নেওয়া (উদাহরণ; আসল Vault ঠিকানা কাল্পনিক)
# vault kv get secret/technova/db
# কখনো secret-কে terminal-এ echo করবেন না, log-এ লিখবেন না

# Image scanning ও Pod নিরাপত্তা যাচাই
kubectl get pods -n prod -o yaml | grep -i "runAsUser\|readOnlyRootFilesystem"
kubectl get networkpolicy -A
```

২২.৯ IaC উদাহরণ: IAM policy Terraform-এ

```
# IAM policy কোডে — হাতে console-এ নয়
# (আসল resource নাম provider-ভেদে বদলায় — ধারণাগত উদাহরণ)
variable "developer_group" { default = "technova-developers" }

# নীতি: developer-রা prod-এ শুধু পড়তে পারে, বদলাতে পারে না
# resource "ibm_iam_access_group_policy" "dev_readonly_prod" {
#   access_group_id = var.developer_group
#   roles           = ["Viewer"]
#   resources { service = "containers-kubernetes" }
# }

# Kubernetes RBAC — namespace-ভিত্তিক Role
# apiVersion: rbac.authorization.k8s.io/v1
# kind: Role
# metadata: { name: pod-reader, namespace: prod }
# rules: [{ apiGroups: [""], resources: ["pods"], verbs: ["get", "list"] }]
```

২২.১০ প্রোডাকশন পরিস্থিতি ও incident response

পরিস্থিতি — API key ফাঁস: একটি API key Git-এ চলে গেছে। পদক্ষেপ: (১) key revoke করুন, (২) নতুন key তৈরি করে Vault-এ রাখুন, (৩) Git history থেকে মুছুন (শুধু ফাইল মুছলেই হবে না — history-ও পরিষ্কার করতে হয়), (৪) audit log-এ ওই key দিয়ে কী করা হয়েছে তা দেখুন, (৫) pre-commit hook-এ secret scanning বসান। **Incident response-এর ক্রম:** শনাক্ত → বিচ্ছিন্ন (আক্রান্ত workload আলাদা করা) → প্রমাণ সংরক্ষণ (log, snapshot) → নির্মূল (key বদলানো, দুর্বলতা ঠিক করা) → পুনরুদ্ধার → শিক্ষা (post-mortem)। Multi-cloud-এ incident-এর সময় সবচেয়ে বড় সমস্যা: log দুই জায়গায় ছড়িয়ে থাকা — তাই কেন্দ্রীয় log আগেই সাজাতে হয়, incident-এর সময় নয়।

REAL WORLD: TechNova-র একবার একটি developer-এর laptop থেকে একটি দীর্ঘমেয়াদি API key চুরি যাওয়ার আশঙ্কা হয়েছিল। তখন দেখা গেল ওই key-টি তিনটি সেবায় ব্যবহৃত হচ্ছে, আর কোনটি কোথায় তা লেখা নেই। শিক্ষা: প্রতিটি workload-এর জন্য আলাদা, স্বল্পমেয়াদি credential — একটি key দিয়ে সব সেবা চালাবেন না। Workload identity-তে গেলে এই সমস্যা অনেকটাই কমে।

২২.১১ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: MFA সবার জন্য, least privilege সব জায়গায়, secret Vault-এ, encryption দুই স্তরে, audit log কেন্দ্রীয়ভাবে, incident runbook লিখে রাখা। **খরচ:** KMS, Secrets Manager ও audit log-এর খরচ ব্যবহারের উপর বাড়ে — কিন্তু নিরাপত্তা ঘটনার খরচের তুলনায় তা নগণ্য। কখন ব্যবহার করবেন: যখন একাধিক provider, একাধিক দল, বা সংবেদনশীল ডেটা আছে — অর্থাৎ প্রায় সব prod পরিবেশে। কখন এড়াবেন: Zero Trust-এর সব স্তর একদিনে নেবেন না — ধাপে ধাপে নিন, না হলে দল থেমে যাবে। বিকল্প: provider-এর managed IAM ও security সেবা, SaaS secret manager, SaaS SIEM। Trade-off: নিজে চালালে নিয়ন্ত্রণ বেশি, পরিচর্যা বেশি; managed নিলে পরিচর্যা কম, কিন্তু provider-এর উপর নির্ভরতা বাড়ে।

WARNING: “আমাদের VPN আছে, তাই আমরা নিরাপদ” — এটি ভুল। VPN নেটওয়ার্কের একটি স্তর, পরিচয়ের প্রমাণ নয়। একজন আক্রমণকারী VPN-এর ভিতরে ঢুকতে পারলে, বা একটি Pod আপস হলে, VPN তাকে থামাবে না। তাই mTLS, workload identity ও least-privilege IAM — এই তিনটি VPN-এর পাশাপাশি লাগে।

২২.১২ সাধারণ ভুল

- দীর্ঘমেয়াদি API key কোডে, Git-এ বা image-এ রাখা।
- একটি key বা role দিয়ে সব সেবা চালানো।
- Pod-কে node-এর অনুমতি দেওয়া।
- MFA শুধু কিছু ব্যবহারকারীর জন্য রাখা।
- Audit log কেন্দ্রীয়ভাবে না পাঠানো।
- Image scanning ছাড়া deploy করা।
- Policy শুধু prod-এ পরীক্ষা করা, CI-তে নয়।

২২.১৩ ট্রাবলশুটিং

TROUBLESHOOTING: “Pod storage পড়তে পারছে না — 403” — ক্রম: (১) Pod-এর ServiceAccount কী; (২) তার সাথে যুক্ত federated identity কী; (৩) ওই identity-র policy-তে storage পড়ার অনুমতি আছে কি না; (৪) token-এর মেয়াদ আছে কি না; (৫) audit log-এ deny-র কারণ কী লেখা আছে। সবচেয়ে সাধারণ কারণ: policy-তে resource-এর নাম ভুল, বা token-এর scope ভুল। মানুষের IAM দিয়ে Pod-এর সমস্যা মেটাবেন না — Pod-এর নিজের identity যাচাই করুন।

২২.১৩ক গভীরতর আলোচনা: supply chain, audit trail ও নিরাপত্তার ধাপে ধাপে যাত্রা

Supply chain নিরাপত্তা মানে: আপনার অ্যাপে যা যা ঢুকছে — base image, library, Helm chart, Terraform module — তার প্রতিটির উৎস যাচাই করা। একটি দুর্বল বা আপস করা library আপনার কোডের চেয়েও বেশি ক্ষতি করতে পারে, কারণ তা আপনার অ্যাপের ভিতরে চলে। নিয়মগুলো: image-কে sign করা ও verify করা, dependency-কে নিয়মিত scan করা ও update করা, আর অজানা registry থেকে image না নেওয়া। SBOM (Software Bill of Materials) ধারণাটি এখানে কাজে আসে — আপনার অ্যাপে কী কী component আছে, তার তালিকা — যাতে কোনো library-তে দুর্বলতা ধরা পড়লে আপনি জানতে পারেন আপনার অ্যাপ আক্রান্ত কি না। ছোট দলের জন্য SBOM-কে জটিল করবেন না — image scanning ও dependency update-এর নিয়মিত অভ্যাসই যথেষ্ট শুরু।

Audit trail-এর ক্ষেত্রে প্রশ্নটি হলো: “কে, কখন, কী করল, আর তা কি অনুমোদিত ছিল?” প্রতিটি provider-এর audit log কেন্দ্রীয় জায়গায় পাঠান, আর সেই log-কে immutable রাখুন — যে log বদলানো যায়, তা প্রমাণ নয়। নিয়মিত audit করুন: কারা অতিরিক্ত অনুমতি পেয়ে আছে, কোন service account দীর্ঘদিন ব্যবহার হয়নি, কোন key rotate হয়নি। এই পর্যালোচনাটি ত্রৈমাসিক করুন — “একবার দিয়ে ভুলে যাওয়া” IAM-ই সবচেয়ে সাধারণ নিরাপত্তা গর্ত। Zero Trust-এর যাত্রাটিকে ধাপে ভাগ করুন: ধাপ ১ — SSO + MFA সবার জন্য; ধাপ ২ — IAM Terraform-এ, হাতে নয়; ধাপ ৩ — workload identity; ধাপ ৪ — mTLS service-to-service; ধাপ ৫ — device ও প্রসঙ্গ-ভিত্তিক নীতি। প্রতিটি ধাপের পরে পরিমাপ করুন — কতটি workload এখনো দীর্ঘমেয়াদি key ব্যবহার করে, কতটি service এখনো mTLS ছাড়া কথা বলে — সংখ্যাটি কমছে কি না, তা দেখুন।

নিরাপত্তা ঘটনার পরে সবচেয়ে গুরুত্বপূর্ণ কাজটি হলো শিক্ষা — কিন্তু শিক্ষা মানে শুধু post-mortem লেখা নয়, মানে নিয়ম বদলানো। একটি key ফাঁস হলে শুধু key বদলানো যথেষ্ট নয় — pre-commit hook-এ secret scanning বসানো, Vault-এর ব্যবহার বাধ্যতামূলক করা, আর দলকে জানানো — এই তিনটি একসাথে করতে হয়। আর মনে রাখুন: নিরাপত্তা ও ব্যবহারযোগ্যতার মধ্যে ভারসাম্য লাগে। MFA, Vault, policy — প্রতিটি পদক্ষেপ দলের কাজকে একটু ধীর করে, কিন্তু একটি ঘটনা দলের কাজকে সপ্তাহের জন্য থামিয়ে দেয়। এই trade-off-টি দলকে বুঝিয়ে বলুন — নিরাপত্তা “না” বলার জন্য নয়, নিরাপদে “হ্যাঁ” বলার জন্য।

২২.১৪ সারসংক্ষেপ

Multi-cloud নিরাপত্তা মানে: কেন্দ্রীয় SSO + federation, প্রতিটি workload-এর নিজস্ব স্বল্পমেয়াদি identity, least-privilege IAM, secret Vault-এ, encryption দুই স্তরে, mTLS, image scanning, policy governance, আর কেন্দ্রীয় audit log। Zero Trust একটি যাত্রা — একদিনে নয়, ধাপে ধাপে। Incident-এর প্রস্তুতি আগেই নিন: runbook, কেন্দ্রীয় log, আর key revoke-এর পদ্ধতি।

২২.১৫ পর্যালোচনা প্রশ্ন

1. Identity federation কী, এবং তা ছাড়া multi-cloud IAM-এ কী সমস্যা হয়?
2. Workload identity কেন দীর্ঘমেয়াদি API key-এর চেয়ে নিরাপদ?
3. Zero Trust-এর মূল ধারণা কী, এবং তা ধাপে ধাপে কীভাবে নেবেন?
4. Secrets rotation কেন জরুরি, এবং তা কীভাবে automation-এ করবেন?
5. Incident response-এর পাঁচটি ধাপ কী, এবং multi-cloud-এ কেন্দ্রীয় log কেন জরুরি?

২২.১৬ ব্যবহারিক অনুশীলন

1. TechNova-র জন্য একটি IAM matrix লিখুন: developer, platform engineer, auditor — প্রতিটি role কোন provider-এ, কোন namespace-এ, কী করতে পারে। Least privilege নীতি মেনে লিখুন।
2. একটি Kubernetes Role ও RoleBinding লিখুন যা prod namespace-এ শুধু Pod পড়তে দেয়, বদলাতে দেয় না। `kubectl auth can-i` দিয়ে যাচাইয়ের ধারণা লিখুন।

অধ্যায় 23 • PART V — Enterprise Multi-Cloud Operations

Chapter 23 — Disaster Recovery, Reliability and FinOps

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) RPO ও RTO ব্যাখ্যা ও পরিমাপ করতে; (২) backup strategy সাজাতে; (৩) cross-region ও cross-cloud recovery-র পার্থক্য বলতে; (৪) failure domain চিনতে; (৫) data consistency-র trade-off বুঝতে; (৬) data egress cost হিসাব করতে; (৭) cost allocation ও governance সাজাতে; এবং (৮) operational readiness (game day, runbook) পরিকল্পনা করতে।

২৩.২ পূর্বশর্ত ঝালাই

আপনার জানা থাকা উচিত: Backup ও restore-এর ধারণা, database replication-এর ধারণা (primary/replica), DNS-এর ধারণা, এবং cloud billing-এর ধারণা (ব্যবহারের উপর খরচ)। এই অধ্যায়ে আমরা ধরে নিচ্ছি আপনি একটি ডেটাবেসের backup নিতে পারেন, আর এখন প্রশ্ন হলো — পুরো সিস্টেম বন্ধ হলে কতক্ষণে ফিরবে, কত ডেটা হারাবে, আর তার খরচ কত?

২৩.৩ প্রথম নীতি থেকে: RPO, RTO ও failure domain

RPO (Recovery Point Objective) মানে: কতটুকু ডেটা হারানো সহ্য করা যায় — সময়ে মাপা। RPO ১ ঘণ্টা মানে শেষ ১ ঘণ্টার ডেটা হারানো সহ্য করা যায়; RPO ৫ মিনিট মানে শুধু শেষ ৫ মিনিটের ডেটা হারানো যায়। **RTO (Recovery Time Objective)** মানে: কতক্ষণে সেবা ফিরবে। RTO ৪ ঘণ্টা মানে outage-এর ৪ ঘণ্টার মধ্যে সেবা চালু করতে হবে। RPO ও RTO যত ছোট, খরচ তত বেশি — কারণ তার জন্য ক্রমাগত replication, standby infrastructure ও পরীক্ষা লাগে। ব্যবসার সাথে কথা বলে RPO/RTO ঠিক করুন — ইঞ্জিনিয়ার একা ঠিক করবেন না।

Failure domain মানে: একটি ব্যর্থতা কতদূর ছড়ায়। একটি availability zone একটি failure domain, একটি region একটি বড় failure domain, একটি provider সবচেয়ে বড় failure domain। DR পরিকল্পনা মানে: কোন failure domain-এর জন্য প্রস্তুতি নিচ্ছি, তা স্পষ্ট করা। Zone failure-এর জন্য multi-zone, region failure-এর জন্য cross-region, provider failure-এর জন্য cross-cloud — প্রতিটির খরচ আলাদা, আর সব failure domain-এর জন্য প্রস্তুতি নেওয়া প্রায়শই অসম্ভব ব্যয়বহুল।

> **CONCEPT:** Data consistency-র trade-off: ডেটা দুটি জায়গায় একই সময়ে একই থাকা (strong consistency) কঠিন ও ধীর — বিশেষ করে cross-cloud-এ, যেখানে network latency বেশি। Eventual consistency মানে ডেটা কিছু সময় পরে মিলে যাবে — দ্রুত, কিন্তু failover-এর মুহূর্তে দুটি জায়গায় ডেটা ভিন্ন হতে পারে। কোনটি নেবেন তা নির্ভর করে workload-এর উপর: billing-এর মতো ডেটায় strong consistency লাগতে পারে, analytics-এর মতো ডেটায় eventual যথেষ্ট। DR-এর সময় এই পার্থক্যই নির্ধারণ করে কত ডেটা হারাবে।

২৩.৪ Backup strategy ও cross-region / cross-cloud recovery

Backup-এর নিয়ম: 3-2-1 ধারণা — ডেটার ৩টি copy, ২টি ভিন্ন মাধ্যমে, ১টি ভিন্ন জায়গায়। TechNova-র জন্য: primary ডেটাবেস IBM Cloud-এ, replica অন্য region-এ, backup object storage-এ (অন্য region), আর গুরুত্বপূর্ণ archive on-prem-এ। Backup নেওয়া যথেষ্ট নয় — restore পরীক্ষা করতে হয়। একটি backup যা restore করা হয়নি, তা backup নয়, আশা মাত্র।

Cross-region recovery: একই provider-এর অন্য region-এ failover — নেটওয়ার্ক ও IAM একই provider-এর, তাই সহজ। Cross-cloud recovery: অন্য provider-এ failover — নেটওয়ার্ক, IAM, ডেটা ফরম্যাট — সব বদলায়, তাই কঠিন ও ব্যয়বহুল। বেশিরভাগ কোম্পানির জন্য cross-region যথেষ্ট; cross-cloud শুধু তখনই যখন provider failure-এর ঝুঁকি ব্যবসার জন্য অসহনীয়। Recovery-র ধাপ: (১) ডেটা replicate হয়েছে কি না যাচাই, (২) infrastructure Terraform/GitOps দিয়ে পুনর্নির্মাণ, (৩) DNS বদলে ট্রাফিক সরানো, (৪) স্বাস্থ্য যাচাই, (৫) ব্যবহারকারীকে জানানো।

২৩.৫ আর্কিটেকচার ডায়াগ্রাম: DR ও FinOps একসাথে

```
Primary (IBM Cloud, us-south)
- App (multi-zone) + PostgreSQL (primary)
- Backup → Object Storage (অন্য region)
|  async replication (RPO 15 মিনিট — উদাহরণ)
v
DR (IBM Cloud, অন্য region — cross-region)
- Standby app (ছোট, স্কেল করা যায়) + DB replica
- DNS failover (health check-এর উপর)
|
Cross-cloud DR (শুধু গুরুত্বপূর্ণ ডেটার জন্য)
- Provider B-তে backup copy (দৈনিক, সংক্ষিপ্ত)
- পুনর্নির্মাণ Terraform + GitOps দিয়ে (ঘণ্টার কাজ, মিনিটের নয়)

FinOps স্তর (সব provider জুড়ে):
- Tag: team / workload / environment (বাধ্যতামূলক)
- Cost dashboard: প্রতি workload-এর খরচ, egress আলাদা
- Alert: বাজেটের 80% হলে সতর্কতা
- মাসিক পর্যালোচনা: অব্যবহৃত resource মুছে ফেলা
```

২৩.৬ Data egress cost — সবচেয়ে অবহেলিত খরচ

Egress মানে cloud থেকে ডেটা বের করা — অন্য provider-এ, internet-এ, বা on-prem-এ। Ingress (ভিতরে আনা) প্রায়শই বিনামূল্যে, egress-এর খরচ আছে। Cross-cloud-এ egress সবচেয়ে বড় চমক: analytics-এর জন্য প্রতিদিন বড় ডেটা টানলে, বা backup প্রতিদিন অন্য provider-এ পাঠালে, egress বিল compute বিল ছাড়িয়ে যেতে পারে। সমাধান: (১) ডেটা সরানোর বদলে compute-কে ডেটার কাছে নেওয়া; (২) সংক্ষিপ্ত (aggregated) ডেটা পাঠানো, কাঁচা ডেটা নয়; (৩) replication-এর frequency ও পরিমাণ হিসাব করা; (৪) egress-কে dashboard-এ আলাদাভাবে দেখা। Egress খরচের কোনো নির্দিষ্ট সংখ্যা এখানে দেওয়া হয়নি — provider-এর বর্তমান মূল্য তালিকা দেখুন, আর নিজের ট্রাফিক পরিমাপ করুন।

২৩.৭ FinOps — cost allocation, governance ও operational readiness

FinOps মানে অর্থ ও পরিচালনার সমন্বয়: খরচ দেখা যায়, দায়িত্ব বোঝা যায়, আর অপচয় কমানো যায়। তিনটি স্তম্ভ: **Visibility** — প্রতিটি workload-এর খরচ tag দিয়ে দেখা যায়; **Optimization** — অব্যবহৃত resource, oversized VM, অপ্রয়োজনীয় replication কমানো; **Governance** — বাজেট, অনুমোদন, policy (যেমন "prod-এ বড় VM-এর জন্য অনুমোদন লাগে")। Operational readiness মানে: runbook লেখা, game day (ইচ্ছাকৃতভাবে ব্যর্থতা তৈরি করে পরীক্ষা) করা, on-call rotation, আর incident-এর পরে post-mortem। DR পরিকল্পনা যা পরীক্ষা করা হয়নি, তা পরিকল্পনা নয়।

২৩.৮ CLI/API উদাহরণ

```
# Backup যাচাই — শেষ backup কখন, সফল কি না
# (কমান্ড provider/টুল-ভেদে বদলায় — ধারণা)
kubectl get volumesnapshots -n prod
# PostgreSQL replica lag যাচাই (উদাহরণ)
# SELECT now() - pg_last_xact_replay_timestamp() AS lag;

# খরচ যাচাই — tag দিয়ে (ধারণা)
# ibmcloud billing ... / provider cost dashboard
# অব্যবহৃত resource খোঁজা
kubectl top nodes
kubectl top pods -n prod --sort-by=cpu

# DNS failover যাচাই
dig +short technova.example
curl -s -o /dev/null -w "%{http_code}\n" https://technova.example/health
```

২৩.৯ IaC উদাহরণ: backup ও budget alert Terraform-এ

```
# Backup নীতি — কোডে লেখা, হাতে নয়
# (আসল resource নাম provider-ভেদে বদলায় — ধারণাগত উদাহরণ)
locals {
  backup = {
    frequency = "daily"
    retention = "30d"
    region    = "us-south-2"  # primary থেকে ভিন্ন region
    rpo       = "15m"         # ব্যবসার সাথে ঠিক করা
    rto       = "4h"          # ব্যবসার সাথে ঠিক করা
  }
}

# Cost tag — প্রতিটি resource-এ বাধ্যতামূলক
# tags = ["team:platform", "workload:web", "env:prod"]
# Tag ছাড়া resource CI-তে ব্যর্থ হবে (policy দিয়ে)
```

২৩.১০ প্রোডাকশন পরিস্থিতি

পরিস্থিতি — Region outage: Primary region বন্ধ। DNS health check ব্যর্থতা ধরে, ট্রাফিক DR region-এ যায়, DB replica primary হয়। RTO ৪ ঘণ্টার মধ্যে ফিরতে পারলে সফল। শিক্ষা: DNS failover-এর TTL কম রাখুন, আর DR-এর app-কে নিয়মিত পরীক্ষা করুন — standby app যা কখনো চালু হয়নি, তা চালু হবে কি না তা কেউ জানে না। **পরিস্থিতি —**

Egress bill: Backup প্রতিদিন cross-cloud-এ পাঠানো হচ্ছিল — বিল দেখে frequency কমিয়ে সাপ্তাহিক করা হলো, আর সংক্ষিপ্ত ডেটা পাঠানো হলো। শিক্ষা: backup-এর frequency RPO দিয়ে ঠিক করুন, "যত বেশি তত ভালো" দিয়ে নয়।

> **REAL WORLD:** TechNova-র game day-তে দেখা গেল DR region-এর ডেটাবেস replica lag ২ ঘণ্টা — RPO ১৫ মিনিটের লক্ষ্য পূরণ হচ্ছে না। কারণ: replication-এর network bandwidth কম ছিল। সমাধান: bandwidth বাড়ানো ও lag-এর alert বসানো। Game day ছাড়া এই সমস্যা আসল outage-এর সময় ধরা পড়ত — তখন অনেক দেরি হয়ে যেত।

২৩.১১ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: Backup-ও encrypt করুন — backup-এ পুরো ডেটাবেস থাকে, চুরি গেলে সব চলে যায়। Backup-এর access সীমিত করুন, আর backup-এর restore পরীক্ষায় sensitive ডেটা যেন staging-এ না ছড়ায় সেদিকে নজর দিন। **খরচ:** DR-এর খরচ: standby infrastructure, replication, backup storage, egress, আর পরীক্ষার সময়। Active-active সবচেয়ে ব্যয়বহুল, backup-and-restore সবচেয়ে সস্তা — RPO/RTO দিয়ে বেছে নিন। কখন ব্যবহার করবেন: যখন ব্যবসা outage-এর খরচ জানে, আর RPO/RTO লিখিতভাবে ঠিক হয়েছে। কখন এড়াবেন: যখন RPO/RTO ছাড়া "সবচেয়ে ভালো DR" চাওয়া হয় — তাতে খরচ অসীম হয়ে যায়। বিকল্প: cross-region (একই provider), backup-and-restore (সবচেয়ে সস্তা), pilot light (ছোট standby), warm standby (মাঝারি)। Trade-off সবসময় একই: দ্রুত recovery বনাম কম খরচ।

> **WARNING:** Backup নেওয়া মানেই DR নয়। Backup থেকে restore করতে কতক্ষণ লাগে, তা পরীক্ষা না করলে RTO একটি অনুমান মাত্র। আর backup-এর ডেটা যদি primary-র মতোই আক্রান্ত হয় (যেমন ransomware-এ), তাহলে backup-ও কাজ করবে না — তাই backup-কে immutable বা ভিন্ন অ্যাকাউন্টে রাখার ধারণা বিবেচনা করুন।

২৩.১২ সাধারণ ভুল

- RPO/RTO ব্যবসার সাথে ঠিক না করে "সবচেয়ে ভালো" চাওয়া।
- Backup নেওয়া, restore পরীক্ষা না করা।
- DR কখনো পরীক্ষা না করা (game day না করা)।
- Egress খরচ হিসাব না করে cross-cloud replication করা।
- Tag ছাড়া resource — খরচের দায়িত্ব জানা যায় না।
- Standby-কে কখনো চালু না করা — চালু হবে কি না জানা নেই।

২৩.১৩ ট্রাবলশুটিং

> **TROUBLESHOOTING:** "Replica lag বাড়ছে" — প্রথমে দেখুন replication-এর network bandwidth, primary-র লেখার পরিমাণ, আর replica-র resource (CPU/disk)। Lag-এর alert বসান — RPO ছাড়িয়ে গেলে জানতে হবে। "Restore ধীর" — backup-এর আকার, storage-এর গতি, আর restore-এর পদ্ধতি (snapshot বনাম logical dump) যাচাই করুন। Restore-এর সময় পরিমাপ করে RTO-তে লিখুন — অনুমান নয়, পরিমাপ।

২৩.১৩ক গভীরতর আলোচনা: game day, FinOps-এর সংস্কৃতি ও খরচের মনস্তত্ত্ব

Game day মানে ইচ্ছাকৃতভাবে ব্যর্থতা তৈরি করে দেখা — সিস্টেম কী করে। ছোট থেকে শুরু করুন: একটি Pod মুছে দেখা, একটি zone বন্ধ করা, একটি dependency-কে ধীর করা। প্রতিটি game day-এর আগে লিখুন: কী পরীক্ষা করছেন, সফলতার মানদণ্ড কী, কে কী করবে, আর কখন থামবেন (abort condition)। Game day-এর সময় সবাইকে জানিয়ে রাখুন — হঠাৎ করে prod-এ ব্যর্থতা তৈরি করা, কাউকে না জানিয়ে, একটি incident তৈরি করার মতোই খারাপ। Game day-এর পরে post-mortem লিখুন — কী শিখলেন, কী ঠিক করবেন, আর পরের game day কবে। TechNova-র অভিজ্ঞতা: প্রথম game day-তে দেখা গেল DR-এর app চালু হতে ৪০ মিনিট লাগে — RTO ৪ ঘণ্টার মধ্যে থাকলেও, কেউ তা জানত না। পরিমাপ ছাড়া RTO একটি অনুমান মাত্র — game day সেই অনুমানকে সংখ্যায় পরিণত করে।

FinOps-এর সংস্কৃতি মানে: খরচকে কারো একার দায়িত্ব ভাববেন না — প্রতিটি দল নিজের workload-এর খরচ দেখতে পায়, আর খরচ কমানোকে ভালো কাজ হিসেবে স্বীকৃতি দেওয়া হয়। মাসিক FinOps পর্যালোচনায় তিনটি প্রশ্ন করুন: কোন workload-এর খরচ সবচেয়ে বেশি বেড়েছে, কেন বেড়েছে, আর তা কি মূল্য দিচ্ছে? অব্যবহৃত resource-এর তালিকা দেখান — পুরোনো snapshot, বন্ধ Pod-এর PV, অপ্রয়োজনীয় LoadBalancer — আর সেগুলো মোছার দায়িত্ব দিন। Reserved capacity বা commitment-এর সিদ্ধান্ত নেওয়ার আগে ব্যবহারের ধারাবাহিকতা পরিমাপ করুন — অস্থির workload-এর জন্য commitment করা মানে ভবিষ্যতের খরচে আটকে যাওয়া। আর egress-কে আলাদা করে দেখুন — egress প্রায়শই “অন্য কারো বিল” ভেবে উপেক্ষিত হয়, কিন্তু cross-cloud-এ এটিই সবচেয়ে বড় চমক।

খরচের মনস্তত্ত্বটিও বুঝুন: মানুষ খরচ দেখতে পায় না বলে অপচয় করে — তাই খরচকে দৃশ্যমান করুন। প্রতিটি pull request-এ Terraform plan-এর সাথে আনুমানিক খরচের পরিবর্তন দেখান (যদি টুল থাকে), প্রতিটি dashboard-এ খরচের গ্রাফ রাখুন, আর প্রতিটি দলের জন্য বাজেটের সীমা স্পষ্ট করুন। কিন্তু খরচ কমানোকে একমাত্র লক্ষ্য করবেন না — reliability ও নিরাপত্তার খরচ কমানো মানে ভবিষ্যতের incident-এর খরচ বাড়ানো। FinOps-এর সঠিক প্রশ্নটি হলো: “এই খরচটি কি মূল্য দিচ্ছে?” — শুধু “এই খরচটি কি কমানো যায়?” নয়।

২৩.১৪ সারসংক্ষেপ

DR মানে RPO ও RTO — ব্যবসার সাথে ঠিক করা, পরিমাপ করা, পরীক্ষা করা। Backup 3-2-1 নীতিতে, restore পরীক্ষাসহ। Cross-region সহজ, cross-cloud কঠিন ও ব্যয়বহুল — শুধু তখনই যখন provider failure-এর ঝুঁকি অসহনীয়। Egress খরচ DR ও analytics-এর নকশায় হিসাব করুন। FinOps মানে tag, visibility, optimization ও governance — আর operational readiness মানে runbook ও game day।

২৩.১৫ পর্যালোচনা প্রশ্ন

1. RPO ও RTO-র মধ্যে পার্থক্য কী, এবং ছোট RPO/RTO-র খরচ কেন বেশি?
2. Failure domain কী, এবং zone, region ও provider failure-এর জন্য প্রস্তুতি কীভাবে আলাদা?
3. Strong ও eventual consistency-র trade-off DR-এ কীভাবে প্রভাব ফেলে?
4. Egress খরচ কীভাবে DR ও analytics-এর নকশা বদলে দেয়?
5. FinOps-এর তিনটি স্তম্ভ কী, এবং tag কেন জরুরি?

২৩.১৬ ব্যবহারিক অনুশীলন

1. TechNova-র একটি workload-এর জন্য RPO ও RTO লিখুন (ব্যবসার যুক্তিসহ), তারপর backup frequency, retention ও DR পদ্ধতি (backup-and-restore / pilot light / warm standby) বেছে নিন।

2. একটি FinOps dashboard-এর খসড়া তৈরি করুন: কোন tag থাকবে, কোন খরচ আলাদাভাবে দেখাবেন (compute, storage, egress), আর বাজেটের 80%-এ কী alert যাবে।

অধ্যায় 24 • PART V — Enterprise Multi-Cloud Operations

Chapter 24 — Enterprise Multi-Cloud Architecture

২৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) enterprise architecture-এর নীতি প্রয়োগ করতে; (২) provider selection-এর সিদ্ধান্ত যুক্তি দিয়ে নিতে; (৩) workload placement, নেটওয়ার্ক, identity, Kubernetes platform, ডেটা, observability, নিরাপত্তা ও FinOps — সব স্তর একসাথে নকশা করতে; এবং (৪) TechNova-র multi-cloud capstone-কে end-to-end হেঁটে দেখাতে — কেন প্রতিটি provider ব্যবহার করা হচ্ছে, কেন কিছু workload এক provider-এই থাকছে।

২৪.২ পূর্বশর্ত ঝালাই

এই অধ্যায়টি Part IV+V-এর সমাপ্তি — অধ্যায় ১৭–২৩-এর সব ধারণা এখানে একত্র হয়। আপনার জানা থাকা উচিত: multi-cloud pattern, hub-and-spoke নেটওয়ার্ক, Kubernetes/GitOps, Terraform/Ansible, observability (OTel/Prometheus), Zero Trust, RPO/RTO ও FinOps। যদি কোনো ধারণা ঝাপসা মনে হয়, তাহলে সেই অধ্যায়ে ফিরে যান — এই অধ্যায়টি সংক্ষিপ্তসার নয়, প্রয়োগ।

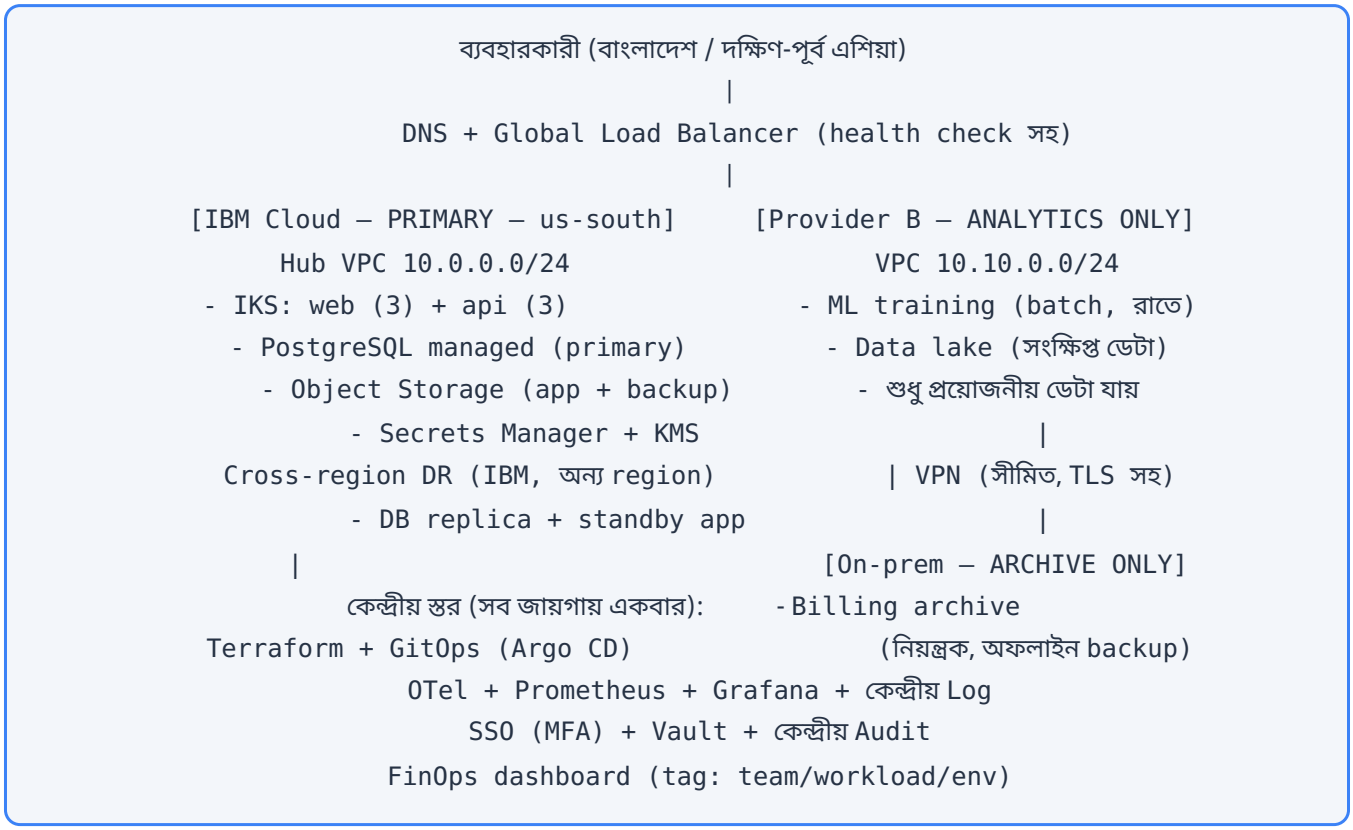
২৪.৩ প্রথম নীতি থেকে: enterprise architecture মানে সিদ্ধান্তের শৃঙ্খলা

Enterprise architecture মানে প্রযুক্তির তালিকা নয় — সিদ্ধান্তের শৃঙ্খলা। প্রতিটি সিদ্ধান্তের পিছনে একটি প্রয়োজন, একটি বিকল্প, একটি trade-off ও একটি পর্যালোচনার তারিখ থাকতে হবে। নীতিগুলো: (১) **সরলতা প্রথম** — যত কম provider, যত কম পদ্ধতি, তত ভালো; নতুন provider শুধু তখনই যখন প্রয়োজন পরিমাপযোগ্য। (২) **একটি সত্যের উৎস** — Git-এ infrastructure, Git-এ workload, কেন্দ্রীয় IAM, কেন্দ্রীয় observability। (৩) **Least privilege ও Zero Trust** — প্রতিটি স্তরে। (৪) **পরীক্ষিত resilience** — দাবি নয়, game day-র প্রমাণ। (৫) **খরচের স্বচ্ছতা** — প্রতিটি workload-এর খরচ জানা যায়। এই নীতিগুলো ভাঙলে architecture-এর যেকোনো স্তরই ভেঙে পড়ে।

> **CONCEPT:** Provider selection matrix মানে একটি টেবিল যেখানে প্রতিটি workload-এর জন্য প্রতিটি provider-কে নির্দিষ্ট মানদণ্ডে (ডেটা residency, সেবা, latency, দক্ষতা, খরচ, নিয়ন্ত্রক) মূল্যায়ন করা হয়, আর সিদ্ধান্তের কারণ লেখা থাকে। Matrix-এর উদ্দেশ্য “সবচেয়ে ভালো provider” খোঁজা নয় — উদ্দেশ্য হলো প্রতিটি workload-এর জন্য “কেন এই provider, কেন অন্য provider নয়” — তা লিখিতভাবে দেখানো। যে workload-এর জন্য কোনো provider-এর বিশেষ কারণ নেই, সেটি প্রধান provider-এই থাকে।

২৪.৪ Capstone — TechNova-র multi-cloud architecture end-to-end

TechNova-র অবস্থা: ১২ জনের প্ল্যাটফর্ম দল, মূল SaaS অ্যাপ, গ্রাহক বাংলাদেশ ও দক্ষিণ-পূর্ব এশিয়ায়, billing ডেটায় নিয়ন্ত্রক বাধ্যবাধকতা, analytics-এ একটি নির্দিষ্ট ML সেবার প্রয়োজন, আর DR-এর জন্য RPO ১৫ মিনিট, RTO ৪ ঘণ্টা (ব্যবসার সাথে ঠিক করা)। দল ছোট — তাই সরলতা অগ্রাধিকার। এই প্রসঙ্গে capstone-এর সিদ্ধান্তগুলো দেখুন।



২৪.৫ Provider selection matrix — কেন প্রতিটি provider, কেন নয়

এই matrix-টিই capstone-এর কেন্দ্র। লক্ষ্য করুন: চারটি provider-এ সব workload ছড়িয়ে দেওয়া হয়নি — প্রতিটি provider-এর একটি নির্দিষ্ট, লিখিত কারণ আছে, আর অনেক workload প্রধান provider-এই আছে।

Workload	Provider	কেন এই provider	কেন অন্য provider নয়
Web + API (মূল অ্যাপ)	IBM Cloud (primary)	দলের গভীর দক্ষতা, IKS + OpenShift বিকল্প, প্রধান region-এ উপস্থিতি	সরানো মানে দক্ষতা ভাগ করা, egress ও দ্বিগুণ পরিচর্যা — কোনো বিশেষ কারণ নেই
PostgreSQL (primary)	IBM Cloud	অ্যাপের কাছে ডেটা — latency ও egress কম, managed backup	অন্য provider-এ নিলে প্রতিটি query-তে cross-cloud latency ও egress
Analytics / ML training	Provider B	নির্দিষ্ট ML সেবা যা primary-তে নেই (পরিমাপযোগ্য প্রয়োজন), batch কাজ — latency কম সংবেদনশীল	মূল অ্যাপ এখানে নয় — দল এখানে গভীর নয়, আর ML-এর বাইরে কোনো কারণ নেই
Billing archive	On-prem	নিয়ন্ত্রক বাধ্যবাধকতা — ডেটা নিজের নিয়ন্ত্রণে রাখতে হয়	Cloud-এ রাখলে নিয়ন্ত্রক শর্ত পূরণ হয় না (এই ক্ষেত্রে)
DR	IBM Cloud, অন্য region (cross-region)	RPO/RTO পূরণ হয়, IAM ও নেটওয়ার্ক একই provider-এর — সহজ ও সস্তা	Cross-cloud DR নয় — provider failure-এর ঝুঁকি ব্যবসার জন্য অসহনীয় নয়, আর খরচ দ্বিগুণ হতো
— (ইচ্ছাকৃতভাবে ফাঁকা)	Provider C / D ব্যবহার করা হয়নি	কোনো workload-এর জন্য তাদের বিশেষ কারণ নেই	

			চতুর্থ provider যোগ করা মানে চতুর্থ IAM, চতুর্থ billing, চতুর্থ দক্ষতা — কোনো লাভ ছাড়া

এই matrix-এর শিক্ষা: “আমরা চারটি provider ব্যবহার করি” গর্বের বিষয় নয় — “আমরা প্রতিটি provider কেন ব্যবহার করি তা জানি” গর্বের বিষয়। Provider C/D-কে ফাঁকা রাখা একটি সচেতন সিদ্ধান্ত, অভাব নয়।

২৪.৬ স্তরে স্তরে নকশা: নেটওয়ার্ক, identity, Kubernetes, ডেটা

নেটওয়ার্ক: Hub-and-spoke (অধ্যায় ১৮) — hub-এ firewall, DNS forwarder, VPN termination; spoke-এ web/API/data; CIDR পরিকল্পনা একটি উৎসে; Provider B-তে শুধু সীমিত VPN, শুধু analytics ট্রাফিক। **Identity:** কেন্দ্রীয় SSO + MFA, federation দিয়ে প্রতিটি provider-এ; workload identity Pod-স্তরে; secret Vault-এ; KMS-এ key। **Kubernetes platform:** IKS প্রধান, Provider B-তে ছোট ক্লাস্টার শুধু analytics-এর জন্য; GitOps দিয়ে দুটি ক্লাস্টারই একই Git থেকে sync; Helm-এ provider-নির্দিষ্ট value আলাদা। **ডেটা:** Data gravity নীতি — ডেটা যেখানে, compute সেখানে; cross-cloud-এ শুধু সংক্ষিপ্ত ডেটা; PostgreSQL primary এক জায়গায়, replica অন্য region-এ; backup 3-2-1।

২৪.৭ Observability, নিরাপত্তা governance ও FinOps — একসাথে

Observability: OTel দিয়ে সব অ্যাপ instrumented, প্রতিটি ক্লাস্টারে Collector, কেন্দ্রীয় Prometheus/Grafana/Log/Trace; একটি dashboard-এ সব provider-এর স্বাস্থ্য; alert শুধু তখনই যখন মানুষের কাজ লাগে। **নিরাপত্তা governance:** Policy (OPA) CI-তে — latest tag নিষিদ্ধ, tag বাধ্যতামূলক; image scanning deploy-এর আগে; audit log কেন্দ্রীয়ভাবে; incident runbook ও game day। **FinOps:** Tag বাধ্যতামূলক, প্রতি workload-এর খরচ dashboard-এ, egress আলাদাভাবে, বাজেট alert ৮০%-এ, মাসিক পর্যালোচনায় অব্যবহৃত resource মোছা। এই তিনটি স্তর আলাদা দলের কাজ নয় — একই প্ল্যাটফর্ম দলের দায়িত্ব, একই pipeline-এ।

২৪.৮ CLI/API ও IaC উদাহরণ: capstone-এর সত্যের উৎস

```
# Capstone-এর Git কাঠামো (একটি সত্যের উৎস)
# technova-infra/
#   terraform/ibm/       — primary (VPC, IKS, DB)
#   terraform/provider-b/ — analytics only
#   terraform/onprem/    — archive (সীমিত)
#   helm/web/            — values-ibm.yaml / values-provider-b.yaml
#   ansible/             — configuration
#   policies/            — OPA policy (tag, image, latest নিষিদ্ধ)
#   runbooks/            — incident + DR runbook

# স্বাস্থ্য যাচাই — একটি কমান্ডে সব স্তর
kubectl --context technova-iks get pods -n prod
kubectl --context technova-provider-b get pods -n analytics
curl -s https://technova.example/health
terraform plan -detailed-exitcode  # drift আছে কি না
```

```
# OPA policy — ধারণা (Rego-র সরল রূপ)
# package technova
# deny[msg] {
#   input.kind == "Deployment"
```

```
#   image := input.spec.template.spec.containers[_].image
#   endswith(image, ":latest")
#   msg := "latest tag নিষিদ্ধ — নির্দিষ্ট version ব্যবহার করুন"
# }
```

২৪.৯ প্রোডাকশন পরিস্থিতি: capstone-এর একটি দিন

সকাল — Deploy: Developer pull request দেয়, CI-তে Terraform plan ও OPA policy পরীক্ষা হয়, পর্যালোচনার পর merge হয়, Argo CD IKS-এ sync করে, Grafana-তে error rate দেখা হয়। **দুপুর — Analytics batch:** রাতে নয়, দুপুরে একটি বড় batch Provider B-তে চলে — সংক্ষিপ্ত ডেটা VPN দিয়ে যায়, egress dashboard-এ দেখা যায়। **রাত — Game day:** একটি zone বন্ধ করে দেখা হয় app multi-zone-এ চলছে কি না; DR region-এ replica lag যাচাই করা হয়। **মাসের শেষ — FinOps পর্যালোচনা:** অব্যবহৃত node, পুরোনো snapshot, আর egress-এর শীর্ষ workload — তিনটি নিয়ে সিদ্ধান্ত হয়। এই দিনটির শিক্ষা: capstone একটি ছবি নয়, একটি অভ্যাস — প্রতিদিনের pipeline, প্রতি মাসের পর্যালোচনা, প্রতি ত্রৈমাসিকের game day।

REAL WORLD: TechNova প্রথমে capstone-এ চারটি provider-এ সব workload ভাগ করতে চেয়েছিল — “তাহলে আমরা সত্যিকারের multi-cloud হব”। Matrix লিখতে গিয়ে দেখা গেল: Provider C/D-এর জন্য কোনো workload-এরই বিশেষ কারণ নেই — শুধু “ভাগ করা”। তারা সিদ্ধান্ত বদলে দুটি cloud + on-prem archive নিল, আর Provider C/D-কে ভবিষ্যৎ পর্যালোচনার জন্য লিখে রাখল। ছয় মাস পরে দল বলেছিল: এই সিদ্ধান্তই তাদের বাঁচিয়েছে — দুটি provider-ই তাদের পুরো সময় নিচ্ছিল, চারটি হলে কিছুই ভালোভাবে চলত না।

২৪.১০ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: Capstone-এর প্রতিটি স্তরে least privilege, encryption দুই স্তরে, audit log কেন্দ্রীয়ভাবে, আর incident response-এর runbook। সবচেয়ে বড় ঝুঁকি: provider-এর সংখ্যা বাড়লে IAM-এর ভুলের সুযোগ বাড়ে — তাই provider-এর সংখ্যা কম রাখা নিজেই একটি নিরাপত্তা সিদ্ধান্ত। **খরচ:** খরচের চারটি ধারা — প্রতিটি provider-এর বিল, egress, duplicated tooling, মানুষের সময়। Capstone-এ মানুষের সময়ই সবচেয়ে বড় খরচ — ১২ জনের দল দুটি provider ভালোভাবে চালাতে পারে, চারটি নয়। কখন এই architecture ব্যবহার করবেন: যখন উপরের matrix-এর মতো পরিমাপযোগ্য প্রয়োজন আছে। কখন এড়াবেন: যখন দল ছোট ও workload সাধারণ — তখন single-cloud + multi-region যথেষ্ট। বিকল্প: single-cloud, hybrid, বা managed platform সেবা। Trade-off সবসময় একই — সরলতা ও গতি বনাম নমনীয়তা ও resilience — আর এই অধ্যায়ের matrix-ই সেই trade-off-কে লিখিতভাবে দেখায়।

WARNING: Capstone-কে “একবার বানালেই শেষ” ভাববেন না। Provider-এর সেবা বদলায়, ব্যবসার প্রয়োজন বদলায়, দলের দক্ষতা বাড়ে। প্রতি ছয় মাসে provider selection matrix পর্যালোচনা করুন — কোনো provider-এর কারণ কি এখনো আছে? কোনো workload-এর placement কি বদলানো উচিত? পর্যালোচনা ছাড়া architecture ধীরে ধীরে ইতিহাস হয়ে যায়, বর্তমান থাকে না।

২৪.১১ সাধারণ ভুল

- কারণ ছাড়া প্রতিটি workload সব provider-এ ভাগ করা।
- Matrix না লিখে “সবাই যা করছে” তা অনুকরণ করা।
- DR ও game day-কে “পরে করব” বলে ফেলে রাখা।

- Tag ও FinOps-কে “billing-এর কাজ” ভেবে অবহেলা করা।
- Observability ও IAM-কে প্রতিটি provider-এ আলাদা রাখা।
- Architecture-কে একবারের দলিল ভাবা, পর্যালোচনা না করা।

২৪.১২ ট্রাবলশুটিং

TROUBLESHOOTING: Capstone-এ সমস্যা হলে স্তর ধরে যাচাই করুন: (১) DNS ও নেটওয়ার্ক (অধ্যায় ১৮) — ট্রাফিক কোথায় যাচ্ছে; (২) Kubernetes (অধ্যায় ১৯) — Pod ও Service ঠিক আছে কি না; (৩) GitOps (অধ্যায় ২০) — ক্লাস্টার Git-এর সাথে মিলছে কি না (drift); (৪) Observability (অধ্যায় ২১) — metrics/trace/log-এ কখন শুরু হলো; (৫) নিরাপত্তা (অধ্যায় ২২) — IAM deny হচ্ছে কি না; (৬) DR/FinOps (অধ্যায় ২৩) — replica lag ও খরচ। একটি স্তরে আটকে থাকবেন না — সমস্যা প্রায়শই স্তরের সীমানায় থাকে।

২৪.১২ক গভীরতর আলোচনা: platform দল, maturity মডেল ও ভবিষ্যৎ পর্যালোচনা

Enterprise architecture-কে টিকিয়ে রাখে platform দল — যারা অন্য দলের জন্য পথ তৈরি করে। Platform দলের কাজটি হলো “golden path” বানানো: নতুন workload deploy করতে হলে কোন Terraform module, কোন Helm chart, কোন pipeline ব্যবহার করতে হবে — তা আগে থেকে ঠিক করা, যাতে প্রতিটি দল নিজে নিজে আবিষ্কার না করে। Golden path-কে বাধ্যতামূলক করবেন না — তাতে দল পালিয়ে যাবে; বরং golden path-কে সবচেয়ে সহজ পথ করুন — যে পথে গেলে নিরাপত্তা, observability ও FinOps স্বয়ংক্রিয়ভাবে পাওয়া যায়, আর অন্য পথে গেলে নিজে সব করতে হয়। TechNova-র platform দলের নিয়ম: নতুন workload-এর জন্য একটি template repository — Terraform, Helm, CI, dashboard, alert, runbook — সব একসাথে। একটি নতুন সেবা শুরু করতে ১ দিন লাগে, ১ সপ্তাহ নয় — কারণ template-এ সবকিছু আগে থেকে আছে।

Maturity-কে ধাপে ভাবুন: ধাপ ১ — একটি provider, Terraform ও GitOps; ধাপ ২ — multi-region, কেন্দ্রীয় observability; ধাপ ৩ — দ্বিতীয় provider শুধু নির্দিষ্ট workload-এর জন্য, federation ও কেন্দ্রীয় IAM; ধাপ ৪ — cross-cloud DR (যদি প্রয়োজন হয়), FinOps-এর পরিপক্কতা, game day-এর নিয়মিত অভ্যাস। প্রতিটি ধাপে যাবেন শুধু তখনই যখন আগের ধাপটি স্থিতিশীল — ধাপ ১-কে এড়িয়ে ধাপ ৪-এ গেলে ভিত্তি দুর্বল থাকে, আর incident-এর সময় তা ধরা পড়ে। Maturity মাপার প্রশ্নগুলো: আমরা কি failover পরীক্ষা করেছি? আমরা কি প্রতিটি workload-এর খরচ জানি? আমরা কি প্রতিটি provider-এর IAM audit করেছি? যদি কোনো প্রশ্নের উত্তর “না” হয়, তাহলে পরের ধাপে যাওয়ার আগে সেই প্রশ্নের উত্তর “হ্যাঁ” করুন।

ভবিষ্যৎ পর্যালোচনার জন্য একটি ক্যালেন্ডার রাখুন: প্রতি ছয় মাসে provider selection matrix, প্রতি ত্রৈমাসিকে IAM audit ও game day, প্রতি মাসে FinOps পর্যালোচনা, প্রতি সপ্তাহে drift check। পর্যালোচনার সময় তিনটি প্রশ্ন করুন: কী বদলেছে (ব্যবসা, provider, দল)? আমাদের সিদ্ধান্তগুলো কি এখনো সঠিক? কী সরল করা যায়? সরল করা — provider কমানো, টুল কমানো, dashboard কমানো — একটি বৈধ architecture সিদ্ধান্ত, দুর্বলতা নয়। সবশেষে, এই বইয়ের Part IV+V-এর মূল বার্তাটি মনে রাখুন: multi-cloud একটি লক্ষ্য নয়, একটি উপায়। লক্ষ্য হলো — ব্যবহারকারীর জন্য নির্ভরযোগ্য, নিরাপদ, আর টেকসই সেবা। যে architecture সেই লক্ষ্য সবচেয়ে সরলভাবে দেয়, সেটিই সঠিক architecture — তা single-cloud হোক, hybrid হোক, বা সাবধানে বেছে নেওয়া multi-cloud হোক।

২৪.১৩ সারসংক্ষেপ

Enterprise multi-cloud architecture মানে সিদ্ধান্তের শৃঙ্খলা: সরলতা প্রথম, একটি সত্যের উৎস, least privilege, পরীক্ষিত resilience, খরচের স্বচ্ছতা। TechNova-র capstone দেখায় — মূল অ্যাপ ও ডেটাবেস IBM Cloud-এ (দক্ষতা ও data gravity), analytics শুধু Provider B-তে (নির্দিষ্ট ML সেবা), archive on-prem-এ (নিয়ন্ত্রক), DR cross-region-এ (সহজ ও সস্তা), আর Provider C/D ইচ্ছাকৃতভাবে ব্যবহার করা হয়নি। নেটওয়ার্ক hub-and-spoke, identity federation, Kubernetes GitOps, ডেটা gravity-ভিত্তিক, observability কেন্দ্রীয়, নিরাপত্তা Zero Trust-এর দিকে ধাপে ধাপে, FinOps tag-ভিত্তিক — সব একসাথে, একই pipeline-এ। Multi-cloud স্বয়ংক্রিয়ভাবে ভালো নয় — ভালো তখনই যখন প্রতিটি provider-এর কারণ লেখা থাকে, আর সেই কারণ নিয়মিত পর্যালোচনা করা হয়।

২৪.১৪ পর্যালোচনা প্রশ্ন

1. Provider selection matrix-এর উদ্দেশ্য কী, এবং এতে “ফাঁকা” provider রাখা কেন একটি বৈধ সিদ্ধান্ত?
2. TechNova কেন cross-region DR বেছে নিল, cross-cloud DR নয়? Trade-off কী ছিল?
3. Data gravity কীভাবে workload placement-কে প্রভাবিত করে? একটি উদাহরণ দিন।
4. Capstone-এর কেন্দ্রীয় স্তরগুলো (GitOps, observability, IAM, FinOps) কেন একবার লেখা হয়, প্রতিটি provider-এ আলাদা নয়?
5. Enterprise architecture-কে প্রতি ছয় মাসে পর্যালোচনা করা কেন জরুরি?

২৪.১৫ ব্যবহারিক অনুশীলন

1. TechNova-র matrix-এর আদলে নিজের একটি কাল্পনিক কোম্পানির জন্য provider selection matrix তৈরি করুন: ৫টি workload, প্রতিটির জন্য provider, কারণ ও “কেন অন্য provider নয়” লিখুন। অন্তত একটি provider-কে ইচ্ছাকৃতভাবে ফাঁকা রাখুন ও তার যুক্তি লিখুন।
2. Capstone-এর Git কাঠামোর খসড়া তৈরি করুন (terraform/, helm/, policies/, runbooks/) এবং একটি DR game day-র পরিকল্পনা লিখুন: কী বন্ধ করবেন, কী পরিমাপ করবেন, সফলতার মানদণ্ড কী, আর কে কী করবে।